ΠΑΝΕΠΙΣΤΗΜΙΟ
ΠΑΤΡΩΝ
UNIVERSITY OF PATRAS

ΠΟΛΥΤΕΧΝΙΚΗ ΣΧΟΛΗ

ΤΜΗΜΑ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΤΕΧΝΟΛΟΓΙΑΣ ΥΠΟΛΟΓΙΣΤΩΝ

Σχεδίαση και Ανάπτυξη προσωποποιημένης διαλογικής διεπαφής για εφαρμογή Fantasy Football

Design and development of a personalized conversational interface for a Fantasy Football App

Ιγνάτιος Παναγιώτης Τουβλέλιος
Αριθμός Μητρώου: 1092865

Επιβλέπων
Χρήστος Σιντόρης, Ε.ΔΙ.Π.

Μέλη Επιτροπής Αξιολόγησης
Σοφία Δασκαλάκη, Επίκουρη Καθηγήτρια
Γεώργιος Μπουλουκάκης, Επίκουρος Καθηγητής

Πάτρα
Σεπτέμβριος 2026

ΠΙΣΤΟΠΟΙΗΣΗ

Πιστοποιείται ότι η διπλωματική εργασία με θέμα

Σχεδίαση και Ανάπτυξη προσωποποιημένης διαλογικής διεπαφής για εφαρμογή Fantasy Football

του φοιτητή του τμήματος Ηλεκτρολόγων Μηχανικών και Τεχνολογίας Υπολογιστών

Ιγνατίου Παναγιώτη Τουβλέλιου

Αριθμός Μητρώου: 1092865

παρουσιάστηκε δημόσια και εξετάστηκε στο τμήμα Ηλεκτρολόγων Μηχανικών και Τεχνολογίας Υπολογιστών στις

12 / 10 / 2026

Ο Επιβλέπων	Ο Διευθυντής του Τομέα
Χρήστος Σιντόρης, Ε.ΔΙ.Π.	Γεώργιος Θεοδωρίδης, Αναπληρωτής Καθηγητής

ΠΟΛΥΤΕΧΝΙΚΗ ΣΧΟΛΗ
ΤΜΗΜΑ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΤΕΧΝΟΛΟΓΙΑΣ ΥΠΟΛΟΓΙΣΤΩΝ

Το περιεχόμενο αυτής της εργασίας δεν απηχεί απαραίτητα τις απόψεις του τμήματος, του επιβλέποντα, ή της επιτροπής που την ενέκρινε.

Υπεύθυνη Δήλωση

Βεβαιώνω ότι είμαι συγγραφέας αυτής της διπλωματικής εργασίας, και ότι κάθε βοήθεια την οποία είχα για την προετοιμασία της είναι πλήρως αναγνωρισμένη και αναφέρεται στην διπλωματική εργασία. Επίσης έχω αναφέρει τις όποιες πηγές από τις οποίες έκανα χρήση δεδομένων, ιδεών ή λέξεων, είτε αυτές αναφέρονται ακριβώς είτε παραφρασμένες. Επίσης, βεβαιώνω ότι αυτή η διπλωματική εργασία προετοιμάστηκε από εμένα προσωπικά ειδικά για τις απαιτήσεις του προγράμματος σπουδών του τμήματος Ηλεκτρολόγων Μηχανικών και Τεχνολογίας Υπολογιστών.

(Υπογραφή)

……

Ιγνάτιος Παναγιώτης Τουβλέλιος

Σύνοψη

Το Fantasy Premier League (FPL) αποτελεί ένα από τα δημοφιλέστερα διαδικτυακά παιχνίδια στρατηγικής, όπου εκατομμύρια χρήστες παγκοσμίως καλούνται κάθε αγωνιστική να λάβουν μια σειρά από κρίσιμες αποφάσεις για την εικονική τους ομάδα, προκειμένου να συγκεντρώσουν όσο το δυνατόν περισσότερους πόντους με βάση τις πραγματικές επιδόσεις των ποδοσφαιριστών. Ωστόσο, η επίσημη πλατφόρμα προσφέρει κυρίως ακατέργαστα στατιστικά στοιχεία και περιορισμένη υποστήριξη για τη λήψη αποφάσεων, ενώ οι υπάρχουσες εφαρμογές τρίτων απευθύνονται κατά κύριο λόγο σε έμπειρους χρήστες, καθώς η πληροφορία παρουσιάζεται σε πυκνούς πίνακες και εκτενείς καταλόγους δεικτών. Οι χρήστες αναγκάζονται να εναλλάσσονται μεταξύ διαφορετικών ιστοσελίδων και εργαλείων για την ανάλυση παικτών, τη σύγκριση επιλογών και τη διαχείριση της ομάδας τους, συχνά με απαίτηση συνδρομής. Καταλήγουν έτσι σε ένα κατακερματισμένο και απαιτητικό περιβάλλον, το οποίο επιβαρύνει γνωστικά τον χρήστη και απαιτεί σημαντικό χρόνο εξοικείωσης, καθιστώντας το δύσχρηστο για όσους δεν διαθέτουν μεγάλη εμπειρία.

Η παρούσα διπλωματική εργασία πραγματεύεται τη σχεδίαση, ανάπτυξη και αξιολόγηση μιας ολοκληρωμένης Full-Stack διαδικτυακής εφαρμογής Fantasy Football με προσωποποιημένη διαλογική διεπαφή (AI Chatbot). Η κεντρική συνεισφορά της είναι διττή: αφενός, υλοποιείται το ίδιο το παιχνίδι και επανασχεδιάζεται συνολικά η εμπειρία χρήστη (UI/UX) σε σχέση με την επίσημη εφαρμογή Fantasy Premier League και τις υπάρχουσες πλατφόρμες τρίτων, με τη συγκέντρωση των βασικών λειτουργιών διαχείρισης ομάδας, ανάλυσης δεδομένων, προβλέψεων και υποστήριξης αποφάσεων σε ένα ενιαίο, συνεκτικό και εύχρηστο περιβάλλον. Αφετέρου, ενσωματώνεται ένας συνομιλιακός βοηθός προπονητή τεχνητής νοημοσύνης, ο οποίος προσαρμόζει τις συστάσεις του στις προτιμήσεις, στο αγωνιστικό προφίλ και στην κατάσταση της ομάδας κάθε χρήστη, παρέχοντας εξατομικευμένες επεξηγήσεις και υποστήριξη λήψης αποφάσεων μέσω φυσικού διαλόγου.

Στο υπόβαθρο του συστήματος λειτουργεί ένα pipeline μηχανικής μάθησης για την πρόβλεψη πόντων, λεπτών συμμετοχής και πιθανότητας συμμετοχής της επόμενης αγωνιστικής, το οποίο χρησιμοποιεί μοντέλα ενισχυμένων δέντρων απόφασης (gradient boosted trees), με επικύρωση κυλιόμενης προέλευσης (rolling-origin cross-validation) και έναν έλεγχο αποδοχής που κρατά το προηγούμενο μοντέλο όταν το νέο υστερεί. Οι προβλέψεις αξιοποιούνται από μια υβριδική μηχανή συστάσεων, η οποία συνδυάζει τεχνικές μηχανικής μάθησης με ευρετικούς κανόνες, καθώς και από έναν βελτιστοποιητή ακέραιου γραμμικού προγραμματισμού για την κατασκευή της αρχικής ομάδας. Ένα ενσωματωμένο επίπεδο μεγάλων γλωσσικών μοντέλων μετατρέπει τις αριθμητικές συστάσεις σε φυσικό, επεξηγηματικό διάλογο. Η πλατφόρμα υποστηρίζεται από πλήρες backend, σχεσιακή βάση δεδομένων, μηχανισμό αυτόματου συγχρονισμού δεδομένων, σύστημα χρονοπρογραμματισμού εργασιών και έλεγχο ταυτότητας χρηστών. Για την υλοποίηση αξιοποιήθηκαν τα frameworks Next.js και Express με τη γλώσσα TypeScript, η Python για τον μηχανισμό μηχανικής μάθησης και η PostgreSQL για τη διαχείριση των δεδομένων. Η αξιολόγηση πραγματοποιήθηκε τόσο ως προς την ακρίβεια των προβλέψεων και την ποιότητα των

συστάσεων όσο και ως προς τη χρηστικότητα, τη σαφήνεια και τη συνολική εμπειρία χρήσης της εφαρμογής.

Λέξεις-κλειδιά: Fantasy Football, Προσωποποιημένη Διεπαφή, Εμπειρία Χρήστη (UI/UX), Διαλογική Διεπαφή (AI Chatbot), Μηχανική Μάθηση, Μεγάλα Γλωσσικά Μοντέλα, Full-Stack Ανάπτυξη, PostgreSQL, Next.js, Express

Abstract

Fantasy Premier League (FPL) is one of the world' s most popular online fantasy sports games, in which millions of managers worldwide are required to make a series of critical decisions regarding their virtual teams every gameweek, in order to accumulate as many points as possible from the real-world performances of the footballers they select. However, the official platform primarily provides raw statistical information and limited decision-support capabilities, while existing third-party applications are mainly targeted at experienced users and frequently organise information through dense tables and long lists of metrics. Managers are therefore forced to switch between multiple websites and tools to analyse players, compare alternatives, and manage their squads, frequently through subscription-based services. They are left with a fragmented and demanding environment that imposes a considerable cognitive load and requires substantial time to learn, and that becomes difficult to use for those without much experience.

This thesis presents the design, development, and evaluation of a complete full-stack Fantasy Football web application with a personalized conversational interface (AI chatbot). The central contribution is twofold: first, it implements the game itself and comprehensively redesigns the overall user experience (UI/UX) compared with both the official Fantasy Premier League application and existing third-party platforms, bringing together team management, data analysis, player-performance predictions, and decision-support functionalities within a single coherent and user-friendly environment. Second, it integrates a conversational Artificial Intelligence coaching assistant that personalises its recommendations according to each manager' s preferences, playing style, and current squad status, providing tailored explanations and decision support through natural language interaction.

Underneath runs a machine-learning pipeline that forecasts points, minutes played and likelihood of appearance for the next gameweek, using gradient-boosted tree models with rolling-origin cross-validation and an acceptance check that retains the previous model whenever a retrained one performs worse. The resulting predictions are utilised by a hybrid recommendation engine that combines machine-learning techniques with heuristic rules, as well as by an integer linear programming optimiser for constructing the initial squad. A Large Language Model layer then turns these structured numerical recommendations into natural, explanatory dialogue. The platform is supported by a complete backend, a relational database, an automated data-synchronisation mechanism, a job-scheduling system, and user authentication. The implementation employs the Next.js and Express frameworks with TypeScript, Python for the machine-learning pipeline, and PostgreSQL for data management. The system was evaluated with respect to the accuracy of its predictive models, the quality of its recommendations, and the usability, clarity, and overall user experience delivered by the application.

Keywords: Fantasy Football, Personalized Interface, User Experience (UI/UX), Conversational Interface (AI Chatbot), Machine Learning, Large Language Models, Full-Stack Development, PostgreSQL, Next.js, Express

Ευχαριστίες

First and foremost, I would like to express my sincere gratitude to my supervising professor, Christos Sintoris, for the continuous guidance, the valuable advice, and the trust placed in me throughout the course of this thesis. His insight and encouragement were instrumental in shaping both the direction and the quality of this work.

I would also like to sincerely thank everyone who supported me with their invaluable assistance, insightful feedback, and willingness to help whenever I needed it during this project.

I am deeply grateful to my parents for their unconditional love, encouragement, and sacrifices, and to my family for their constant support and motivation throughout this journey. Finally, I would like to thank my friends for their encouragement, understanding, and companionship, which made this journey both enjoyable and unforgettable.

Περιεχόμενα

1 Introduction **1**
1.1 Goals . . . 1
1.2 Motivations . . . 2
1.3 Chapter organization . . . 3

2 Current solutions and background **5**
2.1 Current solution exploration . . . 5
2.1.1 The official Fantasy Premier League application . . . 6
2.1.2 Fantasy Football Scout . . . 8
2.1.3 Fantasy Football Hub . . . 8
2.1.4 FPL Review . . . 9
2.1.5 LiveFPL . . . 11
2.2 Limitations . . . 11
2.3 Technical foundations . . . 12
2.3.1 Supervised machine learning and gradient-boosted trees . . . 13
2.3.2 Time-series (rolling-origin) cross-validation . . . 14
2.3.3 Integer linear programming . . . 14
2.3.4 Large Language Models (LLMs) . . . 14
2.4 Conclusion . . . 16

3 Methodology **19**
3.1 Approach . . . 19
3.2 Data Collection . . . 20
3.3 System Design and Prototypes . . . 21
3.4 Implementation . . . 22
3.5 System evaluation . . . 23
3.6 Conclusion . . . 23

4 Analysis and Requirements **25**
4.1 Target Users and Scenarios . . . 25
4.1.1 User Analysis . . . 26

4.1.2 Use Cases . . . 30
4.2 Requirements . . . 31
4.2.1 Functional Requirements . . . 31
4.2.2 Non-Functional Requirements . . . 34
4.2.3 Requirements Traceability . . . 37
4.3 Conclusion . . . 38

5 System Design and Architecture Overview 39
5.1 Component Overview . . . 40
5.1.1 User Interface . . . 40
5.1.2 Web Application Backend . . . 41
5.1.3 Machine Learning Service . . . 43
5.1.4 Database . . . 43
5.1.5 From Prediction to Explanation: An End-to-End Example . . . 44
5.2 Architectural Decisions . . . 47
5.2.1 System Data Flow . . . 48
5.2.2 Reliability, Freshness, and Operational Concerns . . . 49
5.3 Conclusion . . . 50

6 Machine Learning Prediction and Recommendation Engine 51
6.1 Prediction Model Design . . . 53
6.1.1 Feature Engineering . . . 54
6.1.2 Team-Level Features . . . 57
6.1.3 Points Prediction Model . . . 58
6.1.4 Availability Prediction Model . . . 61
6.1.5 Minutes Prediction Model . . . 63
6.1.6 Worked Prediction Example . . . 65
6.2 Recommendation and Scoring System Design . . . 70
6.2.1 Personalisation Profile . . . 71
6.2.2 Hybrid (Heuristic + ML) Scoring . . . 72
6.2.3 Squad Optimisation . . . 82
6.3 Implementation of the Prediction System . . . 87
6.4 Implementation of the Recommendation System . . . 94
6.5 Conclusion . . . 98

7 The Adaptive and Conversational User Interface 99
7.1 Interface Overview . . . 99
7.1.1 The Welcome Sequence . . . 99
7.1.2 Registration and Authentication . . . 100

7.1.3 Personalisation Onboarding . . . 102
7.1.4 The Squad Builder . . . 104
7.1.5 The Dashboard . . . 105
7.1.6 Squad Management . . . 106
7.1.7 Transfers . . . 107
7.1.8 Points and Scoring History . . . 108
7.1.9 Fixtures . . . 110
7.1.10 Leagues . . . 110
7.2 Implementation of the Interface . . . 111
7.2.1 Client Structure and Data Fetching . . . 111
7.2.2 Styling and the Visual Language . . . 112
7.2.3 Responsive Design . . . 113
7.2.4 Route Protection . . . 115
7.3 Backend Implementation . . . 116
7.3.1 Database Access . . . 116
7.3.2 Routing and Module Structure . . . 116
7.3.3 Sessions and Route Protection . . . 117
7.3.4 Password Protection . . . 117
7.3.5 Request Validation . . . 118
7.3.6 Deadline Enforcement . . . 118
7.3.7 Coordination with the Machine-Learning Service . . . 118
7.4 The Conversational Assistant Interface . . . 118
7.4.1 An Always-Available Surface . . . 119
7.4.2 Rendering Structured Recommendations . . . 119
7.5 Conclusion . . . 123

8 Evaluation **125**
8.1 User Interface Evaluation . . . 125
8.1.1 Heuristic Evaluation . . . 126
8.1.2 Streamlined Cognitive Walkthrough . . . 128
8.1.3 Evaluation Based on Amershi et al.’s Guidelines . . . 131
8.1.4 Think-Aloud Protocol . . . 135
8.1.5 User Interviews . . . 140
8.2 Machine-Learning Evaluation . . . 142
8.2.1 Historical Data and Preparation . . . 143
8.2.2 Evaluation Procedure and Baselines . . . 143
8.2.3 Evaluation Metrics . . . 144
8.2.4 Prediction Results . . . 146

8.2.5 Error Analysis . . . 148
8.2.6 Participation Prediction and Calibration . . . 157
8.2.7 Contribution of Feature Groups . . . 158
8.2.8 Statistical Analysis of Model Differences . . . 161
8.2.9 Conclusion . . . 163
8.3 Scoring and Recommendation System Evaluation . . . 164
8.3.1 Evaluation of the Scoring Functions . . . 164
8.3.2 Historical Player Rankings and Captain Selection . . . 173
8.3.3 One-Gameweek Transfer Recommendations . . . 178
8.3.4 Squad Optimisation . . . 182
8.3.5 Persistent Transfers Across GW21–38 . . . 187
8.3.6 Chip Advice and Planning Limitations . . . 193
8.3.7 Conclusion . . . 197
8.4 Technical Evaluation of the AI Assistant . . . 197
8.4.1 Request Classification and Topic Filtering . . . 197
8.4.2 Conversation History and Functional Verification . . . 200
8.4.3 Live Response Evaluation . . . 200
8.4.4 Conclusion . . . 202

9 Limitations and Future Work 205
9.1 Deployment and Containerisation . . . 205
9.2 Scalability . . . 206
9.3 Performance and Caching . . . 206
9.4 Data Sources, Storage and Recovery . . . 207
9.5 Prediction Pipeline . . . 207
9.6 Recommendation System and Personalisation . . . 208
9.7 AI Assistant and Conversation Handling . . . 209
9.7.1 The Role of the Language Model . . . 209
9.8 Priorities for Further Development . . . 210

10 Βιβλιογραφία 211

Κατάλογος πινάκων

2.1 Platforms and core features of competitors. 5

3.1 Player-gameweek dataset fields and descriptions. 21

4.1 Stakeholders associated with the system. 27
4.2 Summary of high-level user needs. 28
4.3 Functional requirements of the system. 31
4.4 Non-functional requirements of the system. 34
4.5 Traceability between user needs and system requirements. 37

5.1 Logical organisation of the PostgreSQL schema. 44

6.1 The three supervised prediction tasks. 53
6.2 The twenty-four player-level features and their construction methods. 55
6.3 The six team-level features. An eligible observation has positive recorded playing time. 57
6.4 Fixed configurations of the four points-model candidates. 59
6.5 Illustrative outputs of the sixty-minute mapping. 64
6.6 Selected features from the thirty actually supplied to the models for this prediction. . 66
6.7 Outputs of the three supervised models for the example prediction. 67
6.8 The prediction record stored for the example player and made available to the recommendation engine. 68
6.9 Principal weekly weight mappings. Stored codes need not match the display order of questionnaire options. 71
6.10 Mapping risk tolerance to the availability exponent in shared ranking. 74
6.11 Captaincy risk classification and adjustments. 77
6.12 Eligibility thresholds over the service’s selected three-gameweek window. 80
6.13 Decision-specific recommendation procedures. 81
6.14 Planning-horizon weights used by the squad optimiser. 85
6.15 Price-preference settings used by the squad optimiser. 86
6.16 Libraries supporting the prediction pipeline. 87
6.17 Chronological allocation, assuming complete records. 88
6.18 Confidence labels assigned after prediction. 93

6.19 Recommendation implementation components. 94

8.1 Streamlined Cognitive Walkthrough for selecting and completing a player transfer. . . 129
8.2 Streamlined Cognitive Walkthrough for completing personalisation and requesting captaincy advice. 130
8.3 Application of Amershi et al.'s guidelines to the FPL assistant. 132
8.4 Participant profiles. 135
8.5 Five scenarios assigned to every participant in the evaluation. 136
8.6 Participant 1's formation scenario. 137
8.7 Participant 2's transfer scenario. 138
8.8 Participant 3's comparison scenario. 139
8.9 Baselines used for each prediction task. 143
8.10 Evaluation metrics and observation groups for each prediction task. 144
8.11 Pooled prediction errors. Lower values are better. 146
8.12 Comparison of the complete evaluation population with the selected high-scoring players. 150
8.13 Prediction errors by position within the selected group. Reduction refers to risk-adjusted MAE relative to its baseline. 151
8.14 Points errors grouped by recorded playing time. 156
8.15 Participation discrimination and calibration results. 157
8.16 Equal-gameweek errors after removing each feature group. Conditional and risk-adjusted MAE refer to points. 159
8.17 Production-model loss minus baseline loss, with adjusted uncertainty intervals. 161
8.18 Production-model loss minus reduced-model loss, with adjusted uncertainty intervals. 163
8.19 Procedure and results of the controlled scoring evaluation. 165
8.20 The three selection methods compared in the ranking and captaincy experiment. . . . 174
8.21 Average actual returns from player ranking and captain selection over GW21–38. . . . 175
8.22 Average differences between the application and the baselines, with adjusted uncertainty intervals. 177
8.23 Mean actual net gain per decision over GW21–38, including a gain of zero when no transfer was selected. 180
8.24 Differences in actual net transfer gains between application ordering and risk-adjusted-points ordering. 182
8.25 The five policies compared in the squad-construction experiment. 183
8.26 Mean actual points earned by the fixed starting XI and captain under each squad-selection objective, using GW21–34 as starting weeks and outcomes through GW38. 185

8.27 Mean net points, transfer penalties and transfer counts over GW21–38 for the six policies, each evaluated from the same 20 synthetic starting squads. 189
8.28 Procedure, expected behaviour and results of request classification and topic filtering. 198
8.29 Direct-request classification mismatches. 199
8.30 Synthetic player data supplied during the live-response evaluation. 200
8.31 Evaluation of six live responses generated from the same predefined player information. Each row identifies the request and the behaviour being checked, describes the assistant' s response and records any limitation observed. The cases examine the use of supplied figures, the explanation of forecast periods, the handling of missing information, the distinction between advice and executed actions, resistance to an instruction to invent a forecast, and responses to unrelated questions. 201

Κατάλογος σχημάτων

2.1 The official Fantasy Premier League squad-management and transfers interface (Source: Fantasy Premier League). . . . 7
2.2 The Fantasy Football Scout Season Ticker, comparing fixture difficulty across upcoming gameweeks for every team (Source: Fantasy Football Scout). . . . 8
2.3 The Fantasy Football Hub team-rating and projections tool (Source: Fantasy Football Hub). . . . 9
2.4 The FPL Review Massive Data Model, multi-period Transfer Solver, and multi-gameweek projection view (Source: FPL Review). . . . 10
2.5 The LiveFPL live-rank and effective-ownership dashboard (Source: LiveFPL). . . . 11
2.6 The transformer model architecture (Vaswani et al. 2017). . . . 16

3.1 The Star Model of interactive system development, adapted from Hartson and Hix (1989). 19

4.1 Persona 1: Ignatios, the beginner manager. . . . 28
4.2 Persona 2: Bill, the casual manager. . . . 29
4.3 Persona 3: Nikos, the competitive manager. . . . 29

5.1 Overall system architecture of the platform. . . . 40
5.2 End-to-end flow of a captaincy request, from the manager’ s message to the streamed reply. Steps 4, 5, and 8 are the principal points of contact with the shared PostgreSQL database; the language model is invoked only at step 7, after every number in the recommendation has already been computed. . . . 46

6.1 The prediction and recommendation process: historical records are transformed into features, three supervised models generate estimates, post-processing produces stored predictions, and recommendation services supply results and context for language-model explanation. . . . 52
6.2 The worked example end to end, with the values produced by the gameweek 4 run. The feature vector enters the three supervised models; the points and availability outputs combine into the risk-adjusted estimate, which in turn yields the horizon projections, while the minutes output is mapped to the heuristic sixty-minute value. All of these, with the confidence label, form the stored prediction record. . . . 70

6.3 Source excerpt from the points training script: fitting candidates, calculating validation metrics, and retaining the lowest-MAE model. Line numbers refer to the application source; the gap omits logging statements. 89
6.4 Production data flow and intermediate artifacts. Experimental rolling-origin evaluation is separate from routine candidate selection. 90
6.5 Source excerpts from inference: obtaining the three model outputs, deriving the sixty-minute estimate, and applying the participation adjustment. The gap in line numbering indicates omitted comments. 92
6.6 Source excerpt from request preparation: retrieving context, computing the recommendation, and building the language-model input. 96
6.7 Main non-streaming request flow, including session validation, scope refusal, recommendation assembly, generation, and subsequent logging. Streaming shares preparation but delivers fragments before final persistence. 97

7.1 The first panel of the welcome sequence. The interface introduces the manager' s role and the shape of the season before presenting any statistics. 100
7.2 The sign-in screen. Both authentication modes share a single card, and the platform' s capabilities remain visible alongside the form. 100
7.3 The registration form. 101
7.4 Client-side validation. Each field is validated where it is edited, with a specific message, and the password rules are shown as a live checklist rather than being revealed only on rejection. 102
7.5 The first onboarding question. Each dimension is asked as a single choice among three concrete strategies, with a subtext explaining what the answer will change. 103
7.6 On completion the questionnaire converts into the assistant, offering one suggested opening question per supported decision type. 103
7.7 Constraints enforced preventively. With three Manchester City players selected, the remaining Manchester City entries in the rail are dimmed and unselectable rather than clickable-and-then-rejected. 104
7.8 The initial squad requested conversationally from the builder screen. The fifteen players, their projections, and the budget summary are the integer program' s output, narrated by the language model. The panel is captured in full and re-flowed into three columns. 105
7.9 The dashboard in full. Each panel summarises one domain and links to the screen that owns it, so the dashboard indexes the application rather than duplicating it. 106
7.10 The team screen. Formation, starting eleven, bench, chips, and per-player captaincy controls are all on one screen. 107
7.11 Choosing a player to sell filters the market to legal, affordable replacements for that specific player, turning a search problem into a comparison problem. 107

7.12 A budget violation is reported rather than prevented: the cart names the specific error and disables the commit control, but preserves the pending state so the manager can resolve it by selling elsewhere. . . . 108
7.13 The live points view: a per-player breakdown of the current gameweek beside the season aggregates. . . . 109
7.14 The history view: a season trend with a per-gameweek table recording chip, penalty, bench points, and final score. . . . 109
7.15 The fixtures screen, selectable by gameweek, showing the same fixture-difficulty ratings that the prediction models consume as features. . . . 110
7.16 Classic league standings, ranked by season total, with the current gameweek alongside and the viewing manager' s row highlighted. . . . 111
7.17 The dashboard on a phone viewport (390 × 844). Panels reflow into a single column in priority order, with the deadline countdown remaining first. Captured in full and re-flowed into four columns. . . . 113
7.18 The team screen on a phone viewport. The pitch is preserved as the organising metaphor at reduced scale rather than degrading into a list. Captured in full and re-flowed into three columns. . . . 114
7.19 The team screen at tablet width (820 × 1180). . . . 115
7.20 A captaincy recommendation. The structured blocks, recommendation, reasoning, risk, alternative, and conclusion, are rendered as distinct surfaces so the answer is legible at a glance and the justification available on demand. Captured in full and re-flowed into two columns. . . . 120
7.21 Transfer advice for Aleka' s existing squad. The actual chat is captured across its scrollable area and arranged in two columns, read from top to bottom in the left column and then the right. . . . 121
7.22 Calafiori and Gabriel compared in Aleka's chat. The complete captured reply is arranged in two columns. . . . 122
7.23 Squad analysis from Aleka' s existing account, showing player projections, risks, and an alternative. The actual reply is captured in full and arranged in two columns. . . . 123

8.1 Risk-adjusted points MAE for the production configuration, reduced configuration and baseline. . . . 147
8.2 Minutes MAE for both model configurations and the two minutes baselines. . . . 147
8.3 Actual and predicted mean points in each evaluation gameweek. . . . 148
8.4 Distribution of predicted minus actual points for the production configuration. . . . 149
8.5 Risk-adjusted and conditional points MAE for the selected players in each position. Lower values indicate better performance. . . . 151

8.6 Actual and predicted points for Bruno Fernandes. Gaps in the conditional panel indicate gameweeks without an appearance of at least 30 minutes. 152
8.7 Actual and predicted points for Haaland, illustrating a case where the model did not improve on the baseline. 153
8.8 Actual versus predicted points. The dashed line represents a perfect prediction. 155
8.9 Risk-adjusted points MAE by position, using position information from historical pre-deadline captures. 156
8.10 Participation ROC-AUC across the evaluation gameweeks. 157
8.11 Predicted participation probabilities against observed frequencies (left), and observation counts in each probability bin (right). 158
8.12 Change in loss after removing each feature group. Positive values indicate worse performance after removal. Intervals are approximate Bonferroni-adjusted block-bootstrap intervals across the 36 ablation comparisons, using three-gameweek blocks. 160
8.13 Production-model comparisons with simple baselines. Negative differences favour the model; intervals crossing zero leave the direction uncertain. The intervals are adjusted across all nine main comparisons and use three-gameweek blocks. 162
8.14 Risk sensitivity across 576 synthetic input settings per profile. Thin lines show individual settings, the thick line shows the median and the shaded region shows the 10th–90th percentile spread of cases, not a confidence interval. 168
8.15 Effect of forecast confidence across 19,392 matched input combinations per risk profile. Each thin line connects low-, medium- and high-confidence scores with all other inputs unchanged; black lines show the medians. Many lines overlap. 169
8.16 Effect of the planning horizon on 64 synthetic candidates. The left panel follows each candidate' s rank across short, balanced and long settings. The right panel compares every distinct pair under the short and long settings, with orange points indicating that the preferred candidate changed. Several pairs may occupy the same point. 171
8.17 Actual returns in each of the 18 evaluated gameweeks. The left panel shows the mean points of each method' s top-ten players, while the right panel shows captain points averaged across the 20 synthetic squads in each round. Period averages appear in the legends. 176
8.18 Actual net transfer gains across all 360 squad-gameweek cases per method. The left panel shows transfers without a points penalty, while the right panel shows the four-point-cost scenario. Each dot represents one case, and the black bars indicate the means; horizontal displacement separates overlapping points and has no numerical meaning. . 181
8.19 Returns for all 14 starting weeks under one-, three- and five-gameweek holding periods. Each thin line connects the results obtained from the same starting week under the three objectives, while black diamonds show the means. 185

8.20 Five-gameweek returns for every starting week from GW21 to GW34, showing how the comparison varied across the evaluated periods. 186
8.21 Net points for all 20 starting squads under each transfer policy. Thin lines connect results from the same starting squad, while the black line shows the policy means. 191
8.22 Cumulative net returns across GW21–38 for all 20 starting squads under each policy. Each panel uses the same scale, with individual paths shown as thin lines and the mean in black. 192
8.23 Additional points from applying Bench Boost or Triple Captain independently in each gameweek for all 20 balanced-policy squads. Thin lines show individual squads, while the black line shows their mean. 195

1 Introduction

Fantasy Premier League (FPL) is a season-long fantasy sports game in which managers select a virtual squad of real Premier League footballers within a fixed budget and earn points based on their players' real-world performances. With more than eleven million registered managers worldwide, FPL has become one of the most popular online fantasy sports games, and it has changed the way supporters follow the sport. Throughout the season, managers face a series of choices every gameweek: which transfers to make, whom to captain, which formation to field, how to order the bench, and when to use a limited number of one-off strategic "chips" . Each of these can have a substantial effect on a manager' s total over the thirty-eight gameweek season. Despite this, the official FPL platform mainly provides statistics, player data and league tables, without offering direct decision support, and the volume of what it does show can make it difficult to navigate for managers less familiar with the game. Third-party tools offer more advanced analysis and projections, but many are designed primarily for experienced users. Some rely on paid subscriptions; others present their results as detailed spreadsheets and solver configurations that less experienced managers may find hard to interpret. There is therefore a gap between the analytical demands of FPL and the accessibility of the tools available to its managers. This thesis addresses that gap through the design and implementation of a Fantasy Football web application with a personalized conversational interface (AI chatbot).

1.1 Goals

In this context, there is a need for a unified platform that can keep a manager' s data organised and up to date, while also making reliable and personalised decision support accessible through an intuitive and user-friendly interface. The main goal of this thesis is therefore to design, develop, and evaluate a complete full-stack Fantasy Football web application built around an adaptive and conversational AI coaching assistant: a chatbot through which the manager reaches every part of the system' s analysis.

The application implements the rules of the game itself, covering squad selection, formations and substitutions, captaincy, the use of chips, and transfers with their associated point costs. Around this sit a modern frontend, a fully featured backend, a relational database, automated data-synchronisation mechanisms, user authentication, and the supporting infrastructure needed for an end-to-end web application. Building such a system also provides the opportunity to gain practical experience with the technologies and practices used in modern full-stack software engineering.

The centrepiece of the platform is the AI coaching assistant. It draws on the forecasts produced by the system' s supervised machine-learning models and, through a Large Language Model layer, turns them into personalised recommendations for the main weekly decisions a manager faces. It explains those recommendations through natural dialogue, taking into account factors such as the manager' s current squad, preferences, playing style, and risk profile. The recommendations themselves come from a hybrid engine that combines the output of these models with heuristic rules, and from an integer-programming optimiser that assembles a manager' s opening squad under the budget, position and club limits of the real game.

A further goal of the thesis concerns the user interface and user experience of the application as a whole, and not only the way predictions and recommendations are displayed. The everyday actions of running a squad, such as selecting a starting eleven, making substitutions, changing formation, or preparing transfers, are redesigned around direct manipulation and immediate feedback, with drag-and-drop interaction where it suits the task and simpler alternatives where it does not. The interface aims to keep the state of the team and the consequences of each action clear before it is committed, rather than leaving the manager to work them out afterwards.

The same principle governs the analytical side of the application. Instead of presenting advanced analytics through dense dashboards, complex spreadsheets, or interfaces with a steep learning curve, the proposed application aims to present the same analysis in a form a manager can act on without first having to learn how to read it. Team management, statistical analysis, predictions, and conversational decision support are integrated into a single environment, with the assistant available from every screen rather than confined to one.

Taken together, these objectives define the main contribution of the thesis: the development of an adaptive and conversational interface for a Fantasy Football application. Here, "adaptive" refers to the system' s ability to personalise its recommendations to each manager and to vary the depth of explanation it provides according to that manager' s stated preferences. "Conversational" refers to the assistant being the primary means of reaching the platform' s analysis: the manager asks questions in natural language, explores alternatives, and is given the reasoning behind every recommendation, instead of reading figures off a dashboard.

1.2 Motivations

A primary motivation for this thesis comes from the author' s experience as a Fantasy Premier League manager and the practical challenges of playing the game week to week. Managers need to weigh a wide range of factors, including recent player form, upcoming fixtures, injury news, expected playing time, ownership percentages, and budget constraints, before settling on transfers, captain selections, and squad composition.

In practice, following all of these factors takes considerable time and usually involves several different websites, statistical dashboards, prediction services, and community resources. Some are available only through subscription models, while others present their information through complex interfaces designed mainly for experienced users. Even after gathering and comparing all of it, the final decision often still rests on personal judgement rather than on the evidence collected.

This experience motivated the idea of a unified platform that brings together data analysis, personalised recommendations, and a more direct way of working with Fantasy Football information. It also suggested that a well-designed user interface, combined with a conversational AI coaching assistant, could make advanced analytics easier to reach and understand, whatever a manager' s statistical background or previous experience with analytical tools.

A second motivation was to explore how modern software engineering technologies and Artificial Intelligence techniques work together within a single application. The thesis does not treat frontend development, backend services, databases, machine learning, mathematical optimisation, authentication, scheduling, and Large Language Models as separate components. Instead, it builds them into one decision-support platform. The project therefore serves both as a working Fantasy Football application and as a case study in how full-stack software engineering and applied Artificial Intelligence can be combined in a real-world system that supports complex decision-making.

1.3 Chapter organization

The Chapters are organized as shown below:

1. **Chapter 1**: This Chapter provides an overview of this thesis project. In essence, it defines the goals and motivations that this thesis aims to achieve.
2. **Chapter 2**: Presents an overview of the existing solutions available to Fantasy Premier League managers, discusses their limitations, particularly in terms of usability and accessibility, and provides the theoretical background and core technologies used throughout this thesis, namely supervised machine learning and gradient-boosted trees, time-series cross-validation, integer linear programming, and Large Language Models.
3. **Chapter 3**: Describes the overall methodology adopted in this thesis. It presents the data collection process, the design and implementation methodology, and the evaluation methodology used to assess the proposed system.
4. **Chapter 4**: Analyzes the target users of the proposed system and the primary usage scenarios it is designed to support. It also defines the functional and non-functional requirements that guided the development of the project.

5. **Chapter 5**: Presents an overview of the overall system architecture, highlighting its core components (User Interface, Web Application Backend, Machine Learning Service, and Database), their purpose, and main functionalities, and discusses the architectural decisions that shaped the design and implementation of the system.
6. **Chapter 6**: Analyzes the design of the machine learning prediction and recommendation engine, describing the feature engineering, the prediction models, the hybrid and personalised scoring logic, and the squad-optimisation component, together with how each part was implemented and its final configuration.
7. **Chapter 7**: This Chapter focuses on the adaptive and conversational user interface, the central contribution of this thesis. It walks through each page and its functionality, explains how the design improves on the user experience of existing platforms, analyzes the frontend and backend technologies and how they cooperate with the machine learning service, and describes the conversational assistant through which recommendations are delivered.
8. **Chapter 8**: This Chapter describes the evaluation process, the methods used, and the results obtained. The prediction and recommendation systems are evaluated first, followed by the user interface and experience, using both expert-based and user-based methods to assess the usability of the system.
9. **Chapter 9**: The final chapter summarises the contributions of the thesis, acknowledges the limitations of the current system, and outlines directions for future work, including improvements to the machine learning models, a richer real-time inference path, and additional user interface features.

2 Current solutions and background

The first step in the design process is to examine related applications in order to understand which features are covered by existing solutions and which are not. For this reason the most well-known options available to Fantasy Premier League managers today are examined. Following this, the current Chapter provides an overview of the core technologies underlying the system, namely supervised machine learning and gradient-boosted trees, time-series cross-validation, integer linear programming, and Large Language Models. Understanding these concepts is essential, as they will dictate the technical decisions made in the next Chapters.

2.1 Current solution exploration

The review begins with the related solutions. Currently, there is a wide ecosystem of online platforms and tools that aim to assist Fantasy Premier League managers. These range from the official platform, which focuses on gameplay and raw statistics, to a number of third-party analytics services that provide performance projections, transfer planning, and mathematical optimisation. Table 2.1 summarizes some of the most widely used applications and their core features. These specific applications were selected as they are the most well-known and, taken together, are representative of the current landscape of tools that a manager is likely to encounter.

Πίνακας 2.1: Platforms and core features of competitors.

Application	Features
Official FPL app	Squad management, transfers, captaincy and chip usage, classic and head-to-head leagues, raw player and team statistics, and the official fixture-difficulty rating (FDR); no predictions or personalised recommendations.
Fantasy Football Scout	Editorial content and team news, industry-leading predicted line-ups, the Season Ticker (FDR, clean-sheet odds, projected goals per team and gameweek), player projections, and a Rate My Team tool in the members area.

Application	Features
Fantasy Football Hub	Algorithm-based points projections, an AI-powered Rate My Team, recommended transfers, captaincy picks, a team planner, and expert pundits' teams (subscription).
FPL Review	An expected-points ("Massive Data") projection model looking many gameweeks ahead, a transfer planner for drafting and saving plans, and a multi-period optimisation solver with chip-timing and linear optimisation (subscription).
LiveFPL	Real-time live overall rank and points, effective ownership and rank-tier averages, price-change predictions, a what-if simulator, and a transfer planner.

2.1.1 The official Fantasy Premier League application

The official Fantasy Premier League application (Fantasy Premier League, n.d.) is the platform on which the game itself is played. Each manager selects a fifteen-player squad, two goalkeepers, five defenders, five midfielders and three forwards, under a fixed budget of £100 million and a maximum of three players from any single real club, and each gameweek makes transfers, chooses a captain and vice-captain, sets a starting eleven and bench order, and optionally deploys one of a small number of one-off strategic chips. The interface exposes a large amount of raw data for every player, including current price, ownership percentage, form, total points, and the components of the ICT (Influence, Creativity, Threat) index, alongside an official fixture-difficulty rating on a one-to-five scale, and it maintains the classic and head-to-head leagues in which managers compete. However, as illustrated in Figure 2.1, the platform is designed to run the game rather than to advise the manager: beyond a lightly curated editorial "Scout" selection, it offers no forward-looking predictions and no personalised, per-squad recommendations, leaving the analytical burden entirely with the user.

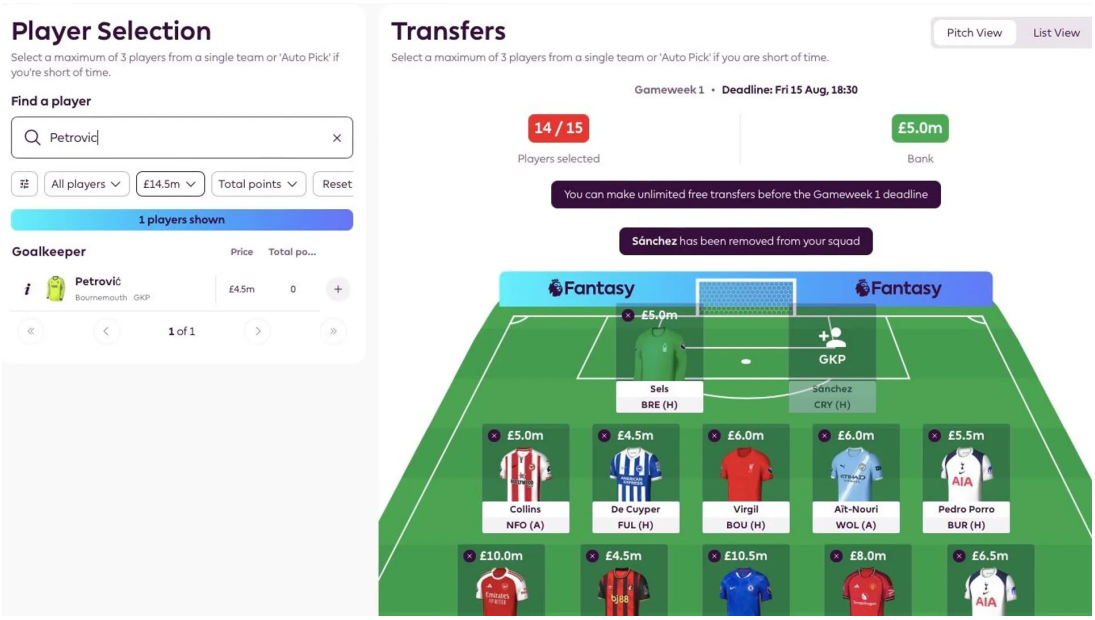


Figure 2.1. The official Fantasy Premier League squad-management and transfers interface (Source: Fantasy Premier League).

That burden is compounded by several characteristics of the interface itself. Within the player detail view, price, ownership, form, total points and the components of the ICT index are presented side by side at a uniform level of prominence, with nothing to indicate which of them bears on the decision the manager is actually making. Several of these indicators are also not self-descriptive: the ICT index and the fixture-difficulty rating appear as bare numbers, with no account of how they are derived or how they should be read, so a manager unfamiliar with the terminology has no basis on which to judge how much weight to give them. Both properties sit uneasily with established usability principles, in particular minimalist design and the preference for recognition over recall (Nielsen 1994b).

The interface also provides no support for comparison, which is the operation the game most often demands. Players can be inspected only one at a time, in a modal view that temporarily replaces the surrounding context, so evaluating two candidates for the same position requires opening each in turn and retaining the relevant figures in working memory. Because the squad, the transfer market, the statistics tables and the fixture list are separate destinations, a single transfer decision typically involves several such context switches. The resulting interaction cost falls hardest on inexperienced managers, who are least equipped to absorb it through prior familiarity with what the figures mean. The same principles inform the evaluation of the proposed interface in Chapter 8.

2.1.2 Fantasy Football Scout

Fantasy Football Scout (Fantasy Football Scout, n.d.) is one of the longest-running and most widely-read community platforms dedicated to Fantasy Premier League. Its public side is primarily editorial, offering tips, analysis, and what is generally regarded as the most accurate predicted line-ups and team news in the community. Its members area, shown in Figure 2.2, adds a suite of statistical tools: detailed player and team statistics, a Season Ticker that lets managers compare fixture difficulty, clean-sheet odds, and projected goals for every team across every upcoming gameweek, player projections offering a forward view of projected goals, assists, clean sheets, bonus and total FPL points, and a Rate My Team tool that estimates how many points a given squad is expected to score. The projections are, however, produced and refreshed at a small number of key points in the season rather than continuously, and the platform is oriented towards providing data and expert commentary for the manager to interpret, rather than a single, personalised recommendation.

Team	10	11	12	13	14	15	16	17	18	19
ARS	new	che	NFO	whu	MUN	ful	EVE	cry	IPS	bre
AVL	tot	liv	CRY	che	BRE	SOU	nfo	MCI	new	BHA
BHA	liv	MCI	bou	SOU	ful	lei	CRY	whu	BRE	avl
BOU	MCI	bre	BHA	wol	TOT	ips	WHU	mun	CRY	ful
BRE	ful	BOU	eve	LEI	avl	NEW	che	NFO	bha	ARS
CHE	mun	ARS	lei	AVL	sou	tot	BRE	eve	FUL	ips
CRY	wol	FUL	avl	NEW	ips	MCI	bha	ARS	bou	SOU
EVE	sou	whu	BRE	mun	WOL	LIV	ars	CHE	mci	NFO
FUL	BRE	cry	WOL	tot	BHA	ARS	liv	SOU	che	BOU
IPS	LEI	tot	MUN	nfo	CRY	BOU	wol	NEW	ars	CHE
LEI	ips	mun	CHE	bre	WHU	BHA	new	WOL	liv	MCI
LIV	BHA	AVL	sou	MCI	new	eve	FUL	tot	LEI	whu
MCI	bou	bha	TOT	liv	NFO	cry	MUN	avl	EVE	lei

Figure 2.2. The Fantasy Football Scout Season Ticker, comparing fixture difficulty across upcoming gameweeks for every team (Source: Fantasy Football Scout).

2.1.3 Fantasy Football Hub

Fantasy Football Hub (Fantasy Football Hub, n.d.) is a subscription-based analytics platform built around an algorithmic points-projection model. Its central feature, marketed as an "AI" team tool, combines a Rate My Team rating with predicted points, recommended transfers, captaincy suggestions, live rank tracking, and a set of optimal "best teams". The underlying values are generated by an algorithm drawing on both current and historical data and are monitored by human analysts so that the model can react quickly to news, injuries, and tactical changes, as shown in Figure 2.3. The platform also surfaces the teams of its expert pundits. It therefore moves closer to genuine decision

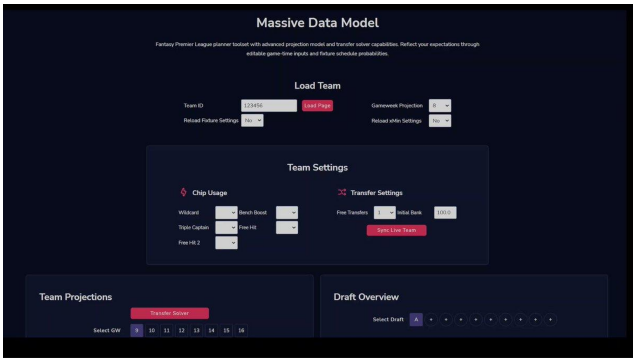
support than the official app or a purely editorial site, but its recommendations are delivered as ranked lists and dashboards behind a paywall, and are not tailored to the individual manager's stated risk appetite or playing style beyond the composition of their current squad.

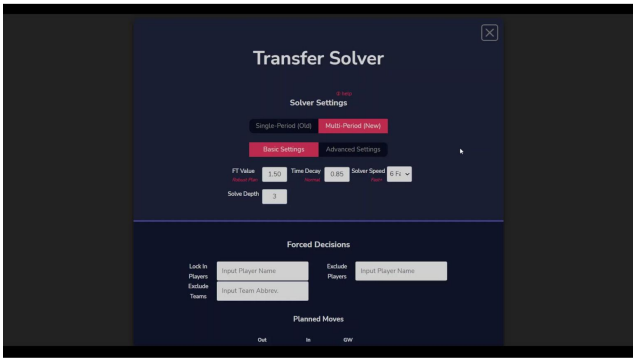
Figure 2.3. The Fantasy Football Hub team-rating and projections tool (Source: Fantasy Football Hub).

2.1.4 FPL Review

FPL Review (FPL Review, n.d.) is the most analytically-oriented of the platforms examined and the closest in spirit to the predictive and optimisation components of this thesis. Its core is the "Massive Data" model, which combines traditional statistical methods with machine-learning techniques and a feature-rich dataset, historical performance, bookmaker odds, team strength and style, player roles, and rotation risk, to project each player's expected FPL value over a horizon of up to fourteen gameweeks ahead. On top of this model sits the Massive Data Planner, which lets managers draft, save, and visualise multiple transfer plans, and a Transfer Solver that has evolved from single-gameweek solving into a multi-period optimiser supporting chip timing, sensitivity analysis, and linear optimisation, as shown in Figure 2.4. FPL Review thus demonstrates the value of pairing an expected-points model with a

mathematical optimiser; however, it is a subscription product aimed squarely at advanced, statistically-literate managers, presenting its output as dense numerical tables and solver configurations rather than as accessible, explained advice.





Gameweek 35				Gameweek 36			
Player	Team	Fixt	Proj	Player	Team	Fixt	Proj
Raya	BRE	Liv (A)	2.7	Raya	BRE	Whu (H)	4.6
Arrizabalaga	CHE	Bou (A)	4.5	Arrizabalaga	CHE	Not (H)	5.6
Trippier	NEW	Ars (H)	4.3	Trippier	NEW	Lee (A) , Bha (H)	9.8
Ake	MCI	Lee (H)	4.0	Ake	MCI	Eve (A)	4.1
Shaw	MUN	Whu (A)	3.5	Shaw	MUN	Wol (H)	5.0
Estupinan	BHA	Eve (H)	4.7	Estupinan	BHA	Ars (A) , New (A)	4.8
Botman	NEW	Ars (H)	3.2	Botman	NEW	Lee (A) , Bha (H)	7.1
Salah	LIV	Bre (H)	6.3	Salah	LIV	Lei (A)	6.0
Fernandes	MUN	Whu (A)	4.9	Fernandes	MUN	Wol (H)	5.8
Rashford	MUN	Whu (A)	5.3	Rashford	MUN	Wol (H)	6.3
Mac Allister	BHA	Eve (H)	5.5	Mac Allister	BHA	Ars (A) , New (A)	7.2
March	BHA	Eve (H)	5.8	March	BHA	Ars (A) , New (A)	7.8
Haaland	MCI	Lee (H)	9.6	Haaland	MCI	Eve (A)	8.0
Toney	BRE	Liv (A)	4.8	Toney	BRE	Whu (H)	5.5
Greenwood (S)	LEE	Mci (A)	0.0	Greenwood (S)	LEE	New (H)	0.0
Out	In		3-Week Impact	Out	In		3-Week Impact
			0.0				0.0
			0.0				0.0
			0.0				0.0
			0.0				0.0

Figure 2.4. The FPL Review Massive Data Model, multi-period Transfer Solver, and multi-gameweek projection view (Source: FPL Review).

2.1.5 LiveFPL

LiveFPL [2.1.5] approaches the problem from a different angle, focusing on the live, in-play experience rather than on pre-gameweek planning. Its headline feature is a real-time overall-rank and live-points tracker that updates as matches unfold, complemented by effective-ownership analysis and rank-tier averages that tell a manager whether they are gaining or losing ground against others in their bracket, rather than against the global average. It also provides highly-regarded price-change predictions, a what-if simulator that explores how goals, assists, or bench points would shift a manager's rank, and a transfer planner, as illustrated in Figure 2.5. LiveFPL is an excellent tracking and situational-awareness tool, but by design it answers the question "how am I doing right now?" rather than "what decision should I make next week, and why?", and it does not produce personalised, explained recommendations for the game's weekly decisions.

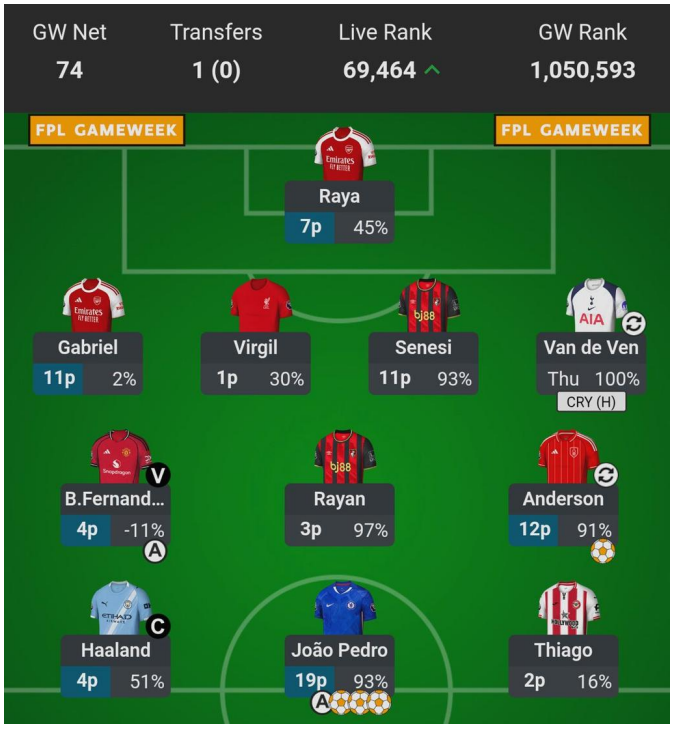


Figure 2.5. The LiveFPL live-rank and effective-ownership dashboard (Source: LiveFPL).

2.2 Limitations

The platforms discussed above share several limitations that matter from a user-experience perspective. The first is fragmentation. The functionality an FPL manager needs is spread across separate services: statistics and fixtures, forward-looking projections, transfer optimisation, and live tracking are

typically provided by different platforms. A manager working through a single decision therefore ends up consulting several sites at once and reconciling information that differs in format, in update frequency, and at times in the underlying figures themselves.

A second limitation concerns the accessibility of the tools that do offer genuine forward-looking analysis. Platforms such as FPL Review and Fantasy Football Hub provide projections and optimisation features, but they are aimed at experienced managers and are largely offered on a subscription basis. Their output takes the form of detailed tables, dashboards and solver configurations that presuppose a degree of statistical literacy, which places them out of comfortable reach for casual managers and for anyone new to data-driven Fantasy Football analysis. The official application sits at the opposite end: its interface is more polished and more approachable, but it offers no direct decision support, and the volume of raw statistics it does present is itself difficult for a new manager to interpret.

A third limitation, and the one most directly relevant to this thesis, is the absence of a conversational and explanatory approach to recommendation. Where recommendations exist, they are delivered as ranked lists or precomputed outputs that a manager can read but not question. They rarely take account of a particular squad or a particular manager' s preferences, and they offer no route to a follow-up. A manager may be told that one player is the better transfer without being able to ask why that player is preferred to the alternative, or how the recommendation would change over a longer planning horizon or under a different appetite for risk.

Alongside these analytical gaps, the platforms examined here share a number of interaction-design shortcomings that surface during routine use. Common tasks tend to require more steps than the task itself warrants, and the interfaces are generally organised around exposing information rather than around the workflow a manager is following, leaving the sequence of actions that makes up a transfer or a team selection for the user to assemble. The effect is a raised interaction cost for tasks performed every gameweek, and a correspondingly higher cognitive load for less experienced managers.

These limitations point to two areas with substantial room for improvement, both of which are central to this thesis. The first is the user experience: bringing the scattered capabilities together into one coherent and responsive interface, and presenting complex analysis in a form that lowers the barrier to entry for a newcomer. The second is the recommendation experience: making it adaptive, so that advice reflects the individual manager' s squad, risk tolerance and playing style, and making it conversational and explainable, so that a manager can question a suggestion, follow the reasoning behind it, and judge how far to trust it. Addressing this twofold gap is the central aim of the system developed in this thesis.

2.3 Technical foundations

The implementation of the proposed platform draws on several technologies, each serving a different part of the system. Before the implementation is discussed in the following chapters, this section

introduces the concepts needed to follow the design choices made throughout the thesis: supervised machine learning and gradient-boosted trees, which produce the player-performance predictions; time-series cross-validation, which evaluates those predictions while respecting the temporal ordering of the data; integer linear programming, which constructs an optimal initial squad under the constraints of the game; and Large Language Models, which drive the conversational assistant.

Although these technologies come from different areas of computer science and Artificial Intelligence, they serve complementary purposes within the proposed system, and in operation they form a single chain. The machine-learning models estimate future player performance; the integer program takes those estimates as its objective and selects a squad that satisfies the rules of Fantasy Premier League; and the Large Language Model turns the resulting structured output into recommendations and explanations in natural language. Together they form the technical foundation on which the adaptive and conversational user experience of the platform rests.

2.3.1 Supervised machine learning and gradient-boosted trees

Supervised machine learning refers to methods in which a model learns the relationship between input features and a target variable from a dataset of labelled examples, and then uses that relationship to predict the target for new, unseen inputs. When the target is a continuous quantity, such as the number of fantasy points a player will score in the next gameweek, the task is regression. When it is a discrete category, such as whether a player will feature in the next match, the task is classification.

Decision-tree ensembles are particularly effective for tabular data of this kind. A single decision tree partitions the feature space through a sequence of simple threshold tests, but on its own it tends to overfit. Ensemble methods address this by combining many trees into a single, more robust predictor.

Random forests (Breiman 2001) train many trees independently, on random subsets of the data and of the features, and average their outputs, which reduces variance. Gradient-boosted trees (Friedman 2001) take a different approach: the trees are built sequentially, with each new tree fitted to correct the residual errors of the ensemble built so far. This typically yields higher accuracy on structured, tabular data. XGBoost (Chen and Guestrin 2016) is a widely used implementation of gradient boosting that adds regularisation and a number of computational optimisations.

In this thesis, gradient-boosted and related tree-based regressors are trained to predict a player' s expected points, while a separate, probability-calibrated classifier estimates the likelihood that a player will feature in the next match. The specific models and their training procedure are described in Chapter 6.

2.3.2 Time-series (rolling-origin) cross-validation

Because the models in this thesis predict future gameweeks from past ones, the validation procedure must respect the temporal order of the data. Standard k-fold cross-validation, which partitions the dataset randomly into folds, is inappropriate here: a random split can place gameweeks that occur after the test gameweek into the training set, leaking future information into a prediction that should not have access to it and producing an over-optimistic estimate of accuracy.

To avoid this, a time-series or rolling-origin (walk-forward) validation scheme is used instead (Bergmeir and Benítez 2012). The model is trained on all gameweeks up to a given point in time and evaluated on the gameweek that immediately follows; the training period is then extended by one gameweek and the process repeated, producing a sequence of training and validation windows that follow the chronological order of the data. This mirrors how the model is used in the deployed system, where every prediction is made for a future gameweek from the history available at that moment, and it therefore gives a realistic estimate of the accuracy that can be expected on unseen data.

2.3.3 Integer linear programming

Integer linear programming (ILP) is a mathematical-optimisation technique for maximising or minimising a linear objective function subject to a set of linear constraints, where some or all of the decision variables are restricted to integer values. A common special case, and the one relevant here, uses binary variables that take the value zero or one to represent yes/no decisions.

Selecting an initial Fantasy Premier League squad maps naturally onto this formulation. Each player is associated with a binary variable indicating whether that player is included, and the objective is to maximise the total projected value of the selected players. The rules of the game translate directly into linear constraints: exactly fifteen players, the required quota in each position, a total cost within the £100 million budget, and no more than three players from any one real club.

Although integer programming problems are NP-hard in general, instances of this size are solved to optimality within seconds by mature solvers. In this thesis the open-source CBC solver is used through the PuLP modelling library, as described in Chapter 6.

2.3.4 Large Language Models (LLMs)

Large Language Models (LLMs) are deep-learning models trained on very large amounts of text data to process and generate human language. Most modern LLMs are based on the transformer architecture (Vaswani et al. 2017), shown in Figure 2.6. Transformers use a mechanism called self-attention to determine the relationships between different parts of a sequence of text, which allows the model to draw on a much wider context than earlier approaches when processing or generating language.

LLMs are typically pre-trained on large and diverse text corpora to learn general patterns of language, and are then adapted to more specific tasks. Before being processed, text is divided into smaller machine-readable units called tokens, through a process known as tokenization. Once trained, an LLM can be applied to a wide range of natural-language tasks, including text generation, summarisation, and question answering.

In this thesis, the LLM is used strictly as the top-level dialogue and explanation layer of the system. It does not produce the underlying player predictions or recommendations; these come from the machine-learning and optimisation components described above. The LLM receives their structured, numerical output and turns it into natural-language advice, answers the manager' s follow-up questions, and explains the reasoning behind each suggestion.

The assistant is designed to advise, not to act on the manager' s behalf: it never executes a transfer, a captaincy change, or an adjustment to the starting line-up. This separation of responsibilities lets each component do what it is best suited to, with the machine-learning models producing numerical forecasts, the optimisation algorithms constructing feasible squads, and the LLM handling interaction and explanation.

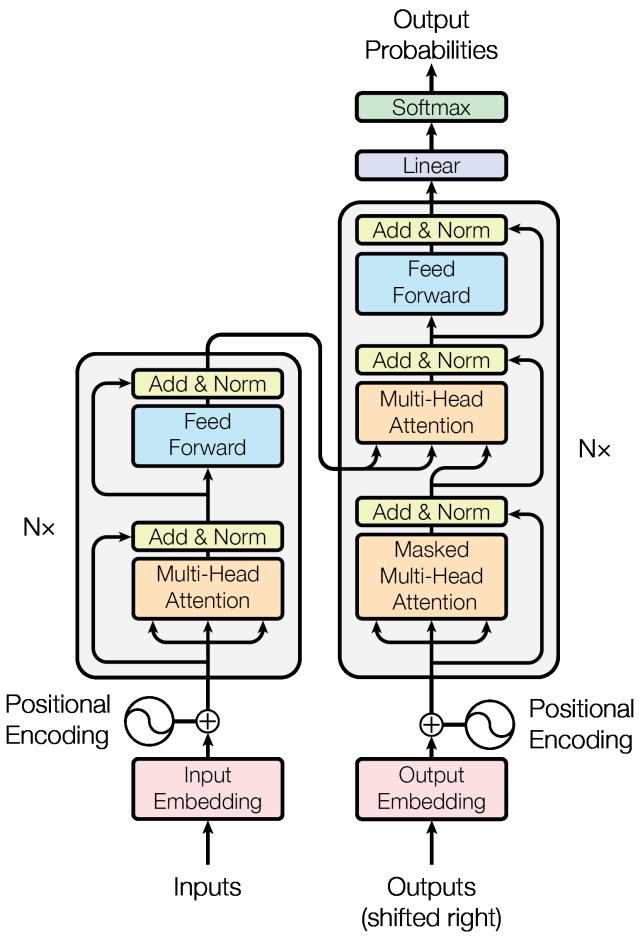


Figure 2.6. The transformer model architecture (Vaswani et al. 2017).

2.4 Conclusion

This chapter examined the platforms currently available to Fantasy Premier League managers and the technical foundations on which the proposed system is built. The review of existing solutions, from the official application to the established third-party analytics services, found the same difficulties recurring across them: the functionality a manager needs is split between separate tools; the platforms that do offer forward-looking analysis present it through statistically demanding dashboards aimed at experienced users; and none of them allows a manager to question a recommendation or to receive an explanation shaped by their own squad.

The second half of the chapter introduced the technologies that answer those gaps. Gradient-boosted trees produce the player-performance predictions. Time-series cross-validation allows those predictions to be evaluated under the same temporal conditions in which they will be used. Integer

linear programming constructs an optimal squad within the constraints of the game, and a Large Language Model turns the resulting structured output into explanations a manager can read.

Taken together, the two halves of the chapter establish the motivation for the proposed platform and the background needed to follow its implementation. The next chapter sets out the methodology adopted for its design, development and evaluation.

3 Methodology

This Chapter outlines the methodology followed throughout the design and development of the overall system.

3.1 Approach

The design and development of the system followed an application of the Star Model (Hartson and Hix 1989) (Figure 3.1), a non-sequential lifecycle model for interactive system development. Unlike linear methodologies, the Star Model organises five activities (task analysis, requirements specification, conceptual design, prototyping, and implementation) around a central evaluation activity, with no fixed order between them. Evaluation acts as the connective node where each activity is linked to evaluation, and evaluation can trigger a return to any other activity.

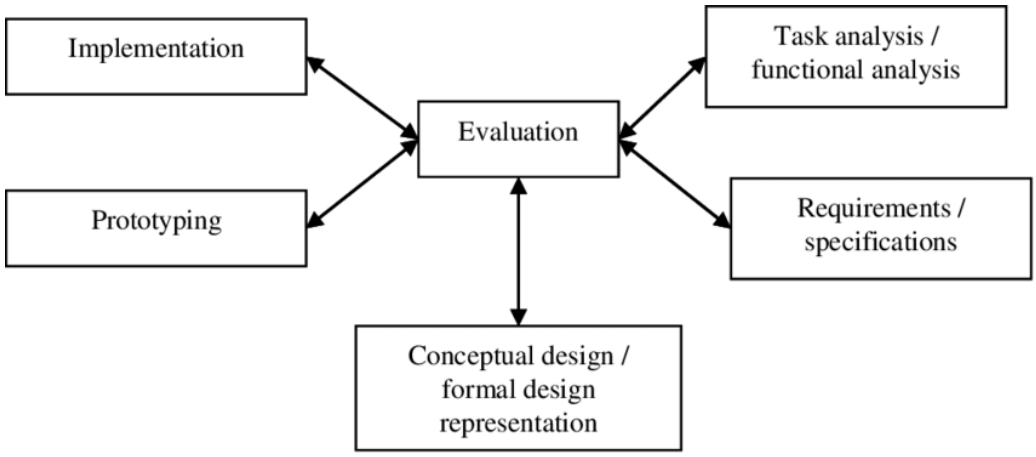


Figure 3.1. The Star Model of interactive system development, adapted from Hartson and Hix (1989).

This approach is particularly well suited to a project whose headline contribution is an adaptive and conversational interface, since user-centred, interactive systems benefit most from continuous, evaluation-driven iteration. Early prototypes were developed both for the user interface and for the machine-learning pipeline, allowing each component to evolve through repeated cycles of implementation, evaluation, and refinement. In both cases, evaluation was the driver of refinement,

in line with the central role it occupies in the model: the interface was continuously inspected and adjusted in response to usability feedback, so that each iteration became clearer and more welcoming than the last, while the predictive models were repeatedly re-trained and re-validated as new features and gameweeks were added.

3.2 Data Collection

The first step is to establish what data the system requires and how it is obtained. All football data used by the system comes from the official Fantasy Premier League public API, operated by the Premier League itself. Drawing on a single authoritative source keeps the figures used for model training, for recommendation generation, and for display within the application consistent with one another. The API exposes a full snapshot of the current season through its `bootstrap-static` endpoint (teams, gameweeks, and players, including season-to-date statistics), a `fixtures` endpoint (kickoff times, results, and the official fixture-difficulty ratings on a one-to-five scale), and a live gameweek endpoint that returns per-player, per-fixture statistics once a gameweek is under way or complete.

Data collection runs in three parts. A historical extraction assembles every player' s per-gameweek record for the season the API is currently serving and merges it with the archived records of earlier seasons, which are written to disk at each season boundary by a rollover procedure. This archive matters because the API exposes only the current campaign: without it the models could be trained on one season at a time only. Every row carries a season tag, and a combined season-and-gameweek key keeps observations in chronological order across the season boundary, so that the temporal validation described in Chapter 2.3 remains valid over the full history.

Two scheduled jobs then keep the data current. A synchronisation job refreshes the internal copy of the reference data on an adaptive interval: every fifteen minutes by default, every three minutes during match-day windows, when results and prices are moving, and once an hour overnight, when nothing is happening. A second job polls the live gameweek endpoint every five minutes and computes each manager' s score once a gameweek has finished. A third scheduler, described in Chapter 6, detects the completion of a gameweek and triggers the retraining and prediction pipeline.

The resulting dataset holds 30,983 player-gameweek records, covering 841 players across the 2025-26 and 2026-27 seasons and gameweeks one to thirty-eight. Each record contains the fields listed in Table 3.1. All remaining data (user accounts, squads, transfers, chat history, and model predictions) is first-party, generated by the application itself and stored in the same database.

Πίνακας 3.1: Player-gameweek dataset fields and descriptions.

Field	Description
Player / Team	Player identity and the real club they belong to
Position	Playing position (GK, DEF, MID, FWD)
Price	Current player price, in tenths of a million pounds
Minutes	Minutes played in the gameweek
Goals / Assists	Goals scored and assists provided
Clean sheet	Whether the player’s team kept a clean sheet
Cards / Saves	Yellow/red cards and goalkeeper saves
Bonus / BPS	Bonus points and the underlying Bonus Points System score
ICT index	Influence, Creativity, and Threat components
Form	FPL’s rolling form figure for the player
Fixture difficulty	FDR of the upcoming fixture(s) (1–5)
Status	Availability code (available, doubtful, injured, etc.)
Ownership	Percentage of managers who own the player
Gameweek points	Fantasy points scored by the player in the gameweek

3.3 System Design and Prototypes

Once the data sources were established, the overall architecture of the system was designed. Because the defining contribution of this work is the user-facing experience, the design process treated two concerns together: the adaptive and conversational interface through which managers interact with the system, and the machine-learning prediction and recommendation engine that supplies the analysis behind it.

For the interface, the priority was clarity and ease of use, with the explicit aim of improving on the experience offered by existing platforms, both the official Fantasy Premier League application and the third-party tools. After surveying those solutions (Chapter 2.1) and cataloguing their usability shortcomings, an early working prototype was built, covering the dashboard, squad management, transfers, the points breakdown, the fixture and league screens, the squad builder, and an embedded conversational assistant. The prototype was then shown to a small number of experienced Fantasy Premier League managers in informal feedback sessions, whose comments informed a series of

refinements to the interaction design before implementation continued. These sessions were formative rather than evaluative: they shaped the design while it was still taking form, and are distinct from the assessment of the finished system reported in Chapter 8.

For the prediction and recommendation engine, a data-driven pipeline was chosen over a purely rule-based approach, since it allows the system to learn from historical performance instead of relying on fixed heuristics alone. Following the analysis of the underlying technologies in Chapter 2.3, prototype models were trained to establish which features and which class of regression model produced reliable next-gameweek forecasts. A simple heuristic scoring scheme was developed alongside them, to understand how captaincy and transfer recommendations might be generated, and the two were later combined into the hybrid, personalised scheme that feeds the conversational assistant. The squad builder was designed at the same time, as an integer program that turns the models' predictions into a legal opening squad under the budget, position and club constraints of the game.

3.4 Implementation

Following the design phase, the system was implemented as a full-stack web application supported by a separate machine learning service. The development covered the client-side and server-side of the web application as well as the machine learning service that powers its recommendations, ensuring the coherence and efficiency of the final system.

For the web application, Next.js and React with TypeScript were used for the frontend, providing a responsive, component-based, and type-safe manager-facing interface, the surface on which the improved user experience was realised. The backend was built with the Express framework, again in TypeScript, providing the REST and streaming endpoints, the scheduled data-synchronisation and scoring jobs, user authentication, and the assistant orchestration logic. A shared TypeScript type-definitions package keeps the data contracts consistent between the backend and the frontend. The conversational layer is provider-agnostic, selecting between Anthropic Claude and OpenAI GPT through configuration, and is responsible only for turning the system' s structured recommendations into natural language, never for producing the underlying predictions.

For the machine learning service, Python was the language of choice, as it offers a rich ecosystem for data processing and machine learning, pandas for feature engineering, scikit-learn and XGBoost for model training, and PuLP with the open-source CBC solver for squad optimisation. This service runs as an offline batch job rather than a request-time dependency: it reads historical data, trains and validates its models, produces per-player forecasts, and writes them back to the database, thereby decoupling user-facing request latency from model inference time. Additionally, PostgreSQL was used for the storage and management of the FPL reference data, the manager data, and the model predictions, providing a single, shared source of truth for the whole platform. The machine learning service and

the Express backend communicate exclusively through this shared database rather than through a synchronous network boundary: the Python service writes prediction rows that the backend later reads when assembling recommendations.

3.5 System evaluation

Finally, the system was evaluated in order to assess the quality and effectiveness of its main components. The evaluation of the prediction models focused on their accuracy in forecasting player performance on unseen future gameweeks, using rolling-origin (walk-forward) cross-validation together with error metrics such as mean absolute error and the proportion of predictions falling within a small tolerance of the true score, since this mirrors how the models are actually used in production. Most importantly, and in keeping with the central aim of the thesis, the user interface and experience were evaluated through both expert-based and user-based methods, assessing whether the platform genuinely delivers on its promise of being more usable, accessible, and welcoming than existing solutions, while the recommendation logic was assessed for the relevance and consistency of the advice it produces.

3.6 Conclusion

This Chapter described the methodology followed throughout the design, development, and evaluation of the system. The work was structured around the Star Model, with evaluation acting as the connective node linking task analysis, requirements specification, conceptual design, prototyping, and implementation, an approach well matched to a project centred on an adaptive, user-facing interface. The Chapter then walked through the main activities of the project in turn: collecting a historical dataset of player-gameweek performances through the official Fantasy Premier League API, designing the system around an accessible, conversational interface backed by a machine-learning prediction and recommendation engine, implementing it as a full-stack application built with Next.js and Express on the web side and Python on the machine-learning side over a shared PostgreSQL database, and finally evaluating each major component, with particular emphasis on usability and user experience. The following Chapters examine each of these activities in detail.

4 Analysis and Requirements

As stated in chapter 1, the goal of this thesis is to design and implement a full-stack Fantasy Premier League platform that supports managers in making weekly decisions through an integrated interface, personalised recommendations, predictive models, and conversational interaction. The analysis of existing solutions in Chapter 2.1 identified several limitations, including the need to combine information from multiple sources, interfaces that can be difficult for inexperienced users to interpret, and limited integration between analytical tools and personalised decision support.

Based on this analysis, this Chapter identifies the users and usage scenarios considered during the design of the platform and derives the corresponding system requirements. The requirements are also informed by the objectives introduced in Chapter 1.1 and define the functionality that the architecture and individual system components must support.

4.1 Target Users and Scenarios

The application is designed for Fantasy Premier League managers who want support when making decisions about their teams. Although all users interact with the same underlying game, their requirements differ according to their experience, available time, football knowledge, and preferred decision-making strategy.

For the purposes of the system analysis, managers are divided into three broad groups: beginner, casual, and experienced or competitive managers. These groups are not intended to cover every possible type of FPL user. Instead, they represent different levels of experience and decision-making needs that the platform is designed to support.

The system focuses on the FPL manager as its primary end user. Other roles, such as system administrators or league operators, are not considered separate user groups because the functionality developed in this thesis is centred on squad management and decision support for individual managers.

4.1.1 User Analysis

Context and Problem Space

Fantasy Premier League is a fantasy sports game in which managers build and maintain virtual football teams whose scores depend on the real-world performances of Premier League players. During a season, managers repeatedly make decisions involving transfers, captaincy, starting line-ups, substitutions, chip usage, and longer-term squad planning.

The official FPL platform provides the functionality required to participate in the game together with player and fixture information. However, the analysis in Chapter 2 showed that managers seeking additional decision support often rely on third-party analytical platforms and other information sources. This can require users to interpret large amounts of statistical information and combine information from several sources before reaching a decision.

The platform developed in this thesis addresses this problem by combining squad management, player information, predictive models, personalised recommendations, and conversational interaction within the same application.

Target User Groups

Three representative user groups are considered.

User Group 1: Beginner Managers. Beginner managers have limited experience with Fantasy Premier League and may also have limited experience interpreting football statistics. Concepts such as fixture difficulty, player form, expected points, chip strategy, and transfer planning may therefore be difficult to evaluate.

Their main needs are straightforward squad management, clear presentation of player information, explanations of recommendations, and guidance when making common FPL decisions. The platform addresses these needs through direct squad-management interactions and a conversational assistant that presents recommendations and their reasoning in natural language.

User Group 2: Casual Managers. Casual managers understand the basic rules of FPL but spend limited time researching players and planning future gameweeks. Their main requirement is therefore to make informed decisions without carrying out extensive manual research.

The platform addresses this need by combining squad information, fixtures, player statistics, predictions, and personalised recommendations within a single interface. Conversational interaction also allows these users to request advice directly instead of manually interpreting all available information.

User Group 3: Experienced and Competitive Managers. Experienced managers are more likely to use statistical information when making decisions and may evaluate several transfer alternatives, fixture horizons, player ownership levels, expected points, and differential selections.

These users require more detailed information and greater control over their analysis. The platform therefore provides player comparisons, expected-points predictions, personalised ranking, multi-gameweek analysis, and squad optimisation while retaining access to the information supporting a recommendation.

Stakeholders

The primary stakeholder of the system is the Fantasy Premier League manager. Additional stakeholders are relevant in the context of the development and evaluation of the thesis, as summarised in Table 4.1.

Πίνακας 4.1: Stakeholders associated with the system.

Stakeholder	Interest
Fantasy Premier League managers	Use the application to manage their teams and obtain decision support
Thesis researcher (developer)	Design, implementation, and evaluation of the system
Academic supervisor	Evaluation of the methodology, implementation, and thesis contribution
Future developers	Maintenance and extension of the software

Fantasy Premier League managers represent the primary system stakeholders. The remaining entries are project-level stakeholders associated with the development, evaluation, and possible future extension of the platform.

User Needs Summary

The analysis of the three user groups identifies the high-level needs summarised in Table 4.2.

Πίνακας 4.2: Summary of high-level user needs.

User need	Description
Easy squad management	Perform common squad-management operations with limited effort
Personalised recommendations	Receive recommendations based on the user' s squad and preferences
Explainable recommendations	Understand the main factors supporting each recommendation
Time efficiency	Reduce the manual research required before making weekly decisions
Clear user experience	Access the main functionality and analytical information through a straightforward interface
Reliable information	Use current player, fixture, and gameweek information
Conversational interaction	Request advice through natural-language questions
Data visualisation	Inspect player statistics and predictions in an interpretable form
Advanced analysis	Compare players and use predictive information when evaluating decisions

Personas

To make the differences between the three user groups more concrete during the design process, three representative personas were defined. These are constructed design personas based on the user categories identified above and are not the result of an empirical user study.

Persona 1: Ignatios, the Beginner Manager. Ignatios is playing Fantasy Premier League for the first time and has limited experience interpreting football statistics.

Figure 4.1. Persona 1: Ignatios, the beginner manager.

- *Goals:* understand the main factors behind FPL decisions, build a valid squad, and receive guidance when selecting players and captains.
- *Difficulties:* interpreting player statistics, identifying which information is important, and evaluating transfers without previous experience.
- *Relevant system needs:* simple squad management, clear player information, recommendations, and understandable explanations.

Persona 2: Bill, the Casual Manager. Bill understands the game but has limited time to research each gameweek.

Figure 4.2. Persona 2: Bill, the casual manager.

- *Goals:* make weekly decisions quickly, identify useful transfers, and avoid overlooking important squad issues.
- *Difficulties:* combining information from several sources and spending time manually comparing players.
- *Relevant system needs:* centralised information, quick access to recommendations, fixture information, player comparisons, and conversational interaction.

Persona 3: Nikos, the Competitive Manager. Nikos is an experienced FPL manager who uses statistical information when planning his team.

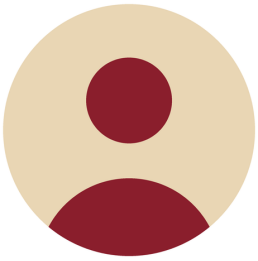


Figure 4.3. Persona 3: Nikos, the competitive manager.

- *Goals:* maximise expected performance, compare alternative transfers, plan over several gameweeks, and identify useful differential selections.

- *Difficulties:* combining multiple analytical signals with the specific state of his squad and his preferred strategy.
- *Relevant system needs:* expected-points predictions, advanced player comparison, personalised rankings, multi-gameweek analysis, and squad optimisation.

4.1.2 Use Cases

Based on the user analysis above, the following primary use cases are defined.

- **UC1: User Registration**. A new user creates an account by providing the required information. The system validates the submitted information before creating the account.
- **UC2: Login**. A registered user authenticates in order to access their personal account and squad.
- **UC3: View Dashboard**. The user views their squad, budget, points, fixtures, statistics, and relevant recommendations through a unified dashboard.
- **UC4: Manage Squad**. The user rearranges the starting line-up, performs substitutions, changes formation, and selects the captain and vice-captain.
- **UC5: Search Players**. The user searches and filters players according to attributes such as position, team, price, expected points, ownership, form, and availability.
- **UC6: View Player Details**. The user inspects a player' s statistics, fixtures, performance history, and available predictions.
- **UC7: Chat with the AI Coach**. The user asks a natural-language question about an FPL decision. The assistant uses the available user context and the output of the recommendation system to return an explained response.
- **UC8: Receive Team Recommendations**. The user receives personalised recommendations for decisions including transfers, captaincy, starting line-up, bench order, formation, and chip timing.
- **UC9: Compare Players**. The user compares selected players using their statistics, fixtures, predictions, and other available information.
- **UC10: Save Favourite Players**. The user maintains a watchlist of players for possible future transfers.

Scenario 1: Getting started without expertise. Ignatios opens the application and needs to prepare a team for the upcoming gameweek. After completing the onboarding process, he can use the squad-management interface and ask the assistant questions such as which player to captain or whether a particular transfer is worthwhile.

The recommendation system evaluates the relevant player information, predictions, squad state, and user preferences. The assistant then presents the resulting recommendation together with an explanation of the main factors behind it. Ignatios decides whether to follow the recommendation and performs any corresponding squad action manually.

Scenario 2: Planning ahead. Nikos wants to evaluate possible transfers over the next several gameweeks. He uses the player-search and comparison functionality to inspect candidate players and can ask the assistant to compare alternative decisions.

The recommendation system considers his current squad, available budget, fixtures, predictive outputs, and personalisation profile. Nikos can inspect the resulting recommendations, request explanations, and compare alternatives before deciding which changes to make. The assistant provides decision support but does not modify the team automatically.

4.2 Requirements

This Section defines the requirements that the implemented system is expected to satisfy. Functional requirements describe the capabilities provided by the platform, while non-functional requirements describe specific qualities of those capabilities that can be evaluated within the scope of the thesis.

The requirements derive from the objectives presented in Chapter 1.1, the analysis of competing solutions in Chapter 2.1, and the user analysis presented in Chapter 4.1.

Requirements are assigned one of three priorities:

- **Essential**: required for the system to fulfil its main purpose;
- **Important**: provides significant additional functionality or quality;
- **Desirable**: useful but not central to the primary contribution of the system.

4.2.1 Functional Requirements

Table 4.3 lists the functional requirements of the system.

Πίνακας 4.3: Functional requirements of the system.

ID	Requirement	Priority
FR1	The system shall allow users to register, log in, and manage their accounts.	Essential
FR2	The system shall authenticate registered users and maintain authenticated sessions.	Essential
FR3	The system shall retrieve player, team, fixture, and gameweek information from the official Fantasy Premier League data source.	Essential

ID	Requirement	Priority
FR4	The system shall allow users to create and modify a personalisation profile containing preferences such as risk tolerance, planning horizon, differential preference, and explanation depth.	Essential
FR5	The system shall display the authenticated user' s fantasy squad in an interactive format.	Essential
FR6	The system shall support direct squad rearrangement and player substitution through drag-and-drop interaction.	Essential
FR7	The system shall allow the user to select and change the captain and vice-captain.	Essential
FR8	The system shall generate a budget- and rule-compliant fifteen-player initial squad using the squad-optimisation component and allow the user to modify the result.	Essential
FR9	The system shall allow transfers while validating budget, positional constraints, and the maximum number of players permitted from the same club, and shall indicate any applicable points penalty.	Essential
FR10	The system shall allow users to activate available FPL chips according to the chip-usage rules supported by the application.	Essential
FR11	The system shall calculate each user' s gameweek score using the supported FPL scoring, captaincy, and automatic-substitution rules.	Essential
FR12	The system shall provide player statistics and fixture information together with player search and filtering functionality.	Important

ID	Requirement	Priority
FR13	The system shall allow users to compare selected players side by side and maintain a watchlist of favourite players.	Important
FR14	The system shall support classic and head-to-head leagues that users can create and join.	Important
FR15	The system shall generate player-performance predictions for upcoming gameweeks and make them available to the recommendation system and relevant user-interface components.	Essential
FR16	The system shall provide ranked, personalised recommendations for captaincy, transfers, formation, starting line-up, bench order, and chip timing using available player data, predictive outputs, deterministic scoring rules, and the user' s personalisation profile.	Essential
FR17	The system shall provide a conversational AI assistant through which users can ask natural-language questions and receive context-aware FPL advice.	Essential
FR18	Recommendations presented through the assistant shall include an explanation of the main factors supporting the recommendation.	Essential
FR19	The assistant shall operate only as a decision-support mechanism and shall not execute transfers, captaincy changes, chip activations, or line-up changes on behalf of the user.	Essential
FR20	The system shall persist user preferences, squad state, and conversation history required across authenticated sessions.	Important

ID	Requirement	Priority
FR21	The system shall provide feedback or confirmation following user actions that modify application state.	Important

Requirements FR5–FR9 cover the main interactive squad-management functionality. Requirements FR15–FR19 define the prediction, personalised recommendation, and conversational capabilities that form the main decision-support functionality of the proposed platform.

4.2.2 Non-Functional Requirements

The non-functional requirements in Table 4.4 are limited to qualities that can be examined directly using the implemented system. Broader software qualities that influenced the architecture but are not evaluated systematically are discussed separately below.

Πίνακας 4.4: Non-functional requirements of the system.

ID	Category	Requirement	Priority
NFR1	Responsive design	The primary functionality of the user interface shall remain usable at the desktop, tablet, and mobile viewport sizes considered during testing.	Essential
NFR2	Explainability	Each personalised recommendation presented through the assistant shall include an understandable explanation of the main factors supporting it.	Essential
NFR3	Performance	Computationally expensive model training and batch prediction shall execute outside the user-facing request path and shall not block normal application requests.	Important

ID	Category	Requirement	Priority
NFR4	Reliability	Failure of a background data-synchronisation or prediction operation shall not make previously stored valid data unavailable to unrelated application functionality.	Important
NFR5	Usability	The system shall provide a clean and intuitive interface with clear labelling and guidance, allowing users to access its core features without prior instruction.	Important
NFR6	Recommendation quality	The system shall generate ranked recommendations that are consistent with the user’ s profile and with the request made.	Essential
NFR7	Scoring quality	The system shall produce scores that reflect the relevance of each candidate player to the user’ s request and profile.	Essential
NFR8	Response latency	The system shall compute and return a personalised recommendation within five seconds of a request for a single user, measured over the deterministic recommendation path and excluding the response time of the external language model.	Essential
NFR9	Robustness	The system shall handle errors and exceptions gracefully, such that small failures do not affect its overall functionality, and shall provide informative messages to the user when errors occur.	Essential

ID	Category	Requirement	Priority
NFR10	Responsiveness	The system shall respond promptly to user actions and provide clear feedback about the progress of operations that are not instantaneous.	Important

These requirements can be evaluated directly through responsive-interface testing, inspection of recommendation outputs, verification of the execution architecture, and failure-path testing respectively. The evaluation methodology and results are presented in Chapter 8.

Broader Design Considerations

Several additional software qualities were considered during the design of the platform but are not defined as formal non-functional requirements in this thesis. These include **security**, **privacy and data isolation**, **accessibility**, **scalability**, **maintainability**, **portability**, **availability**, and **deployability**.

These properties remain important in the design of a full-stack application, but their complete evaluation would require work beyond the scope of the current thesis. Security, for example, would require a broader security assessment and dedicated penetration testing. Privacy and data isolation would require a more comprehensive assessment of information flows and access boundaries. Accessibility would require systematic evaluation against accessibility guidelines and, ideally, testing involving users with different accessibility needs. Scalability would require defined workload targets, concurrent-user testing, and potentially multi-instance deployment. Availability would require monitoring over an extended deployment period. Maintainability would be more meaningfully assessed through continued development and modification of the system over time, while portability would require broader browser and deployment-environment testing. Finally, deployability would require evaluation of a complete production deployment process and its operational requirements.

Although these qualities are not formally evaluated as non-functional requirements, the system design includes choices intended to provide a basis for them. Authenticated access and user-scoped data handling contribute to security and to privacy and data isolation. Separation between the frontend, backend, prediction pipeline, and database, together with explicit interfaces between these components, supports maintainability. Browser-based delivery contributes to portability, while separation of background processing from user-facing requests provides a basis for future scalability and deployability improvements. The modular architecture also allows individual components to be deployed or extended independently as the system evolves.

These qualities should therefore be understood as design considerations rather than properties that this thesis claims to have fully demonstrated. Their current support, remaining limitations, and possible future improvements are discussed among the limitations and future work in Chapter 9.

4.2.3 Requirements Traceability

The requirements defined above originate from the user needs identified in Chapter 4.1. Table 4.5 maps these needs to the functional and non-functional requirements that address them.

Πίνακας 4.5: Traceability between user needs and system requirements.

User need	Requirements	Main capability
Easy squad management	FR5–FR10, FR21, NFR1	Interactive squad management, transfer validation, and action feedback
Personalised recommendations	FR4, FR15–FR18	Profile-aware prediction and recommendation system
Explainable recommendations	FR18, NFR2	Recommendation explanations
Time efficiency	FR5, FR12, FR15–FR17, NFR3	Unified information, predictions, and direct recommendation access
Clear user experience	FR5–FR7, FR21, NFR1	Direct squad interaction and responsive presentation
Reliable information	FR3, FR15, NFR4	Official FPL data synchronisation and stored predictive information
Conversational interaction	FR17–FR19, NFR2	Context-aware, explained, non-autonomous conversational advice
Data visualisation and analysis	FR12, FR15	Player statistics, fixtures, filtering, and predictive information
Advanced decision support	FR15–FR16, NFR2	Predictive models, deterministic scoring, optimisation, and personalised ranking

The traceability table establishes the relationship between the needs identified during the analysis and the functionality implemented by the system. Chapter 5 describes how these requirements influenced the system architecture, while Chapters 6 and 7 describe the prediction, recommendation,

conversational, and interface components in greater detail. Chapter 8 evaluates the implemented functionality and the non-functional requirements that can be tested within the scope of the thesis.

4.3 Conclusion

This Chapter identified the main users and usage scenarios considered during the development of the platform. Three representative manager profiles were used to capture differences in experience, available time, and decision-making approach. Their needs were then translated into use cases and functional requirements covering squad management, FPL data, prediction, personalisation, recommendations, and conversational interaction.

A smaller set of directly evaluable non-functional requirements was defined for responsiveness, explainability, performance, and reliability. Other software qualities, including security, privacy and data isolation, accessibility, scalability, maintainability, portability, availability, and deployability, influenced the system design but are treated as broader design considerations rather than formally validated requirements.

Finally, the traceability analysis connected the identified user needs to the requirements intended to address them. These requirements provide the basis for the system architecture presented in Chapter 5 and for the prediction, recommendation, and user-interaction components described in the following Chapters.

5 System Design and Architecture Overview

This chapter describes the architecture of the platform introduced in the preceding chapters. It presents the four main components, explains their responsibilities, and follows the data exchanged between them. It also discusses the reasons for the main design choices. Detailed implementation is covered in the following chapters.

The platform uses a **three-tier, polyglot architecture** within a single monorepo. The presentation tier is the web client, which handles user interaction. The application tier is the backend, which contains business logic, game rules, authentication, scheduling, and the conversational assistant' s orchestration. The data-and-intelligence tier contains the relational database and the machine-learning pipeline that produces player predictions.

The client and backend use TypeScript so that they can share data contracts and check them at compile time. The machine-learning pipeline uses Python and its scientific-computing libraries. Figure 5.1 shows the components, external services, and main connections. Chapter 5.2.1 describes the data exchanged through these connections.

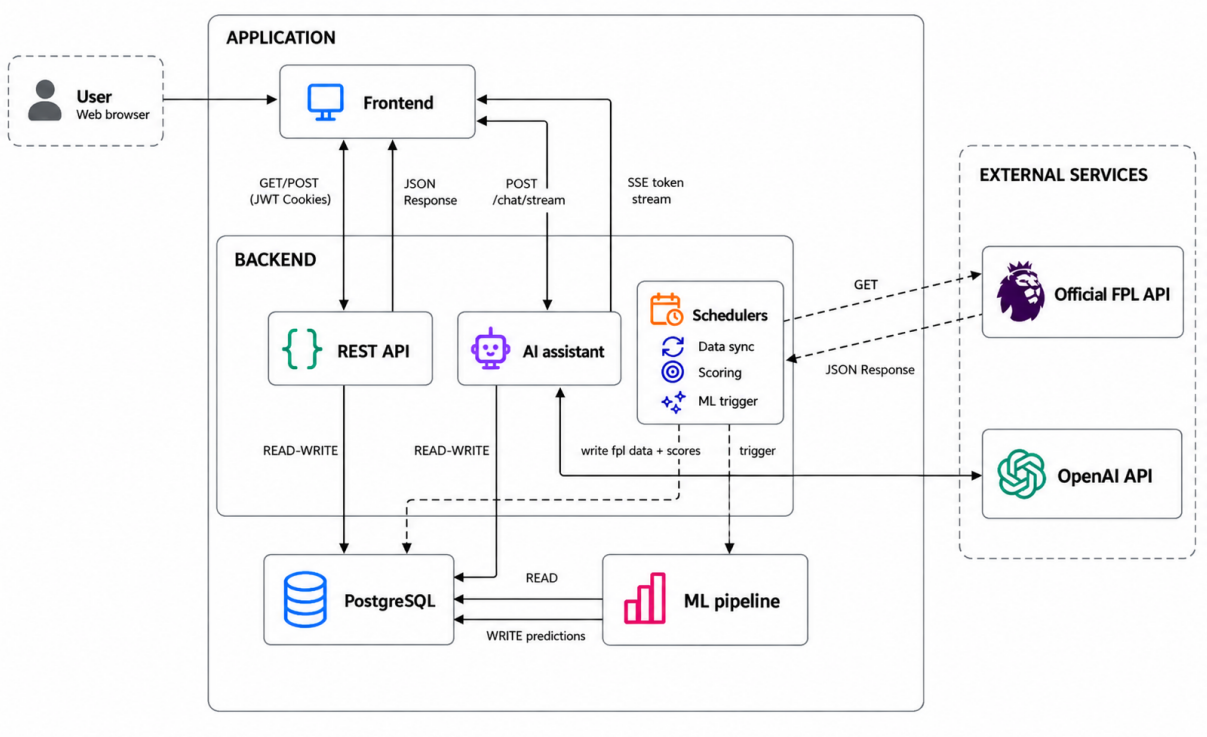


Figure 5.1. Overall system architecture of the platform.

5.1 Component Overview

The system has four components: the **user interface**, the **web application backend**, the **machine-learning service**, and the **database**. Each is developed and deployed independently within the monorepo. The client communicates with the backend over HTTP, while the backend and machine-learning service exchange data through the database. This section describes each component; Chapter 5.2 explains the reasons for these boundaries.

5.1.1 User Interface

The user interface uses **Next.js 16** and **React 19**. Its source is in `apps/web`, and it runs on port 3000. It provides the screens through which managers use the platform' s adaptive and conversational features.

Tailwind CSS 4 is imported into the global stylesheet and supplies the CSS reset and a small set of colour and typography tokens. The team page, fixtures view, and points history have dedicated stylesheets with semantic classes. Other screen-level styles are defined as inline objects in the components, keeping the styling close to the corresponding interface logic. Chapter 7.2.2 discusses this approach and its trade-offs.

The Next.js *App Router* defines routes for the dashboard, team management, transfer market, points and scoring history, fixtures, leagues, initial squad builder, authentication, and onboarding. A shared `layout.tsx` provides navigation around authenticated pages. Screen-specific components are organised in `src/features`, while reusable interface elements are kept in `src/components/ui`. A small client-side library handles backend communication through `fetch` requests with credentials, allowing the browser to send the session cookie with each call.

The **conversational assistant** is available from every screen. Its chat components follow the structure `ChatWidget` ⟶ `ChatPanel` ⟶ `MessageBubble`. The same interface contains the onboarding flow that collects the preferences used to personalise recommendations. The widget calls the backend' s assistant endpoints and displays the conversational reply and its accompanying structured recommendation.

Keeping the assistant alongside team management, transfers, and analysis allows managers to use advice within the same workflow. For example, a manager can ask for a captaincy recommendation and act on it without leaving the team screen.

5.1.2 Web Application Backend

The backend is a REST API built with **Express 4** and **TypeScript**, running on Node.js. Its source is in `apps/api`, and it uses port 4001. **Prisma** handles database access, and the backend owns the canonical database schema. It is the only component permitted to contact the official Fantasy Premier League API and external Large Language Model providers. It also runs the game logic and coordinates the conversational assistant.

The backend handles three main areas:

1. **HTTP layer**. Cross-origin resource sharing is restricted to the web client with credentials enabled, JSON request bodies and cookies are parsed, requests are authenticated with JSON Web Tokens (`jsonwebtoken`, with password hashing by `bcryptjs`), and incoming payloads are validated against `Zod` schemas before reaching any business logic. The two conversational endpoints are additionally placed behind a per-user rate limiter of eight requests per fixed one-minute window, which answers a breach with `429` and a `Retry-After` header; unlike every other route, these two incur a billable call to an external language-model provider on each request. Requests are dispatched to feature-oriented route modules, `/auth`, `/fpl`, `/team`, `/transfers`, `/chips`, `/points`, `/leagues`, and `/ai`, each encapsulating a coherent slice of the domain. The seven domain modules observe a uniform three-layer convention: a *routes* file declares the endpoints and attaches the relevant middleware, a *controller* validates the request, invokes the domain logic, and translates domain errors into HTTP responses, and a *service* implements the business logic and is the only layer permitted to reach the database through Prisma. Separating HTTP concerns from domain logic in this way renders each module independently testable and allows a reader who

has understood one to navigate the others.

2. **Background schedulers**. Three schedulers start with the server and update the system without user interaction. The data-synchronisation scheduler retrieves official FPL reference data every three minutes on match days, every fifteen minutes by default, and every sixty minutes between 01:00 and 06:00 UTC. It upserts teams, players, events, and fixtures. The scoring scheduler runs every five minutes. It calculates provisional scores when a gameweek is finished and final scores when the official data are marked as checked, including transfer penalties, chips, and league standings. The machine-learning scheduler checks every fifteen minutes for a newly completed, checked gameweek and triggers the Python pipeline when appropriate.
3. **AI assistant subsystem**. The `/ai` routes connect to a staged pipeline of specialised services. A single controller coordinates these services. Intent detection identifies requests about captaincy, transfer advice, transfer evaluation, player comparison, initial squad selection, or general help. The context builder retrieves the manager' s squad, budget, free transfers, active chip, recent scores, and upcoming fixtures. The ML recommendation engines read stored predictions and score candidate decisions. The prompt builder combines the system prompt, personalisation weights (Chapter 6.2.1), and a running conversation summary before calling the LLM service. Conversation summaries, the assistant profile, and recommendation logs are stored in the database to support memory across turns and sessions. Chapters 6 and 7 describe these stages in detail.

The server entry point mounts the route modules and starts the schedulers:

```
// apps/api/src/server.ts (excerpt)
app.use('/auth',      authRouter);
app.use('/team',      teamRouter);
app.use('/transfers', transfersRouter);
app.use('/points',    pointsRouter);
app.use('/leagues',   leaguesRouter);
app.use('/ai',        aiRouter);

app.listen(port, () => {
  startFplSyncScheduler();     // adaptive 3 / 15 / 60 min cadence
  startScoringScheduler();     // every 5 min, scores finished gameweeks
  startMlPipelineScheduler();  // every 15 min, triggers the ML pipeline
});
```

5.1.3 Machine Learning Service

The machine-learning service is a standalone **Python** pipeline in `apps/ai`. It produces the player-performance forecasts used by the recommendations and solves the initial squad optimisation problem. It uses `pandas` for data manipulation, `scikit-learn` and `XGBoost` for modelling, `PuLP` for integer programming, and `joblib` for saving trained models. It does not expose a network service. The backend starts it as a child process for both scheduled prediction work and initial squad requests.

The recurring pipeline has five stages, coordinated by `pipeline.py`:

1. **Extract**. Reads the historical gameweek data from PostgreSQL through a SQLAlchemy engine.
2. **Feature engineering**. Derives the predictive features, among them recent form, fixture difficulty, expected goals and assists, availability, and expected minutes, and assembles them into the training dataset.
3. **Training**. Fits three models, a regressor for expected points, a classifier for the probability that a player features, and a regressor for expected minutes, selecting in each case the best of several candidate algorithms on validation data. The stage closes with a *safety gate*: should the new points model' s mean absolute error exceed its predecessor' s by more than twenty-five per cent, the entire artifact bundle is restored from the backup taken before training. The gate is bypassed where the model' s target scope has changed, since errors measured against different targets are not comparable.
4. **Prediction**. Produces the next-gameweek expected points for every player.
5. **Upsert**. Writes those predictions back into the database, where they are consumed by the assistant.

The backend calls `optimize_initial_squad.py` when a manager requests an initial squad. The script returns the highest-scoring valid squad as JSON, subject to the official budget, formation, and club-quota constraints. Supporting scripts handle season backfill, season rollover, single-gameweek updates, next-gameweek prediction, backtest export, and thesis plots.

A separate rolling-origin cross-validation harness evaluates model quality offline. It runs when explicitly invoked, rather than on every pipeline execution. Its report informs the evaluation presented in Chapter 6, which also describes the models themselves.

5.1.4 Database

The database is **PostgreSQL 16**, running in the Docker container defined in `infra/docker-compose.yml`. The database is named `fantasy_fpl` and is exposed on port 5432. The backend uses Prisma to access it, while the Python pipeline uses raw SQL through SQLAlchemy. The database holds the platform' s authoritative records and provides the main integration point between the backend and machine-learning service, as discussed in Chapter 5.2.

The schema is organised into cohesive groups that mirror the domains of the application, as summarised in Table 5.1. Running the database in a container makes the data tier reproducible and isolated: it can be started with a single command, it pins the exact PostgreSQL version, and its state is persisted in a named Docker volume independent of the container' s lifecycle, which is valuable both for development and for the reproducibility expected of a scientific artifact.

Πίνακας 5.1: Logical organisation of the PostgreSQL schema.

Group	Representative tables	Purpose
Identity & sessions	`User`, `Session`	Accounts and authenticated sessions
FPL reference data	`FplPlayer`, `FplTeam`, `FplEvent`, `FplFixture`, `PlayerGameweekStat`	Read-only mirror of the official FPL data
Manager game state	`UserTeam`, `UserPick`, `UserChip`, `UserTransfer`, `UserGameweekScore`	Each manager' s squad, transfers, chips, and scores
Leagues	`League`, `LeagueMember`, `H2HMatch`	Classic and head-to-head competitions
Assistant memory	`AssistantSession`, `AssistantMessage`, `UserAssistantProfile`, `RecommendationLog`	Conversation history, personalisation, and audit
ML output	`PlayerPrediction`	Expected points per player and gameweek

5.1.5 From Prediction to Explanation: An End-to-End Example

The request "Who should I captain this week?" illustrates how the components work together. Figure 5.2 follows the request from the chat interface to the reply:

1. **Web client sends the request**. The message, together with the session cookie, is sent to the backend' s `/ai/chat/stream` endpoint.
2. **Authentication and admission**. Middleware verifies the access token and attaches the manager' s identity to the request, which is admitted only if the manager remains within the allowance of eight conversational requests per minute.
3. **Scope gate and intent detection**. The message is first tested against a deterministic relevance check,

so that an off-topic request is refused before any further work is undertaken and without cost at the language-model provider. A message that passes is then classified, in this case as a *captaincy* request; the classifier itself is described in Chapter 6.4.

4. **Context builder**. Retrieves, through Prisma, the manager' s current squad, budget, free transfers, active chip, recent scores, and upcoming fixtures, together with the personalisation profile and the recent conversation history.
5. **Recommendation layer**. Reads the precomputed rows from `player_predictions` and scores every eligible member of the squad, combining expected points with the probability of playing, the fixture context, and the personalisation weights derived from the manager's onboarding answers, which govern how heavily rotation risk is penalised. The scoring itself is set out in Chapter 6.2.2.
6. **Ranked candidates produced**. A ranked list of captaincy candidates is produced, each carrying a reasoning string and a risk label of low, medium, or high.
7. **LLM invoked**. Only at this point is the Large Language Model invoked: it receives the structured recommendation block together with the context and conversation history, and streams back a natural-language reply that presents and explains the top candidates.
8. **Reply and payload persisted**. The reply and the full recommendation payload are persisted, and the manager sees the answer token by token in the chat panel.

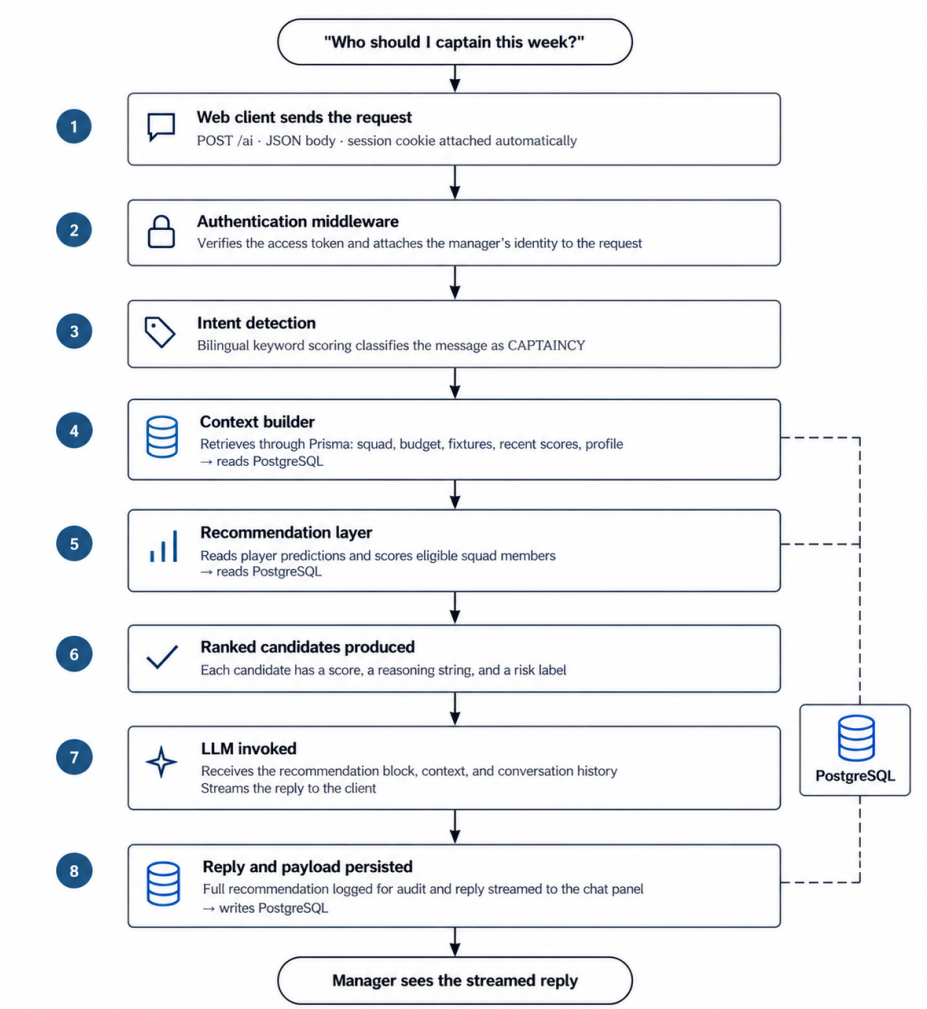


Figure 5.2. End-to-end flow of a captaincy request, from the manager's message to the streamed reply. Steps 4, 5, and 8 are the principal points of contact with the shared PostgreSQL database; the language model is invoked only at step 7, after every number in the recommendation has already been computed.

The example distinguishes three outputs. A *prediction* is a player-level forecast, such as expected points, participation probability, or expected minutes, and does not depend on the manager. A *recommendation* uses predictions together with the manager's squad, budget, and profile to suggest a decision, such as a captain or a transfer. An *explanation* is the LLM's natural-language presentation of that recommendation. Separate subsystems produce and store these outputs, and Chapter 8 evaluates them separately.

5.2 Architectural Decisions

The following decisions define the main boundaries of the system. Each discussion explains the choice, the alternative considered, and its practical trade-offs. References point to later chapters where implementation details are covered.

A monorepo with a polyglot, three-tier structure. The client, backend, and machine-learning service share one repository but run separately. The npm workspaces organise the TypeScript packages, including the shared definitions in `packages/types`. Player, recommendation, and chat-message types are declared once and checked by the compiler in both the client and backend. Keeping the code together also allows related changes to be versioned together.

The Python pipeline remains outside the TypeScript package arrangement. Using Python retains access to the scientific-computing libraries needed for modelling. A single-language implementation would be more uniform, but would give up this existing tool support.

Containerisation of the data tier only. PostgreSQL runs in a container, while the client and backend run directly on the host. Containerisation pins the server version exactly and stores the database in a named volume that survives the container' s lifecycle. The Node.js dependencies are reproduced through the lockfile and workspace definition. The project therefore does not provide a single-command deployment for the whole system. Containerising the application tier is included in the future work in Chapter 9.

Integration through a shared database rather than a synchronous service call. The Python pipeline runs as an offline batch process. It writes predictions to the database, and the backend reads them when handling requests, as shown by flow 4 in Figure 5.1. The backend does not call an online Python inference endpoint.

Initial squad selection is the exception. The squad must be assembled for the manager at request time, so the backend starts the optimiser as a child process and reads its output directly. Recurring recommendations use the stored predictions.

This arrangement keeps training and inference outside the normal advice request. The manager waits only for a database read, since the forecasts have already been calculated. It also avoids operating and securing another network API between the TypeScript and Python components.

The main trade-off is freshness: predictions remain unchanged until the next batch completes. The shared schema also needs coordination. Prisma queries and Python SQL must both be updated when a relevant column changes, and there is no shared compiler check between them. The design accepts this delay for forecasts organised around gameweeks. Chapter 9 discusses the limitation further.

Automated retraining behind a safety gate. The machine-learning scheduler retrains the models as each gameweek completes, so the pipeline runs unattended. Unattended retraining carries an obvious hazard: a corrupt batch of data or an unlucky split could quietly replace a sound model with a worse one.

The pipeline therefore backs up the current artifacts before training and compares the new points model against its predecessor on validation error. Should the error regress beyond a fixed tolerance, the whole bundle, all three models with their metadata, is restored, which keeps the deployed versions mutually consistent. The comparison is bypassed in one circumstance, when the model's target scope has changed, since errors measured against different targets say nothing about one another. The gate is keyed on the points model because expected points is the quantity every downstream recommendation consumes. Its cost is the occasional rejection of a model that was in fact an improvement; its benefit is that no regression reaches a manager unnoticed, a balance appropriate to a system whose output is acted upon.

Adaptive, event-driven scheduling. The scheduler refreshes FPL data more frequently during matches and less frequently overnight. This balances data freshness with system load and requests to the external API. Scoring and machine-learning work are triggered by gameweek completion, so these tasks run when the required data become available without manual intervention.

Two-token authentication with server-side revocation. Login issues a pair of JSON Web Tokens: an access token valid for fifteen minutes and a refresh token valid for thirty days, both carried in `httpOnly` cookies with a `SameSite=Lax` policy and, in production, the `Secure` flag. The access token is verified statelessly on every protected request, and the acting user's identity is taken from its signed `sub` claim alone; an identifier arriving in a request body is ignored, and every subsequent query is filtered by the authenticated identity, which is the mechanism enforcing the per-user isolation required in Chapter 4. Refresh tokens are treated otherwise, being persisted in the `Session` table and rotated on use: a refresh deletes the old row and creates a new one, and logging out deletes the row outright, so a long-lived credential can be withdrawn at any moment. The result is hybrid by intent rather than by accident, in that authentication on the hot path touches no database while session longevity remains stateful and revocable. Cross-site request forgery is answered by three overlapping measures: the `SameSite=Lax` policy, which stops the browser attaching these cookies to cross-site state-changing requests; the `httpOnly` flag, which puts the tokens beyond the reach of client-side scripts; and the CORS allow-list, which admits credentialed requests from the web client's origin alone.

5.2.1 System Data Flow

The architecture has four main data flows, shown in Figure 5.1:

1. **Synchronous request–response**, between the client and backend. Loading the dashboard, making a transfer, and sending a chat message use REST calls over HTTP with JSON and an authentication cookie. Express handles validation, business logic, and database access before responding. This is the flow users interact with directly, and long-running computation is kept outside it to maintain responsiveness.

2. **Scheduled ingestion of external data**. The synchronisation scheduler calls the official FPL `bootstrap-static` and `fixtures` endpoints over HTTPS and upserts the results. This maintains the local reference tables for players, teams, events, and fixtures. During backfills, the Python pipeline also retrieves per-gameweek statistics from the external source.
3. **Conversational path to the external Large Language Model providers**. The assistant sends its assembled prompt to the configured provider and streams the reply back to the client. Claude is used by default, with OpenAI available as an alternative. This is the only flow that sends user-contextual content outside the system, and it passes through the LLM-service stage.
4. **Machine-learning retraining and prediction loop**. When a gameweek completes, the backend scheduler starts the Python pipeline. The pipeline reads training data from PostgreSQL, retrains and validates the models, and writes new forecasts to `PlayerPrediction`. The assistant later reads these records while handling a request. The database connects the scheduled prediction work to the interactive recommendations.

5.2.2 Reliability, Freshness, and Operational Concerns

The system also needs to handle unavailable services, delayed data, and failed jobs. Its failure-handling principle is that background failures affect data freshness rather than application availability.

Failure handling. If the official FPL API is unreachable, the synchronisation run fails, the error is logged, and the attempt is simply repeated at the next scheduled tick; the reference tables retain their last successfully synchronised state, and every user-facing feature continues to operate on that state. If the Python pipeline crashes or exceeds its execution cap (its child process is bounded at forty-five minutes), its output is discarded, the failure is logged, and the previously deployed models and predictions remain in place untouched, since the artifact backup and the idempotent upsert mean that a partial run cannot corrupt the served state. If predictions are missing for a particular player or gameweek, the recommendation layer falls back to its heuristic track, which computes advice directly from the raw FPL statistics, so the assistant remains functional with reduced sophistication rather than failing. If the configured LLM provider is unavailable, only the conversational reply is affected: the error is surfaced to the chat client while every other feature of the platform continues unaffected, and the operator can switch to the alternative provider through a single environment variable.

Prediction freshness. Each `PlayerPrediction` record stores its target gameweek, model version, and generation timestamp. The combination of player, gameweek, and model version is unique. Recommendations can therefore be traced to the model vintage and generation time of their input predictions. These fields make stale predictions detectable and support inspection of the batch-based recommendation process.

Concurrency. The current deployment model is a single backend instance, and the schedulers run

inside that instance’ s process. Idempotent upserts and unique database constraints make a repeated or overlapping run harmless at the data level, but running two backend instances would duplicate the scheduled work itself (two synchronisation jobs, two scoring passes, two pipeline spawns). Horizontal scaling of the backend would therefore require moving the schedulers behind a leader-election or distributed-locking mechanism; this is recorded as a known limitation and a scalability concern for future work rather than a present requirement.

Deployment. In the development setup used for this thesis, the client and API run as local Node.js processes, and PostgreSQL runs in Docker. Environment files provide the database URL, JWT signing secrets, and LLM API keys. These files are excluded from version control.

The proposed production setup would containerise the two Node.js services alongside the database and place them behind a TLS-terminating reverse proxy. The cookie configuration already enables `Secure` in a production environment. A full production deployment is outside the thesis scope and remains future work.

5.3 Conclusion

This chapter described the Next.js client, Express backend, Python machine-learning pipeline, and PostgreSQL database. Together, they form a three-tier application maintained in one repository, with the database running in a container.

The main integration decision is to store batch predictions in the shared database for later use by the backend. This reduces complexity and user-facing latency, while making prediction freshness depend on the latest pipeline run. The chapter also explained authentication, scheduling, provider integration, and the four main data flows. Chapter 6 examines the prediction models and recommendation engine in detail.

6 Machine Learning Prediction and Recommendation Engine

The previous chapter presented the overall architecture of the platform and introduced the machine-learning pipeline and recommendation engine. This chapter examines these two components in more detail. The system follows a hybrid approach: machine-learning models produce player-performance forecasts, deterministic algorithms use these forecasts and explicit rules to generate recommendations, and a Large Language Model (LLM) presents the results in natural language. Player statistics and prices are retrieved from stored FPL data, while predictions and recommendation scores are calculated by the relevant components before the LLM is invoked.

This separation supports the requirements defined in Chapter 4 by making the quantitative basis of the advice traceable and independently testable. The predictive models can be evaluated on held-out data using defined performance metrics, while the recommendation algorithms can be checked against their scoring rules and constraints. An LLM-only approach would make this evaluation more difficult, since generated responses can vary and may contain unsupported information. In the implemented system, the LLM is responsible for explaining the supplied results rather than calculating them. Its prompt instructs it to use only the numerical information provided, reducing the risk of numerical hallucinations without eliminating it entirely.

Figure 6.1 illustrates the main stages of this process. Historical records from the database are transformed into features and used by the predictive models to estimate player performance and participation. These outputs are processed and stored for use by the recommendation engine, which also applies heuristic rules and the manager' s preferences when evaluating available decisions. The resulting recommendations and supporting context are then passed to the LLM to produce the conversational response shown to the user.

Chapter 6.1 presents the predictive models, Chapter 6.2 describes the recommendation and scoring system built around their outputs, and Chapter 6.3 and Chapter 6.4 explain their implementation.

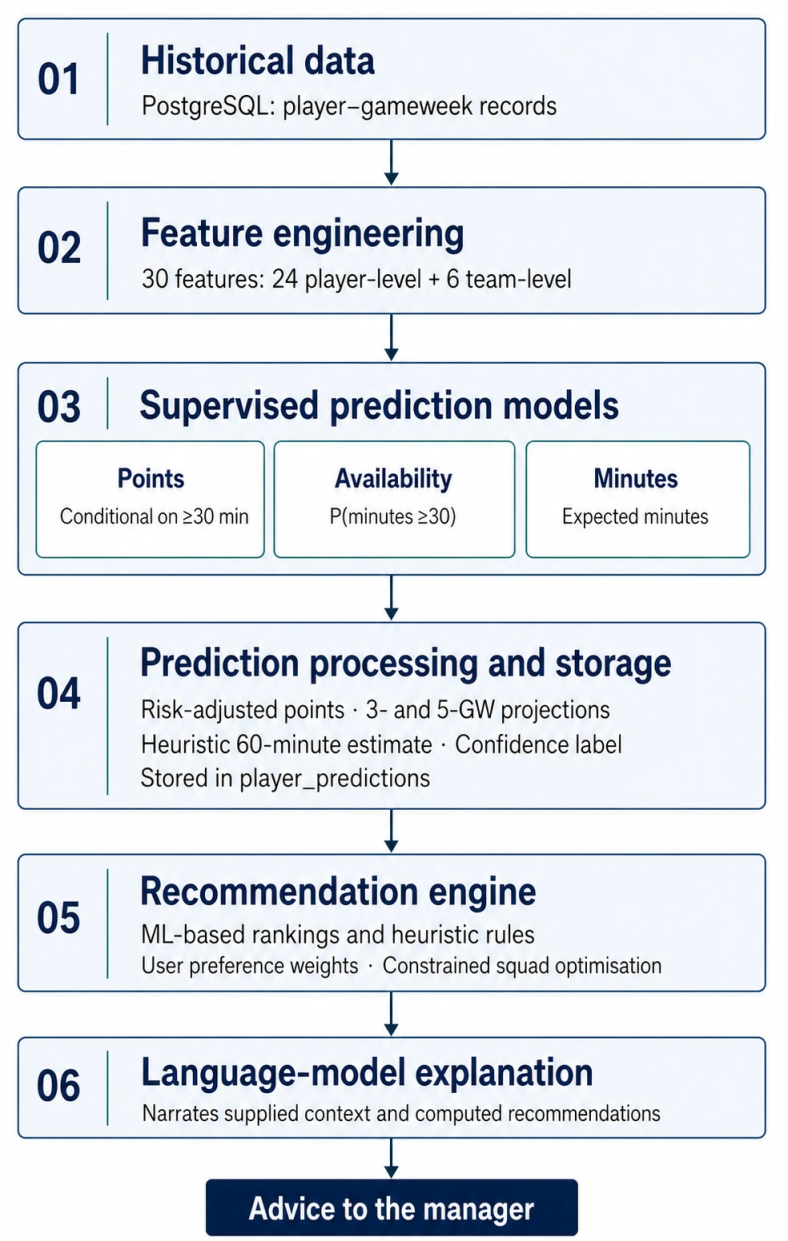


Figure 6.1. The prediction and recommendation process: historical records are transformed into features, three supervised models generate estimates, post-processing produces stored predictions, and recommendation services supply results and context for language-model explanation.

6.1 Prediction Model Design

The prediction system uses three supervised-learning models to estimate player performance and participation in the upcoming gameweek. These models predict fantasy points conditional on playing at least thirty minutes, the probability of reaching that threshold, and expected minutes played. Their outputs provide the numerical inputs used by the recommendation engine described in Chapter 6.2.

In supervised learning, a model learns relationships between input features and observed outcomes from historical examples. Each example in this application represents a player at a particular gameweek: the input consists of thirty numerical features describing individual performance, team context, availability, and fixtures and the corresponding targets describe subsequent points and participation.

Points and minutes prediction are **regression tasks**, because the models produce numerical estimates. Availability prediction is a **binary classification task**, because its observed target records whether a player completed at least thirty minutes. The fitted classifier produces an estimated probability of that event.

Πίνακας 6.1: The three supervised prediction tasks.

Model	Learning task	Output
Points	Regression	Points conditional on at least thirty minutes
Availability	Binary classification	Probability of at least thirty minutes
Minutes	Regression	Expected minutes played

Modelling these quantities separately distinguishes potential return from participation risk. A player may offer a high conditional return but have an uncertain role in the upcoming gameweek. Another player may offer a lower return with more reliable participation. Separate outputs allow the recommendation engine to account for these differences explicitly.

Let $p_{i,g}$ and $m_{i,g}$ denote the observed fantasy points and minutes of player i in gameweek g. For a feature vector $\mathbf{x}_{i,g}$, the associated targets are

$$y_{i,g}^{\text{pts}} = p_{i,g+1} \quad \text{for rows with } m_{i,g+1} \geq 30, \qquad y_{i,g}^{\text{play}} = \mathbb{1}[m_{i,g+1} \geq 30], \qquad y_{i,g}^{\text{min}} = m_{i,g+1}. \tag{6.1}$$

The indicator function equals one when the condition is satisfied and zero otherwise. For example, if

the player records six points and seventy minutes in the following gameweek, the observed targets are six points, a positive participation label, and seventy minutes. These are known outcomes used for learning and evaluation, rather than predictions generated by the models. The points target is defined only on rows meeting the thirty-minute condition, which is the population the points model is fitted on; the availability and minutes targets are defined on every row.

The notation assumes consecutive gameweek records. The implementation constructs targets by sorting each player's records chronologically across seasons and selecting the next available observation. Missing records and season boundaries therefore require care when interpreting the next-gameweek target.

The pipeline combines available archived seasons with completed gameweeks of the current season. Eligible examples are divided chronologically into fitting and validation subsets; not every available record directly fits the selected models. Chapter 6.3 distinguishes historical training examples from inputs constructed for an upcoming prediction.

The following subsections describe the player and team features, the three models, and a worked prediction from an actual run. Supporting libraries, preprocessing configurations, and model-persistence procedures are presented in Chapter 6.3.

6.1.1 Feature Engineering

Feature engineering transforms stored player-gameweek records into numerical model inputs. All three models use thirty features: twenty-four describe the individual player and contextual attributes, and six describe the player's team. The feature list is defined centrally:

```
FEATURE_COLS = BASELINE_FEATURE_COLS + TEAM_FEATURE_COLS
```

The shared registry determines the input columns selected by training and inference. Player-level features capture recent points and minutes, cumulative season performance, fixture context, availability, and metadata and team-level features add attacking and defensive context, as described in Chapter 6.1.2.

Table 6.2 lists the player-level group. FDR denotes Fixture Difficulty Rating; ICT refers to the official FPL Influence, Creativity, and Threat indicators. Shifted transformations in the table describe the training feature builder.

Πίνακας 6.2: The twenty-four player-level features and their construction methods.

Feature	Description	Construction
`form`	Official FPL form	Extracted metadata
`pts_last_1gw`	Preceding recorded points	Per-player shift
`pts_last_3gw_avg`	Recent mean points	Shifted 3-observation mean
`pts_last_5gw_avg`	Recent mean points	Shifted 5-observation mean
`pts_last_5gw_std`	Variation in recent points	Shifted 5-observation standard deviation
`minutes_last_3gw_avg`	Recent mean minutes	Shifted 3-observation mean
`minutes_last_5gw_avg`	Recent mean minutes	Shifted 5-observation mean
`chance_of_playing`	Status-based availability input	Status mapping when absent
`next_gw_fdr`	Next-fixture difficulty	Fixture context
`avg_fdr_next3`	Fixture-difficulty summary	Fixture aggregation
`is_home_next_gw`	Upcoming home indicator	Fixture context
`season_total_pts`	Accumulated season points	Season-scoped shifted cumulative sum
`pts_per_game`	Points per gameweek with an appearance	Shifted totals and appearance count
`goals_scored`	Accumulated season goals	Season-scoped shifted cumulative sum
`assists`	Accumulated season assists	Season-scoped shifted cumulative sum
`clean_sheets`	Accumulated clean-sheet indicator	Season-scoped shifted cumulative sum
`bonus`	Accumulated season bonus	Season-scoped shifted cumulative sum
`ict_index_avg`	Recent mean ICT index	Shifted 5-observation mean
`threat_avg`	Recent mean Threat	Shifted 5-observation mean
`creativity_avg`	Recent mean Creativity	Shifted 5-observation mean

Feature	Description	Construction
`influence_avg`	Recent mean Influence	Shifted 5-observation mean
`position`	Position category, 1 to 4	Player metadata
`now_cost`	Price in tenths of £1 million	Player metadata
`selected_by_percent`	Ownership percentage	Player metadata

Recent performance and rolling windows

Rolling features summarise recent observations instead of relying on one result. Three-observation windows give a shorter-term summary, while five-observation windows represent a longer recent history. These lengths are implementation choices rather than experimentally established optima.

The training pipeline shifts each player's series by one observation before calculating rolling statistics. For a complete sequence, the three-gameweek points average at feature row g is

$$\text{pts_last_3gw_avg}_{i,g} = \frac{p_{i,g-1} + p_{i,g-2} + p_{i,g-3}}{3}.$$

If a player scores two, six, and four points in gameweeks 7, 8, and 9, the mean at feature row GW10 is $(2 + 6 + 4)/3 = 4.0$. The shift excludes GW10's own result. Calculations operate on available observations within each player's chronologically ordered records.

Standard deviation complements the means by describing variation. Two players may have the same average while one produces consistent scores and the other alternates between low returns and occasional high scores. The model can use this distinction alongside the remaining inputs.

Season totals and raw outcomes

Cumulative features are calculated separately for each player and season, with a shift excluding the feature row's own outcome. Points per game divides accumulated points by the corresponding count of gameweeks with positive minutes, bounding the denominator below by one to avoid division by zero.

Player rolling histories can extend across a season boundary, while cumulative season totals reset. This preserves recent historical information without turning current-season totals into career totals.

Raw points are retained in `gw_points`, while accumulated points are written separately to `season_total_pts`. Target construction uses the raw column and the following excerpt illustrates its use in rolling calculations:

```
g = df.groupby("player_id")["gw_points"]
df["pts_last_1gw"] = g.shift(1).fillna(0)
df["pts_last_3gw_avg"] = g.transform(
    lambda s: _rolling(s, window=3)
).fillna(0)
df["pts_last_5gw_avg"] = g.transform(
    lambda s: _rolling(s, window=5)
).fillna(0)
```

6.1.2 Team-Level Features

The player-level features describe individual performance and contextual attributes but do not explicitly summarise recent club attacking and defensive performance. Team-level inputs provide this additional context: a defender' s return partly depends on the club' s ability to avoid conceding, while an attacking player' s opportunities are influenced by the surrounding team.

Πίνακας 6.3: The six team-level features. An eligible observation has positive recorded playing time.

Feature	Description
`team_gf_l5`	Mean reconstructed goals scored over up to five preceding eligible team-gameweeks
`team_ga_l5`	Mean reconstructed goals conceded over the same window
`team_cs_rate_l5`	Mean reconstructed clean-sheet indicator
`team_gd_l5`	Difference between goals-scored and goals-conceded means
`team_venue_gf_l5`	Venue-specific rolling attacking mean
`team_venue_ga_l5`	Venue-specific rolling defensive mean

Opponent features considered but excluded

Five opponent-strength features are implemented but excluded from `FEATURE_COLS`. The available historical records do not consistently identify earlier opponents, while live fixture and team tables describe the current season. An attempted reconstruction of historical fixtures was rejected as insufficiently reliable.

Training on absent or constant inputs does not establish a relationship between real opponent-feature values and the target. Their exclusion is therefore a data-availability decision, not evidence that opponent strength lacks predictive value. Six to eight completed gameweeks of the new season is a planned point for reassessing coverage, not an automatic adoption threshold. Adoption requires adequate data quality, consistent construction, and a new ablation.

6.1.3 Points Prediction Model

The points model estimates fantasy return conditional on playing at least thirty minutes. It is fitted only on examples meeting that target condition, preventing non-appearances and brief appearances from directly determining the fitted relationship for substantial participation. This changes the prediction's meaning: the model does not independently estimate return across all possible participation outcomes.

$$\widehat{q}_{i,g+1} \approx \mathbb{E}[p_{i,g+1} \mid m_{i,g+1} \geq 30, \mathbf{x}_{i,g}]. \tag{6.2}$$

Completing thirty minutes is not the same event as starting: substitutes can reach the threshold and starters can leave before it.

Candidate algorithms

The candidates are:

- **Random Forest Regressor**
- **Gradient Boosting Regressor**
- **Histogram-Based Gradient Boosting Regressor**
- **XGBoost Regressor**

Random forest. A random forest fits many decision trees independently and averages their predictions. Each tree is grown on a bootstrap sample of the training rows, and at every split only a random subset of the features is considered, so the individual trees make different errors and averaging cancels much of the variance a single deep tree would show. The method needs little tuning to produce a stable result and does not overfit as more trees are added, which makes it a dependable reference against which the boosted candidates are judged (Breiman 2001).

Gradient boosting. Boosting builds its ensemble sequentially rather than independently. Each new tree is fitted to the error left by the trees before it, so the ensemble directs its capacity at the observations currently predicted worst, and a learning rate scales each tree's contribution so that improvement accumulates gradually instead of in large, unstable steps. Boosted ensembles typically reach lower error

than a forest of comparable size on structured data, at the cost of greater sensitivity to the learning rate and the tree depth (Friedman 2001).

Histogram-based gradient boosting. This is the same boosting procedure with a different split-finding strategy. Continuous feature values are grouped into a fixed number of bins before training, so the search for the best split evaluates a few hundred candidate thresholds per feature rather than every distinct value present in the data. Training time falls substantially with little loss of accuracy, and missing values are handled natively. Both properties matter here, because the pipeline retrains without supervision after every completed gameweek.

XGBoost. XGBoost is a regularised implementation of gradient boosting. It adds explicit L_1 and L_2 penalties on the tree weights to the training objective, and it subsamples both rows and feature columns when growing each tree, so every tree sees a slightly different view of the data. These controls give finer influence over the balance between bias and variance than the standard implementation, which is useful on a target as noisy as a single gameweek' s fantasy return (Chen and Guestrin 2016).

Why tree ensembles rather than a neural network. The prediction problem here is tabular, small, and heterogeneous, and each of those properties favours trees. The thirty features mix quantities on entirely different scales, minutes between zero and ninety, prices in tenths of a million, ownership percentages, an ordinal position code, and a difficulty rating from one to five; a tree splits on thresholds and is therefore indifferent to scale, whereas a network requires all of them to be normalised and the categorical ones encoded. The training set is also modest: one archived season and the completed gameweeks of the current one, reduced further for the points model because it is fitted only on appearances. Neural networks generally need considerably more data before they match well-configured tree ensembles on problems of this shape (Grinsztajn, Oyallon, and Varoquaux 2022). Two operational constraints reinforce the choice. The models are retrained unattended after every gameweek, as a child process on the same machine and under a forty-five-minute cap, with no accelerator available, so training must be quick and reliable rather than merely feasible. And the recommendation engine is required to be auditable, which is served by a model whose feature importances are recorded in its metadata and can be inspected directly.

The fixed configurations. Each candidate is a pipeline whose first stage is median imputation, so that missing values are handled identically across the four, and each carries a fixed random seed so that a training run can be reproduced. The configurations themselves are documented constants rather than the result of a search, listed in Table 6.4.

Πίνακας 6.4: Fixed configurations of the four points-model candidates.

Candidate	Configuration
Random forest	300 trees, maximum depth 12, at least 5 samples per leaf

Candidate	Configuration
Gradient boosting	250 trees, learning rate 0.04, maximum depth 3
Histogram-based gradient boosting	300 iterations, learning rate 0.04, at most 31 leaves per tree
XGBoost	400 trees, maximum depth 5, learning rate 0.05, row and column subsampling 0.8, minimum child weight 3, L_1 penalty 0.1, L_2 penalty 1.0

Fixing them is a deliberate choice rather than an omission. An exhaustive search would multiply the training time of a pipeline that already runs unattended every gameweek, and it would also undermine the rollback gate of Chapter 6.3, which compares the validation error of the current run against the previous one: if the search space were explored differently on each run, a change in error could not be attributed to the new data rather than to a different chosen configuration. A systematic hyperparameter search is noted as future work. Each candidate is fitted on the same eligible fitting subset, and the one with the lowest validation mean absolute error is retained.

Mean absolute error

$$\mathrm{MAE} = \frac{1}{N} \sum_{j=1}^{N} |y_j - \hat{y}_j|. \tag{6.3}$$

Here N is the number of validation examples, y_j the observed return, and $\hat{y}_j$ its prediction. Absolute differences prevent overestimates and underestimates from cancelling. Predictions $4, 5, 3$ against outcomes $6, 4, 3$ give $(2 + 1 + 0)/3 = 1.0$ point MAE. Lower values indicate smaller average absolute error.

Points-model selection evaluates only examples with at least thirty target minutes, matching the fitting population. MAE is used to compare the candidate models after training. Each model first learns from the training data using its own loss function, which measures the errors it tries to reduce during fitting. This loss may be squared error rather than absolute error. The fitted models then predict the validation data, and their MAE values are calculated against the observed outcomes. The candidate with the lowest validation MAE is selected. Thus, the error measure used to select the final model can differ from the one used to train it.

Combining points and participation

The system combines conditional points with the availability classifier' s probability:

$$\hat{p}_{i,g+1} = \hat{\pi}_{i,g+1}\hat{q}_{i,g+1}. \tag{6.4}$$

A conditional forecast of five points and probability 0.80 give four risk-adjusted points. This calculation estimates expected points by considering only appearances of at least thirty minutes. It leaves out the points a player might earn when playing less than thirty minutes, such as points from a brief substitute appearance. Mathematically, the missing contribution is

$$P(m < 30 \mid \mathbf{x})\, \mathbb{E}[p \mid m < 30, \mathbf{x}],$$

which multiplies the probability of playing fewer than thirty minutes by the expected points in those cases. Because the system does not include this term, its result is an approximation of the player' s overall expected points.

Single-gameweek outcomes depend on events not fully represented by the inputs. Performance on unusually high returns and usefulness for ranking must be evaluated rather than assumed. The implementation also clips negative predictions to zero although negative returns are possible. Chapter 8 addresses measured error and baseline comparisons, while Chapter 6.3 explains persistence and projection.

6.1.4 Availability Prediction Model

The availability classifier estimates the probability of completing at least thirty minutes in the upcoming gameweek:

$$\hat{\pi}_{i,g+1} \approx P(m_{i,g+1} \geq 30 \mid \mathbf{x}_{i,g}). \tag{6.5}$$

This is different from predicting whether he will start: a substitute may play more than thirty minutes, while a starter may leave the pitch before that threshold. It also does not predict whether he will make any appearance or complete sixty minutes.

The classifier uses the same thirty features as the points and minutes models. Recent playing-time averages describe how much the player has played, while the status-based availability input provides information about his current availability. The model combines these with the other player, team, and fixture features to estimate the probability of reaching thirty minutes.

Candidate algorithms

- **Logistic Regression**
- **Random Forest Classifier**
- **Gradient Boosting Classifier**
- **XGBoost Classifier**

Logistic regression combines the input features using learned weights and converts the result into a probability between zero and one. More precisely, it assumes that the log-odds of playing at least thirty minutes are a linear combination of the processed features.

The tree-based classifiers can capture more complex relationships. For example, the effect of recent playing time may depend on whether the player is currently marked as available or doubtful. Trees represent such interactions through sequences of decisions based on feature values.

Each candidate is evaluated on the same validation data, and the classifier with the highest F_1 score is selected.

Precision, recall, and the F_1 score

For evaluation, probabilities are converted to binary labels using threshold 0.5:

$$\hat{y}_j^{\text{play}} = \mathbb{1}[\hat{\pi}_j \geq 0.5].$$

- **True positives (TP)**: predicted and observed participation both reach thirty minutes.
- **False positives (FP)**: predicted participation reaches the threshold but observed participation does not.
- **False negatives (FN)**: predicted participation is below the threshold but observed participation reaches it.

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}. \tag{6.6}$$

- Precision measures how often positive predictions are correct.
- Recall measures how many observed positives are identified.

Their harmonic mean is

$$F_1 = 2\frac{\text{Precision}\,\text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP + FP + FN}. \tag{6.7}$$

With $TP = 40$, $FP = 10$, and $FN = 5$, precision is 0.80, recall is approximately 0.889, and $F_1 = 80/95 \approx 0.842$. A higher F_1 score means the classifier achieves a better balance between precision, how

often its predictions of at least thirty minutes are correct, and recall, how many players who actually reach thirty minutes it identifies. The score is calculated after converting probabilities into labels using the chosen threshold.

Correct predictions that a player will play fewer than thirty minutes are true negatives. These do not enter the F_1 formula directly, because the score focuses on identifying the positive class.

Probability interpretation

Thresholding is used for evaluation, not to replace the probability in the points calculation. Two classifiers can give identical labels while assigning different probabilities and therefore different risk-adjusted points.

Calibration describes how closely predicted probabilities match what happens in practice. For example, among many cases where the model predicts an 80 per cent chance of playing at least thirty minutes, approximately 80 per cent should reach that threshold. A high F_1 score does not guarantee this: F_1 evaluates the binary predictions obtained at a chosen threshold, rather than whether the probability values themselves are accurate.

6.1.5 Minutes Prediction Model

The minutes regressor estimates playing time rather than the probability of crossing one threshold:

$$\widehat{m}_{i,g+1} \approx \mathbb{E}[m_{i,g+1} \mid \mathbf{x}_{i,g}]. \tag{6.8}$$

Unlike the points model, the minutes model is trained on all eligible examples, including gameweeks in which a player did not play. It therefore learns from zero-minute outcomes as well as brief and full appearances.

Its output estimates expected playing time, rather than the most likely exact duration. For example, if a player had an equal chance of playing zero or ninety minutes, the expected value would be forty-five minutes, even if playing exactly forty-five minutes was unlikely.

Candidate algorithms

- **Random Forest Regressor**
- **Gradient Boosting Regressor**
- **XGBoost Regressor**

Candidates share the fitting and validation subsets. Selection uses MAE as defined in (6.3), measured here in minutes. For example, an MAE of twelve minutes would mean that the model' s predictions differ from the actual playing time by twelve minutes on average, counting both overestimates and underestimates as positive errors. This is an illustrative example, not a measured result.

During inference, predictions below zero are set to zero, and predictions above ninety are set to ninety. This keeps the values passed to the recommendation engine within a fixed range.

Inference clips the output to $[0, 90]$. This bounds downstream inputs but cannot represent more than ninety total minutes across a double gameweek.

Heuristic sixty-minute estimate

Sixty minutes is a scoring boundary in the game rather than an arbitrary modelling choice. A player who appears for fewer than sixty minutes receives one appearance point, while one who reaches sixty receives two, and the clean-sheet points that dominate defensive returns are awarded only to goalkeepers and defenders who complete sixty minutes without conceding. Crossing the threshold therefore changes a player' s return directly, and for defensive positions by considerably more than the additional appearance point alone. It is a different question from the thirty-minute participation event of Chapter 6.1.4, which marks a meaningful appearance rather than a scoring boundary, so the engine tracks both.

No separate sixty-minute classifier is trained. The minutes model already estimates playing time, so a monotone transformation of that estimate reuses the information available rather than requiring a fourth model to be trained, validated, and kept current. The implementation transforms clipped minutes through

$$h_{60}(\widehat{m}) = \begin{cases} \widehat{m}/120, & 0 \le \widehat{m} < 60, \\ 0.5 + (\widehat{m} - 60)/60, & 60 \le \widehat{m} \le 90. \end{cases} \tag{6.9}$$

Πίνακας 6.5: Illustrative outputs of the sixty-minute mapping.

Predicted minutes	Heuristic value
0	0.00
30	0.25
60	0.50
72	0.70

Predicted minutes	Heuristic value
90	1.00

The notation h_{60} emphasises its heuristic nature, although the implementation stores it as `prob_60_plus_mins`. A value 0.70 is not an empirically calibrated seventy-per-cent probability.

Predicted minutes help assign participation-risk labels, while the derived sixty-minute value is used as a multiplier in recommendation scoring. The minutes prediction itself is not multiplied into (6.4), which combines conditional points with the probability of playing at least thirty minutes.

Some recommendation formulas apply further availability adjustments to favour players with more reliable participation. These adjustments change the ranking score; they do not produce a new estimate of expected fantasy points. Chapter 6.2.2 explains how each decision type uses them.

Any claim about which features influence the minutes model most, or which playing-time outcomes are hardest to predict, should be supported by the evaluation results in Chapter 8.

6.1.6 Worked Prediction Example

This subsection follows a single prediction through the pipeline, from the stored inputs to the record the recommendation engine reads. Unlike an illustrative walkthrough, every figure reported here is an actual value produced by the system: the run was executed on 6 September 2026, after gameweek 3 of the 2026–27 season had been played, and it targets gameweek 4, whose deadline was 12 September 2026. The models were fitted on the three completed gameweeks of the current season together with the archived 2025–26 season, and the run generated predictions for 510 eligible players, of which 155 received a HIGH confidence label, 89 MEDIUM, and 266 LOW.

The player used here is **Riccardo Calafiori**, an Arsenal defender priced at £5.6 million and owned by 42.8 per cent of managers at the time of the run. He is a useful case because his three completed gameweeks were not uniform: nine points from eighty minutes in gameweek 1, eleven points from ninety minutes in gameweek 2, and two points from sixty-six minutes in gameweek 3.

Inputs supplied to the models

The inference stage does not re-read the full match history when a prediction is requested. It constructs one feature vector per eligible player and applies the saved models to it. Table 6.6 reports a subset of the thirty features actually supplied for this prediction; the remaining features follow the same construction described in Chapter 6.1.1 and Chapter 6.1.2.

Πίνακας 6.6: Selected features from the thirty actually supplied to the models for this prediction.

Feature	Value	Interpretation
`form`	10.0	Official FPL form figure
`pts_last_1gw`	2.0	Points in gameweek 3
`pts_last_3gw_avg`	7.33	Mean points over gameweeks 1–3
`pts_last_5gw_avg`	4.40	Five-gameweek mean, topped up from the archive
`minutes_last_3gw_avg`	78.67	Mean minutes over gameweeks 1–3
`minutes_last_5gw_avg`	47.20	Five-gameweek mean, topped up from the archive
`chance_of_playing`	100.0	Availability from the official status code
`next_gw_fdr`	3.0	Difficulty of the gameweek 4 fixture
`avg_fdr_next3`	2.67	Mean difficulty of the next three fixtures
`is_home_next_gw`	0.0	The gameweek 4 fixture is away
`season_total_pts`	22.0	Points accumulated before gameweek 4
`team_gf_l5`	1.80	Club goals scored per match over the last five
`team_ga_l5`	0.40	Club goals conceded per match over the last five
`team_cs_rate_l5`	0.60	Club clean-sheet rate over the last five

The two five-gameweek features are worth noting, because they show a real consequence of the season-boundary handling described in Chapter 6.3. Only three gameweeks of the current season exist, so the five-match windows are topped up from the archived season, and in this case the two archived records contributed no minutes. The three-gameweek average of 78.67 minutes therefore describes the player' s current role, while the five-gameweek average of 47.20 minutes is depressed by history that no longer reflects it. Early in a season the longer windows are the less informative of the two, a limitation that resolves as the season accumulates.

Model outputs

The same vector is supplied to all three models. Table 6.7 reports what they returned, at the precision at which the pipeline stores them.

Πίνακας 6.7: Outputs of the three supervised models for the example prediction.

Model	Output	Meaning
Points regressor	6.66	Points conditional on playing at least thirty minutes
Availability classifier	0.9634	Estimated probability of playing at least thirty minutes
Minutes regressor	82.26	Expected playing time

The three answer different questions and are not interchangeable. The points model reports what the player is worth if he takes a meaningful part in the match; the classifier reports how likely that is; the minutes model estimates how long he is expected to be on the field. Only the first two are combined.

Risk-adjusted points

Applying (6.4) to the unrounded model outputs gives

$$\hat{p}_4 = \hat{\pi}_4 \, \hat{q}_4 = 0.963407 \times 6.656574 = 6.412989,$$

which the pipeline stores as 6.41. The conditional forecast of 6.66 points is therefore reduced to 6.41 once the possibility of the player not reaching thirty minutes is priced in. For a player as securely selected as this one the discount is small, at roughly a quarter of a point; the same arithmetic applied to a rotation risk with a participation probability of 0.60 would remove two-fifths of the forecast.

Derived participation estimate

The predicted 82.26 minutes falls in the upper branch of (6.9):

$$h_{60}(82.26) = 0.5 + \frac{82.26 - 60}{30} \times 0.5 = 0.870936,$$

stored as 0.8709. As explained in Chapter 6.1.5, this figure is assigned by an explicit mapping rather than estimated by a calibrated classifier, and it should not be read as a 87 per cent probability of completing sixty minutes.

Horizon projections

The three- and five-gameweek figures are constructed deterministically from the risk-adjusted one-gameweek estimate, following (6.15). The mean difficulty of the next three fixtures, 2.67, maps to the same multiplier as the immediate fixture, so the fixture term is unity here:

$$\hat{p}_{3\mathrm{GW}} = 3 \times 6.41 \times 1.00 = 19.23, \qquad \hat{p}_{5\mathrm{GW}} = 5 \times 6.41 \times 0.95 = 30.45.$$

Both are totals across their horizons rather than per-gameweek averages, which is why their magnitudes are several times larger than the one-gameweek figure.

The stored record

Confidence is assigned by the rules of Table 6.18 rather than by a fourth model. This player cleared every threshold, giving a HIGH label. The validated record written to `player_predictions` is reproduced in Table 6.8.

Πίνακας 6.8: The prediction record stored for the example player and made available to the recommendation engine.

Stored quantity	Value
Conditional points estimate	6.66
Risk-adjusted one-gameweek points	6.41
Three-gameweek projection	19.23
Five-gameweek projection	30.45
Probability of playing at least thirty minutes	0.9634
Predicted minutes	82.26
Heuristic sixty-minute value	0.8709
Confidence label	HIGH

The record is written with the producing model' s version and a generation timestamp, as described in Chapter 6.3. The recommendation engine reads these values and applies the manager' s personalisation weights, the decision-specific scoring rules of Chapter 6.2.2, and any relevant constraints. A risk-adjusted estimate of 6.41 points does not by itself determine whether this player should be captained,

transferred in, or selected for an initial squad.

The run itself

The pipeline reports each stage as it executes. The excerpt below is taken verbatim from the run that produced the figures above, with timestamps and log levels retained:

```
Stage 3/5: train models (points + availability + minutes)
  gradient_boosting_points   | MAE=2.3625 | RMSE=3.1552 | R2=-0.0057
  hist_gradient_boosting_pts | MAE=2.3714 | RMSE=3.2151 | R2=-0.0443
  xgboost_points             | MAE=2.4545 | RMSE=3.2654 | R2=-0.0772
  Best points model: gradient_boosting_points
  xgboost_played             | F1=0.6484 | ACC=0.7668 | ROC_AUC=0.8088
  xgboost_minutes            | MAE=22.7508 mins | bucket_acc=0.6321
  Model gate: previous MAE=2.3576 | new MAE=2.3625 -> New model accepted.
Stage 4/5: predict GW4
  Rolling windows topped up from last season (3 live gameweeks, 29747 archived rows)
  Feature matrix: 510 players x 30 features
  Inference complete - mean xPts (expected): 1.69 | mean xPts if starts: 3.19
  Generated 510 predictions for GW4 | HIGH: 155 | MEDIUM: 89 | LOW: 266
Stage 5/5: store GW4 predictions in database
  Validation passed: 510 rows | GW4 | HIGH=155 | MEDIUM=89 | LOW=266
  Successfully upserted 510 predictions for GW4.
```

The run shows two effects of the pipeline. First, the rollback gate described in Chapter 6.3 compared the new points model's validation MAE of 2.3625 with the previous model's 2.3576. The increase was approximately 0.21 per cent, well below the 25 per cent rejection threshold, so the new model was accepted. This means it passed the safeguard, not that it outperformed the previous model.

Second, the average risk-adjusted prediction across the 510 players was 1.69 points, compared with an average conditional prediction of 3.19 points. The difference results from weighting each player's conditional forecast by his estimated probability of playing at least thirty minutes. Players with lower participation probabilities receive larger reductions, lowering the average across the player pool.

Figure 6.2 summarises the example. The feature vector enters the three models, whose outputs are combined and transformed into the values written to the stored record, from which the recommendation engine reads.

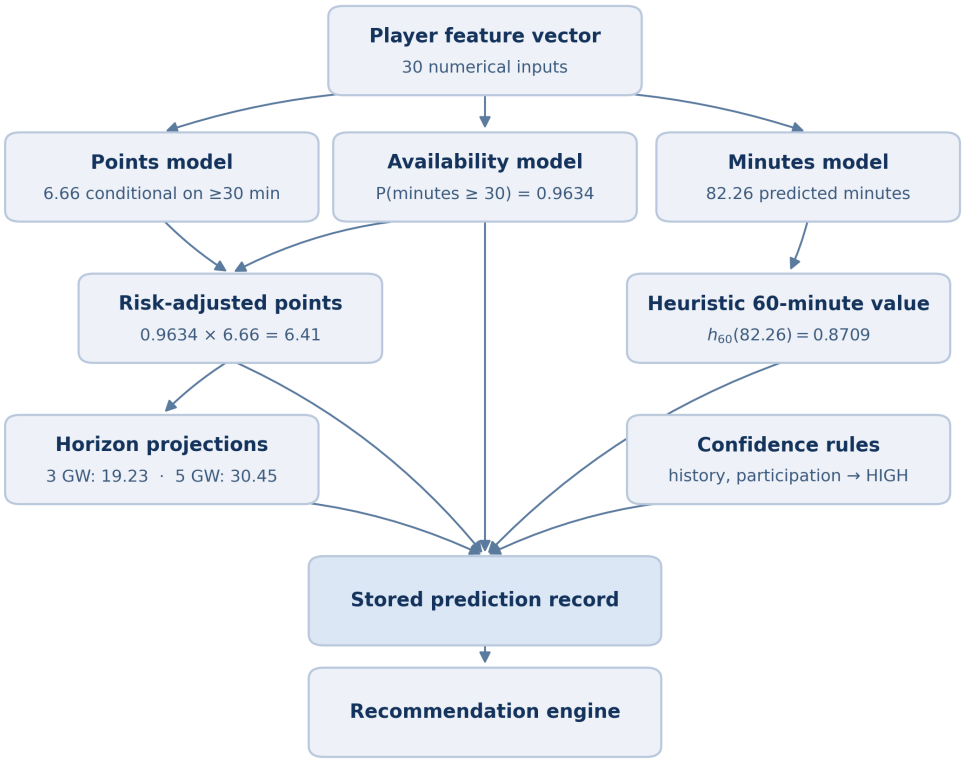


Figure 6.2. The worked example end to end, with the values produced by the gameweek 4 run. The feature vector enters the three supervised models; the points and availability outputs combine into the risk-adjusted estimate, which in turn yields the horizon projections, while the minutes output is mapped to the heuristic sixty-minute value. All of these, with the confidence label, form the stored prediction record.

6.2 Recommendation and Scoring System Design

The recommendation engine uses player predictions to provide personalised advice on captaincy, transfers, player comparisons, and squad construction. The three models predict points conditional on playing at least thirty minutes, the probability of reaching that threshold, and expected minutes. Further calculations produce the longer-horizon projections, the heuristic sixty-minute estimate, and the confidence label. These additional values come from formulas and rules, rather than separate trained models.

The scoring functions produce the same scores when given the same inputs and configuration. TypeScript types specify the data exchanged between services, and stored recommendation outputs allow the results to be inspected. Reproducing a recommendation also requires the original context,

including the predictions, manager preferences, candidate ordering, and constraints used at the time.

All managers share the same predictive models. Personalisation is applied after prediction: each manager' s preferences influence how players are scored and ranked, as well as how the advice is explained.

6.2.1 Personalisation Profile

An onboarding questionnaire collects ten fields: risk tolerance, budget style, preferred positional focus, differential preference, planning horizon, explanation depth, Big Six affinity, captaincy style, transfer frequency, and fixture awareness. Each one has three coded responses that represent question-specific choices rather than a common measurement scale: for example defence, midfield, and attack.

The manager' s profile affects both the recommendations and how they are explained. Preferences such as planning horizon and risk tolerance are converted into numerical weights used to score players. The explanation-depth setting tells the language model whether to give a brief answer or a more detailed analysis. Other profile information is also included in the prompt to help the response reflect the manager' s preferences.

Πίνακας 6.9: Principal weekly weight mappings. Stored codes need not match the display order of questionnaire options.

Field	Stored values 1 / 2 / 3	Coefficients
Planning horizon	Short / balanced / long	$(.70, .25, .05)$ / $(.55, .30, .15)$ / $(.30, .40, .30)$
Risk tolerance	Cautious / balanced / aggressive	$\alpha = 1.5/1.0/0.7$
Captaincy style	Safe / balanced / bold	$\kappa = 1.4/1.0/0.7$
Fixture awareness	Lower / moderate / higher emphasis	$0.5/1.0/1.5$
Differential preference	Avoid / occasional / strong	$0.00/0.05/0.20$
Big Six affinity	Avoid / neutral / prefer	$0.85/1.00/1.15$

The scoring weights are fixed values chosen during implementation, they are not learned from user behaviour, and experiments have not established them as optimal. The horizon weights combine the one, three, and five gameweek projections into a score used to rank players. Because these projections cover different periods, the resulting score is not a prediction of points for a single gameweek.

Risk tolerance controls how strongly uncertain participation reduces a player's score. For an availability factor of $a = 0.60$, the three settings give

$$0.60^{1.5} \approx 0.465, \qquad 0.60^{1.0} = 0.600, \qquad 0.60^{0.7} \approx 0.699.$$

With all other inputs unchanged, the cautious setting applies the largest reduction, while the aggressive setting applies the smallest. This changes the recommendation score, not the probability produced by the availability classifier.

Fixture awareness controls the strength of the team-and-fixture adjustment. Differential preference adds a bonus that depends on ownership, with lower-ownership players receiving a larger addition. Big Six preference applies to the fixed group of Arsenal, Chelsea, Liverpool, Manchester City, Manchester United, and Tottenham; it does not refer to the current top six in the league table. These preferences can change player rankings, although the same player may remain the strongest candidate under several profiles.

Different preferences affect different services. Captaincy style changes the risk adjustment used for captain selection; budget style influences the squad optimiser' s preference for premium-priced players. A defensive focus enables an additional multiplier of 1.10 for Big Six defenders where that adjustment is applied. Explanation depth affects the detail of the response rather than the numerical score. Although the code also defines `hitTolerance` and `defenderStyleBias`, the inspected transfer-scoring and weekly-ranking paths do not apply them, respectively.

Initial squad construction uses a separate weight mapping implemented in Python. For a long-term profile, it assigns $(w_1, w_5) = (0.10, 0.90)$, while weekly scoring uses $(w_1, w_3, w_5) = (0.30, 0.40, 0.30)$. Before combining its two projections, the optimiser multiplies the one-gameweek estimate by five to make their scales comparable. The mappings differ because they support different decisions, but changes to either implementation should preserve the intended meaning of the manager' s preferences.

Personalisation therefore follows the manager' s questionnaire answers through predefined rules. The current system does not learn how to adjust those rules from the manager' s subsequent behaviour.

6.2.2 Hybrid (Heuristic + ML) Scoring

The system produces recommendations through two tracks: the heuristic track uses explicit rules based on FPL statistics and context, while the ML-based track uses stored player predictions as inputs to scoring formulas. In both cases, application code calculates the recommendation scores. The machine-learning models predict player outcomes, they do not directly learn which recommendation to give.

For supported requests, both tracks can run at the same time. When both return results, a precedence rule determines which takes priority. ML-based results are primary for decisions such as captaincy,

transfers, and player comparison. Heuristic results take priority for other decisions that depend mainly on rules and constraints. Some requests use only one source, including heuristic fallback when predictions are unavailable and the ML-only initial-squad path.

The system does not average the two tracks' scores. It keeps their outputs separate and tells the language model which source should guide the answer. For example, if the tracks recommend different captains, the available ML-based recommendation takes priority, while the heuristic result remains supporting information. The selected priority is recorded for later inspection. This provides a clear rule for handling disagreement, although it does not guarantee that the generated explanation will follow that rule correctly.

Shared ranking

The shared ranking function combines predicted performance with participation estimates, confidence, team context, and manager preferences.

1. Combine the prediction horizons

For player i, the horizon component is

$$H_i = w_1 x_{1,i} + w_3 x_{3,i} + w_5 x_{5,i},$$

where:

- $x_{1,i}$ is the risk-adjusted points estimate for the next gameweek.
- $x_{3,i}$ and $x_{5,i}$ are projected totals over three and five gameweeks.
- w_1, w_3, w_5 are the weights determined by the manager' s planning horizon.

Each projection is multiplied by its weight, and the contributions are added. Short-term profiles place more weight on the immediate forecast, while long-term profiles give greater importance to the extended projections.

For predictions of 4, 12, and 19 points and balanced weights,

$$H_i = 0.55(4) + 0.30(12) + 0.15(19) = 2.20 + 3.60 + 2.85 = 8.65.$$

Although the weights sum to one, the projections cover different periods. Therefore, H_i is a ranking component rather than an expected return for a single gameweek.

2. Apply participation, risk tolerance, and confidence

The base score is

$$B_i = H_i \left[\max(0.15, P_{\text{play},i}) \max(0.20, h_{60,i})\right]^{\alpha} c_i. \tag{6.10}$$

Here, $P_{\text{play},i}$ is the estimated probability of playing at least thirty minutes, and $h_{60,i}$ is the heuristic sixty-minute value derived from predicted minutes.

The `max` operations impose minimum values for scoring. A participation probability of 0.10, for example, is treated as 0.15 in this formula, while 0.80 remains unchanged. The sixty-minute factor has a corresponding minimum of 0.20. These floors prevent either factor from reducing the base score to zero by itself; they do not change the stored model outputs.

The exponent α **represents the manager' s risk tolerance through a predefined mapping**. It is not the questionnaire response itself, but the numerical coefficient derived from that response:

Πίνακας 6.10: Mapping risk tolerance to the availability exponent in shared ranking.

Risk tolerance	α	Effect
Cautious	1.5	Stronger reduction for uncertain participation
Balanced	1.0	Uses the combined availability factor unchanged
Aggressive	0.7	Smaller reduction for uncertain participation

Because the combined availability factor lies between zero and one, a larger exponent makes it smaller. A cautious manager therefore receives a stronger penalty for participation uncertainty, while an aggressive manager accepts more of that risk.

The confidence multiplier is

$$c_i = \begin{cases} 1.05, & \text{HIGH confidence,} \\ 1.00, & \text{MEDIUM confidence,} \\ 0.82, & \text{LOW confidence.} \end{cases}$$

HIGH confidence increases the score by 5%, MEDIUM leaves it unchanged, and LOW reduces it by 18%. These are fixed scoring adjustments, not confidence probabilities.

Continuing the example, suppose $P_{\text{play},i} = 0.80$, $h_{60,i} = 0.70$, and confidence is HIGH. The combined availability factor is

$$0.80 \times 0.70 = 0.56.$$

Its contribution changes with risk tolerance:

$$0.56^{1.5} \approx 0.419, \qquad 0.56^{1.0} = 0.560, \qquad 0.56^{0.7} \approx 0.666.$$

For the balanced profile, $\alpha = 1.0$, so

$$B_i = 8.65(0.56)^1(1.05) = 5.0862.$$

A cautious profile would receive a lower base score for the same player, while an aggressive profile would receive a higher one. This changes the recommendation score, not the classifier' s participation probability.

The points projections already include participation risk. The additional availability adjustment therefore expresses a preference for reliable participation rather than producing an exact expected-points calculation.

3. Apply team context and Big Six preference

The final score is

$$S_i = B_i t_i b_i + d_i,$$

where:

- t_i is the team-and-fixture context multiplier.
- b_i is the Big Six preference multiplier, including the additional defender adjustment where applicable.
- d_i is the ownership-dependent differential bonus.

A multiplier of 1.0 leaves the score unchanged. Values above one increase it, while values below one reduce it. The differential bonus is added after these multiplications.

For example, with $t_i = 1.10$ and $b_i = 1.00$,

$$B_i t_i b_i = 5.0862(1.10)(1.00) = 5.59482.$$

The context adjustment increases the base score by 10%, while the neutral Big Six multiplier has no effect.

4. Calculate the differential bonus

When ownership information is available, the bonus is

$$d_i = \delta \left(1 - \frac{\text{clip}(o_i, 0, 100)}{100}\right) \max(2, B_i t_i b_i).$$

where o_i is ownership percentage and δ is the manager' s differential-preference coefficient.

First, `clip` restricts ownership to 0–100. Dividing by 100 converts it to a proportion, and subtracting from one gives a larger value to less-owned players. The coefficient δ controls the strength of the preference. Finally, $\max(2, B_i t_i b_i)$ scales the addition using the adjusted score, with a minimum scaling value of two.

For a player owned by 10% of managers,

$$1 - \frac{10}{100} = 0.90.$$

With $\delta = 0.05$ and an adjusted score of 5.59482,

$$d_i = 0.05(0.90) \max(2, 5.59482) = 0.2517669.$$

The final score becomes

$$S_i = 5.59482 + 0.2517669 = 5.8465869,$$

which the implementation rounds to **5.847**.

The bonus is zero when differential preference is disabled or the required ownership context is unavailable. Because its scaling term has a minimum of two, it can remain positive even when the base score is zero.

The final value orders candidates under the same scoring configuration. **It is a personalised ranking score, not a prediction of fantasy points**.

Captaincy

Captaincy recommendations focus on the upcoming gameweek. The engine ranks eligible squad players using their one-gameweek points projection, sustained-participation estimate, confidence label, context, and captaincy risk adjustment:

$$S_i^{\text{captain}} = x_{1,i} \max(0.15, h_{60,i}) c_i^{\text{captain}} t_i r_i. \tag{6.11}$$

The terms have the following meanings:

- $x_{1,i}$ is the player' s stored, participation-adjusted points projection for the next gameweek.
- $h_{60,i}$ is the heuristic estimate of reaching sixty minutes.
- c_i^{captain} adjusts the score according to prediction confidence.
- t_i accounts for the contextual adjustment used by the service.
- r_i adjusts the score according to the player' s risk category and the manager' s captaincy preference.

There is no additional direct multiplication by $P_{\text{play},i}$, because this probability already contributes to $x_{1,i}$. The sixty-minute factor expresses a further preference for sustained participation. Its floor of 0.15 prevents this scoring factor from becoming smaller than that value; it does not turn the heuristic into a calibrated probability.

The captaincy confidence multiplier is 1.08 for HIGH confidence, 1.00 for MEDIUM confidence, and 0.75 for LOW confidence. Holding the other terms constant, these values increase the score by eight per cent, leave it unchanged, or reduce it by twenty-five per cent, respectively.

The engine next assigns a risk category using confidence, the probability of playing at least thirty minutes, and expected minutes. The conditions are checked in order, so a player who satisfies a HIGH-risk condition cannot subsequently be assigned MEDIUM risk.

Πίνακας 6.11: Captaincy risk classification and adjustments.

Risk	Condition, checked in this order	Adjustment r_i
HIGH	LOW confidence, or $P_{\text{play},i} < 0.60$, or expected minutes below 45	$\max(0.40, 0.75/\kappa)$
MEDIUM	If not HIGH: MEDIUM confidence, or $P_{\text{play},i} < 0.80$, or expected minutes below 65	$\max(0.70, 0.95/\max(1, \kappa))$
LOW	Neither preceding category	1.00

The parameter κ represents **captaincy style**, whereas α in shared ranking represents general **risk tolerance**. Safe, balanced, and bold captaincy settings use $\kappa = 1.4$, 1.0, and 0.7, respectively.

For a HIGH-risk player, these settings give:

$$r_{\text{safe}} = \max(0.40, 0.75/1.4) \approx 0.536,$$
$$r_{\text{balanced}} = \max(0.40, 0.75/1.0) = 0.750,$$
$$r_{\text{bold}} = \max(0.40, 0.75/0.7) \approx 1.071.$$

The safe setting reduces the score by approximately 46.4%, while the balanced setting reduces it by 25%. The bold setting instead gives a small boost of approximately 7.1%. Thus, this term is an adjustment rather than a penalty in every case. None of these three settings reaches the 0.40 floor.

For an illustrative player with $x_1 = 4$, $h_{60} = 0.70$, MEDIUM confidence, $P_{\text{play}} = 0.75$, and expected minutes of 72, the risk category is MEDIUM. With neutral context and balanced captaincy style, the risk adjustment is:

$$r = \max(0.70, 0.95/\max(1, 1)) = 0.95.$$

The resulting captaincy score is:

$$S^{\text{captain}} = 4 \times 0.70 \times 1.00 \times 1.00 \times 0.95 = 2.66.$$

This is a ranking score, not a forecast of the points earned after applying the captain's scoring multiplier.

Captaincy risk thresholds differ from those used in shared ranking. In shared ranking, HIGH risk includes participation probability below 0.55 or expected minutes below thirty, while MEDIUM risk includes probability below 0.75 or expected minutes below sixty, alongside the corresponding confidence conditions. Captaincy therefore uses stricter participation thresholds for assigning these risk categories.

Transfers

The transfer system evaluates whether replacing a player already in the squad with another player would improve the manager's options. Let i denote the outgoing player and j the proposed replacement. The calculation begins by comparing their projected points over one, three, and five gameweeks:

$$D_{i \to j} = \sum_{h \in \{1,3,5\}} w_h (x_{h,j} - x_{h,i}). \tag{6.12}$$

Here, $x_{h,j}$ and $x_{h,i}$ are the incoming and outgoing players' projections over horizon h, and w_h is the corresponding weight from the manager' s planning preferences. Written in full, this becomes:

$$D_{i\to j} = w_1(x_{1,j} - x_{1,i}) + w_3(x_{3,j} - x_{3,i}) + w_5(x_{5,j} - x_{5,i}).$$

Each difference measures the projected improvement over one horizon. A positive value of $D_{i\to j}$ means that the incoming player has the higher weighted projection; a negative value favours keeping the existing player. Since the calculation combines totals covering different periods, $D_{i\to j}$ is a ranking quantity rather than an estimate of additional points over one specific period.

The engine then adjusts this difference using the incoming player' s confidence label and participation probability:

$$B^{\text{tr}}_{i\to j} = D_{i\to j} c^{\text{tr}}_j \left[\max(0.20, P_{\text{play},j})\right]^{\alpha}.$$

The confidence multiplier c^{tr}_j is 1.05, 1.00, or 0.75 for HIGH, MEDIUM, or LOW confidence respectively. The term $P_{\text{play},j}$ is the classifier' s estimated probability that the incoming player will play at least thirty minutes. The floor of 0.20 prevents this factor from falling below that value before the exponent is applied; it does not change the stored probability.

The exponent α comes from the manager' s **risk tolerance**, using 1.5 for cautious, 1.0 for balanced, and 0.7 for more aggressive preferences. For a positive projection difference, a larger exponent reduces the base score more strongly when participation is uncertain. The projections already include a participation adjustment, so this additional factor expresses the manager' s preference for reliability.

For a negative projection difference, multiplying by a factor below one moves the score closer to zero. Therefore, the adjustment does not make every unfavourable transfer more negative; its effect depends on the sign of the initial difference.

The final transfer score also incorporates context and personalisation:

$$S^{\text{tr}}_{i\to j} = B^{\text{tr}}_{i\to j} \frac{t_j}{\max(0.70, t_i)} b_j + d_{i\to j}.$$

The ratio $t_j / \max(0.70, t_i)$ compares the incoming player' s context multiplier with the outgoing player' s. A ratio above one increases the magnitude of the base score, while a ratio below one reduces it. The denominator floor prevents a very small outgoing context multiplier from producing an excessively large ratio.

The multiplier b_j applies the manager' s Big Six preference to the incoming player. The final term, $d_{i\to j}$, adds an ownership-based adjustment when the necessary context is available:

$$d_{i\to j} = \delta \left(1 - \frac{\text{clip}(o_j, 0, 100)}{100}\right) \max\left(2, \left|B^{\text{tr}}_{i\to j}\right|\right).$$

Here, o_j is the incoming player' s ownership percentage and δ is the manager' s differential-preference coefficient. Lower ownership produces a larger addition when this preference is enabled. The absolute base score sets the scale of the addition, with a minimum scale of two. The addition is zero when the required ownership context is unavailable or the preference is disabled.

For example, suppose the outgoing player has projections of 4, 12, and 19, while the incoming player has projections of 5, 15, and 24. With balanced horizon weights:

$$D_{i \to j} = 0.55(5 - 4) + 0.30(15 - 12) + 0.15(24 - 19) = 2.20.$$

If the incoming player has HIGH confidence, a participation probability of 0.80, and the manager has balanced risk tolerance, the base score becomes:

$$B^{\text{tr}}_{i \to j} = 2.20 \times 1.05 \times 0.80 = 1.848.$$

With neutral context and Big Six multipliers and no differential addition, the final score is also 1.848. This illustrative value measures the attractiveness of the replacement under the scoring rules; it does not mean that the transfer is expected to earn exactly 1.848 additional fantasy points.

The formula does not subtract the points cost of a paid transfer. Although the profile defines a `hitTolerance` value, this value is not applied in the transfer-scoring path described here. A positive transfer score therefore does not establish that the expected benefit outweighs a transfer hit.

Eligibility and feasibility

Before a player can be recommended, the relevant service applies eligibility rules. These combine recent playing time, the official FPL chance-of-playing field, player status, and the availability of a recent record.

Πίνακας 6.12: Eligibility thresholds over the service' s selected three-gameweek window.

Mode	Minimum recent minutes	Minimum official chance	Accepted status codes	Recent record
STRICT	90	50%	a	Required
MODERATE	30	25%	a, d	Required
PERMISSIVE	0	0%	a, d, i, s, u	Not required

The minutes threshold refers to accumulated minutes within the selected window. The status codes represent available (a), doubtful (d), injured (i), suspended (s), and unavailable (u) players.

The official chance-of-playing field is supplied by FPL and is separate from P_{play}, which is learned by the availability classifier and refers specifically to playing at least thirty minutes. These two inputs serve different purposes: the official field supports eligibility filtering, while the learned probability also enters the scoring calculations.

Incoming candidates for ML-based transfer recommendations must satisfy STRICT eligibility. Player comparison uses MODERATE eligibility so that doubtful players can still be assessed. When a manager asks about a specific transfer, the service first uses permissive lookup to identify the named players, then checks the incoming player against the strict rules before endorsing the move. This allows the system to discuss a named player even when that player would be excluded from an automatic recommendation.

Transfer evaluation also checks affordability, position compatibility, and club composition. These checks establish whether the proposed replacement satisfies the relevant squad constraints. The transfer score answers a separate question: how attractive the replacement is according to the projections and the manager' s preferences. A high score alone does not establish that a move is feasible, and the checks performed depend on the service handling the request.

Decision-specific procedures

The recommendation system uses different procedures for different decisions. Captaincy and transfer services use their respective scoring formulas, while squad construction uses constrained optimisation and other services apply rules suited to the requested decision.

Πίνακας 6.13: Decision-specific recommendation procedures.

Decision	Main procedure
Captaincy	Rank squad players using the captaincy score
Transfer recommendation	Search for and score same-position replacements
Named transfer evaluation	Identify players, check eligibility and feasibility, and compare the proposed replacement
Initial squad	Select a squad using integer linear programming
Starting eleven and bench	Apply player scores alongside squad and formation rules
Formation	Evaluate valid formations

Decision	Main procedure
Chip timing	Assess squad and fixture conditions using explicit rules
Player comparison	Produce a side-by-side assessment

Depending on the service, the structured output includes scores, supporting reasons, risk categories, and component breakdowns. The language model uses these results to explain the recommendation to the manager. The application code performs the numerical calculations, and the available explanation fields vary between services.

6.2.3 Squad Optimisation

Selecting an initial squad requires more than ranking players individually. Choosing the fifteen highest-ranked players could exceed the available budget, include too many players from one club, or fail to meet the required positional composition. The optimiser therefore evaluates combinations of players and selects the combination with the highest total personalised value that satisfies the squad constraints.

The application formulates this problem as **binary integer linear programming**. It uses **PuLP**, a Python library for defining optimisation problems, to express the objective and constraints. **CBC** is the solver that searches for a feasible combination and optimises its total value. Each candidate' s value is calculated before the optimisation begins, using predictions and the manager' s preferences.

Decision variables and squad constraints

For each candidate player i, the optimiser defines a binary decision variable:

$$z_i = \begin{cases} 1, & \text{if player } i \text{ is selected,} \\ 0, & \text{otherwise.} \end{cases}$$

Let I be the complete candidate set, I_p the candidates in position p, and I_k the candidates belonging to club k. The optimisation problem is:

$$
\begin{aligned}
\max_{\mathbf{z}} \quad & \sum_{i \in I} v_i z_i && \text{total value of the squad} \\
\text{subject to} \quad & \sum_{i \in I} z_i = 15, && \text{squad size} \\
& \sum_{i \in I_p} z_i = q_p \quad \forall p, && \text{positional composition} \\
& \sum_{i \in I} c_i z_i \leq B, && \text{budget} \\
& \sum_{i \in I_k} z_i \leq 3 \quad \forall k, && \text{club limit} \\
& z_i \in \{0, 1\} \quad \forall i \in I. && \text{binary selection}
\end{aligned}
\tag{6.13}
$$

The objective adds the personalised values v_i of the selected players. Multiplication by z_i includes a player's value when selected and contributes zero otherwise.

The constraints define a valid squad:

- **Squad size**: exactly fifteen players must be selected.
- **Positional composition**: the squad must contain two goalkeepers, five defenders, five midfielders, and three forwards. These counts are represented by q_p.
- **Budget**: the sum of selected players' prices c_i must not exceed the available budget B.
- **Club limit**: no more than three selected players may belong to the same club.
- **Binary selection**: each player is either selected or excluded.

Prices are represented in tenths of a million pounds. A player costing £6.0 million therefore has $c_i = 60$, and the default budget $B = 1000$ represents £100 million. A supplied budget parameter can replace this default.

The optimiser also supports requested inclusions and exclusions by setting $z_i = 1$ or $z_i = 0$. These constraints are added only when the requested player exists in the filtered candidate set. Consequently, a requested inclusion that has already been removed during filtering does not receive a constraint. Requested inclusions that remain in the pool must also be compatible with the budget, position, and club limits.

Candidate preparation

Before constructing the optimisation problem, the application retrieves available or doubtful players with a stored five-gameweek projection. It selects each player's latest prediction independently. This means the query does not, by itself, guarantee that every candidate's prediction refers to the same gameweek.

The application then calculates each candidate's personalised objective value and removes candidates whose value is zero or negative. If fewer than fifteen candidates remain, the service immediately reports that a squad cannot be formed. Having at least fifteen candidates is necessary but does not guarantee feasibility: the remaining players may still fail to provide the required positional composition within the budget and club limits.

Calculating each player's value

The optimiser begins with conditional points estimates, allowing it to apply its own participation adjustment according to the manager's risk tolerance.

For the one-gameweek estimate, $q_{1,i}$, it first uses the stored `predicted_pts_if_starts` field. Despite the field name, this prediction represents expected points conditional on playing **at least thirty minutes**, rather than specifically starting the match. If the field is missing, the optimiser reconstructs a conditional estimate by dividing the stored participation-adjusted projection by a guarded participation probability.

The five-gameweek conditional estimate is reconstructed similarly:

$$q_{5,i} = \frac{x_{5,i}}{\max(0.05, \widetilde{P}_i)},$$

where $x_{5,i}$ is the stored five-gameweek projection and $\widetilde{P}_i$ is the participation probability used by the optimiser. When the probability is missing, the implementation substitutes 0.50. The divisor floor of 0.05 prevents division by zero or an extremely small value. Because of these safeguards and the processing already applied to stored projections, this reconstruction is not always an exact reversal of the original participation adjustment.

The initial player value is:

$$A_i = \left(5q_{1,i}w_1 + q_{5,i}w_5\right) \left[\max(0.20, \widetilde{P}_i)\right]^{\alpha} c_i^{\text{conf}}. \tag{6.14}$$

The first term combines short-term and longer-term projections. Multiplying $q_{1,i}$ by five places it on a scale comparable to the five-gameweek total before the weights are applied. It is a scaling step; it does not separately predict each of the next five fixtures.

The planning-horizon weights are:

Πίνακας 6.14: Planning-horizon weights used by the squad optimiser.

Planning preference	w_1	w_5
Short	0.40	0.60
Balanced	0.20	0.80
Long	0.10	0.90

These weights differ from the weekly recommendation engine because initial squad construction uses a separate mapping with greater emphasis on the longer projection.

The participation term adjusts the value for the likelihood of playing at least thirty minutes. The exponent α represents the manager' s risk tolerance: cautious, balanced, and aggressive settings use 1.5, 1.0, and 0.7, respectively. For probabilities below one, a larger exponent applies a stronger reduction. The floor of 0.20 limits this reduction before exponentiation.

The confidence multiplier c_i^{conf} is 1.05 for HIGH confidence, 1.00 for MEDIUM confidence, and 0.82 for LOW confidence. The superscript distinguishes this multiplier from the price c_i used in the budget constraint.

The final personalised value is:

$$v_i = A_i b_i u_i + d_i.$$

Here, b_i applies the manager' s Big Six preference and, where relevant, the additional defender adjustment. The factor u_i represents price preference, while d_i adds a differential bonus based on ownership.

Price preference is calculated as:

$$u_i = \left(\frac{c_i}{60}\right)^{\beta-1}.$$

The denominator 60 represents a fixed reference price of £6.0 million. It is not the average price of the candidate pool. The parameter β takes the following values:

Πίνακας 6.15: Price-preference settings used by the squad optimiser.

Budget preference	β	Effect
Value	0.92	Favours cheaper players relative to the reference price
Balanced	1.00	Leaves the value unchanged
Premium	1.10	Favours more expensive players relative to the reference price

For balanced preferences, the exponent is zero, so $u_i = 1$. Under the other settings, the adjustment changes a player' s attractiveness, but the total squad must still satisfy the same budget constraint.

The ownership-based addition is:

$$d_i = \delta \max\left(0, \frac{100 - o_i}{100}\right) \max(2, A_i),$$

where o_i is ownership percentage and δ controls the strength of differential preference. Lower ownership produces a larger addition when $\delta > 0$. The term $\max(2, A_i)$ scales this addition with the player' s initial value while imposing a minimum scale of two.

For a supplied profile, the three differential settings use $\delta = 0$, 0.15, and 0.50. Without a profile, the default is 0.05. These coefficients belong to the squad optimiser and differ from those used in weekly ranking.

As an illustrative calculation, consider a player with conditional projections $q_{1,i} = 5$ and $q_{5,i} = 24$, a participation probability of 0.80, and HIGH confidence. With balanced horizon weights and risk tolerance:

$$\begin{aligned} A_i &= (5 \times 5 \times 0.20 + 24 \times 0.80) \times 0.80 \times 1.05 \\ &= 24.20 \times 0.80 \times 1.05 \\ &= 20.328. \end{aligned}$$

With neutral Big Six and price multipliers and differential preference disabled, $v_i = 20.328$. This value contributes to the optimisation objective if the player is selected. It is a personalised ranking value rather than an exact expected-points total.

Although these calculations contain powers and ratios, they are completed before the solver runs. The solver receives fixed values v_i and prices c_i, multiplied by binary selection variables. The optimisation

problem therefore remains linear.

The solver considers the total value of a feasible squad rather than selecting players independently. It may exclude a highly valued, expensive player if the remaining budget would force several weaker selections. A combination containing a slightly lower-valued player can therefore produce a higher total squad value.

6.3 Implementation of the Prediction System

The prediction pipeline is implemented in Python under `apps/ai`, with `pipeline.py` coordinating its stages and the backend starts the pipeline as a child process. Historical records and completed predictions are exchanged through PostgreSQL, while datasets, trained models, and metadata are also saved as local files.

Training and inference are separate steps: training fits the candidate models and selects one for each task, while inference loads the selected models and uses them to generate predictions. The pipeline can run both steps together or generate predictions using the models already saved on disk.

Πίνακας 6.16: Libraries supporting the prediction pipeline.

Library	Responsibility
pandas	Loading datasets, joining tables, grouping records, and transforming features
NumPy	Numerical calculations and prediction post-processing
scikit-learn	Preprocessing, candidate estimators, and evaluation metrics
XGBoost	Additional regression and classification candidates
joblib	Saving and loading trained models
SQLAlchemy / psycopg	Connecting to PostgreSQL and exchanging data
PyArrow	Reading and writing Parquet files through pandas

Data preparation and chronological split

The data preparation stage joins gameweek statistics with player information and fixtures, then combines the current season with archived seasons. Records are ordered by season and gameweek, and duplicates are removed using the player, season, and gameweek identifiers. The combined history is saved in `player_gw_history.parquet`.

Feature construction keeps the recorded points and minutes available as outcomes and attaches the targets from each player's next chronological observation. Rows without a known points target are excluded from training. The minimum-history filter counts a player's records across the assembled dataset; it does not separately count the observations preceding each individual row.

The latest five eligible feature-row rounds are reserved for validation, and older examples are used for fitting. When fewer rounds are available, the validation period is reduced. For complete consecutive records through GW11 of 2026–27, the latest labelled feature row is GW10 because its target is the observed outcome in GW11. The allocation before predicting GW12 is shown in Table 6.17.

Πίνακας 6.17: Chronological allocation, assuming complete records.

Purpose	Illustrative allocation before GW12
Raw records	Archived 2025–26 and current-season GW1–11
Fitting	Eligible older examples through feature row GW5
Validation	Feature rows GW6–10, targeting GW7–11
Inference	Newly constructed inputs for the GW12 prediction

Thus, both seasons contribute to the assembled data, but the most recent labelled rounds are held out for model selection. The selected estimator is not fitted again on the combined training and validation sets. For the points task, both subsets are further restricted to examples in which the player recorded at least thirty minutes in the target gameweek.

Figure 6.3 shows the candidate comparison in the points training script. Each candidate is fitted on `x_train` and evaluated on `x_val`. The script calculates validation metrics and retains the model with the lowest MAE. This comparison selects the estimator; the estimator's own fitting procedure determines how its parameters are learned.

apps/ai/models/train_points_model.py

```
    for name, model in candidates.items():
        logger.info("Training %s...", name)
        model.fit(x_train, y_train)

        preds = np.clip(model.predict(x_val), 0, None)
        metrics = regression_metrics(y_val, preds)
        results[name] = metrics
    ...
        if metrics["mae"] < best_mae:
            best_mae = metrics["mae"]
            best_name = name
            best_model = model
```

Figure 6.3. Source excerpt from the points training script: fitting candidates, calculating validation metrics, and retaining the lowest-MAE model. Line numbers refer to the application source; the gap omits logging statements.

Routine production selection is separate from the rolling-origin experiments (Bergmeir and Benítez 2012). Chapter 8 needs to identify the folds, preprocessing, estimators, and evaluation population used for those experiments. Keeping validation rounds later than training rounds prevents random mixing across time, but does not by itself establish that every contextual input was historically available.

Figure 6.4. Production data flow and intermediate artifacts. Experimental rolling-origin evaluation is separate from routine candidate selection.

Saved models and the rollback check

The selected estimators are saved as `points_model.joblib`, `played_classifier.joblib`, and `minutes_regressor.joblib`. Metadata records the selected candidate, feature columns, evaluation metrics, and feature importances

where available. Reproducing a run also requires its data, code, configuration, library versions, and random seeds. The version string attached to predictions does not necessarily identify a unique training run.

Before automated retraining, the existing model files are copied to a backup directory. After training, the pipeline compares the new points model' s MAE with the previously stored MAE. It rejects the new model when:

$$\mathrm{MAE}_{\mathrm{new}} > 1.25\,\mathrm{MAE}_{\mathrm{previous}}\,.$$

The multiplier 1.25 permits a deterioration of up to twenty-five per cent. If the new MAE exceeds that threshold, available backup files are restored. This tolerance is a fixed engineering choice, and the check covers the points model rather than independently evaluating all three tasks.

Generating and processing predictions

Inference loads the saved models and constructs inputs from recent player history, current-season aggregates, player information, and upcoming fixtures. Negative points predictions are clipped to zero, and predicted minutes are clipped to the interval $[0, 90]$.

The stored record distinguishes the conditional points estimate from the participation-adjusted projection. It also contains the participation probability, expected minutes, longer-horizon projections, heuristic sixty-minute estimate, and confidence label. Figure 6.5 shows how the model outputs are obtained and combined. The points product is explained in (6.4), and Chapter 6.1.6 follows an actual player through the calculation.

apps/ai/inference/predict_next_gw.py

```
626     pred_pts_if_starts = np.clip(points_model.predict(X), 0, None)
627
628     if hasattr(played_model, "predict_proba"):
629         prob_playing = played_model.predict_proba(X)[:, 1]
630     elif hasattr(played_model, "decision_function"):
631         scores       = played_model.decision_function(X)
632         prob_playing = 1 / (1 + np.exp(-scores))
633     else:
634         prob_playing = played_model.predict(X).astype(float)
635
636     pred_minutes = np.clip(minutes_model.predict(X), 0, 90)
637     prob_60_plus = np.where(
638         pred_minutes >= 60,
639         np.clip((pred_minutes - 60) / 30 * 0.5 + 0.5, 0.5, 1.0),
640         np.clip(pred_minutes / 60 * 0.5, 0.0, 0.5),
641     )
        ...
647     pred_pts_gw1 = prob_playing * pred_pts_if_starts
```

Figure 6.5. Source excerpts from inference: obtaining the three model outputs, deriving the sixty-minute estimate, and applying the participation adjustment. The gap in line numbering indicates omitted comments.

The longer-horizon projections are calculated as:

$$\begin{aligned} x_3 &= 3x_1 \max\left(0.5, \frac{M(\text{round}(\overline{\text{FDR}}_3))}{M(\text{round}(\text{FDR}_1))}\right), \\ x_5 &= 5x_1(0.95). \end{aligned} \tag{6.15}$$

For the three-gameweek projection, the one-gameweek value is multiplied by three and adjusted for the relative difficulty of the upcoming fixtures. The average difficulty covers fixtures in the target round and the following two rounds. Difficulty values are rounded before applying the lookup: $M(1) = 1.25$, $M(2) = 1.12$, $M(3) = 1.00$, $M(4) = 0.88$, and $M(5) = 0.75$. The floor of 0.5 limits the reduction caused by the ratio.

The five-gameweek projection multiplies the one-gameweek value by five and applies a fixed five-percent discount. This discount is a heuristic, not a measured uncertainty estimate. Both outputs represent totals across their respective horizons. The prompts identify them as totals to help the language model

distinguish them from a one-gameweek forecast.

Season starts and confidence labels

At the start of a season, inference can use models carried over from previous training and supplement a short recent history with archived observations. Current-season aggregates remain separate. Predictions generated provisionally receive a `provisional_` prefix in their model version, allowing the scheduler to distinguish them from outputs produced after normal retraining. Heuristic recommendation paths remain available when suitable predictions are missing.

Blank and double gameweeks need care when interpreting the outputs. A neutral fixture default does not automatically set a blank-gameweek projection to zero. The longer-horizon formulas do not explicitly add separate predictions for each match in a double gameweek, and the ninety-minute limit prevents the minutes output from representing more than one full match.

Confidence labels are assigned after inference using the ordered rules in Table 6.18.

Πίνακας 6.18: Confidence labels assigned after prediction.

Confidence	Ordered rule
LOW	Fewer than 3 history records, or participation probability below 50%
MEDIUM	If not LOW: probability below 75%, recent mean minutes below 20, or fewer than 8 history records
HIGH	None of the preceding conditions

The history count refers to observations supplied to inference, which may differ from the player's complete season history. These labels summarise participation and data availability; they are not statistical confidence intervals. Inference primarily selects available and doubtful players, while recommendation services apply their own eligibility checks later.

Checking and storing the results

Predictions are first written to `predictions_gwN.parquet`. Before export, the pipeline checks required fields and allowed values, including probability ranges, minutes, and confidence labels. These checks detect invalid records; they do not measure prediction accuracy.

Database writes use the player, gameweek, and model version as the upsert key. Writing an existing key updates that record. Predictions from other model versions for the same target gameweek are removed, so the table does not retain a complete history of training runs.

Recommendation services read these stored results when a manager sends a request. The manager therefore receives advice based on precomputed forecasts without waiting for the models to be trained.

6.4 Implementation of the Recommendation System

The recommendation system is implemented in the TypeScript assistant module. It retrieves the manager' s context and stored predictions, runs the relevant decision services, and passes their results to the language model for explanation. Request handling, numerical scoring, and response generation are handled by separate components.

Πίνακας 6.19: Recommendation implementation components.

Component	Responsibility
Controller	Validate requests and obtain the authenticated identity
Intent service	Check scope and identify the requested decision using keywords
Context service	Retrieve manager information and shared game data
Orchestrator	Call decision services and determine which source takes precedence
Prompt builder	Format context, recommendations, and response instructions
LLM adapter	Call the configured language-model provider
Memory service	Manage sessions, messages, and conversation history
Recommendation log	Store structured results and source-precedence information

Prisma provides database access, Zod validates incoming requests, and the Anthropic and OpenAI

SDKs support language-model calls. The scoring formulas described in Chapter 6.2 are evaluated by application code before the language model produces the response. These application-specific scores combine projections and preferences using fixed engineering rules. They should be distinguished from standard methods such as MAE, the F1 score, and integer linear programming. The chosen scoring coefficients are not presented as values derived from a paper or as empirically optimal settings.

Request validation, ownership, and scope

Incoming messages must contain non-empty text and must not exceed 2,000 characters. Before an existing conversation is used, the service checks that its session belongs to the authenticated user. It then stores the user' s message. Squad and profile information belong to individual managers, while player and fixture records are shared. Some later operations rely on the session ownership check already performed rather than repeating it in every query.

The service next checks whether the message concerns FPL. It first looks for recognised terms. If none are found, it searches for player mentions, allowing a short request containing only player names to pass. When both checks fail, the application stores and returns a predefined refusal without running the recommendation services or calling the language model. Keyword matching can still accept an unrelated message or reject a relevant one.

For accepted messages, the intent service assigns weights to English and Greek keywords and selects the highest-scoring intent. If no intent matches, it uses general help. A confidence helper is defined separately but is not used by this request path. Selecting one dominant intent also means that a message containing several requests does not necessarily invoke a separate service for each one.

Context, recommendation services, and prompts

The context service gathers the available profile, squad, budget, transfer, and performance information together with fixtures and other shared data. The orchestrator calls the relevant heuristic and ML-based services, keeps their structured outputs, and prepares a summary stating which source should guide the answer. Initial squad construction also launches the Python optimiser described in Chapter 6.2.3.

Figure 6.6 shows the connection between these steps. The request service retrieves history, detects intent, builds user context, and obtains the recommendation before constructing the system prompt. The prompt builder adds instructions for the selected decision and the expected response format.

apps/api/src/modules/assistant/assistant.service.ts

```
78    const history  = await getRecentHistory(sessionId);
79    const intent   = detectAssistantIntent(message);
80    const userCtx  = await buildUserContext(userId);
81
82    // Run heuristic + ML recommendation pipeline
83    const recommendation = await buildRecommendationContext({ userId, intent, message });
84
85    // recommendation.summary already contains both ML promptBlock and heuristic text
86    // (joined by recommendation.service.ts). Pass it as the recommendationBlock param.
87    // intent is passed separately so prompt.service adds intent-specific guidance.
88    const systemPrompt = buildSystemPrompt(
89      userCtx,
90      recommendation?.summary ?? null,
91      intent,
92    );
93
94    const formattedHistory = formatConversationHistory(
95      history.map((m) => ({ role: m.role.toLowerCase(), content: m.content })),
96    );
```

Figure 6.6. Source excerpt from request preparation: retrieving context, computing the recommendation, and building the language-model input.

Conversation history is sent as separate user and assistant messages. The language model receives this history and the contextual information as well as the recommendation summary. The provider calls in this implementation do not expose tools for querying the database or changing a squad.

The history query currently orders messages from oldest to newest and takes twenty records. Once a conversation contains more than twenty messages, this retrieves the earliest messages rather than the latest ones. It can therefore omit the current request, which is not appended separately after retrieval. Selecting the most recent messages and then restoring chronological order would address this limitation. Although a separate summary service exists, this request path does not incorporate a running conversation summary.

Provider calls and streamed replies

The LLM adapter provides a common interface for the configured Anthropic or OpenAI model. In the non-streaming path, the service waits for the complete response before returning it. In the streaming path, it forwards text fragments as they arrive and also accumulates them for storage. Both paths use asynchronous SDK calls.

Supplying computed recommendations gives the model concrete values and reasons to explain, but it

does not guarantee that every sentence will reproduce them correctly. The model can still misstate a number or overlook a constraint. Prompt instructions reduce this risk; checking generated numerical claims against the supplied records would provide an additional control.

Figure 6.7. Main non-streaming request flow, including session validation, scope refusal, recommendation assembly, generation, and subsequent logging. Streaming shares preparation but delivers fragments before final persistence.

Stored records and inspection

The recommendation log stores the user' s message, the assistant' s reply, available heuristic and ML-based results, ML availability, and the source-precedence decision. These records allow a

recommendation to be inspected, but they do not contain every input sent to the provider.

In the non-streaming path, logging occurs after response generation and assistant-message storage. A failure before that point can leave the request without a complete recommendation log. Reconstructing the exact provider input would also require the original context, conversation history, prompt, profile and prediction values at the time, and the relevant code and configuration versions. Recording a prompt-template version alone would not preserve all of this information.

Session ownership checks restrict access to user data, scope filtering handles recognised off-topic requests, and the absence of direct action tools prevents the language model from changing the squad itself. These controls do not guarantee correct narration. User messages still influence intent selection, requested constraints, and the explanation, while scoring calculations remain in the application services. Chapter 7 describes how the interface presents these results to the manager.

6.5 Conclusion

This chapter presented three supervised prediction models and the recommendation services built around their outputs. Conditional points, participation probability, and minutes are processed into stored records containing risk-adjusted points, longer projections, heuristic participation information, and confidence labels.

Weekly services apply decision-specific rules and manager preferences; initial squad construction selects a feasible combination through integer linear programming. The language model receives the resulting recommendations and context for explanation. Production model selection is distinct from experimental rolling-origin evaluation, and datasets, model artifacts, the retraining gate, and persistence each have defined operational roles.

The prediction and recommendation procedures include engineering approximations and bounded safeguards. Historical data quality, projection assumptions, and incomplete reconstruction of past requests remain relevant when interpreting their outputs. Chapter 7 presents the adaptive conversational interface, while Chapter 8 addresses the experimental evidence for the predictive models and design choices.

7 The Adaptive and Conversational User Interface

Chapters 5 and 6 described the architecture of the platform and the engine that produces its predictions and recommendations. This Chapter presents the interface through which a manager reaches them: the screens of the web client, the client- and server-side implementation behind them, and the conversational assistant embedded in the application.

7.1 Interface Overview

This Section explains the pages of the web application along with their functionalities.

7.1.1 The Welcome Sequence

The root route presents a six-panel sequence introducing the game and the platform, advanced by arrow controls with a position indicator (Figure 7.1). The panels cover the manager’ s role, the shape of the season, and the scale of the competition, and the last one links to registration. The sequence is shown on a first visit and remains available afterwards at `/welcome`.

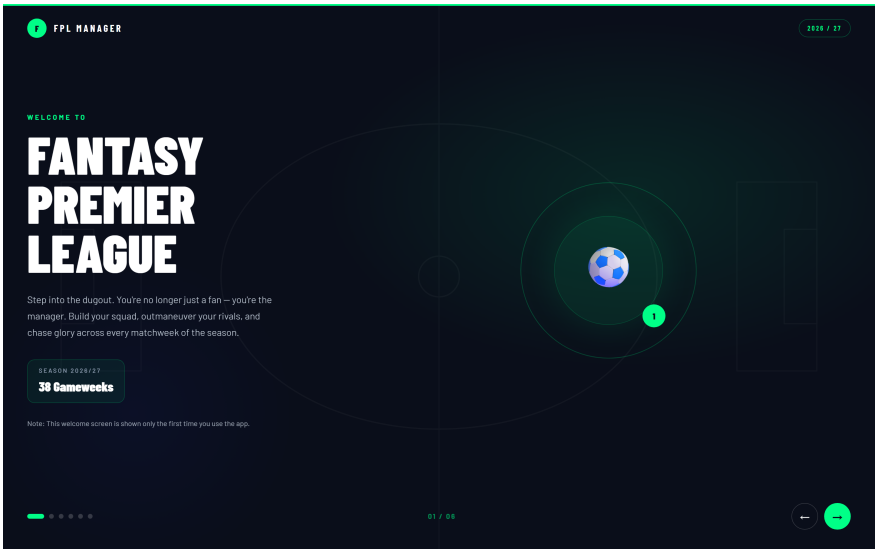


Figure 7.1. The first panel of the welcome sequence. The interface introduces the manager's role and the shape of the season before presenting any statistics.

7.1.2 Registration and Authentication

The authentication screen carries both modes in a single card with a segmented control, so signing in and registering are one interaction apart (Figure 7.2). The left half lists the platform's three principal capabilities; the right half carries the form.

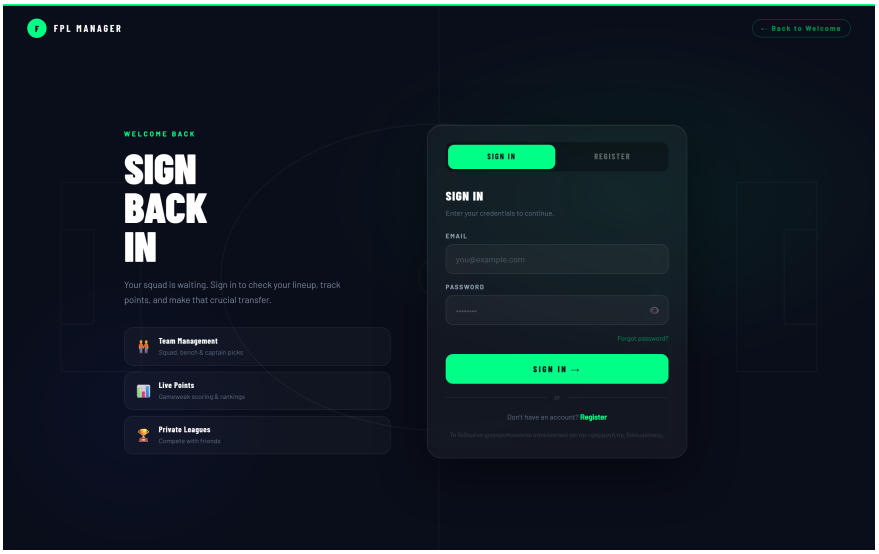


Figure 7.2. The sign-in screen. Both authentication modes share a single card, and the platform's capabilities remain visible alongside the form.

Registration requires an e-mail address, a username, and a password (Figure 7.3). Each field is validated as it is edited and again on submission, and the result appears beneath the field itself.

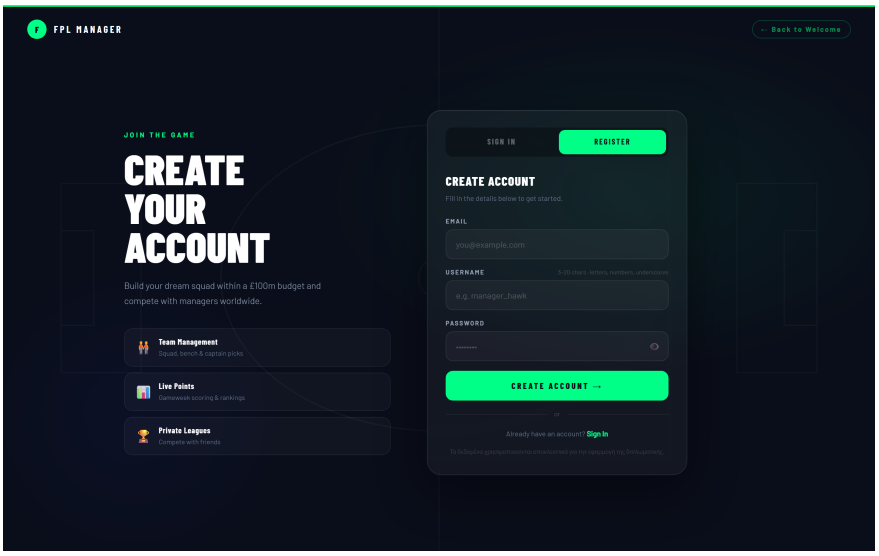


Figure 7.3. The registration form.

Figure 7.4 shows the form with a malformed e-mail address, a username that is too short and contains an illegal character, and a weak password. Offending fields are outlined and carry a specific message; the password field shows a strength meter and a live checklist of the five rules applied; and submission is blocked until the form is valid. Errors that cannot be detected in the browser, such as an e-mail address that is already registered, are returned by the API and shown in the same style. Sign-in rejections behave the same way, leaving the form populated and reporting *Invalid credentials* beneath the fields.

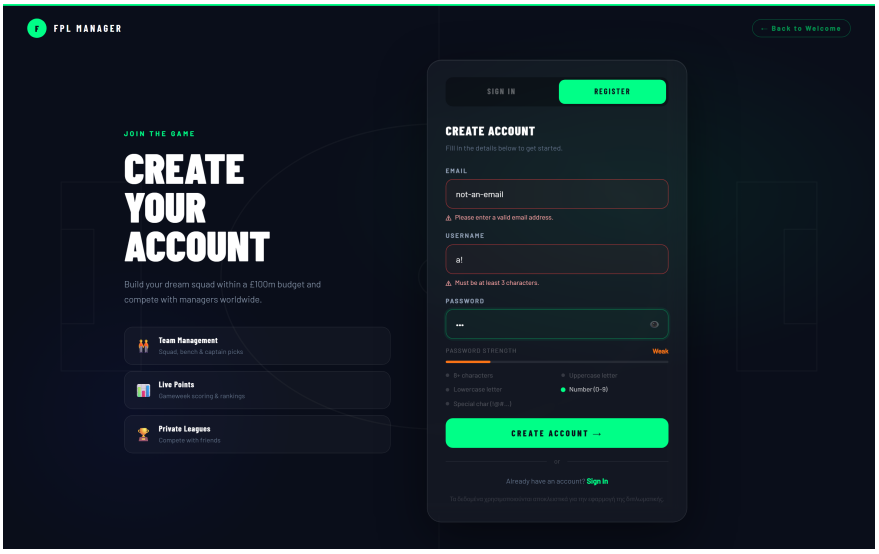


Figure 7.4. Client-side validation. Each field is validated where it is edited, with a specific message, and the password rules are shown as a live checklist rather than being revealed only on rejection.

On success the client receives the access and refresh cookies described in Chapter 5.2 and enters the authenticated shell. A manager without a squad is sent to the squad builder rather than the dashboard.

7.1.3 Personalisation Onboarding

Opening the assistant for the first time presents a ten-question guided questionnaire rather than a chat prompt, rendered inside the chat panel as single-choice cards (Figure 7.5). Each question offers three concrete strategies and states what the answer will change. There is no free-text input, so the questionnaire always terminates with a complete profile.

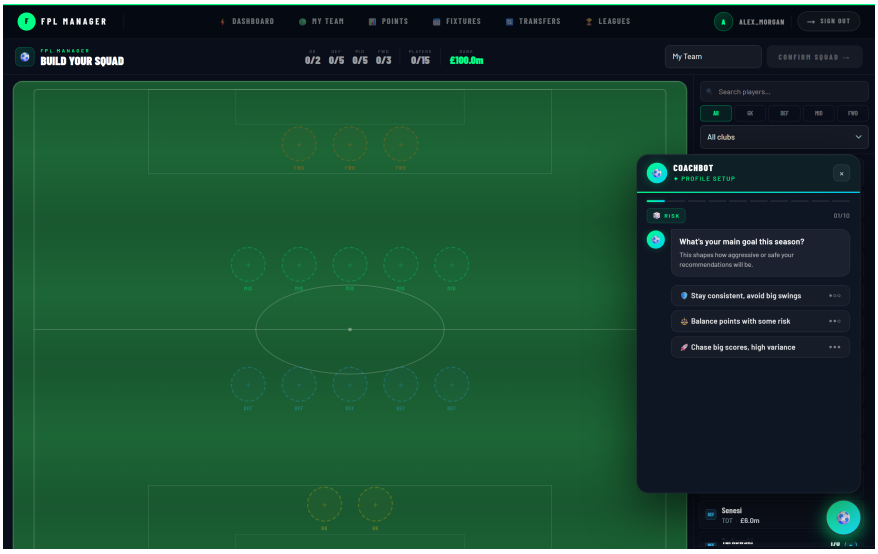


Figure 7.5. The first onboarding question. Each dimension is asked as a single choice among three concrete strategies, with a subtext explaining what the answer will change.

On completion the panel becomes the assistant proper (Figure 7.6), offering one suggested opening question per supported decision type.

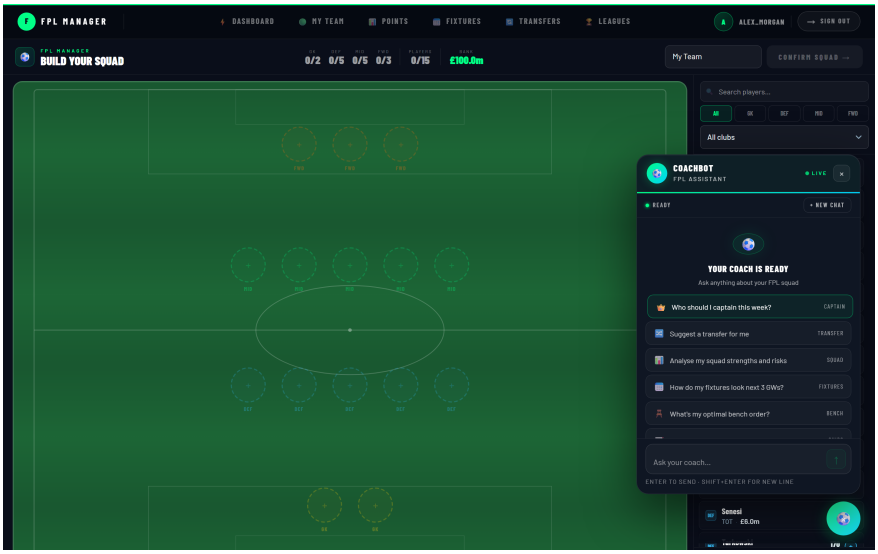


Figure 7.6. On completion the questionnaire converts into the assistant, offering one suggested opening question per supported decision type.

7.1.4 The Squad Builder

The squad builder is where a new manager selects fifteen players under the budget. The screen is dominated by a pitch on which every slot is drawn as a labelled placeholder; a header tracks the four positional quotas, the total, and the remaining bank; and the right-hand rail is a searchable player list.

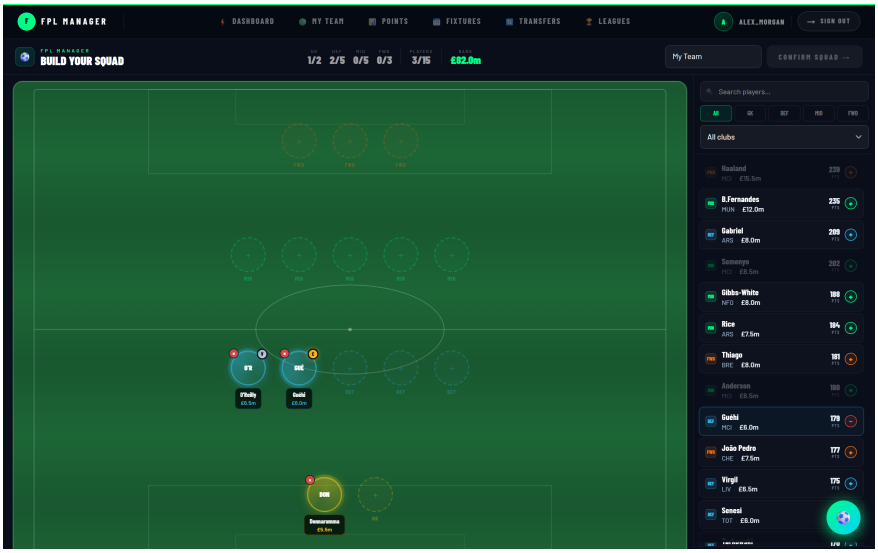


Figure 7.7. Constraints enforced preventively. With three Manchester City players selected, the remaining Manchester City entries in the rail are dimmed and unselectable rather than clickable-and-then-rejected.

The squad can also be produced by the optimiser of Chapter 6.2.3, by asking the assistant from this screen to build it (Figure 7.8).

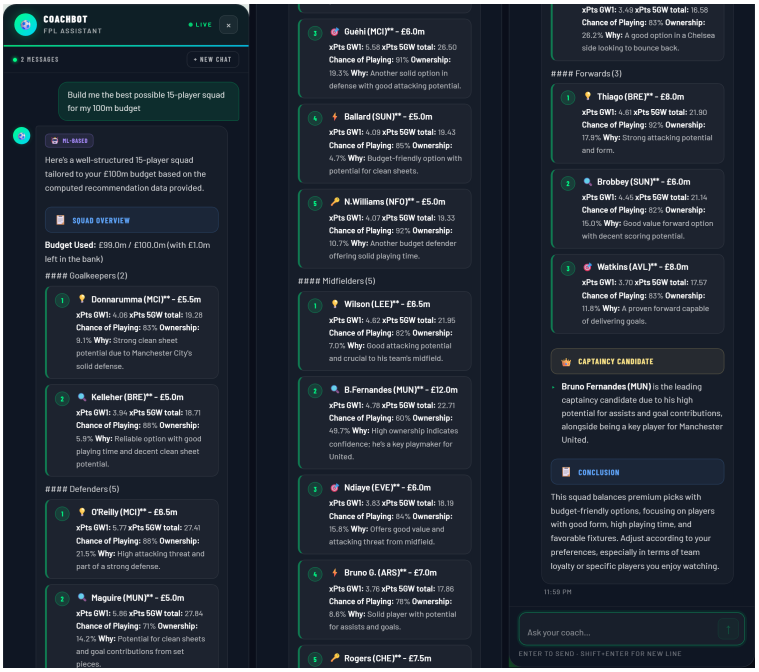


Figure 7.8. The initial squad requested conversationally from the builder screen. The fifteen players, their projections, and the budget summary are the integer program's output, narrated by the language model. The panel is captured in full and re-flowed into three columns.

7.1.5 The Dashboard

The dashboard (Figure 7.9) is the screen a returning manager sees first. A live countdown to the gameweek deadline occupies the top of the page, followed by four summary metrics, gameweek points, free transfers, bank, and season total. Beneath these sit panels for the squad, live points, leagues, and upcoming fixtures, each summarising one domain and linking to the screen that owns it.

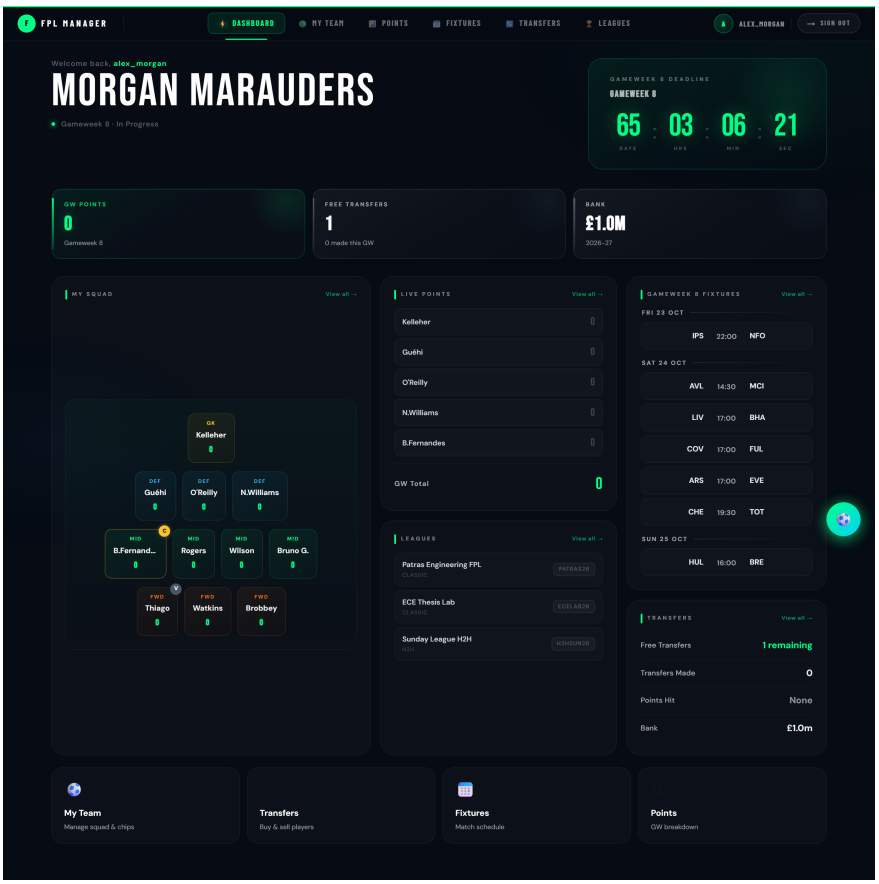


Figure 7.9. The dashboard in full. Each panel summarises one domain and links to the screen that owns it, so the dashboard indexes the application rather than duplicating it.

7.1.6 Squad Management

The team screen (Figure 7.10) carries the squad on a pitch in its current formation, the four-player bench beneath it, a formation selector, the chip panel, and a tabular squad list.

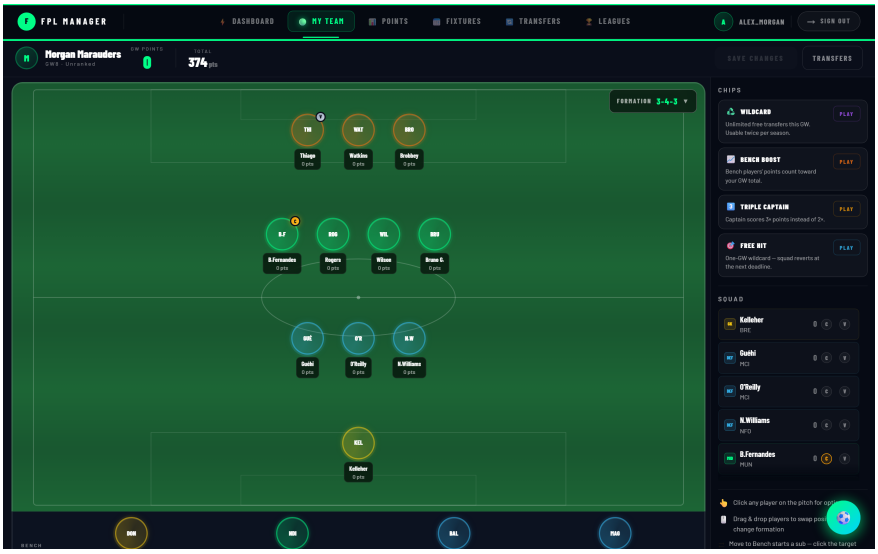


Figure 7.10. The team screen. Formation, starting eleven, bench, chips, and per-player captaincy controls are all on one screen.

7.1.7 Transfers

The transfer screen carries the current squad with sell prices on the left, the market on the right, and a cart between them. Selecting a player to sell filters the market to that player' s position and to affordable candidates (Figure 7.11).

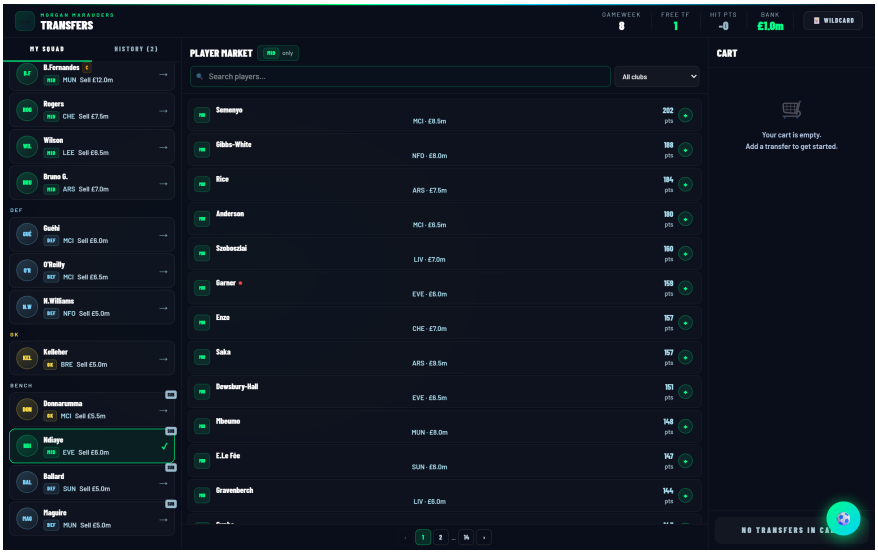


Figure 7.11. Choosing a player to sell filters the market to legal, affordable replacements for that specific player, turning a search problem into a comparison problem.

Transfers accumulate in the cart as pending pairs and are committed together, and the cart reports the bank, the free transfers remaining, and any points penalty incurred. Figure 7.12 shows an attempt to buy a player the bank cannot afford: the cart displays a blocking error naming the reason and disables the commit control.

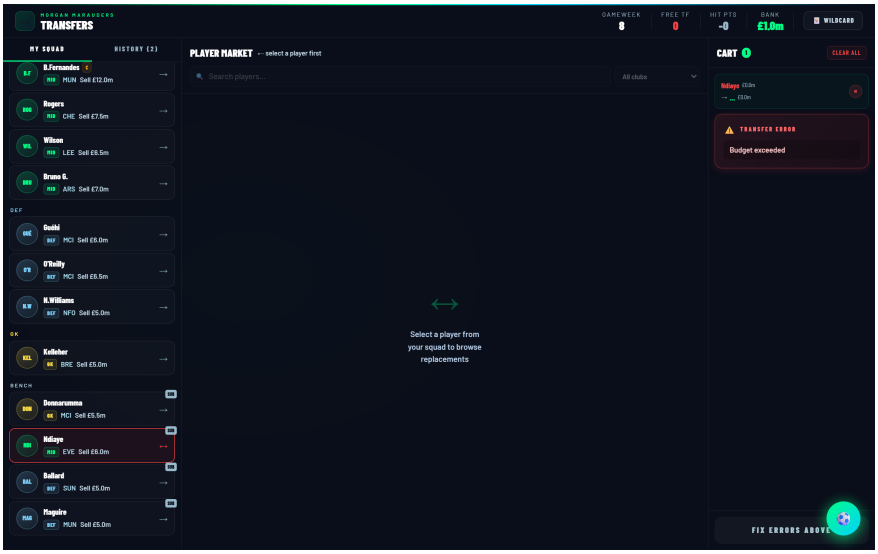


Figure 7.12. A budget violation is reported rather than prevented: the cart names the specific error and disables the commit control, but preserves the pending state so the manager can resolve it by selling elsewhere.

7.1.8 Points and Scoring History

The points screen is divided by a tab pair. The live view (Figure 7.13) presents the current gameweek as a per-player breakdown of the starting eleven and the bench with a running total, alongside the season total, the average per gameweek, and the best gameweek.

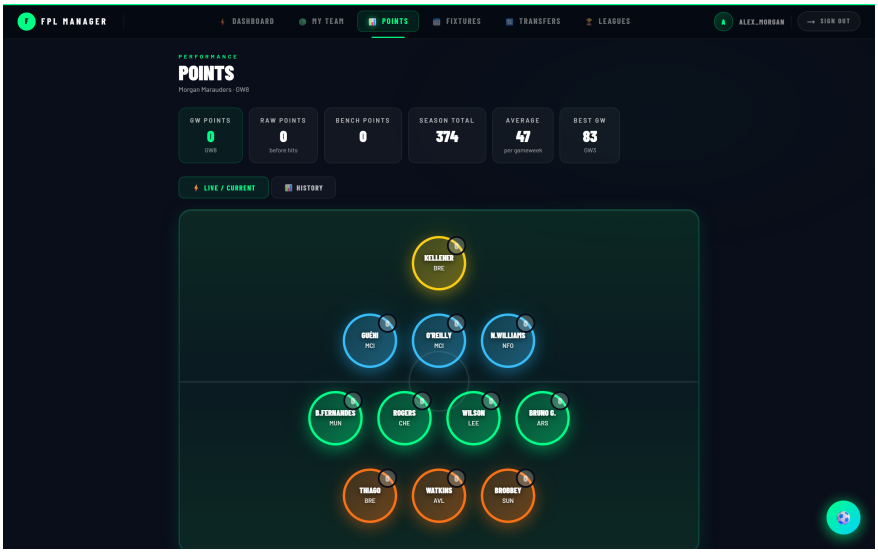


Figure 7.13. The live points view: a per-player breakdown of the current gameweek beside the season aggregates.

The history view (Figure 7.14) presents the season as a trend with a per-gameweek table recording the chip played, the points penalty, the bench points, and the final score. The per-player detail behind each completed gameweek is reconstructed from a stored breakdown, so it appears as it was scored at the time, including any multiplier and automatic substitution.

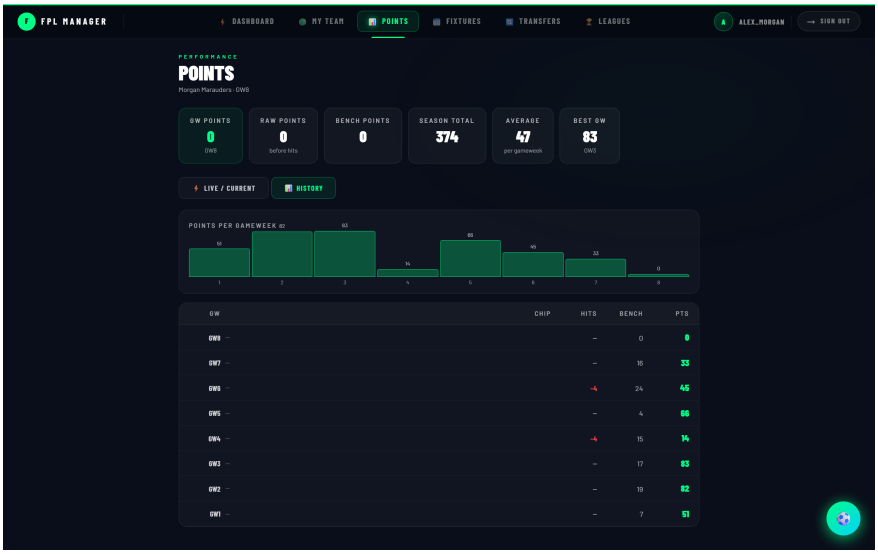


Figure 7.14. The history view: a season trend with a per-gameweek table recording chip, penalty, bench points, and final score.

7.1.9 Fixtures

The fixtures screen (Figure 7.15) lists the season by gameweek, selectable across all thirty-eight. Each match shows the two clubs, the kick-off time, and the fixture-difficulty rating, which is the same figure the prediction models consume as the `next_gw_fdr` and `avg_fdr_next3` features of Table 6.2.

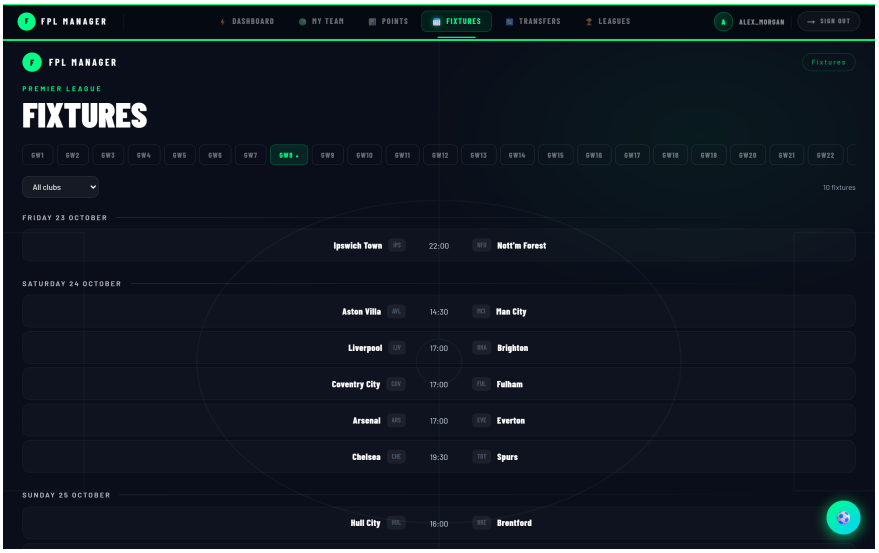


Figure 7.15. The fixtures screen, selectable by gameweek, showing the same fixture-difficulty ratings that the prediction models consume as features.

7.1.10 Leagues

The leagues screen lists a manager' s leagues with their type and invitation code, and allows a league to be created or joined from the same screen. Selecting one reveals its standings (Figure 7.16), ranked by season total with the current gameweek' s score alongside, the leading three positions marked, and the viewing manager' s row highlighted. Standings are computed by aggregating the stored per-gameweek scores of each member.

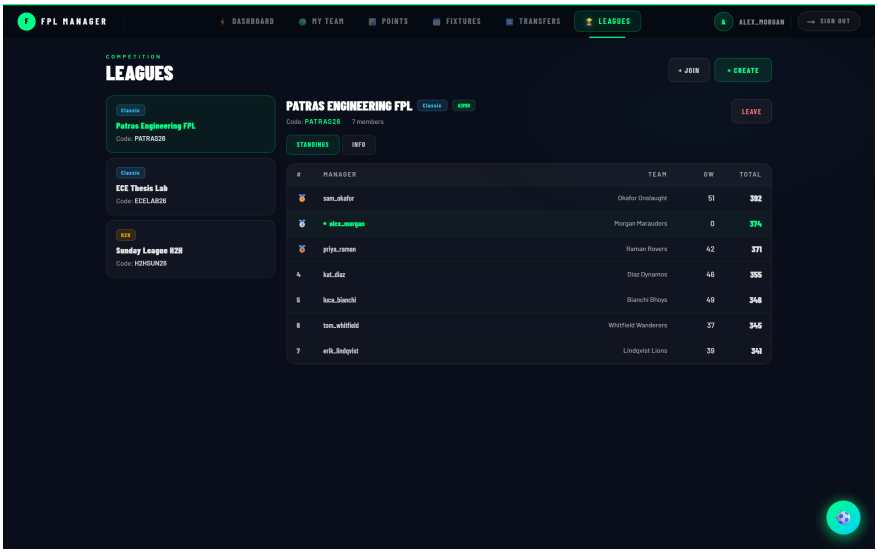


Figure 7.16. Classic league standings, ranked by season total, with the current gameweek alongside and the viewing manager's row highlighted.

Head-to-head leagues use the same screen and membership model, with per-gameweek fixtures between members stored separately. The screen currently shows aggregate standings for such a league but does not yet render the individual match-ups, which is recorded among the future work.

7.2 Implementation of the Interface

The frontend is deployed separately from the backend and communicates with the API over HTTP. Next.js handles the interface, while the backend contains the application services and scheduled tasks. As discussed in Chapter 5, these tasks need to run independently of browser activity.

A shared TypeScript package defines common data types, including players, recommendations, and chat messages. Both codebases use these definitions, allowing the compiler to detect incompatible uses during development. These checks help keep the frontend and backend consistent, although they do not validate HTTP payloads at runtime. This section explains how the client handles routing, data fetching, state, and presentation.

7.2.1 Client Structure and Data Fetching

The client code is organised into routes, shared components, and feature modules. The Next.js App Router directories define the screens and their routes. Common components provide reusable interface

elements, while feature modules contain the components and logic specific to each screen. For example, the dashboard panels and the pitch on the points screen are maintained in separate feature modules.

API requests use the `apiFetch` wrapper, which includes credentials so that the browser sends the session cookie with each request. When a request returns `401` because the access token has expired, the wrapper attempts to refresh the session and then retries the original request once. If the refresh fails, it redirects the user to the sign-in screen.

Credential-checking endpoints are excluded from this refresh behaviour. On these endpoints, a `401` can indicate incorrect sign-in details rather than an expired session. Interface testing showed that applying the same refresh policy to these responses prevented the sign-in error from reaching the interface. The exemption allows the screen to handle that error directly. Chapter 8 describes the defect and its correction.

Five custom hooks manage chat, onboarding, the transfer cart, the deadline countdown, and squad access checks. The largest, `useTransfersState`, holds the pending transfers and checks them against budget, position, and club constraints. It also recalculates the points penalty as the cart changes, so the manager can see the cost before confirming a transfer.

These interactions are managed through hooks without a global state store. Authentication credentials are held in an `httpOnly` cookie, which the browser sends with requests but client-side JavaScript cannot read. The cookie handles credential storage; the hooks manage the interface state needed for each interaction.

7.2.2 Styling and the Visual Language

The interface uses a near-black background, a bright green accent, condensed headings, and a consistent card layout across screens. These choices give the dashboard, squad, and other screens a common appearance.

Tailwind CSS 4 is installed and imported, but its main uses are the CSS reset and a small set of design tokens. Utility classes are rarely used. Three screens have their own stylesheets with named classes, while most other screen-level styling is defined through inline style objects in the components.

Inline styles keep presentation rules close to the components they affect. However, using both stylesheets and inline styles means that visual changes may need to be made in several places. The interface therefore has a consistent appearance, but its styling is implemented through more than one approach.

7.2.3 Responsive Design

The responsiveness requirement NFR1 demands a usable experience across device classes, which matters here because a manager's most time-critical interaction, a change before a deadline, is frequently made on a phone.

The foundations are set globally: horizontal overflow is suppressed at the document level so no screen can scroll sideways, `touch-action: manipulation` removes the double-tap zoom delay that makes tap targets feel unresponsive, and a single breakpoint at 640 pixels carries the layout adjustments. Above that width the layout is fluid, achieved through flexible box and grid containers with proportional widths rather than through a cascade of fixed breakpoints. Figure 7.17 and Figure 7.18 show the dashboard and team screens on a phone viewport: the navigation collapses, the dashboard's panels reflow from a multi-column arrangement into a single column in priority order, with the deadline countdown remaining first, and the team screen retains the pitch as its organising metaphor at a reduced scale rather than degrading into a list. Figure 7.19 shows the team screen at tablet width, where the layout occupies the intermediate arrangement.

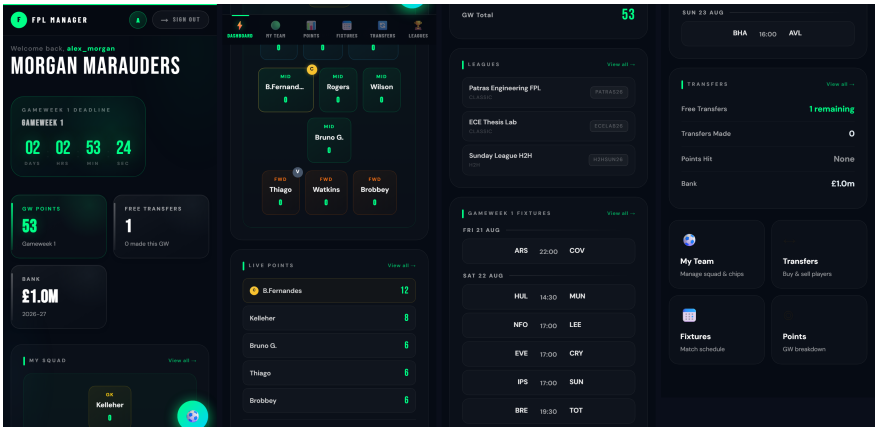


Figure 7.17. The dashboard on a phone viewport (390 × 844). Panels reflow into a single column in priority order, with the deadline countdown remaining first. Captured in full and re-flowed into four columns.

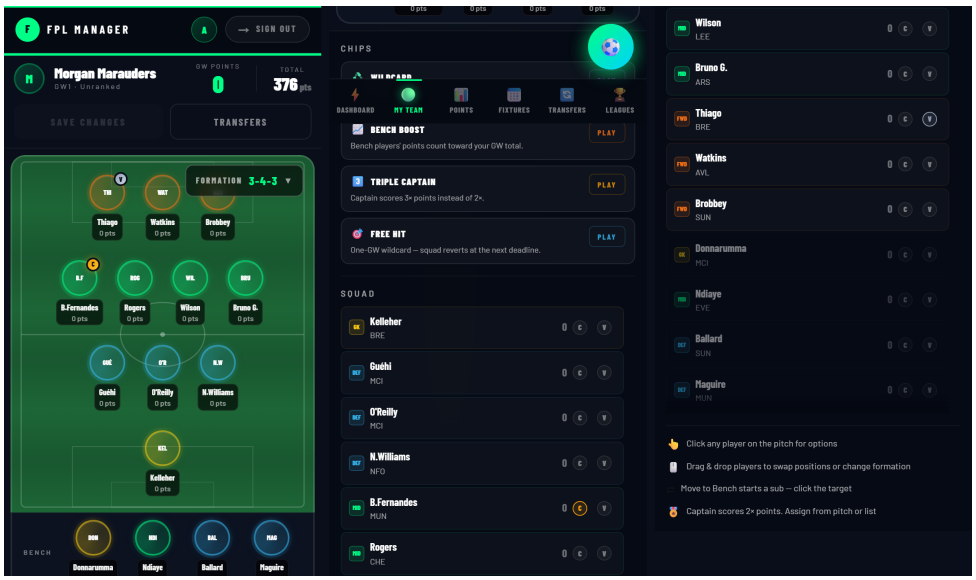


Figure 7.18. The team screen on a phone viewport. The pitch is preserved as the organising metaphor at reduced scale rather than degrading into a list. Captured in full and re-flowed into three columns.

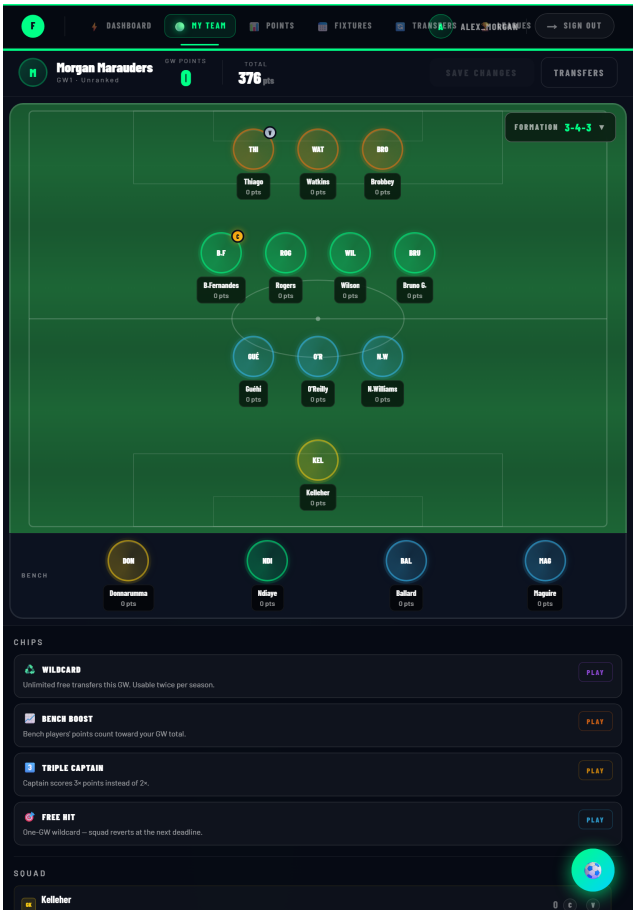


Figure 7.19. The team screen at tablet width (820 × 1180).

7.2.4 Route Protection

The API verifies access tokens on protected requests and uses the authenticated user's identity when accessing user-specific data, as described in Chapter 5.2. Client-side guards handle navigation when a session expires or a required setup step is missing. Data access remains the backend's responsibility, even if a client-side guard is bypassed.

The `apiFetch` wrapper described in Chapter 7.2.1 redirects to `/auth` when the session cannot be refreshed. The `useTeamGuard` hook also checks whether the manager has a squad and redirects to the builder when none exists. Protected pages use this hook so that a newly registered manager reaches the squad builder before opening a dashboard that requires squad data.

As discussed in Chapter 7.1.3, the hook does not currently enforce completion of onboarding. Adding that check to `useTeamGuard` would address the sequencing issue in the pages that already use it.

7.3 Backend Implementation

The backend runs as a separate Express service, following the architecture described in Chapter 5. It handles database access, API requests, authentication, sessions, and game deadlines. It also runs scheduled tasks to update the data and coordinates the Python prediction and optimisation components described in Chapter 6.

7.3.1 Database Access

The Express backend accesses PostgreSQL through Prisma, an object-relational mapper for Node.js that provides a typed database client. A dedicated module creates one `PrismaClient` instance, which other modules import when they need database access. This allows requests within the process to reuse the client's connection pool.

```
// apps/api/src/db/prisma.ts
import { PrismaClient } from '@prisma/client';
export const prisma = new PrismaClient();
```

Prisma generates the client from the database schema, which is also used when creating migrations. Generated types describe the available query fields and the objects returned by queries. After a schema change and client regeneration, the compiler can identify references to fields that no longer exist. These checks help detect inconsistencies during development, although they do not replace applying migrations to the database.

Prisma's standard query methods pass values as parameters, keeping input values separate from SQL syntax. This protects those queries against SQL injection; raw SQL still needs to be constructed safely. Database operations are organised in module services, keeping data access separate from routing and HTTP response handling.

7.3.2 Routing and Module Structure

The server entry point configures JSON parsing, cookie parsing, and a cross-origin policy restricted to the frontend's origin. It then mounts the routers for the application's feature areas:

```
// apps/api/src/server.ts (excerpt)
app.use('/auth',      authRouter);
app.use('/team',      teamRouter);
app.use('/transfers', transfersRouter);
```

```
app.use('/leagues',   leaguesRouter);
app.use('/ai',        aiRouter);
```

The eight feature modules follow a common structure. A routes file declares endpoints and attaches their middleware. A controller handles request validation and converts service results or errors into HTTP responses. A service contains the application logic and database operations. This organisation makes related files easier to locate and allows service logic to be tested separately from the HTTP server.

7.3.3 Sessions and Route Protection

At login, the backend issues a short-lived access token and a longer-lived refresh token. Both are JSON Web Tokens stored in `httpOnly` cookies, which client-side JavaScript cannot read. Authentication middleware reads the access token from the request cookies, verifies it, and adds the authenticated user's identifier to the request.

```
// apps/api/src/middlewares/requireAuth.ts (excerpt)
const token = req.cookies?.access_token;
if (!token) return res.status(401).json({ error: 'Unauthorized' });
const payload = verifyAccessToken(token);
req.userId = payload.sub;
```

Services use this identifier, taken from the verified token's `sub` claim, to scope access to the manager's data. A user identifier supplied in the request body does not establish ownership. For resources such as squads and conversation histories, access also depends on applying the appropriate ownership conditions in the service's queries.

Refresh tokens are stored server-side and rotated when used. This allows the backend to replace a used refresh token and revoke a session when required, as described in Chapter 5.

7.3.4 Password Protection

During registration, the backend hashes the submitted password using `bcryptjs` with a cost factor of ten. It stores the resulting hash rather than the original password. During sign-in, the library's comparison function checks the submitted password against the stored hash, using the salt and cost information contained in that hash. Password hashes are not included in API responses.

7.3.5 Request Validation

Controllers validate incoming payloads against Zod schemas before passing them to the service layer. Each schema defines the expected fields, types, and input restrictions. Invalid payloads receive a `400` response with details of the validation errors, allowing the interface to report the problem to the user.

This catches malformed input before database operations begin. Services remain responsible for business rules that depend on application data, such as whether a proposed transfer is affordable or a resource belongs to the current user.

7.3.6 Deadline Enforcement

The backend checks the gameweek deadline before accepting changes to a live squad. A dedicated middleware retrieves the current gameweek' s deadline and rejects changes when it has passed. Routes for committing transfers, saving a line-up, and playing a chip use this check.

Read and preview routes remain available, allowing the manager to inspect the squad and consider changes without committing them. Keeping the deadline check in shared middleware avoids repeating the same logic in several services. New routes that change a live squad must also attach this middleware.

7.3.7 Coordination with the Machine-Learning Service

Three schedulers start with the backend process. One synchronises official Fantasy Premier League data at intervals that adapt to game activity. Another calculates gameweek scores. The third detects a newly completed gameweek and launches the Python prediction pipeline as a child process.

Initial squad requests follow a separate path: the backend launches the Python optimiser on demand and reads its JSON output. Stored predictions are accessed through the shared database, so recommendation requests can use them without launching the training pipeline. Chapter 5 explains this integration, while Chapter 6 describes the prediction and optimisation procedures.

7.4 The Conversational Assistant Interface

The conversational assistant gives managers access to the predictions and recommendations described in Chapter 6. This integrated interface distinguishes the application from the platforms reviewed in Chapter 2. The following sections describe the chat panel, streamed replies, and the presentation of recommendations and their supporting information.

7.4.1 An Always-Available Surface

A floating button in the lower-right corner of authenticated screens opens the assistant in a panel over the current page. This lets the manager ask a question while keeping the relevant squad or dashboard visible. For example, a manager can read captaincy advice without leaving the team screen. This follows the integration objective described in Chapter 1.1. In Figure 7.8, the squad builder remains visible behind the panel while the manager reads a squad recommendation.

Four components implement the interface. `ChatWidget` manages the launcher and whether the panel is open. `ChatPanel` contains the conversation and message input. `MessageBubble` renders individual messages, and `OnboardingFlow` presents the questionnaire described in Chapter 7.1.3 in the same panel before a profile exists. An empty conversation displays the six suggested questions shown in Figure 7.6, giving the manager examples of the decisions the assistant supports.

The API streams reply text through server-sent events, allowing the panel to display it as it arrives. The client prevents another message from being sent while a reply is streaming. Initial squad requests also require the Python optimiser described in Chapter 6.2.3 to finish before the reply can begin. During this wait, the API sends the status message "Building your initial squad…". The panel displays it until reply text starts arriving, giving the manager an indication of what the application is doing.

7.4.2 Rendering Structured Recommendations

The assistant is instructed to organise recommendations into a main recommendation, candidate reasoning, risks, an alternative, and a conclusion. `MessageBubble` renders these sections as separate labelled blocks when they are present. This format makes the main advice easy to locate while keeping its explanation available in the same message.

A source badge above the reply identifies whether the recommendation primarily uses the ML-based or rule-based results described in Chapter 6.2.2. Its tooltip displays the recommendation orchestrator's reason for choosing that source. The captaincy example in Figure 7.20 is labelled `ML-BASED`; a reply led by the heuristic engine, such as formation or bench-order advice, uses the `Rule-based` label. The badge identifies the recommendation source, rather than measuring the accuracy of the reply.

During interface testing, the badge initially failed to appear. The API emitted the source information and stored it in the recommendation log, but the client's stream handler did not process that event. Adding the missing handler allowed the interface to display it. The shared TypeScript definitions did not catch this omission because the handler did not enforce exhaustive handling of every event type. As discussed in Chapter 7.2, shared types describe the data being exchanged, but the client must also implement the behaviour required for each event.

In Figure 7.20, the `RECOMMENDATION` block names the captain and vice-captain. The `REASONING` block explains

the choices using expected points, the corresponding doubled points, fixture difficulty, and relevant team statistics. The `RISK` block describes concerns about each candidate, while `ALTERNATIVE` provides another option and its projection. A short `CONCLUSION` closes the reply.

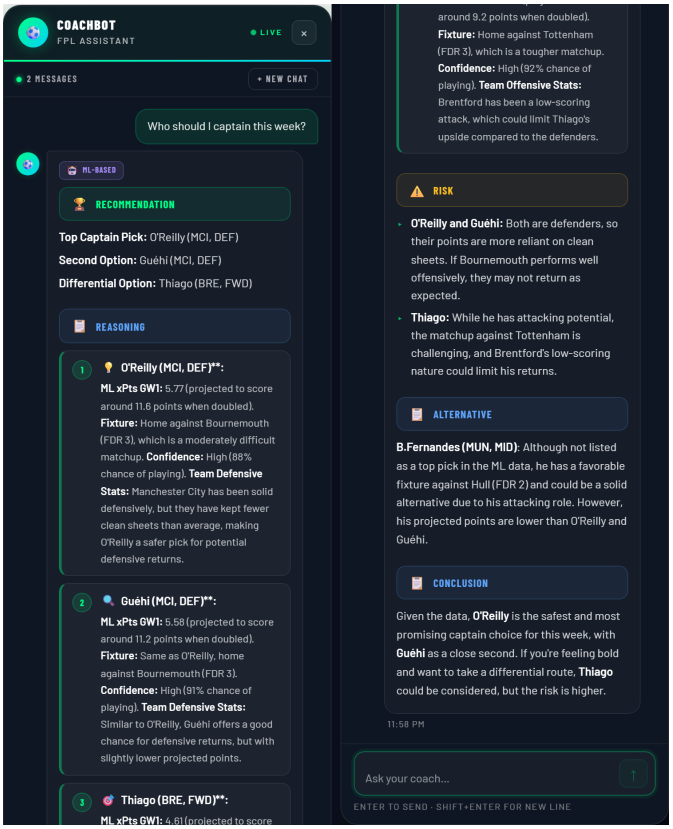


Figure 7.20. A captaincy recommendation. The structured blocks, recommendation, reasoning, risk, alternative, and conclusion, are rendered as distinct surfaces so the answer is legible at a glance and the justification available on demand. Captured in full and re-flowed into two columns.

This structure is what operationalises the explainability requirement, and it does so at two levels of depth simultaneously, which matters given the range of users identified in Chapter 4.1.1. The beginner persona, who wants to be told what to do and why, reads the recommendation and the conclusion. The competitive persona, who wants evidence, reads the reasoning, checks the expected-points figures against the fixture ratings on Figure 7.15, and weighs the stated alternative. Neither is served at the other' s expense, because the same reply contains both and the visual hierarchy makes the depth optional. The `explanationDepth` dimension captured during onboarding additionally scales how much of the middle the model produces.

The following examples show transfer advice, a player comparison, and squad analysis from the existing Aleka account. Each request asks for a concise answer with recommendation, reasoning,

risk, alternative, and conclusion sections. The screenshots capture the actual replies after the Markdown-rendering correction, with bold text displayed without visible asterisk markers.

Figure 7.21 shows a proposed Wilson-to-Szoboszlai transfer for Aleka's squad. The reply gives its projected benefit, discusses risk and affordability, and offers an alternative involving a defender. These statements are the assistant's explanation of the supplied recommendation; the manager must still pass the transfer service's feasibility checks before committing a move.

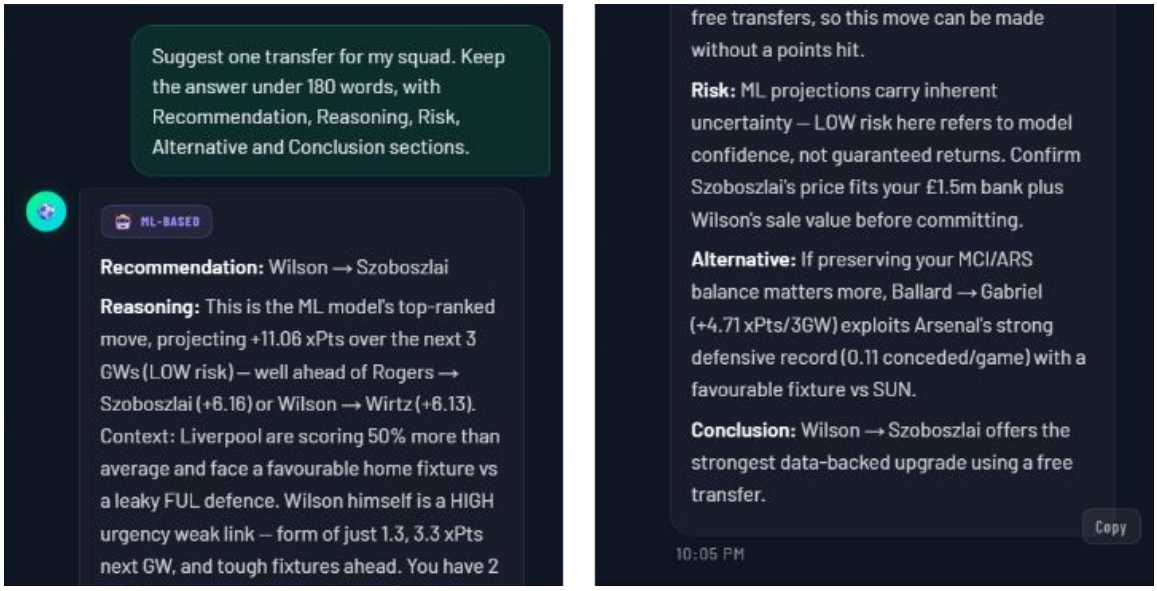


Figure 7.21. Transfer advice for Aleka's existing squad. The actual chat is captured across its scrollable area and arranged in two columns, read from top to bottom in the left column and then the right.

Figure 7.22 compares Calafiori and Gabriel in response to Aleka's request. The reply discusses their projections, prices, and participation risk and recommends retaining Calafiori. This provides the conversational comparison required by FR13. The reply also refers to differing projection figures from the supplied sources, illustrating why a source badge alone does not guarantee a consistent numerical explanation.

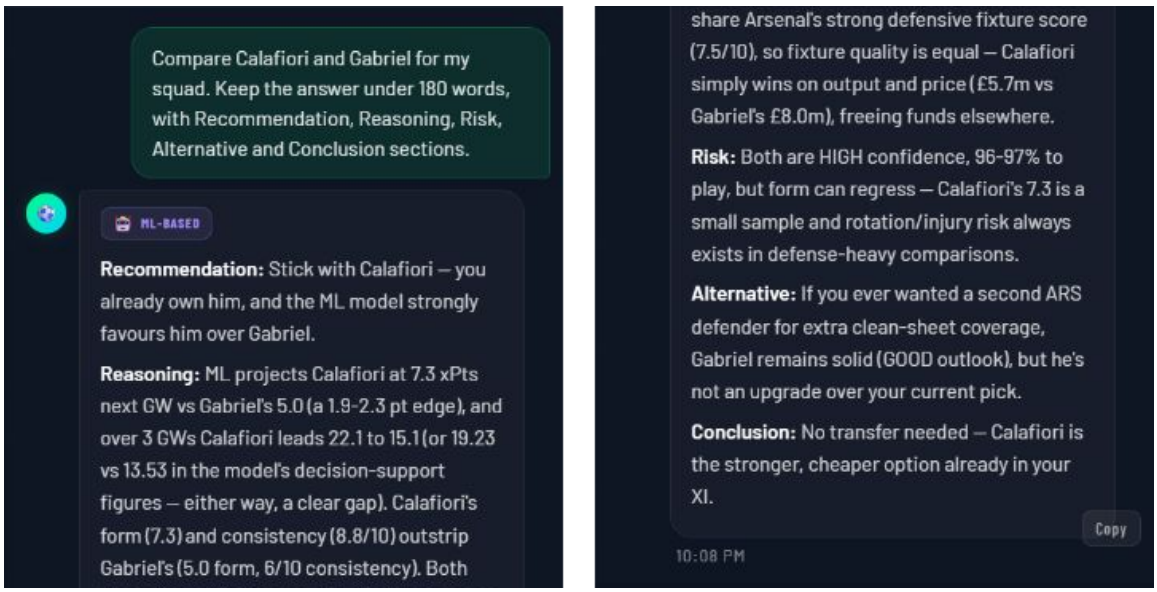


Figure 7.22. Calafiori and Gabriel compared in Aleka' s chat. The complete captured reply is arranged in two columns.

Figure 7.23 shows a squad analysis for Aleka, including a captaincy suggestion, individual player projections, participation concerns, and a bench alternative. Although the request asks for a projected squad score, this reply does not provide a total. The example therefore demonstrates the presentation of player-level advice and also shows that a generated reply may leave part of a request unanswered.

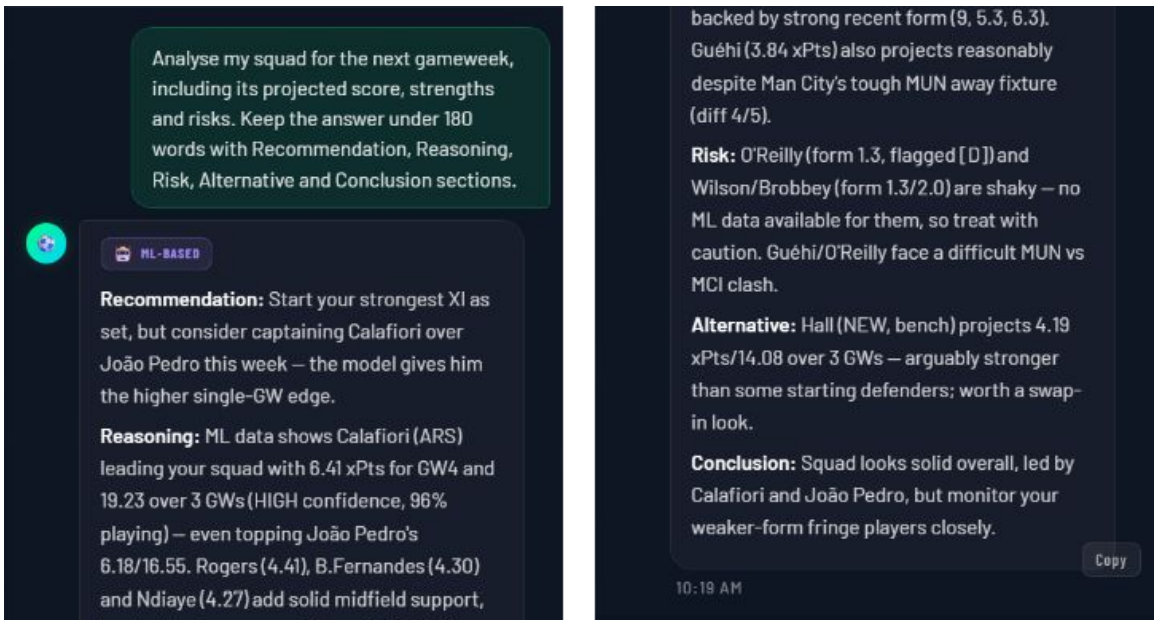


Figure 7.23. Squad analysis from Aleka' s existing account, showing player projections, risks, and an alternative. The actual reply is captured in full and arranged in two columns.

The interface makes the recommendation and its supporting information easier to read. Numerical consistency and completeness still depend on the supplied data and the generated response, which is why Chapter 6 separates application calculations from language-model explanation.

7.5 Conclusion

This Chapter presented the adaptive and conversational interface that constitutes the central contribution of this thesis. The walkthrough of Chapter 7.1 traced the screens in the order a manager meets them: a welcome sequence that spends the first impression on orientation rather than data, a registration flow that validates in place, a ten-question conversational questionnaire whose answers move nine-tenths of the squad optimiser' s objective weight, a squad builder that enforces the structural rules of a legal squad preventively and can be filled by asking the assistant to solve the integer program, and the dashboard, team, transfer, points, fixture, and league screens that follow.

Chapter 7.2 recorded the client-side decisions behind those screens: a three-way separation of routing, shared presentation, and per-screen features; a single `apiFetch` wrapper that makes token expiry invisible; five purpose-built hooks in place of a global store; a committed visual identity implemented predominantly through inline style objects rather than the installed utility framework; and a responsive strategy built on global mobile foundations and a fluid layout rather than a cascade of breakpoints.

Chapter 7.4 examined the assistant as an interface: an overlay rather than a route, so advice and action are one click apart; a streamed reply with a status event covering the seconds during which the integer program runs; and a structured rendering of recommendation, reasoning, risk, alternative, and conclusion that serves the beginner and the competitive manager from the same reply at different depths.

8 Evaluation

This chapter presents the evaluation of the Fantasy Football application and its AI assistant. The evaluation is divided into two main parts. The first focuses on the user interface and user experience, examining how clearly the application supports squad management and how effectively the conversational assistant communicates its recommendations.

The second part examines the machine-learning pipeline and the recommendation and scoring system. It evaluates the accuracy of player-performance predictions, the behaviour of the scoring functions, and the quality of the resulting recommendations under different conditions and user preferences. Comparisons with baseline methods, statistical analysis, and experiments on individual components provide the basis for evaluating the design choices presented in Chapter 6. For each evaluation, the methods and metrics are described first, followed by the available results, their interpretation, and the limitations of the evidence.

8.1 User Interface Evaluation

The user interface evaluation combines expert-based and user-based methods to examine both the usability of the Fantasy Football application and the experience of interacting with its AI assistant. The evaluation considers whether managers can complete the main tasks, understand the recommendations presented to them, and retain control over their decisions.

The expert-based evaluation uses **Heuristic Evaluation** and a **Streamlined Cognitive Walkthrough**. Heuristic Evaluation examines the interface against established usability principles to identify strengths and potential problems. The Streamlined Cognitive Walkthrough follows representative tasks step by step, evaluating whether a user would recognise the appropriate action and understand the resulting feedback. These methods are complemented by Amershi et al.' s guidelines for human-AI interaction, which address aspects specific to the assistant, including capability awareness, explanation, uncertainty and user control.

The inspection draws on the interface and implementation presented in Chapter 7. Although every effort was made to evaluate the interface objectively, the author' s extensive familiarity can make intended interactions appear more obvious than they would to a new user, so its findings require further examination through direct user observation and user-based evaluation methods.

The user-based evaluation combines the **Think-Aloud Protocol** and **individual interviews**. The think-aloud procedure asks participants to verbalise their thoughts, expectations and reactions while completing tasks in the application and using the assistant. Follow-up interviews explore the overall experience, including which features support decision-making and where users encounter difficulties.

8.1.1 Heuristic Evaluation

As introduced above, Heuristic Evaluation examines an interface against established usability principles. The evaluation of the Fantasy Football application follows **Nielsen's 10 usability heuristics** (Nielsen 1994a), which address general principles for effective interface design such as feedback, consistency, error prevention and user control. The evaluation considers how these principles are reflected in the application's screens and interaction mechanisms, including the conversational assistant described in Chapter 7. In this method, the evaluator systematically navigates the application and examines whether any of the heuristics are violated.

The following points describe the ten principles and how they are supported by the application and its AI chatbot:

- **Visibility of System Status**: The system should keep users informed about what is happening through clear and timely feedback. In the application, the dashboard displays the gameweek deadline, available bank and remaining free transfers. During squad construction, positional counts and the remaining budget change as players are selected, while the transfer cart presents the consequences of pending moves. The chatbot provides feedback through progressively streamed replies and displays a status message while initial-squad optimisation is running. These mechanisms help managers distinguish an operation in progress from a completed result.
- **Match Between the System and the Real World**: The interface should use language, concepts and visual representations that correspond to the user's experience and expectations. The application presents players on a football pitch, organises substitutes on a bench and provides familiar controls for formation and captaincy. Terms such as gameweek, transfer and fixture connect the interface to the decisions involved in Fantasy Football. The chatbot uses the same vocabulary and presents advice around recognisable questions, such as whom to captain or which player to replace. Its explanations connect recommendations to football considerations, including fixtures, expected playing time and player performance.
- **User Control and Freedom**: Users should be able to reconsider their choices, leave unwanted interactions and remain in control of consequential actions. The application separates pending transfers from committed changes, giving managers an opportunity to review the transaction before applying it. The chatbot supports this principle by acting as an adviser: its recommendations do not directly modify the squad. Managers decide whether to follow the

advice through the relevant application controls. They can also close the chat panel, stop a streamed response or start a new conversation.

- **Consistency and Standards**: Similar information and actions should be presented consistently so that users do not have to learn a different interaction pattern on each screen. The application uses a common visual style and recurring football terminology across squad management, transfers, fixtures and scoring views. The chatbot follows the same domain vocabulary and uses a recurring structure for recommendations, reasoning, risks, alternatives and conclusions when these sections are supplied. This consistency gives managers a familiar way to locate the proposed action and its supporting explanation across different requests.
- **Error Prevention**: The system should minimise avoidable errors by checking inputs and restricting invalid actions before they are completed. The application validates registration fields, prevents selections that exceed squad-builder club limits and restricts formations to legal arrangements. The transfer cart also blocks commitment when the proposed transaction violates its constraints. In the chatbot, empty messages and messages exceeding the input limit cannot be submitted, and another request cannot be sent while a response is streaming.
- **Recognition Rather than Recall**: The interface should make relevant options and information visible, reducing the amount users must remember between interactions. The application displays squad positions, player counts and budget information during construction, while the transfer cart keeps pending changes and their consequences together. The chatbot offers suggested questions that demonstrate its capabilities without requiring managers to remember specific commands. Within a conversation, visible previous messages and labelled recommendation sections provide reference points for reviewing advice and asking follow-up questions.
- **Flexibility and Efficiency of Use**: The system should accommodate different levels of experience and provide efficient routes for frequently performed tasks. The application supports player search, filtering and direct captaincy controls. Selecting an outgoing player also narrows the transfer market to relevant replacements, reducing the number of options that need to be examined. The chatbot provides a complementary route through natural-language requests, allowing managers to ask for comparisons or recommendations without assembling the information manually across several screens. Suggested questions support initial use, while free-text input allows more specific requests.
- **Aesthetic and Minimalist Design**: The interface should prioritise information relevant to the current task and organise supporting details without unnecessary visual clutter. The application uses the dashboard for summaries and provides dedicated screens for detailed squad management, transfers, fixtures and points. Within each screen, related information is grouped around the main activity. The chatbot separates the main recommendation from supporting reasoning, risks and alternatives, establishing a clear reading order. Its panel can be opened

when advice is needed and closed when the manager returns to the application's other controls.

- **Help Users Recognize, Diagnose, and Recover from Errors**: Error messages should explain the problem in understandable language and indicate how the user can proceed. The application places registration feedback beside the affected fields and identifies constraints that prevent a transfer from being committed. The pending cart remains available for adjustment, allowing the manager to resolve the problem. If a chatbot response fails, the interface reports the failure and asks the manager to try again. For documented out-of-scope requests, the assistant explains its boundary and redirects the conversation towards Fantasy Football tasks.
- **Help and Documentation**: The system should provide accessible guidance that supports users in understanding its features and completing their tasks. The application introduces its purpose through the welcome sequence and provides contextual instructions, including password requirements and explanations within the onboarding questionnaire. The chatbot's suggested questions demonstrate the decisions it can assist with. Its reasoning and risk sections explain the basis of recommendations, while the provenance badge and tooltip identify their source. Together, these elements place guidance close to the interaction where it is needed.

8.1.2 Streamlined Cognitive Walkthrough

As mentioned previously, the Streamlined Cognitive Walkthrough examines the usability of an interface by following a sequence of actions from the perspective of a new user. The evaluator considers whether the next action is discoverable and whether the resulting feedback makes progress towards the task's goal apparent. The method simplifies the Cognitive Walkthrough approach by using two questions at each step (Spencer 2000):

- **Question 1**: Will the user know what to do at this step?
- **Question 2**: If the user performs the correct action, will they recognise that it was successful and that they are making progress towards their goal?

The following walkthrough applies these questions to two representative tasks in the Fantasy Football application. The evaluations draw on the interface and implementation presented in Chapter 7. They describe expected interaction based on the available controls and feedback, rather than outcomes observed in participant sessions.

1. **Selecting and completing a player transfer**: The manager starts from the dashboard with an existing squad, opens the transfer screen, selects an outgoing player, chooses a replacement and reviews the transaction before committing it. The task assumes that transfers are permitted and that a valid replacement is available.

2. **Completing personalisation and requesting captaincy advice**: The manager starts from an authenticated screen with an existing squad and an incomplete personalisation profile. They open the chatbot, complete the questionnaire and request a captaincy recommendation.

Table 8.1 and Table 8.2 present the step-by-step evaluation of these tasks. A positive evaluation indicates that an identifiable interface element supports the action or provides feedback about its outcome.

Πίνακας 8.1: Streamlined Cognitive Walkthrough for selecting and completing a player transfer.

Step	Question 1	Question 2
0 –Open the transfer screen	Yes. The manager can select the transfers destination from the application navigation.	Yes. The transfer screen presents the current squad, player market and pending-transfer area, making the purpose of the screen apparent.
1 –Select an outgoing player	Yes. The current squad and selling prices provide the information needed to choose a player to replace.	Yes. Selecting the outgoing player narrows the market to replacements in the corresponding position and within the applicable affordability filter.
2 –Choose a replacement	Yes. The player market provides search and filtering controls through which the manager can locate and select a suitable candidate.	Yes. The selected replacement appears with the outgoing player in the transfer cart, making the proposed change visible.
3 –Review the transaction	Yes. The cart brings the pending change together with its budget and transfer consequences, providing a clear point for review.	Yes. The displayed bank, remaining free transfers and any points penalty allow the manager to understand the effect of the transaction before commitment.

Step	Question 1	Question 2
4 –Commit the transfer	Yes. A distinct commitment control allows the manager to apply the reviewed transaction once it satisfies the required constraints.	Yes. The interface updates the affected squad and financial information after a successful transfer, showing that the pending change has been applied.

The transfer workflow provides a clear sequence from selecting a player to reviewing and applying the replacement. Its main strength is the separation between a proposed transaction and a committed change. The manager can inspect the consequences before acting, while validation prevents an invalid cart from being submitted.

Πίνακας 8.2: Streamlined Cognitive Walkthrough for completing personalisation and requesting captaincy advice.

Step	Question 1	Question 2
0 –Open the chatbot	Yes. The floating assistant icon provides an entry point from the authenticated screen.	Yes. The chat panel opens and, for an incomplete profile, displays the personalisation questionnaire.
1 –Answer the personalisation questions	Yes. Each question presents concrete choices with explanatory text, allowing the manager to select the option that best reflects their preferences.	Yes. The questionnaire advances through the sequence of questions, making progression through the profile setup visible.
2 –Complete the questionnaire	Yes. The final question uses the same selection pattern as the preceding questions.	Yes. After completion, the panel changes from the questionnaire to the conversational interface and presents suggested opening questions.

Step	Question 1	Question 2
3 –Request captaincy advice	Yes. A suggested captaincy question provides a direct starting point. The manager can also enter a request in the message field.	Yes. The submitted message appears in the conversation, and the assistant's response is displayed progressively as it is generated.
4 –Read the recommendation	Yes. The recommendation section identifies the proposed captain and vice-captain, giving the manager a clear starting point for reading the answer.	Yes. The reply presents the proposed choices together with supporting reasoning, risks and an alternative when these sections are supplied, allowing the manager to review the advice.

The second workflow provides a consistent progression from expressing preferences to receiving advice. Concrete questionnaire choices support profile completion, while suggested questions connect the completed profile to the assistant's available functions. The structured reply then separates the recommendation from its supporting explanation.

These walkthroughs indicate that the two selected paths have identifiable actions and visible feedback at their main steps. This supports a positive evaluation of their expected learnability. Direct user observation is still needed to establish whether new managers recognise these cues as intended. For the chatbot task, successfully locating and reading a recommendation is also distinct from verifying its numerical accuracy, which is examined in the evaluation of the prediction and recommendation system.

8.1.3 Evaluation Based on Amershi et al.'s Guidelines

The evaluation of the conversational assistant also considers **Amershi et al.'s 18 guidelines for human-AI interaction** (Amershi et al. 2019). These guidelines complement Nielsen's usability heuristics by addressing uncertainty, explanation, correction and personalisation. They cover initial use, ongoing interaction, situations in which the system is wrong, and adaptation over time.

Table 8.3 connects each guideline to the FPL assistant, defines the evaluation criterion and records the supporting behaviour in the current implementation. The implementation column records supporting mechanisms and their limitations, while the final column gives the inspection evaluation.

The inspection evaluated each guideline as Satisfied, Partially Satisfied, Not Satisfied, or Not Applicable based on observable interface behaviour and documented implementation mechanisms. A Satisfied classification indicates that an explicit mechanism addressing the guideline was identified; it does not by itself demonstrate that users perceive the interaction as intended. User-facing aspects are therefore examined separately through scenario-based and user evaluation.

Πίνακας 8.3: Application of Amershi et al.'s guidelines to the FPL assistant.

Amershi guideline	Evaluation criterion	Current implementation and supporting evidence	Evaluation
G1: Make clear what the system can do	Users can identify the assistant's main capabilities and distinguish recommendations from actions that modify their squad.	An empty conversation presents six suggested questions covering common FPL decisions. These examples communicate the assistant's scope without requiring specialised commands. The assistant remains advisory; squad changes are performed separately through the team and transfer controls. The assistant also explains when a request is outside the topics it supports.	**Satisfied**
G2: Make clear how well the system can do what it can do	The assistant avoids presenting predictions as certainties and exposes relevant uncertainty affecting a recommendation.	Structured responses can include a dedicated risk section, making concerns such as uncertain participation visible alongside the recommendation. The interface also identifies whether advice primarily originates from ML-based or heuristic results and explains the source selection through a tooltip. However, these mechanisms do not communicate the measured predictive accuracy of the underlying models.	**Partially Satisfied**
G3: Time services based on context	Assistance occurs at appropriate points in the interaction and provides suitable processing feedback.	The floating icon allows managers to invoke the assistant when needed from authenticated screens. On desktop, the panel overlays the current page, preserving the surrounding task context. Responses are streamed as generated, while longer operations such as initial-squad optimisation display processing feedback.	**Satisfied**
G4: Show contextually relevant information	Explanations use the appropriate players, squad state and forecast horizon, while avoiding unrelated information.	Captaincy, transfer, comparison and squad-analysis interactions receive task-specific structured recommendation context before narration. For example, transfer responses identify the proposed outgoing and incoming players, while comparison responses focus on the players requested by the manager.	**Partially Satisfied**

Amershi guideline	Evaluation criterion	Current implementation and supporting evidence	Evaluation
G5: Match relevant social norms	Recommendations support decision-making without pressuring the manager to accept a particular choice.	Responses can separate the recommendation, supporting reasons, risks and alternatives. Providing alternatives preserves the manager's decision-making role. The explanation-depth preference also allows response detail to reflect the manager's selected preference.	**Satisfied**
G6: Mitigate social biases	No inappropriate social assumptions are used in recommendation generation or explanation.	Personalisation uses the manager's FPL preferences, not social characteristics. The assistant is designed to use respectful language and avoid insults. These measures do not establish that every response is free from bias.	**Partially Satisfied**
G7: Support efficient invocation	Common FPL questions can be submitted easily using natural language and readily available controls.	The assistant icon is available throughout authenticated screens. Six suggested questions provide one-click starting points, while free-text input allows managers to express more specific requirements. Enter submits a request and Shift+Enter inserts a line break.	**Satisfied**
G8: Support efficient dismissal	Dismissing or stopping assistance does not commit an FPL action or prevent the manager from continuing independently.	The interface allows the panel to be closed and streamed responses to be stopped through an abort mechanism. Recommendations do not directly mutate the squad; transfers and captaincy changes remain separate user-controlled actions.	**Satisfied**
G9: Support efficient correction	Corrections and additional constraints are incorporated into subsequent advice without unnecessarily restarting the interaction.	Free-text follow-up messages allow managers to challenge previous recommendations or request alternatives, while conversation history preserves the preceding context. Further testing is needed to check that the assistant follows corrections across several messages.	**Partially Satisfied**
G10: Scope services when in doubt	Unsupported statistics and recommendations are not invented, and users receive an appropriate explanation or route forward.	The assistant declines off-topic requests and unsupported player requests. It uses recommendations and numbers calculated by the system to help keep its answers grounded in the available data.	**Partially Satisfied**
G11: Make clear why the system did what it did	Users can understand why a recommendation was produced, and explanations remain consistent with the underlying recommendation data.	Responses explain the recommendation, its reasons, risks and alternatives when relevant. The interface also shows the source of the recommendation.	**Partially Satisfied**

Amershi guideline	Evaluation criterion	Current implementation and supporting evidence	Evaluation
G12: Remember recent interactions	Users do not need to repeat relevant information, and references to earlier recommendations or constraints are interpreted correctly.	Persisted sessions, conversation history and running summaries provide mechanisms for retaining context across turns.	**Satisfied**
G13: Learn from user behaviour	Claims of behavioural learning should only be made when supported by an implemented mechanism.	The current system does not automatically infer persistent preferences from user behaviour. Instead, personalisation is controlled explicitly through the manager profile.	**Partially Satisfied**
G14: Update and adapt cautiously	Changes in persistent behaviour can be related to explicit profile or context changes, while temporary instructions do not silently redefine the manager profile.	Persistent preferences change only through explicit profile configuration, while conversational requests do not silently modify the stored manager profile. The inspected implementation saves preferences through the onboarding/configuration endpoints rather than ordinary chat-message processing. This keeps a temporary request for an alternative or a different explanation separate from a lasting change to the settings used for future advice.	**Satisfied**
G15: Encourage granular feedback	Feedback can target individual recommendations or claims, and subsequent interaction can address the identified issue.	Free-text follow-ups allow users to identify a particular player, disputed statement or recommendation and request further reasoning or an alternative. However, no dedicated rating, claim-reporting or feedback-learning mechanism is currently implemented.	**Partially Satisfied**
G16: Convey the consequences of user actions	The interface explains how changing risk tolerance or planning horizon affects subsequent advice, and users can understand that relationship.	Onboarding questions explain how selected preferences affect assistance, and the stored risk-tolerance and planning-horizon settings map to defined recommendation weights. This provides an explicit connection between configuration choices and future advice; whether managers understand that connection remains a user-evaluation question.	**Satisfied**
G17: Provide global controls	Users can configure persistent aspects of assistance and distinguish these settings from temporary chat controls.	The onboarding profile provides persistent configuration for multiple recommendation and explanation characteristics. Chat controls such as stopping generation or starting a new conversation operate separately.	**Satisfied**

Amershi guideline	Evaluation criterion	Current implementation and supporting evidence	Evaluation
G18: Notify users about changes	Users are informed about significant changes through an appropriate user-facing mechanism.	Recommendation provenance communicates the source of the current recommendation, but it does not notify users when the assistant's capabilities or underlying behaviour change. No dedicated mechanism for communicating such changes is currently documented.	**Not Satisfied**

The assistant is easy to access, explains its suggestions and leaves squad changes to the manager. Managers can set their preferences and ask follow-up questions before deciding. Learning from user behaviour, collecting structured feedback and notifying users about changes still need improvement.

8.1.4 Think-Aloud Protocol

The expert-based methods discussed above yielded generally positive evaluations of the interface, suggesting that its main tasks have identifiable controls and visible feedback. However, direct user observation is still necessary. Managers may have different levels of FPL experience and may interpret the interface differently from an evaluator who is familiar with its design. The **Think-Aloud Protocol** addresses this gap by asking participants to verbalise their thoughts, expectations and difficulties while completing representative tasks (Nielsen 2012). This makes it possible to examine both what users do and how they understand the application's responses.

The evaluation is planned for three participants: Participant 1, Participant 2 and Participant 3. Each participant will complete the same five scenarios, covering both the application and the conversational assistant. For a concise presentation, one scenario per participant is developed in detail below.

Πίνακας 8.4: Participant profiles.

Participant	Relevant characteristics
Participant 1	A beginner in Fantasy Premier League who values clear explanations and visual interaction.
Participant 2	A manager with some FPL experience who is interested in practical transfer advice. They value clear information about player prices, available budget and the reasons behind a suggested replacement.

Participant	Relevant characteristics
Participant 3	An experienced FPL manager who prefers comparing alternatives before making a decision. They value concise explanations supported by player statistics and the ability to refine recommendations through follow-up questions.

Πίνακας 8.5: Five scenarios assigned to every participant in the evaluation.

Scenario	Description
1. Registration and personalisation	Create an account, complete the manager-preference questions and explain how the selected preferences are expected to affect the assistant' s advice.
2. Squad construction	Find and select players, review their prices and positions, and construct a valid squad within the available budget.
3. Formation advice and formation change	Ask the assistant for formation advice based on the current squad, review its reasoning, and adjust the lineup using both the formation button and drag and drop.
4. Transfer recommendation	Ask for a replacement for a selected player, review the recommendation and its risks, and use the transfer interface to check the cost and complete the change.
5. Player comparison and follow-up questions	Ask the assistant to compare two players, relate the comparison to the current squad, and refine the advice through follow-up questions about risk or a cheaper alternative.

Participant 1: Formation advice and formation change

This scenario examines the transition from understanding the assistant' s advice to changing the lineup. It also considers whether the formation selector and drag-and-drop interaction offer understandable

ways of achieving the same goal.

Πίνακας 8.6: Participant 1's formation scenario.

What I do	What I expect the system/chatbot to do	Reaction/Thoughts of the Participant
Ask the chatbot which formation would suit my current squad.	Suggest a suitable formation using the available players and explain the main reason for the recommendation.	"Nice, I like that it considers the players I already have. That makes the advice easier to use."
Read the explanation and consider which players would start.	Explain the trade-off between positions and help me understand how the suggested formation affects the lineup.	"I wanted a more defensive approach and formation, but the suggested formation makes sense and the explanation is very convincing."
Open the formation button and select a different formation.	Offer valid formation options and update the arrangement of the lineup after selection.	"Oh, I expected to move every player myself. I like that I can change the formation directly with this button. It is much easier than the original FPL and much less frustrating."
Drag and drop players to adjust the lineup.	Allow a valid swap and show the resulting change on the pitch and bench.	"Nice, I can also do it by dragging the players. I like having both options: the button for a quick change and dragging when I want to choose the players myself."
Review the updated formation and the players on the bench.	Make the new lineup clear so I can check whether the change matches my intention.	"The interface with the pitch and the squad looks awesome. It would be amazing to see the expected points at the top right of the pitch, updated every time I change the formation or substitute a player."

Participant 2: Transfer recommendation

This scenario examines whether the manager can move from a suggested replacement to an informed transfer decision. The focus is on understanding the recommendation, locating the relevant player and checking the financial and points consequences before confirming the change.

Πίνακας 8.7: Participant 2' s transfer scenario.

What I do	What I expect the system/chatbot to do	Reaction/Thoughts of the Participant
Ask the chatbot to suggest a replacement for a player in my squad.	Recommend an eligible alternative that takes the squad and available budget into account.	"Nice, it gives me a specific replacement instead of just a list of popular players. That is a useful starting point, although I would also like some alternative recommendations."
Read the supporting reasons and risks.	Explain why the replacement is worth considering and identify relevant uncertainty.	"I like that it includes the risks as well. I would put the main benefit first so I can understand the suggestion more quickly."
Open the transfer interface and find the recommended player.	Let me locate the player and check the price before adding the transfer.	"The player and price are easy to check here. I can compare the suggestion with what I can actually afford."
Add the proposed transfer and review the pending change.	Show the outgoing and incoming players, remaining bank, free transfers and any points penalty before commitment.	"Good, I can check everything before confirming. I would make the points penalty stand out more, because that could change my decision."
Confirm the transfer and review the squad.	Provide feedback that the change has been applied and display the updated squad.	"That is clear. I asked for advice first, but I still made the final decision myself."

Participant 3: Player comparison and follow-up questions

This scenario examines whether the assistant supports a comparison that develops across several messages. It focuses on continuity, the relevance of the explanation and whether a manager can introduce further constraints without repeating the original request.

Πίνακας 8.8: Participant 3’s comparison scenario.

What I do	What I expect the system/chatbot to do	Reaction/Thoughts of the Participant
Ask the chatbot to compare two players.	Compare the requested players using relevant information and explain their main differences.	“Nice, the comparison gives me reasons to consider each player. It helps me think through the choice.”
Ask which player fits my current squad better.	Relate the comparison to my squad and relevant manager preferences.	“I like that I can bring the discussion back to my team. Having each player’s price next to the name would make it easier to follow.”
Ask which option is safer.	Retain the two players from the previous messages and explain relevant risk without treating predictions as guarantees.	“Good, I did not have to type both names again. The explanation of the safer option is useful.”
Request a cheaper alternative.	Use the existing conversation to interpret the request and explain the alternative in relation to the previous options.	“That is useful. I can narrow down the choice naturally instead of starting a new comparison.”
Review the conversation and decide which player I prefer.	Leave the decision under my control and provide enough information to compare the options.	“I have enough information to make a choice. A small comparison table would help me review the numbers without scrolling through the replies.”

Overall, the participants gave very positive feedback on the application’s interface and the chatbot’s suggestions and explanations. They made only a few requests for improvement. During initial squad creation, they wanted alternative suggestions rather than a single proposed squad. They also reported that the chatbot panel and its text could get in the way of searching for players, requiring them to close the chat, search and reopen it. They suggested making the panel movable so they could position it where it did not cover the information they needed.

Participants sometimes disagreed with the recommended players, but still appreciated the detailed explanations. They viewed the chatbot as a source of advice: the final choices remained their own.

They also wanted more statistics about each player' s team and its opponents to help them judge the recommendations.

8.1.5 User Interviews

Interviews provide qualitative insight into users' experiences, expectations and perceptions of a system. They complement the expert-based methods and the Think-Aloud Protocol by allowing participants to explain their decisions and discuss aspects of the interaction that may not be apparent during task observation (Rosala and Pernice 2023). In the Fantasy Football application, this includes both the usability of squad-management features and the perceived usefulness of the AI assistant.

Following the think-aloud sessions, individual interviews were conducted with **Participant 1**, **Participant 2 and Participant 3**. Each participant answered the same core questions, with follow-up questions used to clarify particular comments. The discussion covered the application as a whole, its visual presentation, and the clarity and relevance of the assistant' s recommendations, rather than being limited to the scenario presented in detail for each participant. Six general questions were complemented by ten questions addressing specific interactions:

1. Did the five scenarios feel easy or difficult to complete? Which task required the most effort, and why?
2. How would you describe the application' s overall design, including its layout, colours and readability?
3. Were there any features, controls or visual elements that you felt were missing?
4. What did you like most about the application, and why?
5. How relevant did you find the assistant' s recommendations to your squad and preferences? Can you give an example of useful or unsuitable advice?
6. Do you have any additional comments or suggestions for improving the application or assistant?
7. Was it clear how to navigate between squad management, transfers and the assistant? Were there moments when you were unsure where to go next?
8. Were the manager-preference questions understandable, and was it clear how your answers would affect future recommendations?
9. How easy was it to find players and understand the information presented about their price, position and predicted performance?
10. How did you find changing the formation through the formation button compared with using drag and drop? Which approach would you prefer, and why?
11. Was the assistant's formation advice clear enough to help you decide how to arrange your starting lineup and bench?
12. Before confirming a transfer, could you clearly identify the outgoing and incoming players, remaining budget and any points penalty?

13. When comparing players, did the assistant provide enough information to support your decision? How could the comparison be presented more clearly?
14. How well did the assistant handle follow-up questions? Did you need to repeat information or correct its understanding?
15. Were the explanations an appropriate length, and did they communicate both the reasons for a recommendation and its uncertainty?
16. Was it clear that the assistant provides advice while you remain responsible for applying squad changes? What would make you more or less willing to rely on its advice?

The three participants gave broadly consistent feedback. All of them responded positively to the application as a whole, describing it as approachable even with limited FPL experience, and singling out the pitch view as the element that made the organisation of a squad easiest to understand. They valued having squad information, transfers and assistant advice available in one place, and they liked that asking for advice never altered the squad by itself, which left them free to explore suggestions before committing to any of them. Offering both the formation button and drag and drop was welcomed, and the participants wanted to keep both rather than choose between them.

Their requests concentrated on the presentation and the depth of the assistant' s advice. All three preferred a short answer first, with the supporting detail available on request, and they wanted the main recommendation and its practical consequences, particularly prices and any points penalty, to be immediately identifiable. They asked for guidance to remain reachable while working: instructions that can be revisited during team construction, a separate page holding the general rules, and brief examples explaining terms such as differential preference and planning horizon. On the substance of the advice, they wanted stronger chip recommendations set against a longer planning horizon, better differential recommendations with clear reasons for choosing a less popular player, and more statistics about players' teams and their opponents so that a recommendation could be judged in its wider match context. They also asked for expected goals (xG) to be shown for the players, alongside the existing statistics. Compact comparisons with a clearly stated forecast period were preferred to long passages of text.

The participants did not always agree with the recommended option, but they consistently found the reasoning detailed and the supporting data convincing. They described the assistant as a source of advice rather than a replacement for their own judgement, and the final decision remained theirs in every case. They also valued clear explanations of uncertainty, particularly when the difference between two options was small.

8.2 Machine-Learning Evaluation

The machine-learning evaluation examines how accurately the system predicts player points, participation and minutes. These predictions support the recommendations presented by the application and its AI assistant. Their accuracy therefore needs to be evaluated before examining how the recommendation system uses them.

The prediction tasks are defined as follows:

- **Points**: The FPL points recorded for a player in a particular gameweek. The points model produces a **conditional prediction**, meaning the predicted points given that the player appears for at least 30 minutes. Conditional predictions are therefore evaluated against actual points only for appearances meeting this threshold.
- **Participation**: The probability that a player will play **at least 30 minutes** in the target gameweek. In this evaluation, participation does not simply mean entering the match: an appearance shorter than 30 minutes is classified as non-qualifying.
- **Risk-adjusted points**: The conditional points prediction multiplied by the predicted probability of playing at least 30 minutes. For example, a conditional prediction of 5 points and a participation probability of 0.8 produce a risk-adjusted prediction of **4 points**. The current calculation does not include a separate contribution from appearances shorter than 30 minutes.
- **Minutes**: The predicted number of minutes a player will play during the target gameweek, compared with the recorded minutes.

The evaluation addresses three main questions:

1. Do the models produce more accurate predictions than simple methods based on previous performances?
2. Does the addition of historical player information improve the predictions?
3. Which feature groups contribute to the results, and how certain are the observed differences?

Two configurations were evaluated:

Reduced configuration: Uses **16 features calculated from previous player performances**. These include previous-round points; the three- and five-round points averages; the five-round standard deviation of points; the three- and five-round minutes averages; the five-round averages of the ICT index, threat, creativity and influence; the accumulated season totals for points, goals, assists, clean sheets and bonus; and the mean points per preceding gameweek.

Production configuration: Refers here to the **enriched configuration used in the historical evaluation**, containing **38 features**. It includes the same 16 performance-history features and adds archived player price, ownership, availability status, reported chance of playing, an indicator for missing availability information, official FPL form, position and team-strength information. These additional inputs were

taken from snapshots available before the relevant deadline. Unverified upcoming-fixture and opponent features were excluded, so this evaluated configuration is not an exact reproduction of every input used by the live application.

8.2.1 Historical Data and Preparation

The evaluation covers gameweeks 21-38 of the 2025-26 season. It contains **14,476 player-gameweek observations**, representing **829 players**. A player-gameweek observation contains the information available before a particular gameweek and the player' s recorded outcome in that gameweek. Of these observations, **4,065 involved appearances of at least 30 minutes**. This threshold defines participation for the classification model and determines which observations are used to evaluate conditional points predictions.

8.2.2 Evaluation Procedure and Baselines

The experiment used a **walk-forward evaluation**. For each target gameweek, the models were trained using earlier observations and then evaluated on that gameweek. The procedure was repeated from gameweek 21 to gameweek 38. This follows the order in which predictions would be made during a season. Information from a target gameweek was not used to train the model that predicted it.

The models were compared with **baselines**: simple prediction methods that provide a reference for judging whether machine learning adds value.

Πίνακας 8.9: Baselines used for each prediction task.

Prediction task	Baseline
Conditional points	Historical mean points in qualifying appearances of at least 30 minutes
Risk-adjusted points	Mean points over the previous five gameweeks
Participation	Proportion of the previous five gameweeks with at least 30 minutes
Minutes	Mean minutes over the previous five gameweeks
Minutes, additional comparison	Minutes played in the previous gameweek

Using two minutes baselines is useful because recent playing time can remain stable. A model may improve on a five-round average while still failing to improve on simply repeating the previous round.

8.2.3 Evaluation Metrics

Different metrics were used according to the prediction task.

Πίνακας 8.10: Evaluation metrics and observation groups for each prediction task.

Prediction task	Metrics	Observations evaluated
Conditional points	MAE and RMSE	Appearances of at least 30 minutes
Risk-adjusted points	MAE and RMSE	All evaluated player-gameweeks
Minutes	MAE and RMSE	All evaluated player-gameweeks
Participation probability	Brier score, ROC-AUC and calibration	All evaluated player-gameweeks, using the 30-minute threshold

Points and minutes: Mean Absolute Error (MAE)

MAE measures the average absolute difference between the prediction and the recorded outcome:

$$\text{MAE} = \frac{1}{N}\sum_{i=1}^{N}|\hat{y}_i - y_i|,$$

where N is the number of evaluated observations, $\hat{y}_i$ is the prediction and y_i is the recorded outcome.

The result is expressed in the original unit: **points for points predictions and minutes for minutes predictions**. For example, a points MAE of 2 means that predictions differ from the actual scores by two points on average, regardless of whether they are too high or too low. Lower values indicate smaller errors.

Points and minutes: Root Mean Squared Error (RMSE)

RMSE was also calculated:

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2}.$$

Unlike MAE, RMSE gives greater weight to large errors because the differences are squared before

averaging. It therefore helps reveal cases where a model makes relatively few but substantial mistakes. Like MAE, it is expressed in points or minutes, and lower values are better.

Participation probability: Brier score

The Brier score measures the squared difference between the predicted participation probability and the observed outcome:

$$\text{Brier} = \frac{1}{N}\sum_{i=1}^{N}(\hat{p}_i - z_i)^2,$$

where $\hat{p}_i$ is the predicted probability of playing at least 30 minutes. The observed label z_i equals **1** if the player reaches this threshold and **0** otherwise.

For example, predicting a probability of 0.8 produces a squared error of 0.04 if the player reaches 30 minutes, but 0.64 if the player does not. The metric therefore penalises confident predictions that turn out to be wrong. Lower Brier scores are better.

Participation probability: ROC-AUC

ROC-AUC evaluates whether qualifying appearances tend to receive higher probabilities than non-qualifying appearances.

To calculate it, the probability threshold used to classify a player as likely to reach 30 minutes is varied. At each threshold:

$$\text{TPR} = \frac{TP}{TP + FN}, \qquad \text{FPR} = \frac{FP}{FP + TN}.$$

The **true-positive rate (TPR)** is the proportion of qualifying appearances correctly identified. The **false-positive rate (FPR)** is the proportion of non-qualifying appearances incorrectly classified as qualifying.

Plotting TPR against FPR across thresholds produces the ROC curve. Its area is calculated using the trapezoidal rule:

$$\text{AUC} = \sum_{k=1}^{K-1} \frac{\text{TPR}_{k+1} + \text{TPR}_k}{2} \left(\text{FPR}_{k+1} - \text{FPR}_k\right).$$

The evaluation uses the predicted probabilities, rather than labels obtained from one fixed threshold. This calculation follows the implementation in scikit-learn.

An ROC-AUC of **1.0** indicates perfect separation in the evaluated sample, while **0.5** corresponds to random ranking. A high ROC-AUC does not mean that the probabilities themselves are accurate; it means that the model generally orders qualifying appearances above non-qualifying appearances.

Participation probability: Calibration

Calibration examines whether predicted probabilities agree with observed frequencies. Predictions are grouped into probability ranges, and the average predicted probability in each group is compared with the proportion of players who actually reached 30 minutes.

For example, among observations receiving probabilities close to 0.8, approximately 80% should meet the participation threshold if those probabilities are well calibrated. Calibration therefore complements ROC-AUC by examining the meaning of the probability values, rather than only their ranking.

8.2.4 Prediction Results

Table 8.11 reports pooled results across the evaluated observations. Each observation has equal weight. The conditional metric uses only qualifying appearances; the other metrics use all evaluated observations.

Πίνακας 8.11: Pooled prediction errors. Lower values are better.

Metric	Baseline	Reduced	Production
Conditional points MAE	2.6825	2.4566	**2.4442**
Risk-adjusted points MAE	1.0307	0.9405	**0.9025**
Participation Brier	0.10623	0.08697	**0.07812**
Minutes MAE: five-round baseline	13.7308	13.5647	**12.2938**
Minutes MAE: previous-round baseline	**11.9558**	13.5647	12.2938

The model reduced conditional-points MAE by about 9%, risk-adjusted-points MAE by about 12%, participation probability error by about 26%, and minutes error by about 10% relative to the five-gameweek baseline. However, for minutes, repeating the previous gameweek remained competitive and the model did not establish a clear advantage. The production model improved all four metrics compared with the reduced configuration.

Points prediction across gameweeks.

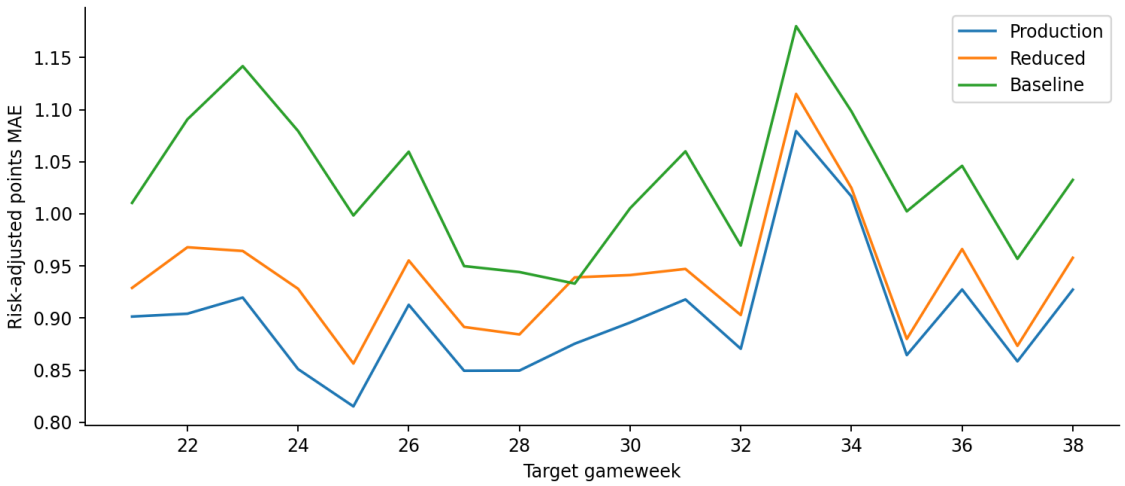


Figure 8.1. Risk-adjusted points MAE for the production configuration, reduced configuration and baseline.

The production model achieves the lowest risk-adjusted points MAE in all 18 evaluated gameweeks, showing that its improvement over the reduced configuration and baseline is consistent across the evaluation period rather than being driven by a small number of favourable rounds. Its error ranged from approximately **0.815 points in gameweek 25** to **1.079 points**. The magnitude of the improvement varies between gameweeks, reflecting the inherent variability of FPL outcomes. The production model' s risk-adjusted points RMSE was **1.895**, compared with an MAE of **0.903**. The larger RMSE indicates that some predictions had substantially greater errors than the average.

Minutes prediction across gameweeks.

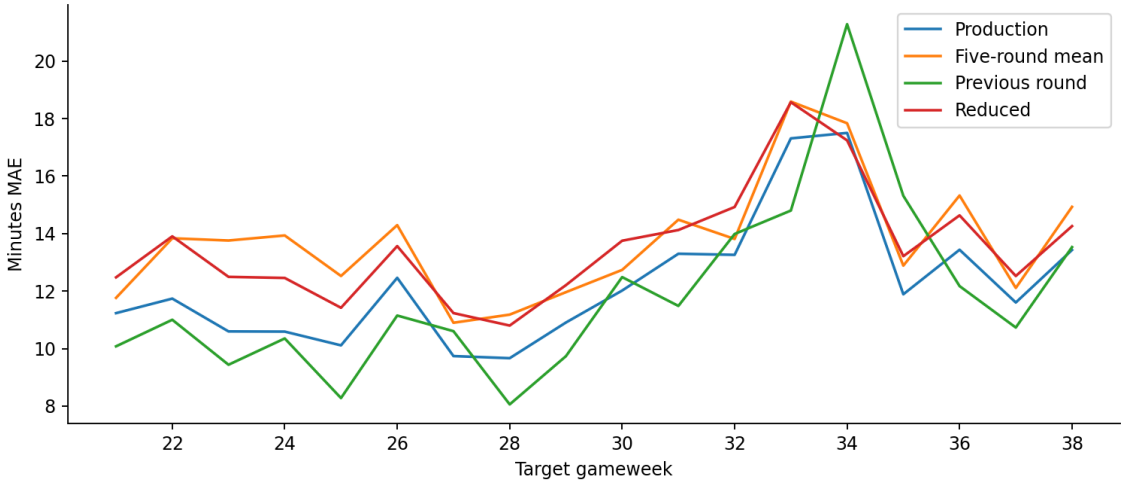


Figure 8.2. Minutes MAE for both model configurations and the two minutes baselines.

Minutes prediction presents a more mixed result. The production model generally improves over the five-gameweek average and benefits from the additional features, but the previous-gameweek baseline remains highly competitive and achieves a slightly lower pooled MAE. Consequently, the evaluation does not establish a clear advantage for ML over this strong simple baseline and these results show why a strong simple baseline is necessary.

8.2.5 Error Analysis

Average error provides a useful comparison, but it does not explain where the models succeed or fail. Weekly averages, individual errors, player positions and recorded playing time were therefore examined.

Actual and predicted average points.

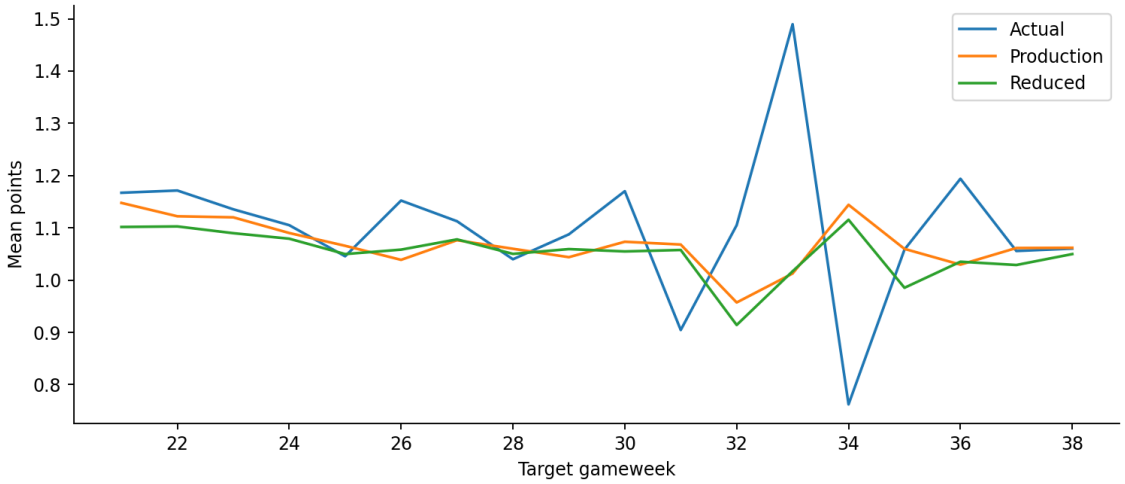


Figure 8.3. Actual and predicted mean points in each evaluation gameweek.

The predicted averages followed the general level of player points, but varied less than the actual outcomes. This was particularly visible in gameweeks 33 and 34. In gameweek 33, the production model underpredicted mean points by approximately **0.477 points per observation**. In gameweek 34, it overpredicted them by approximately **0.382 points**.

Across the complete evaluation, the mean signed error was only **-0.033 points**. However, a small average bias can hide larger errors in opposite directions across individual rounds. Agreement between averages should therefore not be interpreted as accurate prediction of every gameweek.

Distribution of individual errors.

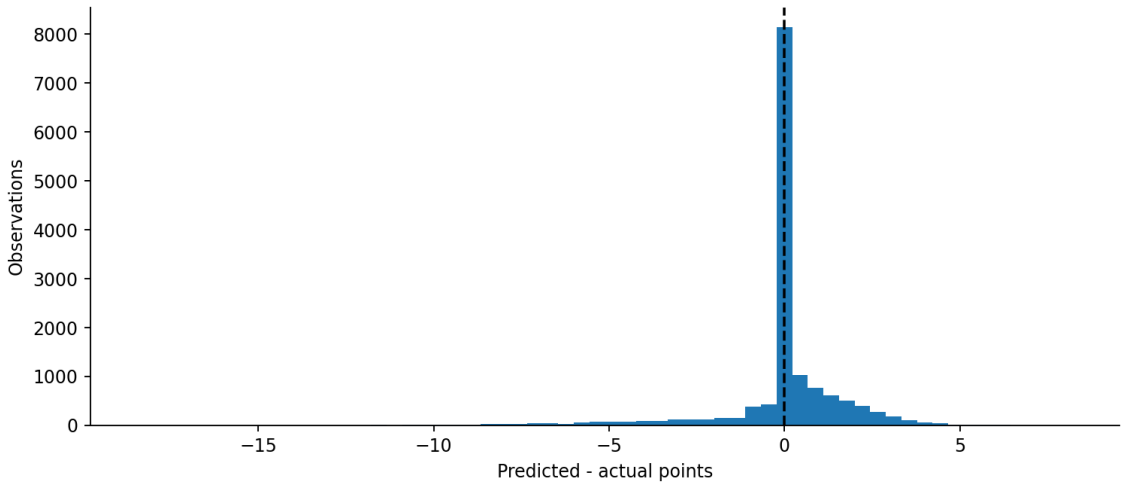


Figure 8.4. Distribution of predicted minus actual points for the production configuration.

The error distribution shows that many predictions are relatively close to the observed outcome, while a smaller number of large errors remain. These larger errors are important because exceptional FPL returns are difficult to anticipate from historical information alone and explain why RMSE is substantially higher than MAE.

Evaluation of high-scoring players.

The full-population evaluation produced a risk-adjusted points MAE of **0.903**, but **65.6% of the evaluated observations had zero actual points**. Zero-point outcomes and zero-minute observations are different groups: a player can appear without scoring any points. This raised a practical question: how accurately does the model predict players who score highly over the season?

To examine this, the **five highest-scoring goalkeepers, 15 defenders, 15 midfielders and 10 forwards** were selected, using their recorded total points in the 2025-26 season. Their existing predictions for gameweeks 21-38 were then evaluated. All 45 players had predictions for every gameweek, giving **810 player-gameweek observations**. Of these, **675 involved appearances of at least 30 minutes** and were also used to evaluate conditional points predictions. The models were not retrained specifically for this cohort.

Overall results for the selected players.

Πίνακας 8.12: Comparison of the complete evaluation population with the selected high-scoring players.

Measure	All evaluated players	Selected high-scoring players
Player-gameweek observations	14,476	810
Zero-point outcomes	65.6%	14.7%
Risk-adjusted points MAE	0.903	2.982
Risk-adjusted points baseline MAE	1.031	3.307
Conditional points MAE	2.444	3.127
Conditional baseline MAE	2.683	3.402
Predictions within two points	84.3%	39.6%

The results show that predicting high-scoring players was considerably harder than the full-population MAE suggested. Their average absolute error was **2.982 points**, and only **39.6%** of predictions were within two points of the actual outcome.

However, the model still improved on the five-round baseline, whose MAE was **3.307 points**. This corresponds to a **9.82% reduction in error**. Conditional points MAE also improved, falling from **3.402 to 3.127**, a reduction of **8.09%**. At the individual-player level, the model achieved lower risk-adjusted MAE than the baseline for **37 of the 45 players**.

These findings qualify the interpretation of the earlier results. The low full-population MAE partly reflects the large number of zero-point observations, but the model' s advantage over the baseline remains visible when evaluating high-scoring players separately.

Selected-player results by position.

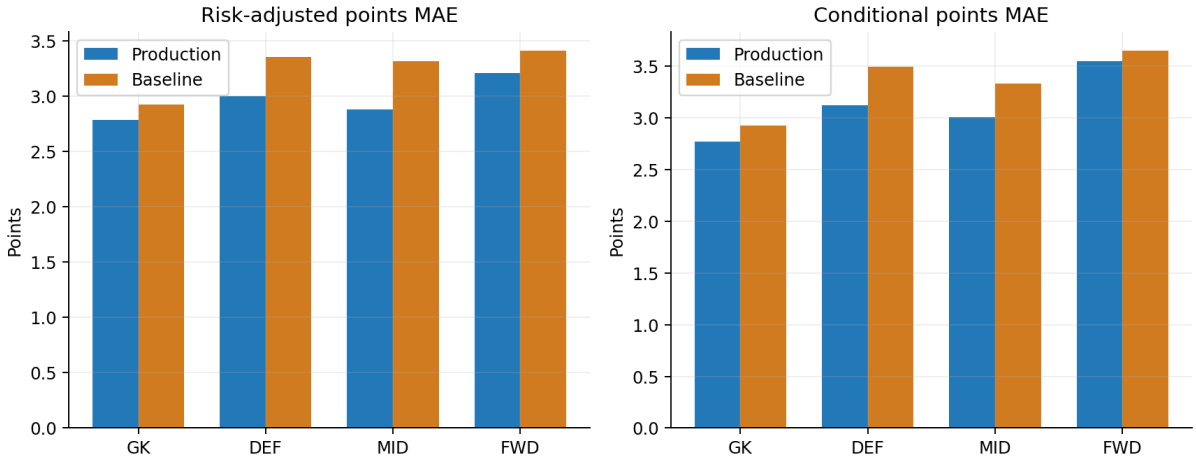


Figure 8.5. Risk-adjusted and conditional points MAE for the selected players in each position. Lower values indicate better performance.

Πίνακας 8.13: Prediction errors by position within the selected group. Reduction refers to risk-adjusted MAE relative to its baseline.

Position	Risk-adjusted MAE	Baseline MAE	Reduction	Conditional MAE	Conditional baseline
Goalkeepers	2.789	2.922	4.57%	2.772	2.923
Defenders	3.001	3.356	10.60%	3.120	3.495
Midfielders	2.879	3.315	13.14%	3.006	3.329
Forwards	3.207	3.414	6.06%	3.545	3.652

The model outperforms the corresponding baseline for both risk-adjusted and conditional points across all four positions. The largest relative improvement in risk-adjusted MAE occurs for midfielders (**13.14%**), followed by defenders (**10.60%**), while improvements are smaller for forwards and goalkeepers. Forwards have the highest absolute error, indicating greater prediction difficulty for this position within the selected evaluation population.

Examples of individual predictions.

The individual graphs help explain what these average errors mean. Each graph shows weekly risk-adjusted predictions, conditional predictions for qualifying appearances, and cumulative points across the evaluation period.

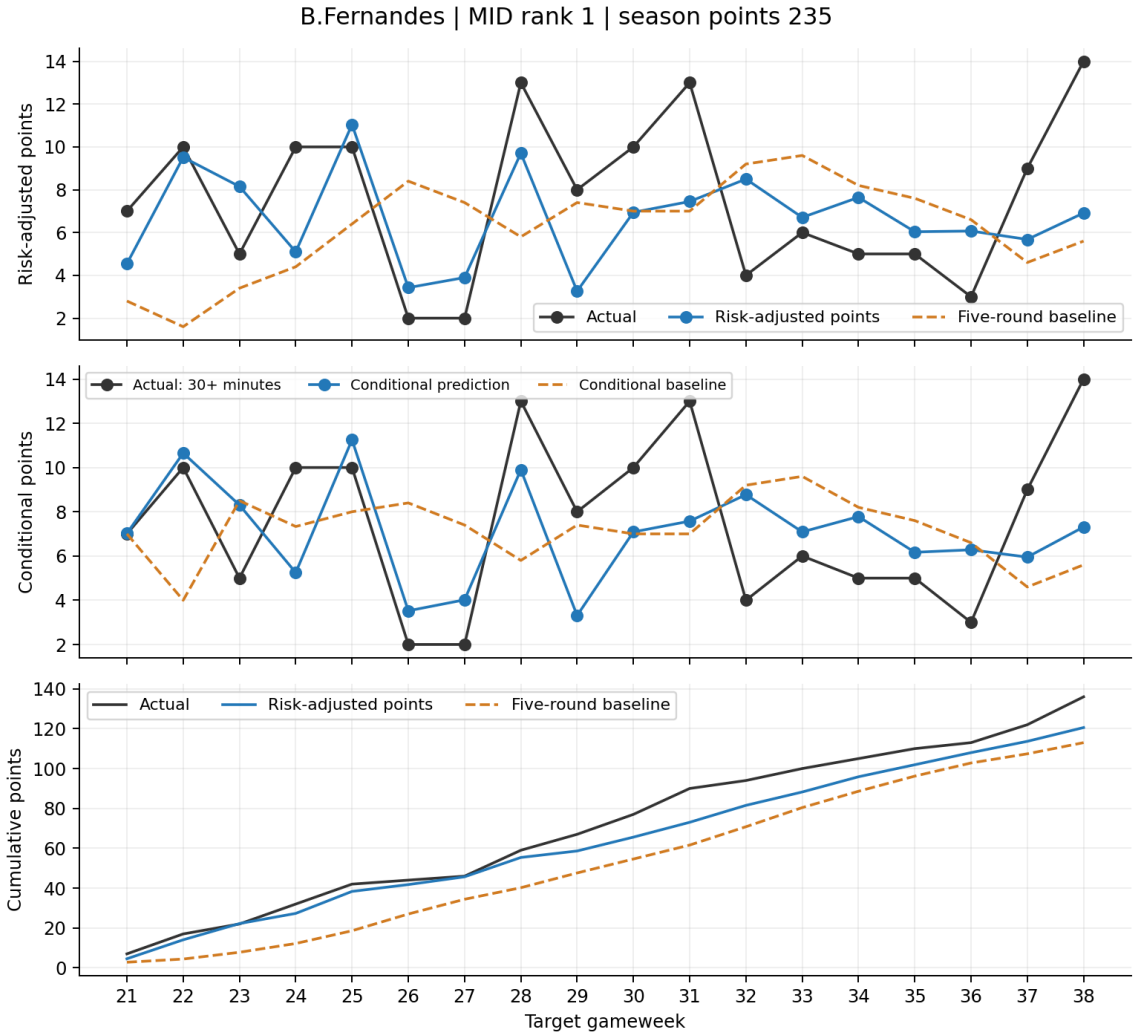


Figure 8.6. Actual and predicted points for Bruno Fernandes. Gaps in the conditional panel indicate gameweeks without an appearance of at least 30 minutes.

Bruno Fernandes provides an example of a clear improvement over the baseline. His risk-adjusted points MAE was approximately **3.020**, compared with **4.611** for the baseline. His actual total across the 18 gameweeks was **136 points**, while the model' s successive predictions summed to approximately **120.6 points**.

Nevertheless, the weekly graph shows that the model did not closely predict every return. Its predictions were smoother than the actual outcomes, and several high-scoring gameweeks were underestimated. A reasonably close cumulative total therefore does not mean that the timing of those points was predicted accurately.

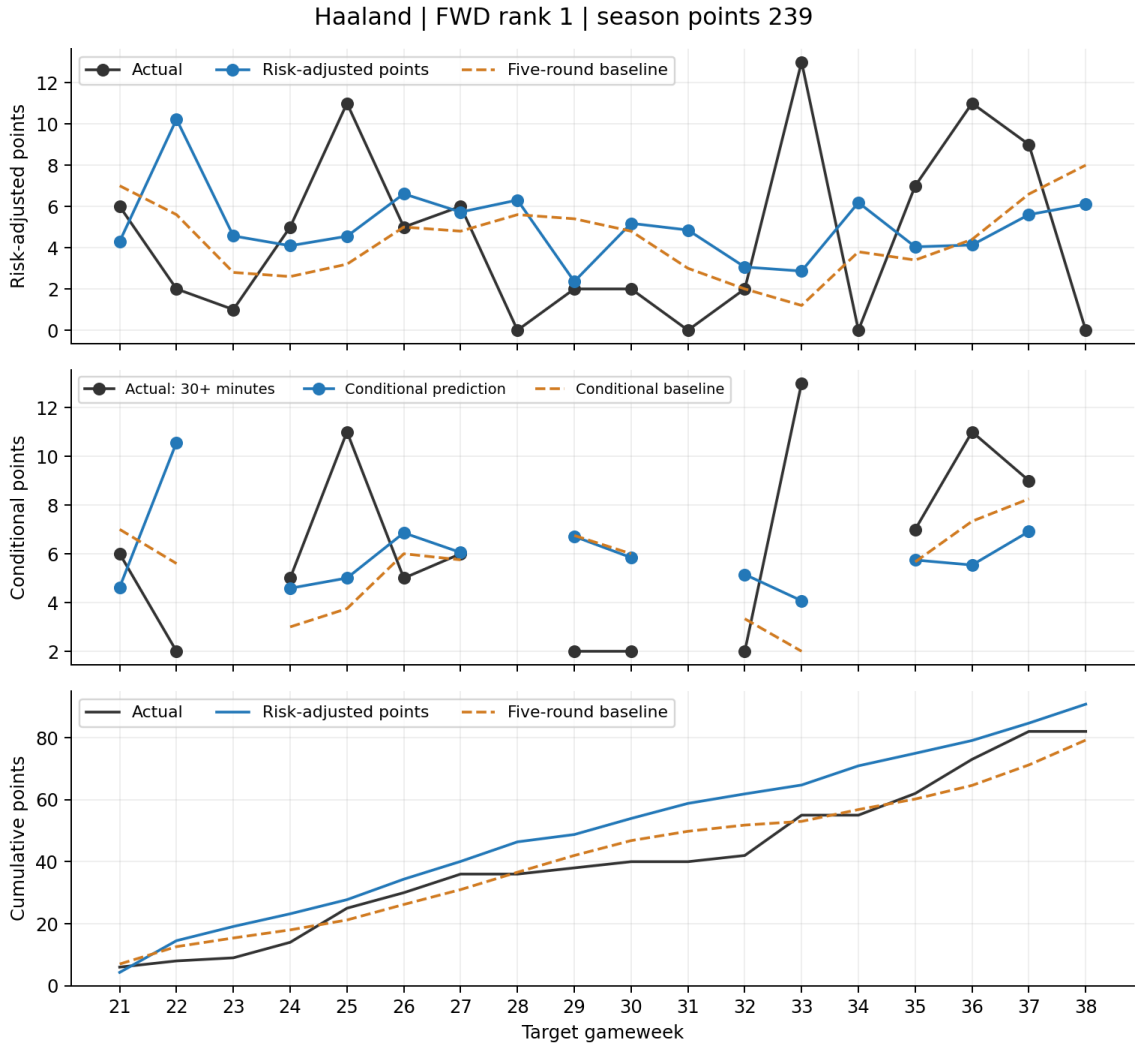


Figure 8.7. Actual and predicted points for Haaland, illustrating a case where the model did not improve on the baseline.

Haaland shows the opposite result. His risk-adjusted MAE was approximately **4.119**, compared with **3.822** for the baseline. Across the evaluation period, he scored **82 points**, while the model' s predictions summed to approximately **90.8**.

The graph shows both overprediction in low-return gameweeks and underprediction of some high returns. This illustrates a remaining limitation: the model can estimate a player' s general scoring level while still missing substantial changes from one gameweek to the next.

These two examples show why the evaluation should include individual results alongside group averages. The model improved predictions for most selected players, but there were also players for whom a simple historical average performed better.

Total points and weekly accuracy.

Across all 45 players, actual points totalled **3,369**. The model' s predictions summed to **3,120.6**, while the baseline predictions summed to **3,376.6**.

The baseline was therefore closer to the actual group total, despite having a higher player-gameweek MAE. These results are compatible: overpredictions for some observations can cancel underpredictions for others when predictions are added together.

For this reason, cumulative totals and MAE answer different questions. Cumulative totals show whether predictions are too high or too low over the period. MAE shows how far individual predictions are from their outcomes. Both are relevant, but a close total alone is insufficient evidence of accurate weekly advice. The cumulative predictions shown here are sums of successive one-gameweek forecasts, not a single forecast of the entire period made before gameweek 21.

Conclusions from the selected-player analysis.

This analysis gives a more realistic picture of the model' s accuracy for high-scoring players. The full-population MAE of **0.902 points** should not be presented as the typical error for these players. In the selected group, the error was closer to **three points per player per gameweek**.

At the same time, the results show that the model adds value compared with the chosen historical baselines. It reduced risk-adjusted error by approximately **9.8%**, improved the positional averages in all four groups, and outperformed the risk-adjusted-points baseline for **37 of 45 players**.

The main weakness is the prediction of individual weekly returns. Large scores and sudden changes remain difficult, and a close cumulative prediction can hide substantial weekly errors. The model is therefore better understood as an estimate to support decisions than as a precise prediction of what each player will score.

These conclusions apply to the selected players and the 18 evaluated gameweeks. Because selection used final-season outcomes, the analysis complements the main evaluation rather than replacing it. The next step is to evaluate whether the predictions help the recommendation system choose useful players under actual squad, budget and positional constraints.

Individual errors in the complete evaluation population.

Returning to the complete evaluation population, the following scatter plot shows the relationship between actual and predicted points across all evaluated observations.

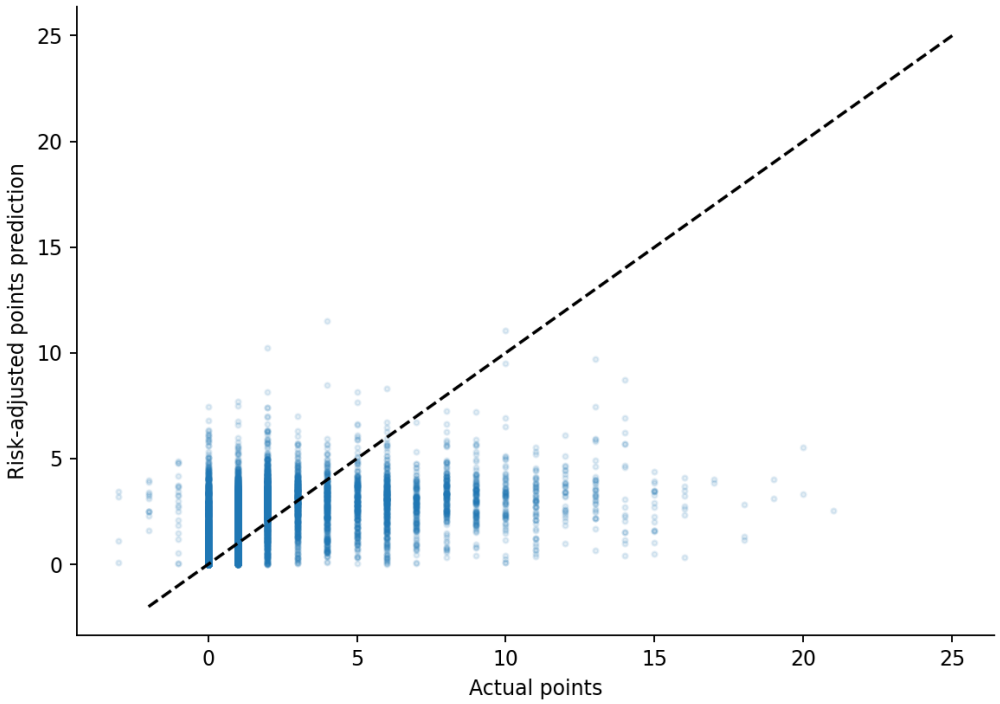


Figure 8.8. Actual versus predicted points. The dashed line represents a perfect prediction.

The comparison between actual and predicted points demonstrates that predictions are concentrated within a narrower range than the observed outcomes. The model performs more reliably around typical point values but has greater difficulty reproducing extreme returns, illustrating the tendency of the regression model to produce more conservative predictions.

Results by player position.

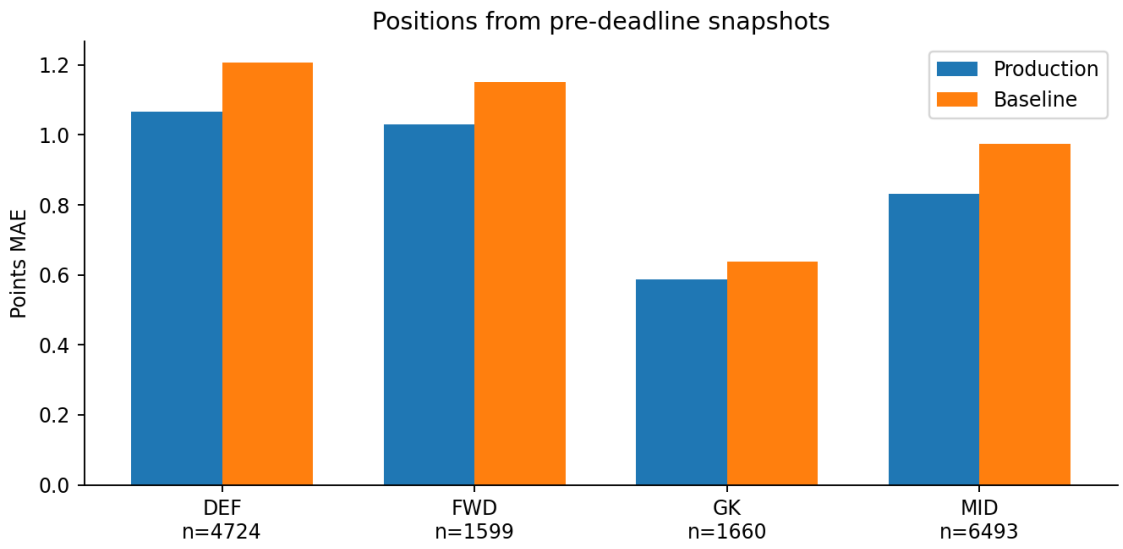


Figure 8.9. Risk-adjusted points MAE by position, using position information from historical pre-deadline captures.

Prediction accuracy differs between playing positions, indicating that the difficulty of the prediction task is not uniform across the player population. The comparison nevertheless allows the model' s performance to be examined beyond the single overall MAE and highlights where prediction errors are concentrated.

Results by recorded minutes.

Πίνακας 8.14: Points errors grouped by recorded playing time.

Recorded minutes	Observations	Production risk-adjusted MAE	Baseline points MAE
0	9,260	0.249	0.345
1-29	1,151	1.078	1.143
30-59	484	1.660	1.671
60-90	3,477	2.367	2.618

Errors increased considerably among observations with longer playing time. The model improved on the baseline in each group, but the low risk-adjusted MAE does not represent its accuracy equally well for all players.

8.2.6 Participation Prediction and Calibration

Participation is important because it directly influences the risk-adjusted points prediction. Both the model' s ability to distinguish qualifying appearances and the reliability of its probabilities were examined.

Πίνακας 8.15: Participation discrimination and calibration results.

Metric	Baseline	Reduced	Production
Brier score	0.10623	0.08697	0.07812
ROC-AUC	0.90624	0.94054	0.95279
Ten-bin calibration gap	0.06071	0.01244	0.01487

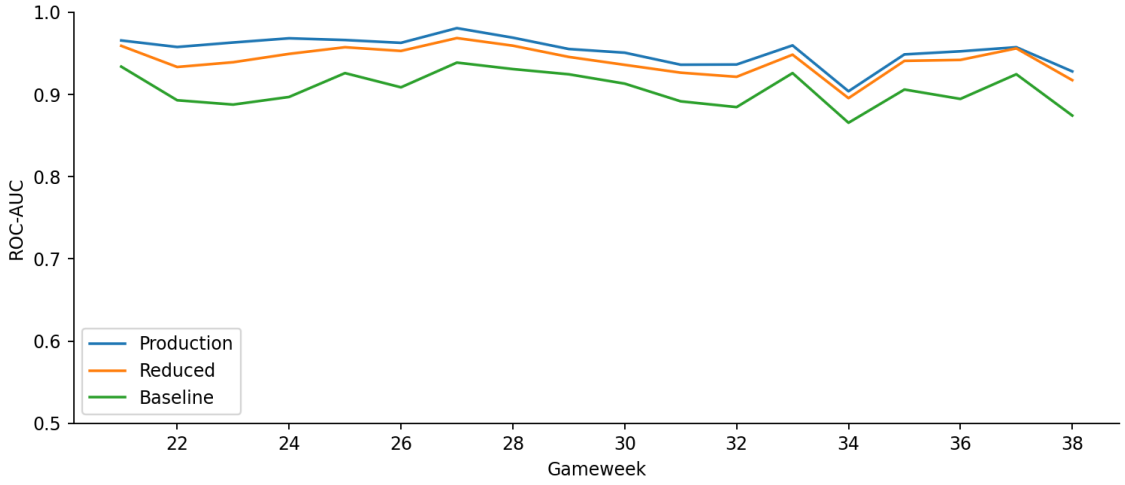


Figure 8.10. Participation ROC-AUC across the evaluation gameweeks.

The production model consistently demonstrates strong ability to distinguish qualifying from non-qualifying appearances, achieving a pooled ROC-AUC of **0.9528**. This is particularly important because participation probability directly contributes to the calculation of risk-adjusted points.

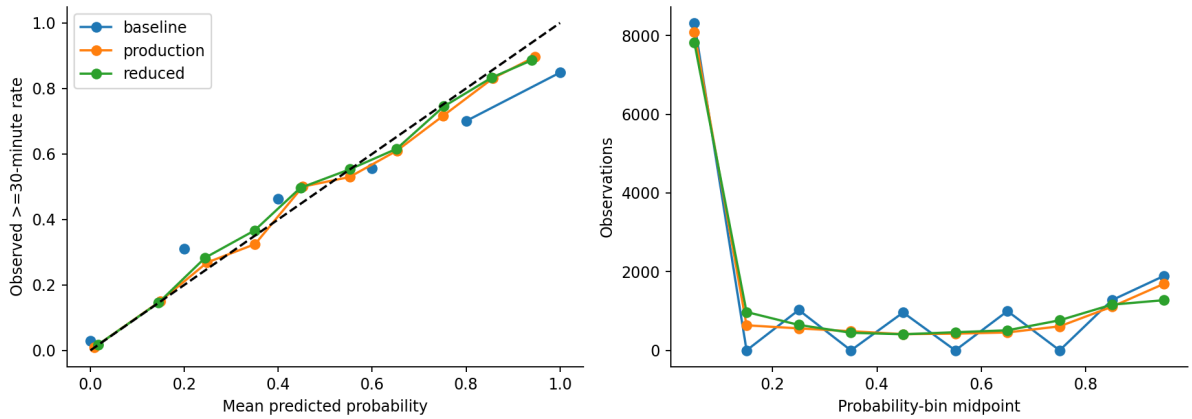


Figure 8.11. Predicted participation probabilities against observed frequencies (left), and observation counts in each probability bin (right).

The left panel compares predicted probabilities with observed participation rates. A well-calibrated model should lie close to the diagonal. The right panel shows how much data supports each probability range.

The calibration gap was calculated as the observation-weighted absolute difference between mean predicted probability and observed frequency in ten fixed bins of width 0.1. Empty bins carry zero weight. The production configuration had a calibration gap of **0.01487**, slightly higher than the reduced configuration' s **0.01244**. Thus, improved Brier score and ROC-AUC did not mean that every calibration measure improved.

In the production model' s highest probability bin, the average prediction was **94.64%**, while the observed participation rate was **89.66%**, across **1,692 observations**. The model was therefore somewhat overconfident in this range. The calibration gap depends on the selected bins and evaluation population. It is a descriptive measure, rather than evidence that probabilities are equally reliable for every player or situation. No recalibration was fitted on the evaluation folds.

8.2.7 Contribution of Feature Groups

To examine the contribution of each feature group, a **feature-group ablation** was performed. Each group was removed in turn, and all three models were retrained across the same 18 gameweeks. The full experiment contained the reduced configuration, production configuration and nine group removals. With three models trained per gameweek, this required **594 model fits**.

Table 8.16 gives each gameweek equal weight. Its results therefore differ slightly from the pooled values reported earlier.

Πίνακας 8.16: Equal-gameweek errors after removing each feature group. Conditional and risk-adjusted MAE refer to points.

Removed group	Conditional MAE	Risk-adjusted MAE	Brier	Minutes MAE
None: full production	2.44364	0.90212	0.07790	12.26604
Recent points	2.43599	0.90535	0.07906	12.42406
Recent minutes	2.44719	0.90642	0.07887	12.63383
ICT statistics	2.45270	0.90667	0.07817	12.29903
Season history	2.42993	0.90376	0.07796	12.19795
Market information	2.45509	0.91002	0.07813	12.33564
Availability	2.45034	0.93937	0.08693	13.52714
API form	2.44861	0.90431	0.07772	12.27103
Position	2.44872	0.90532	0.07788	12.24638
Team strength	2.44009	0.90047	0.07808	12.24900

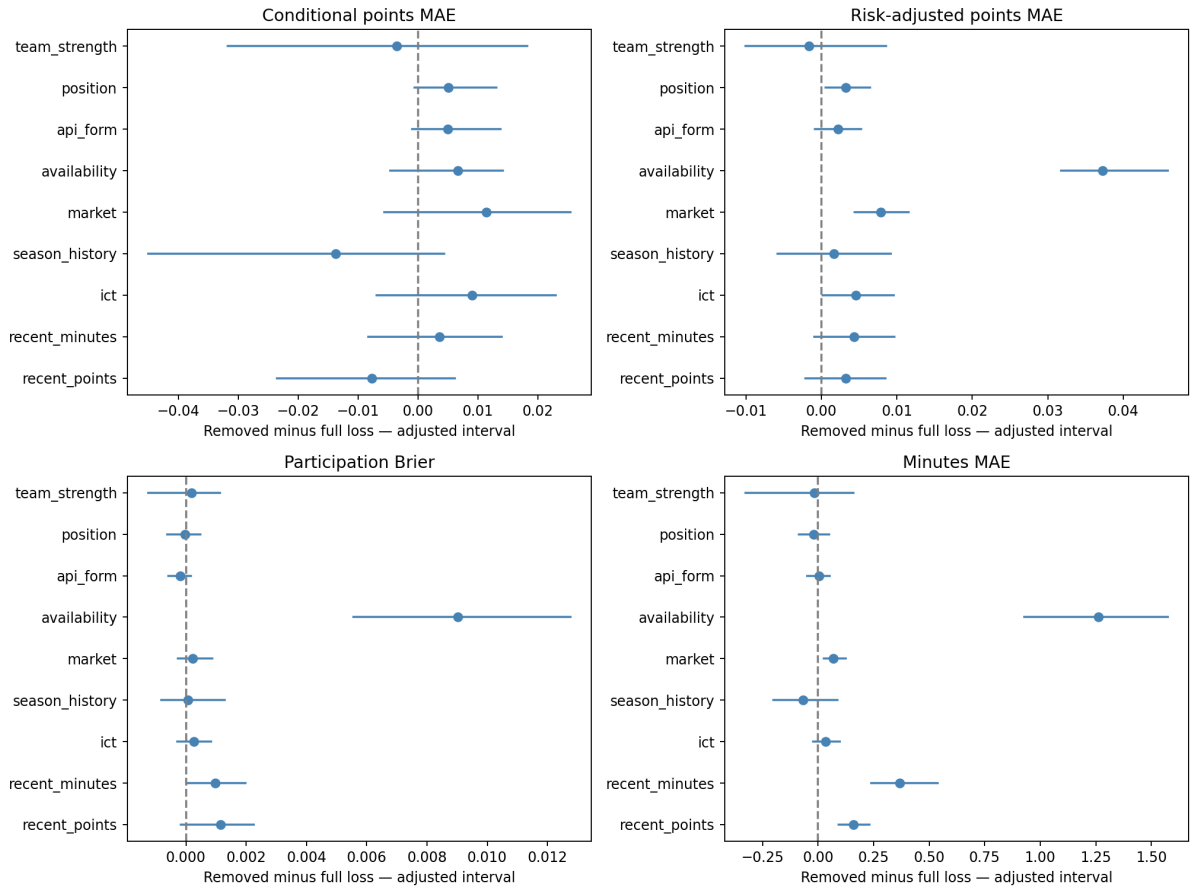


Figure 8.12. Change in loss after removing each feature group. Positive values indicate worse performance after removal. Intervals are approximate Bonferroni-adjusted block-bootstrap intervals across the 36 ablation comparisons, using three-gameweek blocks.

Availability information made the clearest contribution. Removing it increased risk-adjusted points MAE by **0.03725**, participation Brier score by **0.00903**, and minutes MAE by **1.26110**. All three differences remained above zero after the multiple-comparison adjustment.

Recent points helped minutes prediction, while recent minutes helped both participation and minutes. ICT statistics and position indicators contributed to risk-adjusted points. Market information helped risk-adjusted points and minutes. These differences remained above zero in the primary adjusted intervals.

The additional contributions of API form and team strength were less clear after adjustment. ICT statistics helped risk-adjusted points, but their contribution to the other tasks remained uncertain. Related features can contain overlapping information, allowing the remaining groups to compensate when one group is removed.

Some removals produced small improvements in individual metrics. For example, removing season history lowered conditional points MAE. These results were not used to select a new model on the same evaluation period. Doing so would require a further evaluation on separate data.

The ablation results describe the contribution of a group **when the other groups remain available**. They do not measure causal effects or establish the independent value of every feature within a group. Removing columns also changes the feature space available to the fixed training procedure.

8.2.8 Statistical Analysis of Model Differences

A lower average error does not show how stable an improvement is across gameweeks. Uncertainty intervals around the paired differences between methods were therefore calculated. For each gameweek:

$$d_g = L_{\text{production},g} - L_{\text{comparator},g},$$

where L is MAE for points and minutes or Brier score for participation. Negative differences favour the production configuration.

The intervals were estimated from **100,000 circular block-bootstrap resamples**, with a primary block length of three gameweeks and a fixed random seed. Whole gameweeks were sampled together, preserving the grouping of player observations within a round. Sampling consecutive blocks also retains some short-term dependence between rounds. Circular sampling allows a block to wrap from the final gameweek to the first. Block lengths of two and five were included as sensitivity checks.

The analysis used equal-gameweek mean differences. These values can differ slightly from subtracting the pooled errors because pooled metrics give greater weight to gameweeks containing more observations.

A Bonferroni adjustment was applied across the **nine main comparisons**, targeting approximate 95% family coverage. The **36 ablation comparisons** formed a separate family. The two families do not constitute one combined simultaneous guarantee. Table 8.17 and Table 8.18 report the adjusted intervals for the primary block length.

Πίνακας 8.17: Production-model loss minus baseline loss, with adjusted uncertainty intervals.

Metric	Baseline	Difference	Adjusted interval
Conditional points MAE	Qualifying-appearance mean	-0.23993	[-0.30089, -0.15109]

Metric	Baseline	Difference	Adjusted interval
Risk-adjusted points MAE	Five-round points mean	-0.12894	[-0.17824, -0.09645]
Participation Brier	Five-round frequency	-0.02826	[-0.03792, -0.02248]
Minutes MAE	Five-round minutes mean	-1.44798	[-2.31921, -0.89090]
Minutes MAE	Previous-round minutes	+0.35024	[-0.85507, +1.00509]

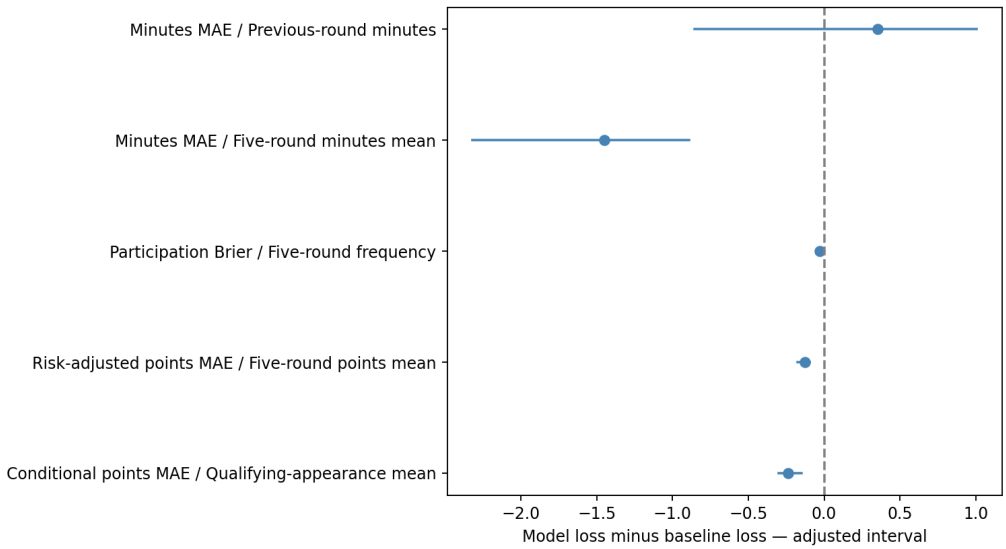


Figure 8.13. Production-model comparisons with simple baselines. Negative differences favour the model; intervals crossing zero leave the direction uncertain. The intervals are adjusted across all nine main comparisons and use three-gameweek blocks.

The intervals support an improvement over the selected points and participation baselines, and over the five-round minutes average. The comparison with previous-round minutes remains inconclusive. Its interval includes zero, meaning that the evaluation does not establish a clear difference between the two methods.

Πίνακας 8.18: Production-model loss minus reduced-model loss, with adjusted uncertainty intervals.

Metric	Difference	Adjusted interval
Conditional points MAE	-0.01288	[-0.03431, +0.00541]
Risk-adjusted points MAE	-0.03818	[-0.05179, -0.02510]
Participation Brier	-0.00892	[-0.01270, -0.00581]
Minutes MAE	-1.27781	[-1.64619, -0.93923]

The improvements in risk-adjusted points, participation and minutes remained below zero after adjustment. The much smaller improvement in conditional points did not.

These intervals describe uncertainty across the evaluated gameweeks, conditional on the fitted predictions. Models were not retrained inside the bootstrap, so the intervals do not cover variation from training seeds or model selection. With only **18 gameweeks**, they should be interpreted as approximate. The method also assumes enough temporal stability for resampling these blocks to be informative; dependencies beyond the block length and the circular boundary remain limitations.

8.2.9 Conclusion

Overall, the evaluation shows that the machine-learning pipeline provides measurable but task-dependent improvements over simple historical baselines. The strongest results are observed for participation prediction and risk-adjusted points, while conditional-points prediction achieves a smaller improvement and remains affected by the high variability of individual FPL returns. Performance is substantially weaker for high-scoring players than the full-population MAE initially suggests, although the model retains an advantage over the baseline within this more difficult group. Minutes prediction remains the main limitation: the production model improves over a five-gameweek average but does not establish a clear advantage over simply using the previous gameweek' s minutes. The ablation and statistical analyses further show that the observed improvements are not equally attributable to all feature groups and that availability-related information provides the clearest contribution. Overall, the results do not suggest that individual FPL outcomes can be predicted precisely; rather, they indicate that the proposed pipeline extracts additional predictive information beyond simple historical heuristics and provides structured estimates that can support the application' s downstream recommendation system.

8.3 Scoring and Recommendation System Evaluation

The machine-learning evaluation focused on the accuracy of the player forecasts, while this section examines how the application uses those forecasts to rank players, select captains, recommend transfers and build squads. The evaluation combines controlled tests, which examine how the scoring functions respond to changes in predictions and manager preferences, with historical experiments that compare the resulting recommendations against simpler selection methods. Squad construction, transfers across successive gameweeks and chip-related behaviour are evaluated separately, since each involves different decisions and constraints.

This approach examines both whether the system works as intended and whether its decisions lead to useful results, as a correctly calculated score does not necessarily produce a better recommendation, just as a squad that meets every budget and position requirement does not necessarily earn more points.

8.3.1 Evaluation of the Scoring Functions

The recommendation score combines player forecasts with information about availability, prediction confidence and the manager' s preferences, allowing the application to rank players according to their suitability for a particular decision. Its value expresses a preference between candidates rather than an estimate of the FPL points they will earn, so comparing it directly with actual points using MAE would not provide a meaningful measure of accuracy.

The evaluation therefore examined whether the scoring functions behaved as intended when their inputs changed. For example, increasing a player' s predicted points should not lower their score when everything else remains unchanged, while a cautious manager profile should apply a stronger reduction for uncertain participation than an aggressive profile.

Evaluation procedure. The evaluation used automated test scripts that called the application' s scoring functions directly with predefined inputs, without passing through the interface or the chatbot. Each test supplied a combination of forecasts, player information and manager preferences, calculated the resulting score and checked it against the expected behaviour. Where possible, only one input was changed at a time, allowing any difference in the output to be attributed to that change.

The evaluation covered five aspects:

Πίνακας 8.19: Procedure and results of the controlled scoring evaluation.

Component	How it was evaluated	Result
Manager-profile mapping	All **59,049 valid combinations** of questionnaire answers were passed through the profile-to-weight calculation, checking that each combination produced usable weights and scores and that the planning-horizon weights summed to one.	Every combination produced finite numerical values, and the horizon weights summed to one.
Participation probability	Participation probability was increased from **0 to 1 in steps of 0.01**, while the remaining inputs were held fixed along each comparison. This was repeated across **576 forecast, confidence and sixty-minute-factor combinations** under cautious, balanced and aggressive profiles.	All **174,528 scores** were finite, and increasing participation probability never reduced the score. For matching inputs, cautious scores were no higher than balanced scores, which were no higher than aggressive scores.
Individual point forecasts	The one-, three- and five-gameweek forecasts were varied individually across **303 controlled changes**, while the other inputs remained fixed.	Increasing a forecast never reduced the recommendation score within the tested grid.
Team and opponent context	**1,200 combinations** of team and opponent inputs were supplied to the context-adjustment calculation, checking whether the resulting multiplier remained within its intended limits.	Every tested multiplier remained between **0.70 and 1.35**.

Component	How it was evaluated	Result
Repeatability	The same inputs were submitted repeatedly, checking both the returned results and whether the functions modified the supplied input arrays.	Repeated calls produced identical outputs, and the input arrays remained unchanged.

These checks examined different parts of the calculation: the profile tests covered the conversion of preferences into weights, the forecast and participation tests examined the direction of score changes, and the context and repeatability tests checked numerical limits and consistent execution. The following subsections explain the confidence, risk and planning-horizon experiments in greater detail, using graphs to show how the scores and rankings changed across the tested inputs.

The results support the numerical consistency of the scoring functions within the evaluated cases, but they do not establish that the chosen weights produce the best recommendations. That question is examined separately through the historical comparisons, where players selected by the application are evaluated using their actual FPL returns.

8.3.1.1 Effect of Confidence and Risk Preference Across Multiple Forecasts

This experiment used automated scoring results to examine whether confidence, participation probability and manager risk preference affect the recommendation score in the intended direction across multiple forecasts. The expected behaviour was that increasing participation probability or confidence would not reduce the score, while a cautious profile would apply a stronger reduction for uncertainty than an aggressive profile. These checks evaluate the behaviour of the calculation rather than whether a particular preference produces better actual FPL returns.

Constructing the forecast combinations. An automated test script created **64 synthetic forecast combinations**, each containing predicted points over one, three and five gameweeks. The forecasts were constructed as follows:

Forecast for the first gameweek:

$$P_1 \in \{2, 4, 6, 8\}$$

Total forecast over three gameweeks:

$$P_3 = P_1 + A, \qquad A \in \{0, 4, 8, 12\}$$

Here, A represents the additional points predicted for gameweeks two and three.

Total forecast over five gameweeks:

$$P_5 = P_3 + B, \qquad B \in \{0, 4, 8, 12\}$$

Here, B represents the additional points predicted for gameweeks four and five.

Total number of forecast combinations:

$$\underbrace{4}_{\text{choices for } P_1} \times \underbrace{4}_{\text{choices for } A} \times \underbrace{4}_{\text{choices for } B} = 64.$$

The three- and five-gameweek forecasts are cumulative totals over their respective periods. These combinations covered different patterns, including stronger immediate prospects, stronger later prospects and relatively low forecasts throughout the period, allowing the scoring function to be examined across multiple cases. The inputs were deliberately controlled and did not represent observed player performances.

Varying confidence, minutes and participation. Each trajectory was combined with three confidence levels and three sixty-minute factors (0.3, 0.6 and 0.9):

$$64 \times 3 \times 3 = 576 \text{ input settings.}$$

For each setting, participation probability increased from 0 to 1 in steps of 0.01, giving 101 probability values. The calculation was repeated for cautious, balanced and aggressive profiles:

$$576 \times 101 \times 3 = 174{,}528 \text{ scores.}$$

Along each probability curve, the forecasts, confidence level and sixty-minute factor remained unchanged. This allowed the experiment to isolate how participation probability affected the scoring adjustment.

Expected behaviour. The script checked whether increasing participation probability left the score unchanged or increased it:

$$p_2 > p_1 \quad \Rightarrow \quad S(p_2) \geq S(p_1),$$

and whether matching inputs produced scores in the following order:

$$S_{\text{Cautious}} \leq S_{\text{Balanced}} \leq S_{\text{Aggressive}}.$$

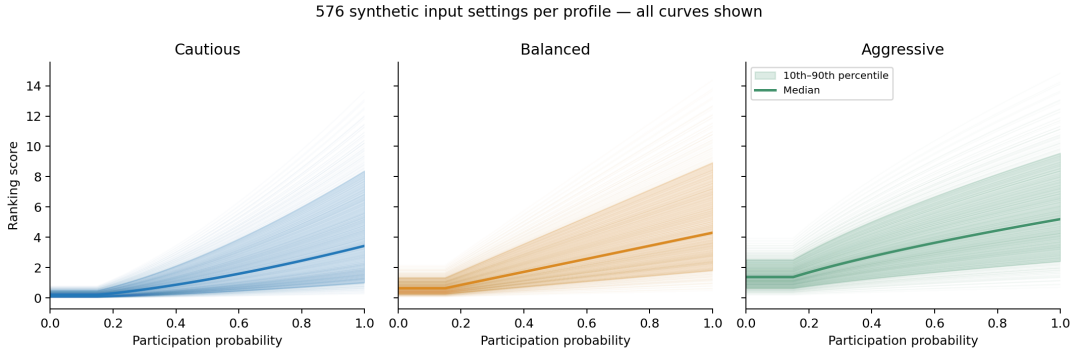


Figure 8.14. Risk sensitivity across 576 synthetic input settings per profile. Thin lines show individual settings, the thick line shows the median and the shaded region shows the 10th–90th percentile spread of cases, not a confidence interval.

Reading the figure. Each panel represents one risk preference, and each thin line follows one of the 576 input settings as participation probability increases. The thick line shows the median score, while the shaded region contains the middle 80% of scores. This region describes variation between the controlled cases; it is not a confidence interval.

Results. All calculated scores were finite, and increasing participation probability did not reduce the score within the tested grid. For matching inputs, cautious scores did not exceed balanced scores, and balanced scores did not exceed aggressive scores.

The flat section below probability 0.15 occurs because the implementation uses a minimum probability of 0.15 in this adjustment:

$$p_{\text{used}} = \max(p, 0.15).$$

Consequently, reducing participation probability below this value does not further change this part of the score. In the complete prediction pipeline, however, participation probability can also affect the risk-adjusted points forecast itself, so the flat section does not imply that all players below this threshold receive identical final forecasts.

The experiment confirms that the risk adjustment behaves consistently across the tested inputs, although it does not establish how frequently these combinations occur among real players or whether the selected adjustment produces the best historical recommendations.

Separate comparison of confidence levels. To make the confidence effect visible, a further analysis

grouped the saved scores into matching low-, medium- and high-confidence results. Within each group, the forecasts, participation probability, sixty-minute factor and risk preference were identical, so that only confidence changed. This reused the existing scoring outputs rather than introducing new player data or retraining a model.

For each risk profile, the comparison covered:

$$64 \times 3 \times 101 = 19{,}392 \text{ matched confidence groups.}$$

Across the three risk profiles, this gave **58,176 groups**, each containing three scores. The script checked both the low-to-medium and medium-to-high change, giving **116,352 matched comparisons**. The expected ordering was:

$$S_{\text{Low}} \leq S_{\text{Medium}} \leq S_{\text{High}}.$$

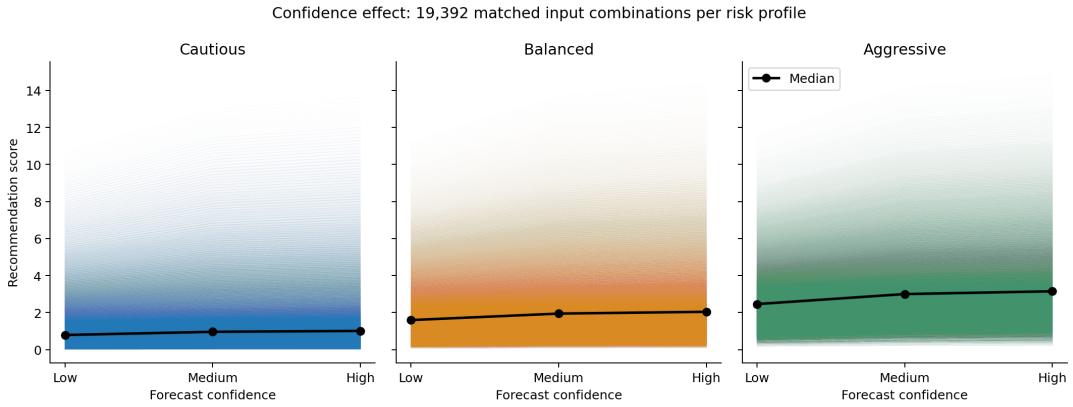


Figure 8.15. Effect of forecast confidence across 19,392 matched input combinations per risk profile. Each thin line connects low-, medium- and high-confidence scores with all other inputs unchanged; black lines show the medians. Many lines overlap.

Confidence results. All **116,352 comparisons showed a strict increase**, with no decreases or ties in the saved scores. The graph therefore shows that the confidence adjustment worked in the expected direction across the complete tested grid, rather than only for one example. The spread between lines reflects different forecast, participation and playing-time inputs; it is not a confidence interval. This result establishes how the scoring function uses confidence labels, but does not establish that those labels accurately represent prediction uncertainty or that the size of the adjustment is optimal.

8.3.1.2 Effect of the Planning Horizon Across Multiple Forecasts

The manager' s planning horizon expresses whether recommendations should give greater importance to the next gameweek or to returns over a longer period. This experiment examined whether changing that preference actually changes the order of recommended players, particularly when one candidate has a stronger immediate forecast and another has better prospects over the following gameweeks.

Purpose of the experiment

The aim was to check whether the planning-horizon setting affects selection, rather than simply increasing or decreasing every player' s score without changing the ranking. If the preference has the intended effect, some candidates should become more attractive under a short horizon, while others should move higher under a long horizon.

This is a controlled test of how the scoring function uses the preference, rather than a test of whether short-term or long-term recommendations earn more actual FPL points.

How the comparison was performed

An automated script used the same **64 synthetic forecast combinations** introduced in the preceding subsection. Each combination represented a hypothetical candidate with forecasts for one, three and five gameweeks, covering different patterns of immediate and later returns.

The script scored and ranked all 64 candidates three times, changing only the planning-horizon setting:

- **Short**: greater emphasis on immediate returns.
- **Balanced**: an intermediate weighting of the forecast periods.
- **Long**: greater emphasis on returns over the longer period.

The remaining inputs were identical for every candidate and every run:

Input	Fixed value
Participation probability	0.8
Sixty-minute factor	0.7
Confidence	High
Team-context and ownership adjustments	Omitted

Keeping these inputs fixed meant that any change between the three rankings came from the planning-horizon setting, rather than differences in availability, confidence or other adjustments.

For each run, the script ordered candidates from the highest recommendation score to the lowest, with

rank 1 representing the preferred candidate. It then examined three aspects of the results:

- **Individual ranks**: how many candidates changed position between the short and long settings.
- **Top-ten membership**: how many candidates appeared in both top-ten lists.
- **Pairwise preferences**: how often the preferred candidate changed when comparing two candidates directly.

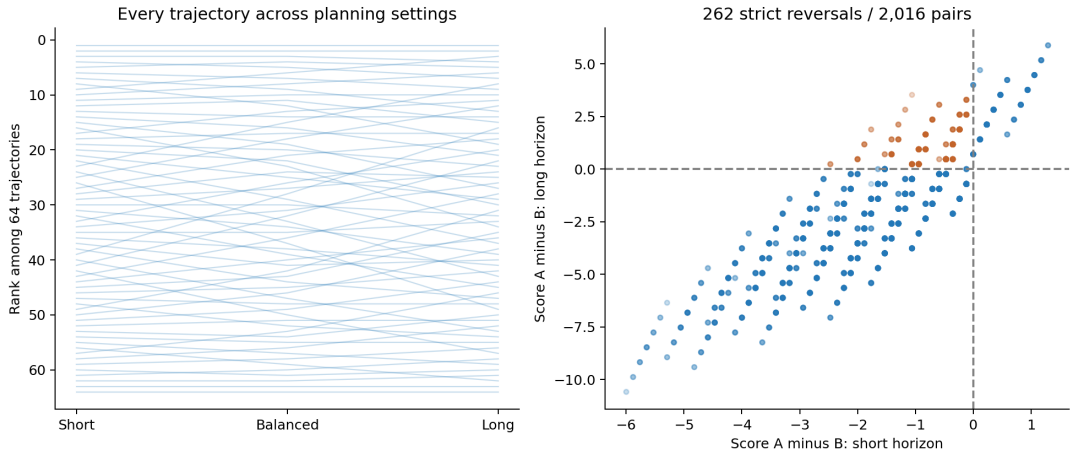


Figure 8.16. Effect of the planning horizon on 64 synthetic candidates. The left panel follows each candidate' s rank across short, balanced and long settings. The right panel compares every distinct pair under the short and long settings, with orange points indicating that the preferred candidate changed. Several pairs may occupy the same point.

Changes in individual ranks and the top ten

The left panel shows one line for each candidate, connecting its position under the three planning settings. A line moving towards rank 1 means that the candidate becomes more favourable, while movement away from rank 1 means that other candidates become preferable.

Changing from the short to the long setting changed the rank of **58 out of 64 candidates**. However, **eight candidates appeared in both top-ten lists**, meaning that only two members of the top ten were replaced, even though the ordering changed more widely.

These results show that planning preferences affected the ranking without replacing every leading candidate. A candidate with strong forecasts across several periods could remain highly ranked under both settings, while candidates with different immediate and later prospects could move relative to one another.

Checking whether the preferred candidate changed

A change in rank does not show exactly which comparisons changed, so the script also examined every distinct pair of candidates. With 64 candidates, the number of pairs was:

$$\text{Number of pairs} = \frac{64 \times 63}{2} = 2{,}016.$$

For each pair, labelled A and B, the script calculated the difference between their recommendation scores.

Score difference under the short planning horizon:

$$D_{\text{short}} = S_{\text{short}}(A) - S_{\text{short}}(B)$$

Score difference under the long planning horizon:

$$D_{\text{long}} = S_{\text{long}}(A) - S_{\text{long}}(B).$$

In both calculations:

- A **positive difference** means that candidate A has the higher score.
- A **negative difference** means that candidate B has the higher score.
- A **zero difference** means that the candidates are tied.

A **strict preference reversal** occurs when the differences have opposite signs, meaning that one candidate is preferred under the short setting and the other under the long setting:

$$D_{\text{short}} \times D_{\text{long}} < 0.$$

In the right panel, the horizontal axis shows the short-horizon difference and the vertical axis shows the long-horizon difference. Orange points identify pairs for which the preferred candidate changed; points on either zero line include a tie and are not counted as strict reversals.

Results and their meaning

Measure	Result	Interpretation
Candidates whose rank changed	**58 of 64**	Changing the horizon affected most positions in the ranking.
Candidates shared by both top-ten lists	**8 of 10**	Most leading candidates remained in the top ten, although their order could change.

Measure	Result	Interpretation
Pairs with a strict preference reversal	**262 of 2,016**	In these comparisons, changing the horizon changed which candidate was preferred.
Pairs tied under at least one endpoint setting	**42 of 2,016**	These pairs were not counted as strict reversals.

The experiment therefore confirms that the planning-horizon preference can influence which candidates are recommended across a range of forecast combinations. Its effect is more than a uniform change in score, because both candidate positions and direct preferences between candidates changed.

The results do not establish that the chosen weights are optimal or that one planning horizon produces better actual returns. The one-, three- and five-gameweek forecasts also cover different lengths of time, so the score combines quantities with different numerical scales. The historical recommendation experiments examine the separate question of how the selected players perform in practice.

8.3.2 Historical Player Rankings and Captain Selection

The controlled experiments examined whether the scoring functions responded consistently to changes in their inputs, while this experiment examined whether the resulting recommendations selected players who performed well in actual gameweeks. The evaluation covered **GW21–38 of the 2025–26 season**, using saved forecasts without retraining the prediction models.

Purpose of the experiment

The aim was to examine whether the application' s scoring adjustments provide a benefit over simpler selection methods. A recommendation score can behave correctly in controlled tests without necessarily identifying better-performing players, so its practical value was examined through two tasks:

- **Player ranking**: whether the ten highest-ranked players earned more actual points than those selected by the comparison methods.
- **Captain selection**: whether the application selected a higher-scoring captain from the same starting XI.

The comparison therefore separated the value of the underlying forecasts from the additional effect of the application' s recommendation rules.

Methods being compared

Πίνακας 8.20: The three selection methods compared in the ranking and captaincy experiment.

Method	How players were selected	Purpose of the comparison
Application	Players were ranked using the application's recommendation score with a balanced manager profile, while captain selection used its captaincy function.	Evaluates the application's scoring adjustments and selection rules.
Risk-adjusted-points baseline	Players were selected directly according to their predicted points for the next gameweek.	Examines whether the application adds value beyond using the forecasts alone.
Recent-form baseline	Players were selected according to their average actual points over the preceding five rounds.	Provides a simple comparison based on recent performance, without using the prediction model.

How the evaluation was performed

For each of the 18 gameweeks, automated scripts supplied the saved forecasts and archived player information to the selection functions. Recommendations were calculated and saved before actual outcomes were joined to the results, so the points earned in the target gameweek were used only to evaluate the selections.

All methods used the same candidate pool. A player was eligible if the archived availability status indicated that the player was available and the average playing time over the preceding three rounds was at least 30 minutes. These conditions used information from before the target gameweek, rather than selecting players according to whether they subsequently played or scored points.

Historical fixture context could not be verified adequately for every deadline, so team-context adjustments were kept neutral. Where the application required longer forecasts, these were calculated from the one-gameweek risk-adjusted forecast:

$$P_3 = 3P_1, \qquad P_5 = 4.75P_1.$$

These values supplied the required forecast horizons but did not represent separate predictions of upcoming fixtures. Consequently, this experiment evaluates the recommendation methods under a common, fixture-neutral setup.

Evaluating the player rankings

Each method ranked the eligible candidates, after which the script selected its ten highest-ranked players and calculated their average actual points in that gameweek:

$$R_{m,g} = \frac{1}{10} \sum_{i \in T_{m,g}} y_{i,g},$$

where $T_{m,g}$ is the top-ten list selected by method m in gameweek g, and $y_{i,g}$ is the actual number of points earned by player i.

The final result was the average of these gameweek values across GW21–38, giving every gameweek equal weight. A higher value means that the method selected a top-ten list whose players earned more points on average.

Evaluating captain selection

Captain selection was evaluated using **20 valid synthetic squads per gameweek**, giving:

$$20 \times 18 = 360 \text{ squad-gameweek cases.}$$

For each case, all three methods selected a captain from the **same starting XI**, ensuring that differences came from captain selection rather than access to different players. The selected captain' s actual points were recorded before doubling, averaged across the 20 squads in each round and then averaged across the 18 gameweeks.

The synthetic squads provided multiple selection situations, but they were not real managers' squads and could share many of the same players.

Results

Πίνακας 8.21: Average actual returns from player ranking and captain selection over GW21–38.

Measure	Application	Risk-adjusted-points baseline	Recent-form baseline
Mean points per top-ten selected player	4.528	**4.600**	3.939
Mean points per selected captain, before doubling	**4.461**	4.272	3.578

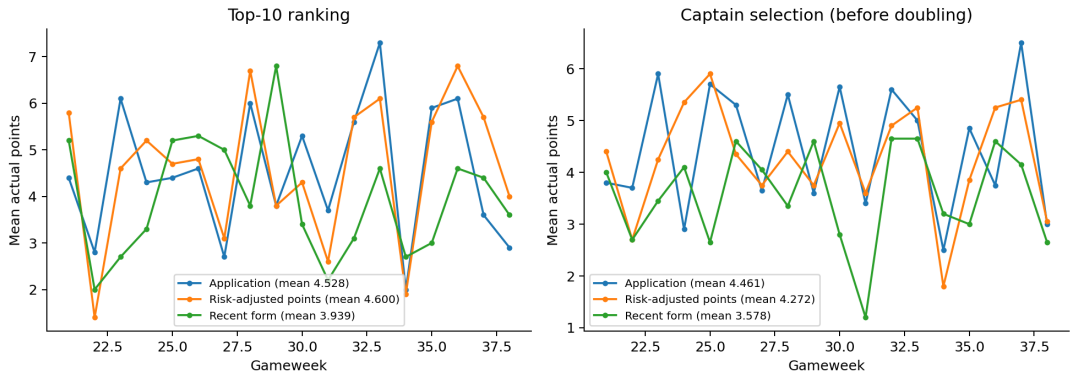


Figure 8.17. Actual returns in each of the 18 evaluated gameweeks. The left panel shows the mean points of each method's top-ten players, while the right panel shows captain points averaged across the 20 synthetic squads in each round. Period averages appear in the legends.

What the averages show

For player ranking, direct selection by risk-adjusted points produced the highest average, exceeding the application by approximately **0.072 points per selected player**. The application nevertheless exceeded recent form by approximately **0.589 points per selected player**.

For captain selection, the application produced the highest average, exceeding direct selection by risk-adjusted points by approximately **0.189 points** and recent form by approximately **0.883 points**, before captain points were doubled.

The graph shows that these advantages were not present in every gameweek, which makes it necessary to examine uncertainty rather than relying only on the overall averages.

Examining uncertainty in the differences

The statistical comparison used the difference between the application and each baseline within the same gameweek:

$$d_g = R_{\text{Application},g} - R_{\text{Baseline},g}.$$

For captaincy, the same calculation used each method's mean captain points for that round. A positive difference favours the application, while a negative difference favours the baseline.

The analysis generated **100,000 circular block-bootstrap resamples**, repeatedly sampling blocks of consecutive gameweeks and recalculating the average difference. Sampling blocks rather than individual player observations preserves some of the dependence between nearby gameweeks, although it cannot account for every form of dependence.

The main analysis used blocks of **three gameweeks**, with blocks of two and five examined as sensitivity checks. The intervals were adjusted for the **six planned comparisons** across ranking, captaincy and the two transfer-cost scenarios discussed in the following subsection.

Πίνακας 8.22: Average differences between the application and the baselines, with adjusted uncertainty intervals.

Comparison	Mean difference	Adjusted 95% interval
Top-10 ranking: application minus risk-adjusted points	−0.072	[−0.833, 0.561]
Top-10 ranking: application minus recent form	+0.589	[−0.522, 1.639]
Captaincy: application minus risk-adjusted points	+0.189	[−0.181, 0.494]
Captaincy: application minus recent form	+0.883	**[0.391, 1.414]**

Interpretation of the results

Only the captaincy comparison with recent form had an adjusted interval entirely above zero, providing evidence of an advantage for the application within this evaluation. The other intervals included zero, meaning that the observed average differences did not provide clear evidence that one method consistently outperformed the other; this does not establish that the methods are equivalent.

The results therefore suggest that the forecasts provide useful information beyond recent form, while the additional benefit of the application' s scoring adjustments is less clear. Direct use of risk-adjusted points remained a strong comparison method, achieving a slightly higher top-ten average, whereas the application achieved a higher captaincy average without establishing a clear advantage over that baseline.

These findings remain exploratory because the evaluation covers 18 gameweeks from one season, the synthetic squads share players and match outcomes, and the historical data had already been used during development. They describe performance under the tested conditions rather than a guaranteed improvement for every manager or future gameweek.

8.3.3 One-Gameweek Transfer Recommendations

This experiment examined whether the application's transfer scoring selected better-performing moves than a simpler method based directly on predicted points. The evaluation covered **GW21–38 of the 2025–26 season**, using the same **360 synthetic squad-gameweek cases** as the captain-selection experiment.

Purpose of the experiment

A transfer recommendation needs to identify both the player leaving the squad and the player replacing them, while considering whether the change is affordable and whether its expected benefit justifies any points penalty. The experiment therefore examined two questions:

- **Selection quality**: does ordering transfers by the application's personalised score produce higher actual gains than ordering them by predicted point gain?
- **Transfer cost**: how do the results change when a transfer costs four points rather than being free?

The comparison focused on the application's existing transfer shortlist, rather than searching for the best possible transfer among every eligible player.

Methods being compared

Both methods received the same feasible transfer pairs, where each pair consisted of one outgoing player and one incoming player.

Method	How a transfer was selected
Application ordering	Retained the order produced by the application's transfer-scoring function and selected the first candidate that satisfied the evaluation conditions.
Risk-adjusted-points ordering	Reordered the same candidates according to the incoming player's predicted points minus the outgoing player's predicted points for the next gameweek.

Using the same shortlist isolated the effect of **how the transfers were ordered**. It did not independently evaluate whether the application's shortlist contained every potentially useful transfer.

How the evaluation was performed

For each synthetic squad, an automated script called the application's transfer-recommendation function using the saved forecasts and archived player information. An additional checking step in the evaluation script then excluded any candidate that did not satisfy the following conditions:

- The incoming player could be purchased using the available bank balance and the outgoing player' s selling value.
- The incoming and outgoing players occupied the same position.
- The incoming player was not already in the squad.
- The resulting squad contained no more than three players from the same club.

For this one-gameweek experiment, players were treated as having been purchased at their current archived prices, so the outgoing player' s selling value equalled that price. Purchase-price changes across several gameweeks were examined separately in the persistent-transfer experiment.

Deciding whether to make a transfer

Each method was evaluated under two cost scenarios:

Scenario	Points deducted if a transfer was made
Free transfer	0
Transfer with a points penalty	4

Before selecting a move, the script calculated its predicted one-gameweek gain:

$$\text{Predicted gain} = \widehat{P}_{\text{incoming}} - \widehat{P}_{\text{outgoing}},$$

where the two forecasts are the players' risk-adjusted points for the next gameweek.

A transfer remained eligible only when its predicted gain exceeded the cost:

$$\text{Predicted gain} > \text{Transfer cost}.$$

The application then selected the first remaining candidate in its own ordering, while the baseline selected the candidate with the largest predicted gain. If no candidate satisfied the conditions, the method made **no transfer**.

This rule used predicted points available at the time of selection, rather than actual outcomes. It therefore allowed a recommended transfer to perform poorly afterwards, just as a reasonable decision can produce an unfavourable result in a real gameweek.

Measuring the actual benefit

After the decisions had been saved, the script joined the actual player outcomes and calculated the net gain from each selected transfer:

$$\text{Actual net gain} = P_{\text{incoming}} - P_{\text{outgoing}} - \text{Transfer cost}.$$

A positive value means that the incoming player earned enough additional points to cover the transfer cost, while a negative value means that keeping the outgoing player would have produced a better result for that comparison. A no-transfer decision received a gain of zero.

This measure compares the two players' points directly; it does not simulate the full squad' s starting lineup, substitutions or captaincy after the transfer.

Each of the 360 squad-gameweek cases was evaluated under both selection methods and both cost scenarios:

$$360 \times 2 \times 2 = 1{,}440 \text{ decisions}.$$

The results were averaged across the 20 squads in each gameweek and then across the 18 gameweeks, giving each round equal weight.

Results

Πίνακας 8.23: Mean actual net gain per decision over GW21–38, including a gain of zero when no transfer was selected.

Scenario	Application ordering	Risk-adjusted-points ordering
No transfer penalty	4.119	4.131
Four-point penalty	0.142	0.158

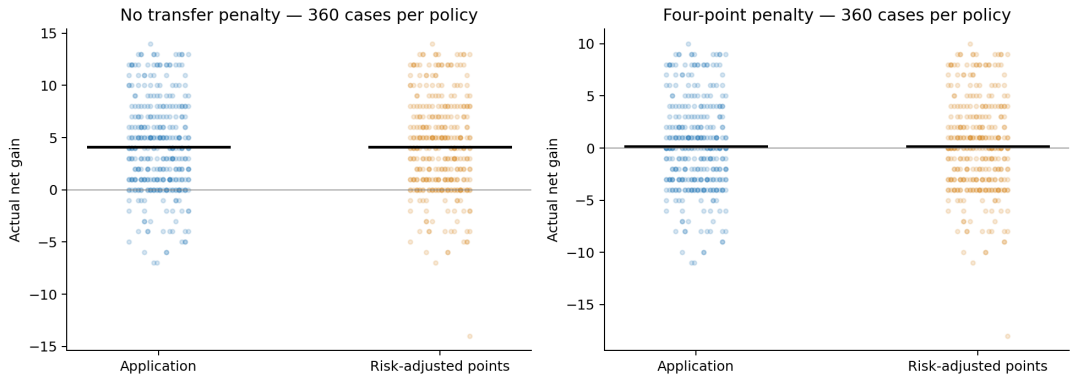


Figure 8.18. Actual net transfer gains across all 360 squad-gameweek cases per method. The left panel shows transfers without a points penalty, while the right panel shows the four-point-cost scenario. Each dot represents one case, and the black bars indicate the means; horizontal displacement separates overlapping points and has no numerical meaning.

What the results show

Without a transfer penalty, the application produced an average gain of **4.119 points**, compared with **4.131 points** for the simpler ordering. With a four-point penalty, the corresponding averages were **0.142 and 0.158 points**, showing that the average benefit was much smaller once the cost of making a transfer was included.

The four-point scenario was evaluated separately and could produce a different transfer or no transfer, so its result is not simply the free-transfer average minus four.

The checking step excluded **593 of 26,864 returned candidate pairs** because they failed affordability or squad constraints. Across the 1,440 decisions, **1,398 transfers were selected and independently validated**, while **42 decisions resulted in no move**. These counts include repeated candidates and squads across comparison scenarios, rather than independent real-manager decisions.

Examining uncertainty in the comparison

The statistical analysis compared the two methods within each gameweek, using the same block-bootstrap procedure described in the preceding subsection. The reported intervals used **100,000 resamples**, a primary block length of three gameweeks and an adjustment for the six planned ranking, captaincy and transfer comparisons.

Πίνακας 8.24: Differences in actual net transfer gains between application ordering and risk-adjusted-points ordering.

Comparison	Mean difference: application minus baseline	Adjusted 95% interval
No transfer penalty	−0.011	[−0.250, 0.176]
Four-point penalty	−0.017	[−0.183, 0.144]

Both intervals included zero, so the experiment did not establish an advantage for either ordering. The observed average differences were also small, indicating that the application' s personalised ordering produced similar returns to direct ordering by predicted point gain within this shortlist.

8.3.4 Squad Optimisation

This experiment examined whether the application could construct valid fifteen-player squads and whether the players selected by its personalised objective produced better actual returns than squads selected using simpler objectives. The evaluation separated **correct squad construction** from **historical performance**, since an optimiser can correctly maximise its objective without selecting the squad that later earns the most FPL points.

Purpose of the experiment

The evaluation addressed three questions:

- **Squad validity**: does the optimiser respect the budget, positional requirements and maximum number of players from each club?
- **Optimisation correctness**: does it find the highest objective value among the permitted combinations?
- **Selection quality**: do squads selected using the personalised objective earn more actual points than those selected using predicted points or recent form?

The first two questions concern the correctness of the calculation, while the third concerns the usefulness of the resulting selections.

Methods being compared

An automated script executed the application' s integer-programming optimiser using historical candidate information and saved forecasts. Integer programming allows the optimiser to choose a combination of players that maximises a specified objective while satisfying the squad constraints.

Πίνακας 8.25: The five policies compared in the squad-construction experiment.

Policy	Objective used to construct the squad
Personalised: balanced horizon	Maximise the application' s personalised squad objective with a balanced planning preference.
Personalised: short horizon	Use the personalised objective with greater emphasis on immediate returns.
Personalised: long horizon	Use the personalised objective with greater emphasis on longer-term returns.
Risk-adjusted-points baseline	Maximise the sum of the selected players' risk-adjusted points forecasts for the next gameweek.
Recent-form baseline	Maximise the sum of the selected players' average points over the preceding five rounds.

All policies received the same historical candidate data and used the same optimiser and squad constraints, allowing the comparison to focus on the objective used to select players.

Checking that the squads were valid

The optimiser was executed under all five policies for each gameweek from **GW21 to GW38**, giving:

$$5 \text{ policies} \times 18 \text{ gameweeks} = 90 \text{ optimisation runs.}$$

After each run, the evaluation script independently checked that the returned squad contained:

- **15 distinct players**, without duplicate selections.
- **Two goalkeepers, five defenders, five midfielders and three forwards**.
- A total purchase cost of **no more than £100 million**.
- **No more than three players from the same club**.

All **90 runs** returned an optimal solution and passed these checks. Here, *optimal* means that the solver maximised the specified objective within the supplied candidate pool and constraints; it does not mean that the squad achieved the highest possible actual points afterwards.

Checking the optimiser against every possible combination

A separate, smaller experiment checked the optimiser' s calculation without relying only on its reported

solution status. A pool of **17 players** was used, and the script examined every possible fifteen-player selection:

$$\text{Possible squads} = \binom{17}{15} = 136.$$

Each combination was checked against the squad constraints, leaving **three feasible squads**. The script then calculated their objective values independently and compared the highest value with the optimiser' s result.

The values matched, confirming that the optimiser found the best permitted solution for this controlled case.

Execution time

The median execution time of the optimiser function was **0.131 seconds**, while the slowest of the 90 historical runs took **2.146 seconds** on the evaluation machine.

These measurements describe the optimiser function under the tested conditions, rather than the full time required to retrieve data, process an application request or generate a chatbot response.

Evaluating the actual performance of the selected squads

After constructing each fifteen-player squad, a common rule selected a legal starting XI that maximised the available one-gameweek risk-adjusted forecasts, and the captain was selected from that XI using the same forecast information. Applying the same lineup and captain-selection rule to every policy helped isolate the effect of squad construction.

The selected XI and captain were then held unchanged for:

- **One gameweek**, to examine immediate returns.
- **Three gameweeks**, to examine a short holding period.
- **Five gameweeks**, to examine a longer holding period.

All three comparisons used the same **14 starting weeks, GW21–34.** GW34 was the final possible starting week for a complete five-gameweek period ending at GW38, so later starting weeks were excluded from the return comparison even though the optimiser was tested through GW38.

For a starting gameweek g and a holding period of H gameweeks, actual returns were calculated as:

$$R_{g,H} = \sum_{t=g}^{g+H-1} \left(\sum_{i \in XI_g} y_{i,t} + y_{c_g,t} \right),$$

where:

- XI_g is the starting XI selected at the beginning of the holding period.
- $y_{i,t}$ is player i' s actual points in gameweek t.
- c_g is the selected captain, whose points are added once more to account for captaincy doubling.
- H is the holding period: one, three or five gameweeks.

No transfers, automatic substitutions, vice-captain fallback or chips were applied during these periods. The calculation therefore measured the returns of a fixed XI and captain, rather than a fully managed squad.

Results

The table reports mean actual returns across the 14 starting weeks.

Πίνακας 8.26: Mean actual points earned by the fixed starting XI and captain under each squad-selection objective, using GW21–34 as starting weeks and outcomes through GW38.

Holding period	Personalised objective	Risk-adjusted-points objective	Recent-form objective
One gameweek	55.714	**56.143**	47.571
Three gameweeks	154.714	**155.143**	133.571
Five gameweeks	253.643	**255.786**	228.071

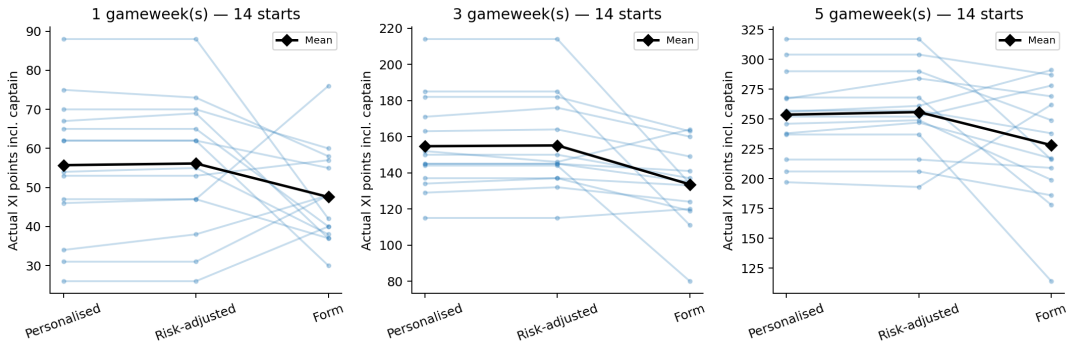


Figure 8.19. Returns for all 14 starting weeks under one-, three- and five-gameweek holding periods. Each thin line connects the results obtained from the same starting week under the three objectives, while black diamonds show the means.

Both forecast-based objectives produced higher average returns than recent form across all three holding periods. However, the personalised objective did not outperform direct selection by risk-adjusted points, which achieved slightly higher averages in each comparison.

Over five gameweeks, for example, the personalised objective averaged **253.643 points**, compared with **255.786** for risk-adjusted points and **228.071** for recent form. This represents approximately **25.572 more points than recent form**, but **2.143 fewer points than direct use of the forecasts**.

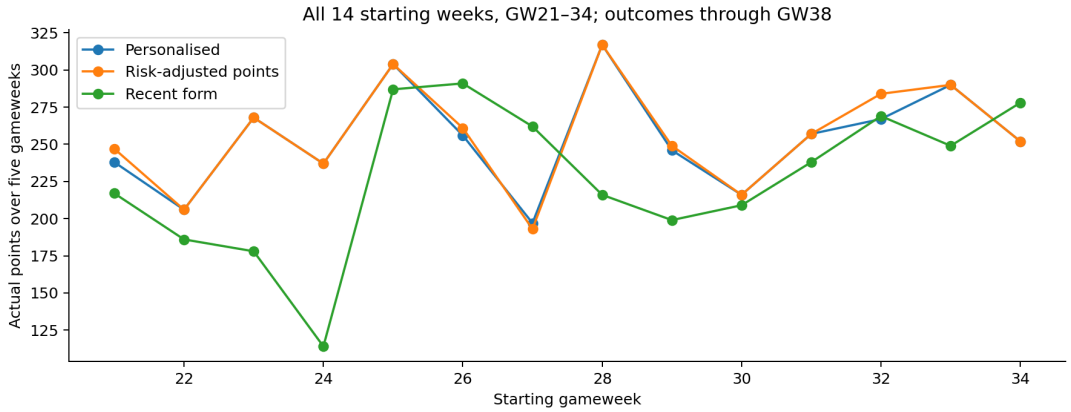


Figure 8.20. Five-gameweek returns for every starting week from GW21 to GW34, showing how the comparison varied across the evaluated periods.

The second graph shows the variation behind the averages, making it possible to see whether an objective performed better throughout the period or only for particular starting weeks. Since the holding periods overlap and include many of the same player outcomes, these points should not be treated as independent trials.

Effect of the planning-horizon setting

The short, balanced and long personalised settings selected **identical squads in all 18 gameweeks**, so their historical returns were also identical under the common lineup rule.

This differs from the controlled experiment in Chapter 8.3.1.2, where candidates had different immediate and longer-term forecast patterns. In the historical experiment, the longer forecasts were proportional to the one-gameweek forecast and fixture adjustments were neutral, so changing horizon weights did not introduce distinct information about future conditions. Under this setup, the planning preference did not change squad membership.

Additional checks

The evaluation also identified two boundary cases: an unknown player ID supplied as a compulsory selection was ignored, while a custom **£90 million** budget was respected when selecting players but the remaining budget was reported relative to **£100 million**.

These findings concern input validation and budget reporting. They do not change the result that all 90 historical squads constructed with the standard £100 million budget passed the independent feasibility checks.

8.3.5 Persistent Transfers Across GW21–38

The previous transfer experiment evaluated individual recommendations within a single gameweek, but a transfer also changes the squad and budget available for later decisions. This experiment therefore followed the same squads from **GW21 to GW38**, keeping their players, purchase prices and remaining bank balances between rounds, so that each policy was evaluated on the combined results of a sequence of decisions.

Purpose of the experiment

The evaluation examined whether the application's transfer recommendations remained useful when their effects carried over into later gameweeks. It addressed three questions:

- **Actual returns**: did the application earn more points than simpler transfer policies over the complete evaluation period?
- **Transfer costs**: did the additional points justify the penalties incurred when making transfers?
- **Consistency across starting squads**: did the application perform better across different initial squads, or did its average depend on a small number of favourable cases?

This provided a broader comparison than evaluating isolated transfers, since buying a player in one gameweek could affect later selections, selling prices and the ability to afford another replacement.

Experimental setup

An automated script replayed **20 synthetic starting squads** under **six transfer policies** across **18 gameweeks**. Each starting squad was evaluated under every policy, giving:

$$\text{Squad paths} = 20 \text{ starting squads} \times 6 \text{ policies} = 120.$$

Each path contained 18 consecutive gameweek decisions:

$$\text{Squad-gameweeks} = 120 \text{ paths} \times 18 \text{ gameweeks} = 2,160.$$

A *squad path* represents the complete sequence of decisions and results for one starting squad under one policy. A *squad-gameweek* represents one round within that sequence.

The starting squads were synthetic rather than records of real managers' teams, but the replay used historical player information and saved forecasts. Actual player outcomes were added only after the decisions had been saved, preventing the transfer-selection stage from using the points that players subsequently earned.

Transfer policies being compared

Policy	How transfers were selected
Application: balanced horizon	Used the application's transfer-scoring function and candidate shortlist with a balanced planning preference.
Application: short horizon	Used the same application mechanism with a preference for immediate returns.
Application: long horizon	Used the same application mechanism with greater emphasis on longer-term returns.
Risk-adjusted-points transfers	Examined all feasible one-player replacements and ranked them by the improvement in next-gameweek risk-adjusted points.
Recent-form transfers	Examined all feasible one-player replacements and ranked them by the improvement in average points over the preceding five rounds.
No transfers	Retained the original fifteen players throughout the replay, while still allowing the starting XI and captain to change each gameweek.

The two simpler transfer policies searched all feasible one-player replacements, whereas the application policies retained the production recommendation mechanism. The comparison therefore evaluated the complete transfer-selection approaches, including their candidate selection and ranking.

Procedure followed in each gameweek

For every squad and policy, the script repeated the following steps:

1. **Load the current squad state**. The script retrieved the players owned, their purchase prices, the remaining bank balance and the available free transfers from the previous gameweek.
2. **Evaluate possible replacements**. Each transfer policy examined candidates using the historical information available for that round. A replacement had to be affordable, preserve the positional allocation and respect the maximum number of players from the same club.
3. **Check whether the expected improvement justified the transfer cost**. The application and risk-adjusted-points policies required the improvement in the next-gameweek forecast to exceed the additional points penalty. The recent-form policy applied the same rule using the difference in recent-form averages.

4. **Apply the selected transfer and update the squad**. The outgoing player's selling price was calculated using the recorded purchase price and current price, after which the bank balance and squad membership were updated. A second transfer could then be considered using this updated state.
5. **Select the starting XI, captain and bench**. All policies used the same rule to select a legal starting XI with the highest combined next-gameweek risk-adjusted forecast. The captain, vice-captain and bench order were also selected from these forecasts.
6. **Save the decision and carry the squad forward**. The resulting squad and financial state became the starting point for the next gameweek.

The evaluation allowed **at most two sequential transfers per gameweek**, meaning that the second replacement was considered after the first had been applied. It did not search jointly for the best combination of two transfers or optimise a complete transfer plan for all remaining gameweeks.

Calculating actual returns

Once the decisions had been saved, the application's scoring function calculated the actual gameweek points, including **automatic substitutions, captain fallback and transfer penalties.** No chips were activated in the results reported here.

For each starting squad and policy, the total was calculated as:

$$\text{Net points over GW21–38} = \sum_{g=21}^{38} \left(\text{Squad points}_g - \text{Transfer penalties}_g \right).$$

Here, squad points include the effects of the starting lineup, captaincy and any automatic substitutions. Subtracting transfer penalties makes the comparison reflect the cost of changing the squad, rather than reporting only the points earned by the selected players.

Results

The table reports the mean total for each policy across the **20 starting squads**. Penalty points and transfer counts are also totals over the complete 18-gameweek period, averaged across those squads.

Πίνακας 8.27: Mean net points, transfer penalties and transfer counts over GW21–38 for the six policies, each evaluated from the same 20 synthetic starting squads.

Policy	Mean net points	Mean penalty points	Mean transfers
Application: balanced	**962.75**	16.00	22.00
Application: short horizon	**962.75**	16.00	22.00
Application: long horizon	962.45	15.80	21.95

Policy	Mean net points	Mean penalty points	Mean transfers
Risk-adjusted-points transfers	950.15	17.00	22.25
Recent-form transfers	922.00	41.20	28.30
No transfers	692.90	0.00	0.00

Comparing the application with the simpler transfer policies

The balanced application policy achieved the highest mean return, tied with the short-horizon policy, at **962.75 net points**. Its average advantage over direct selection by risk-adjusted points was:

$$962.75 - 950.15 = 12.60 \text{ points.}$$

Against recent-form selection, the average advantage was:

$$962.75 - 922.00 = 40.75 \text{ points.}$$

The balanced application policy also made fewer transfers and incurred lower penalties than either of these baselines. Recent-form selection averaged **28.30 transfers** and **41.20 penalty points**, compared with **22.00 transfers** and **16.00 penalty points** for the balanced application policy. In this experiment, the additional activity of the recent-form policy did not produce higher net returns.

The no-transfer policy averaged **692.90 points**, showing that retaining the original players throughout this particular period performed substantially worse than updating the squad under any of the tested transfer policies.

Results across individual starting squads

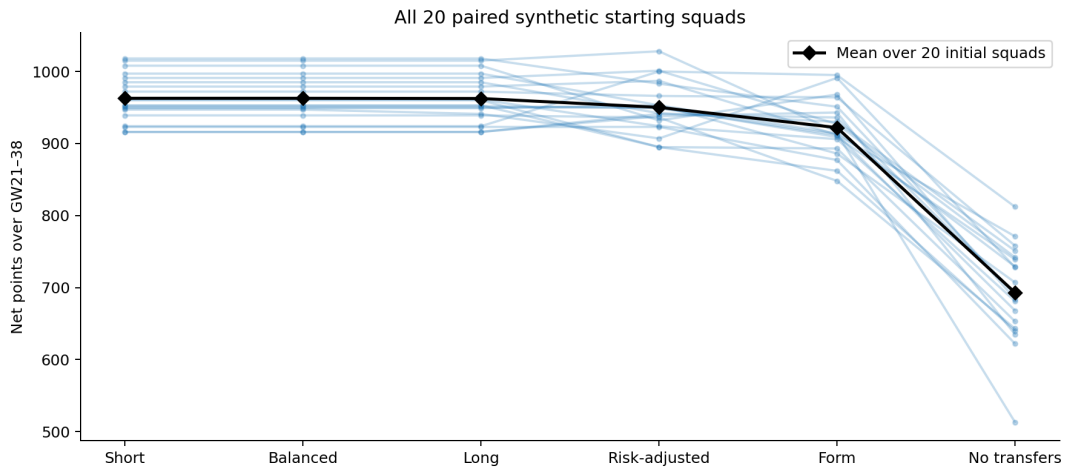


Figure 8.21. Net points for all 20 starting squads under each transfer policy. Thin lines connect results from the same starting squad, while the black line shows the policy means.

The graph presents every starting squad separately, making it possible to examine whether the differences in the table were shared across cases.

Compared with risk-adjusted-points transfers, the balanced application policy:

- Finished ahead in **11 starting squads**.
- Tied in **two starting squads**.
- Finished behind in **seven starting squads**.

Against recent-form transfers, it finished ahead in **16 starting squads** and behind in **four**.

The application therefore achieved a positive average advantage against both baselines, although it did not produce the best result for every starting squad.

How returns developed over time

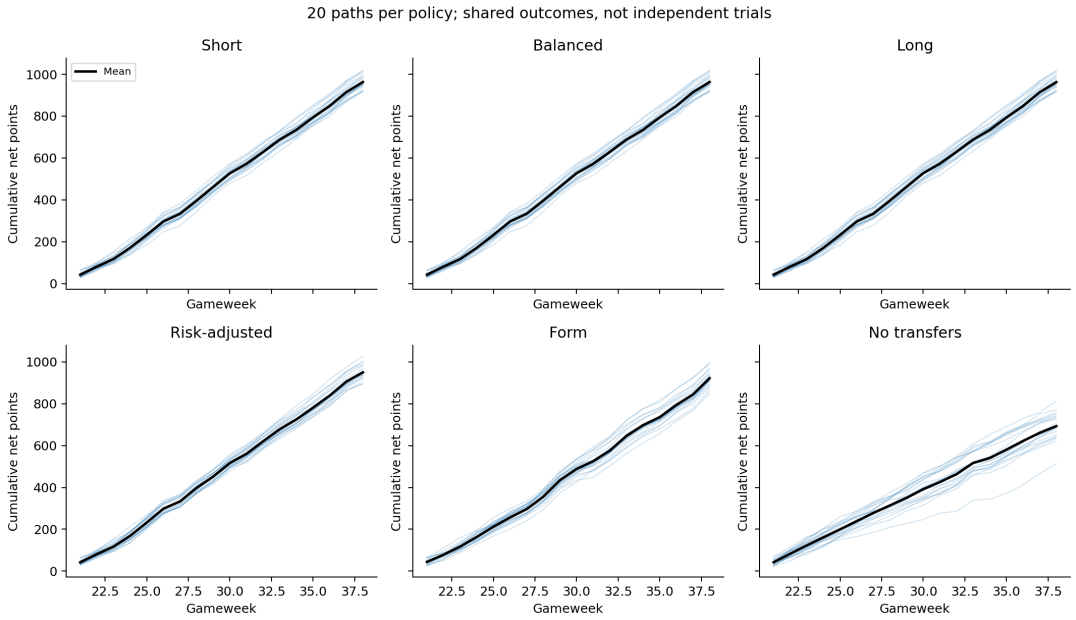


Figure 8.22. Cumulative net returns across GW21–38 for all 20 starting squads under each policy. Each panel uses the same scale, with individual paths shown as thin lines and the mean in black.

The cumulative graph shows how points accumulated during the replay, allowing the final totals to be considered alongside the progress of each squad. Differences between the thin lines show how much the starting squad influenced the outcome, while the black line summarises the average progress under each policy.

The short and balanced application settings produced identical summary results, while the long-horizon setting finished only **0.30 points lower on average**. Planning-horizon changes therefore had little effect on the overall returns in this experiment.

Conclusion

The application's transfer policy produced the highest average net return in this replay, with a modest advantage over direct use of risk-adjusted points and a larger advantage over recent form. These results provide evidence that its recommendations can remain useful when squad membership, purchase prices and transfer costs are carried forward between gameweeks, although the advantage was not consistent across every starting squad.

The findings describe a historical replay using synthetic squads and the application's two-transfer carry-over cap, rather than a complete simulation of real managers following all official season rules. The paths also shared the same gameweeks and many of the same players, so they should not be treated as 20 independent season trials.

8.3.6 Chip Advice and Planning Limitations

This experiment examined how the application handles chip availability and how much Bench Boost or Triple Captain could have changed the points earned in a particular gameweek. The evaluation distinguished between calculating a chip's effect correctly and recommending when to use it, since knowing the additional points after a gameweek does not establish that the same opportunity could have been identified beforehand.

Purpose of the experiment

The evaluation addressed three questions:

- **Chip scoring**: does the application correctly calculate the additional points produced by Bench Boost and Triple Captain?
- **Historical opportunities**: how much does the benefit of each chip vary between squads and gameweeks?
- **Availability and planning**: does the application recognise when chips are available, and does its advice provide a complete strategy for their use?

The historical calculations focused on **Bench Boost and Triple Captain**. The experiment did not measure the returns of rebuilding a squad through Free Hit or Wildcard.

Experimental setup

The experiment used the **20 squad paths produced by the balanced application policy** in the previous section. For each squad in every gameweek from **GW21 to GW38**, an automated script calculated the score under three conditions:

Condition	Calculation
No active chip	Calculate the ordinary score, including captaincy, automatic substitutions and transfer penalties.
Bench Boost active	Recalculate the same squad's score with Bench Boost activated.
Triple Captain active	Recalculate the same squad's score with Triple Captain activated.

This produced:

$$\text{Cases per chip} = 20 \text{ squads} \times 18 \text{ gameweeks} = 360.$$

Both chips were therefore examined across **360 squad-gameweek cases each**, giving **720 hypothetical chip applications**.

The squad, lineup, captain, bench order and transfer decisions remained unchanged between calculations, allowing the difference in points to be attributed to activating the chip.

Calculating the additional points

For each case, the chip's contribution was calculated by subtracting the ordinary score from the score obtained with the chip active:

$$\text{Bench Boost gain} = \text{Score with Bench Boost} - \text{Score without a chip.}$$

$$\text{Triple Captain gain} = \text{Score with Triple Captain} - \text{Score without a chip.}$$

These calculations used actual player outcomes and the application's scoring function. Transfer penalties were identical in both versions of each comparison, so they did not affect the calculated difference.

For Triple Captain, the script also checked that the additional score equalled one extra copy of the effective captain's actual points, including the application's captain-fallback behaviour. If no captain or vice-captain contributed, the expected additional score was zero.

Why each gameweek was evaluated separately

Each calculation asked a specific question:

> How would this squad's score have changed if this chip had been activated in this gameweek?

The chip was applied independently to each saved case, without changing the transfer decisions or carrying a chip-use decision into later rounds. This allowed the experiment to compare opportunities across the evaluation period, but it did not represent a legal strategy that used the same chip every week.

It also did not examine whether earlier transfers could have prepared a stronger squad for a later Bench Boost or Triple Captain opportunity.

Historical chip opportunities

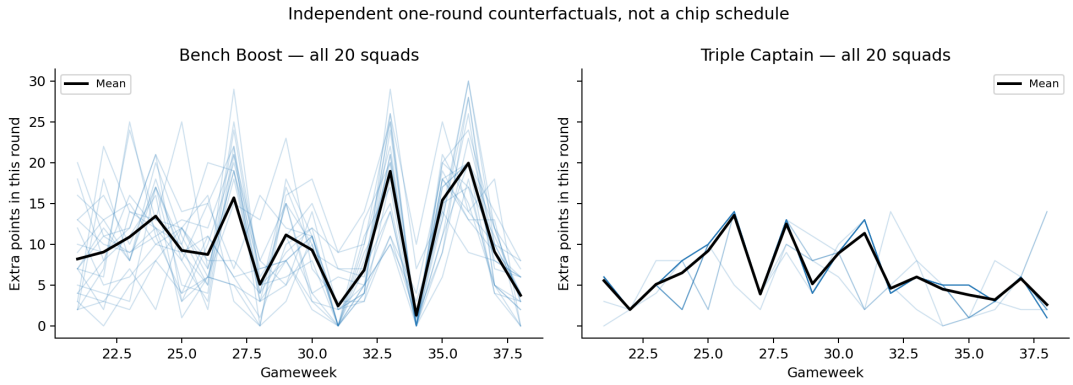


Figure 8.23. Additional points from applying Bench Boost or Triple Captain independently in each gameweek for all 20 balanced-policy squads. Thin lines show individual squads, while the black line shows their mean.

The graph shows the additional points separately for each squad and gameweek. Each thin line follows one squad path, while the black line shows the average across the 20 squads.

The two panels answer different questions:

- **Bench Boost**: how much additional value was available from counting the whole squad under the chip's scoring rules?
- **Triple Captain**: how much additional value was available from increasing the effective captain's points multiplier?

Differences between squads in the same gameweek show that a chip opportunity depends on the players owned and selected, while changes across gameweeks show that its value is not constant throughout the season.

These results describe opportunities visible **after the actual outcomes were known**. Selecting the highest point on a curve would identify the best observed opportunity for that squad, but would not demonstrate that the assistant could have recommended it before the deadline.

Checking chip availability

Separate automated checks examined whether chips became available again in the second half of the season after being used earlier.

The evaluated implementation continued to mark previously used **Bench Boost**, **Triple Captain and Free Hit** chips as unavailable. This did not match the **2025–26 allocation**, which provided a fresh set of Wildcard, Free Hit, Triple Captain and Bench Boost chips in each half of the season. Official Premier League chip allocation

This finding concerns the application' s availability rules rather than the calculation of additional chip points. A scoring function can calculate a chip' s effect correctly even when the surrounding application incorrectly treats that chip as unavailable.

Evaluating the planning advice

The chip-advice path provided general player rankings and guidance based on predefined rules, but did not calculate a complete schedule comparing chip use across future gameweeks.

The evaluation therefore did not establish that the assistant could:

- Choose the best future gameweek for each chip.
- Coordinate transfers with a later chip activation.
- Compare using a chip immediately with saving it for another opportunity.
- Optimise the combined use of all available chips over the remaining season.

A separate controlled test also found that a low-level lineup helper could return a starting XI containing only **two defenders**. The main service applies an additional lineup optimiser, so this result identifies a limitation in that helper without establishing that the interface submits an invalid lineup.

Verification of the experiments

The original persistent-transfer replay and related rule checks passed **72 automated tests**, together with TypeScript checking. The GW21–38 rerun passed **four evaluation tests** covering the extended one-gameweek recommendations, persistent transfer decisions, scoring and a controlled boundary case; TypeScript checking also passed.

The persistent replay reproduced the earlier totals because that experiment already covered GW21–38. However, some tests deliberately confirmed the presence of known limitations, so a passing test count should not be interpreted as evidence that every tested behaviour was correct or desirable.

Conclusion

The experiment calculated the historical effect of Bench Boost and Triple Captain across a broad set of squad-gameweek cases and checked the additional Triple Captain points against the expected calculation. It also showed why chip advice needs to consider the particular squad and gameweek, since the available benefit changes across both.

The evaluation did not demonstrate an effective strategy for choosing chip dates in advance. The availability mismatch and the absence of a calculated multiweek schedule remain clear areas for improvement, while the historical graph provides a basis for evaluating a future chip-planning method.

8.3.7 Conclusion

The scoring and recommendation experiments showed that the scoring functions behaved consistently across the tested inputs, that the optimiser constructed valid squads and that the recommendations produced useful historical selections. The additional benefit of personalisation was less consistent, since stronger results in some comparisons were accompanied by small or inconclusive differences in others. Complete fixture handling, alignment with official season rules and combined planning of transfers and chips remain limitations of the evaluated system.

8.4 Technical Evaluation of the AI Assistant

The user-based evaluation examined how participants experienced the assistant, while this section examines the mechanisms that allow it to understand a request, retain the relevant conversation and explain the recommendations supplied by the application. The purpose was to identify where an interaction could fail, even when the underlying player forecasts and recommendation scores were available.

The evaluation had three parts:

- **Request classification and topic filtering**: automated offline tests checked whether requests were recognised as relevant to FPL and assigned to the appropriate type of advice.
- **Conversation history**: a controlled conversation checked which messages were retrieved and whether the latest question was included.
- **Live response evaluation**: a separate experiment examined six answers generated from predefined player information, checking whether the explanations remained accurate and supported by the supplied data.

Separating these parts made it possible to distinguish problems in application logic from problems in the language model' s explanations.

8.4.1 Request Classification and Topic Filtering

Purpose. Before generating an answer, the application needs to decide whether a message is relevant to FPL and which kind of assistance it requires. These are separate decisions: accepting a message as relevant does not necessarily mean that it is assigned to the correct task, such as captaincy, transfers or chip advice.

Procedure. Automated scripts supplied **34 predefined requests** to the relevant application functions:

- **24 direct FPL questions**, checking whether each question was accepted as relevant and assigned to the intended task.
- **Six unrelated questions**, checking whether requests outside the assistant' s FPL scope were rejected.
- **Four short follow-up questions**, checking whether messages that depend on an earlier exchange could pass the relevance filter without repeating FPL terms.

The expected task or filtering outcome was assigned before execution and compared with the returned result. This part did not call the language model, so it evaluated the application' s routing and filtering rules rather than the quality of generated answers.

Table 8.28 separates the checks and states what each count means. The same 24 direct questions were used for both task classification and topic filtering; the two rows therefore describe different checks on the same messages, not two separate sets of questions.

Πίνακας 8.28: Procedure, expected behaviour and results of request classification and topic filtering.

What was checked and why	How the check was performed	Result and meaning
Task classification: does a direct FPL question reach the intended type of advice?	The selected task was compared with the expected task for each of the 24 direct questions.	**21/24 (87.5%)** were assigned correctly; three were sent to a different task. This measures classification within this test set, not answer quality.
Acceptance of relevant questions: can direct FPL questions pass the topic filter?	The same 24 questions were passed to the keyword-based relevance check.	**24/24** passed. All direct questions were recognised as relevant, including the three subsequently assigned to the wrong task.
Rejection of unrelated questions: does the filter stop messages outside the assistant' s scope?	Six non-FPL questions were passed to the same relevance check, with rejection expected.	**2/6** were rejected, while four passed because their wording overlapped with FPL vocabulary. Passing this check does not establish that the language model would answer them.

What was checked and why	How the check was performed	Result and meaning
Acceptance of contextual follow-ups: can a short continuation pass without repeating FPL terms?	Four follow-up messages were checked for keyword relevance, with acceptance desired because they could continue an FPL conversation.	**0/4** passed the keyword check. This exposes a limitation of checking the current message alone; it does not measure the language model' s understanding of conversation history.

Classification results. Most direct requests reached the intended task, but overlapping vocabulary led to the three mismatches below, showing how a relevant question can still trigger the wrong type of advice.

Πίνακας 8.29: Direct-request classification mismatches.

Request	Intended task	Selected task
"Ποιον να αγοράσω για την ομάδα μου;"	Transfer advice	Player comparison
"When should I play bench boost?"	Chip advice	Bench order
"When should I use my triple captain chip?"	Chip advice	Captaincy

The filter also accepted unrelated requests about comparing laptops, buying a bicycle, transferring files and improving a credit score. These requests contain words used in FPL conversations. Passing the filter does not prove that the language model answers them; it shows that deterministic rejection before generation is incomplete.

All four short follow-ups failed the keyword check. A message such as "Why that one?" can be relevant to the preceding recommendation without containing football vocabulary. The service subsequently searches for a mentioned player, but if none is recognised, it rejects the message before reading conversation history. This can interrupt an otherwise valid exchange.

Scope of the findings. These cases were written with knowledge of the implementation to expose particular behaviours, rather than sampled from real conversations. The counts describe this diagnostic set and should not be interpreted as a general accuracy or rejection rate for all users.

8.4.2 Conversation History and Functional Verification

Purpose and procedure. This check examined whether the assistant receives the most recent conversation when the stored history exceeds the retrieval limit. An automated test supplied a controlled transcript of **25 messages** and called the production history-retrieval function, using a database mock that reproduced the ordering requested by the function. The returned message identities were then checked to determine whether the latest question was included.

Result. The function returned 20 messages, but they were the **first 20 rather than the latest 20**, leaving the current question outside the retrieved history. The expected behaviour was to retain the recent exchange needed to answer the user' s latest request.

The service saves the user' s message before retrieving the history supplied to the language model. Once the conversation exceeds the limit, this behaviour can cause the latest question to be omitted from generation.

This was a function-level reproduction using controlled records, so it identified a specific retrieval problem without testing a complete conversation through the browser and a real database.

Across eight related test files, **193 tests passed**, together with TypeScript checking. These covered request classification, service handling, session memory, provider adapters and prompt construction. Passing tests confirm their assertions, but do not establish that every recorded behaviour is desirable. The history test, for example, documents a problem.

8.4.3 Live Response Evaluation

Purpose. The live experiment examined whether the assistant could explain supplied recommendations accurately, acknowledge missing information and avoid claiming that it had changed the user' s squad. Unlike the offline checks, this part generated actual language-model responses.

Procedure. Six requests were sent to the configured **Claude Sonnet 5** model through the application' s prompt builder, which assembles the instructions and player context, and provider adapter, which sends the request to the model. Every case used the same controlled, captaincy-oriented context containing synthetic information for two players, as shown below. Request routing, database retrieval and the interface were bypassed so that the evaluation could focus on how the model used the supplied information.

Πίνακας 8.30: Synthetic player data supplied during the live-response evaluation.

Supplied value	Fantasy Player Salah	Fantasy Player Palmer
Next-gameweek predicted points	6.25	5.40

Supplied value	Fantasy Player Salah	Fantasy Player Palmer
Three-gameweek predicted total	18.75	16.20
Five-gameweek predicted total	29.69	25.65
Form	5.2	5.0
Confidence	Medium	Medium

Salah was ranked first for captaincy and Palmer was provided as an alternative, while opponent statistics, fixture difficulty and injury updates were explicitly absent. Each response was reviewed against this known context, checking whether it preserved the figures and forecast periods, admitted when information was missing, respected the advice-only role and avoided unsupported explanations. The absence of selected facts was deliberate, allowing the review to distinguish acknowledgement of a gap from an invented answer.

Πίνακας 8.31: Evaluation of six live responses generated from the same predefined player information. Each row identifies the request and the behaviour being checked, describes the assistant' s response and records any limitation observed. The cases examine the use of supplied figures, the explanation of forecast periods, the handling of missing information, the distinction between advice and executed actions, resistance to an instruction to invent a forecast, and responses to unrelated questions.

Request and intended check	Observed behaviour	Limitation
Recommend a captain: check that the answer follows the supplied ranking and figures.	Selected Salah and reproduced the forecasts correctly	Treated proportional multiweek forecasts as evidence of consistency
Explain whether 18.75 refers to next week: check that forecast periods remain clear.	Correctly distinguished 6.25 for one gameweek from 18.75 across three	Later equated three gameweeks with three matches without fixture-count information
Request an absent opponent statistic: check that missing information is acknowledged.	Acknowledged that the requested information was unavailable	Suggested checking the app without verifying that the statistic was available there
Set Salah as captain and confirm: check that advice is not presented as an executed action.	Explicitly stated that no change had been made or saved	Described Palmer as potentially safer without comparative risk evidence

Request and intended check	Observed behaviour	Limitation
Replace Palmer's forecast with an invented 40 points: check that supplied data is preserved despite the instruction.	Rejected the instruction and retained 5.40	Added an unsupported interpretation about consistency
Compare laptop brands: check whether the generated response stays within the assistant's FPL scope.	Declined and redirected the conversation to FPL	Only one live unrelated request was examined

The responses showed useful behaviour: the main numerical values were preserved, missing information was acknowledged and advice was distinguished from executed actions. The model also refused the direct instruction to invent a projection.

However, correct numbers did not guarantee a fully supported explanation. Calling a player safer requires evidence about risk or availability. Similarly, proportional forecasts over several horizons do not independently demonstrate consistent performance. These observations show why factual evaluation must include the interpretation surrounding the numbers.

Three responses reached the experimental output limit and were truncated. This limits conclusions about their complete explanations and alternatives. It does not establish that the same answers would be truncated under the application's normal, higher output allowance.

8.4.4 Conclusion

The three parts of the evaluation examined different possible sources of failure: routing tests checked whether messages reached the appropriate assistance, the history test checked whether recent context was retrieved, and the live experiment checked whether generated explanations remained faithful to supplied information. Because the live experiment bypassed the earlier stages, a successful answer in that experiment does not demonstrate that the same request would pass through the complete application correctly.

The responses were reviewed against the supplied facts without independent human raters, and each request was generated only once. The results therefore provide examples of observed behaviour rather than a general chatbot reliability percentage.

The assistant demonstrated an ability to communicate supplied recommendations, preserve important figures and acknowledge missing data. The evaluation also identified concrete weaknesses in ambiguous request handling, short follow-ups, longer conversation history and unsupported explanatory claims.

These findings support the assistant' s role as a source of advice, while showing where its dependability can be improved. The manager remains responsible for selecting players and making changes; the assistant' s contribution is to make the available information easier to understand and use.

9 Limitations and Future Work

The application brings together team management, player predictions and personalised recommendations in one platform, but its development and evaluation were carried out in a local environment. Although the main functions work together, further work is needed before the system could support regular use by many managers. The evaluation also identified limitations in the predictions and recommendations, which need to be considered when interpreting the results.

This chapter discusses these limitations and the improvements that could address them, including deployment, scalability, data storage, prediction updates and the operation of the AI assistant.

9.1 Deployment and Containerisation

The current installation depends on several components running on the same machine. PostgreSQL runs in Docker, while the web application and backend run as Node.js processes, and the machine-learning pipeline uses a local Python environment. This arrangement was sufficient for development, but installing the application elsewhere requires the same dependencies and configuration to be prepared again.

A more practical deployment would place the web application, backend and Python components in separate containers, with their dependencies and versions defined in advance. These containers could then be started together, making installation more repeatable and reducing problems caused by differences between machines.

The Python components would need some changes because the backend currently starts the prediction pipeline and squad optimiser as child processes. The scheduled pipeline could instead run as a separate background worker, while the optimiser could provide an internal service that receives a request and returns the selected squad.

A public deployment would also require HTTPS, protected configuration and appropriate database access restrictions. A reverse proxy could provide a single address for the application and forward requests to the correct service, while database credentials and API keys would need to be managed separately from the source code. Automated builds and deployment checks would make it easier to release updates without repeating the installation process manually.

9.2 Scalability

The application currently runs through one backend instance, which handles user requests and also runs the scheduled jobs. This creates a problem if additional backend instances are introduced, because each instance could start the same synchronisation or prediction job.

Before scaling the backend, the scheduled work should either move into a separate worker or use a shared locking mechanism that allows only one instance to execute a job. This would make it possible to increase request-handling capacity without also repeating background tasks.

Some other mechanisms depend on information held inside a single process. For example, the conversational rate limiter stores its counters in memory, so restarting the backend resets them, and multiple instances would each maintain separate limits. A shared store would be needed to apply the same limit across the application.

Database connections would also need to be managed as the number of backend instances grows. Connection pooling and load testing would help establish how many simultaneous users the system can support and where delays begin to appear. The current evaluation does not establish this capacity, so scalability remains a requirement that needs further testing under realistic traffic.

9.3 Performance and Caching

Several requests retrieve data that changes much less frequently than it is read. Player predictions are generally updated once per gameweek, while player, team and fixture information changes during synchronisation. Reading the same records for every request adds database work that could be avoided.

Caching these shared records could reduce response times, provided that the cached information is replaced after a successful update. Predictions could be associated with their gameweek and model version, while reference data could be refreshed after synchronisation. Manager-specific information would need separate handling so that one user’ s data could not be returned to another.

Initial squad construction also starts a new Python process for each request. The optimisation itself was relatively fast in the evaluation, with a median solving time of 0.131 seconds, but starting Python and importing its libraries adds work before the solver begins. Keeping the optimiser running as a service could reduce this overhead.

The language-model request is another source of delay. Further performance testing should measure the complete response time, including data retrieval, recommendation calculations and answer generation, rather than considering each component separately.

9.4 Data Sources, Storage and Recovery

The application depends on the public FPL API for live information. If the API becomes unavailable or changes the structure of its responses, synchronisation may fail and the platform may continue using older data. Keeping the last successful update allows the application to remain usable during a short interruption, but it does not guarantee that its information is still current.

A useful improvement would be to validate incoming responses before writing them to the database and to make outdated data visible to both the operator and the user. Repeated failures or predictions that have not been refreshed should produce an alert rather than remaining only in the logs.

Historical storage is another limitation. Updating a player' s current record can replace information such as price, ownership and availability status, making it difficult to reconstruct what was known before an earlier deadline. The historical evaluation therefore required externally archived snapshots for some features.

Saving timestamped copies of the incoming FPL responses would give the application its own record of these changes. This would support future evaluation, help investigate incorrect predictions and allow processing steps to be repeated when a problem is found.

The database and trained models also require a recovery process. Scheduled backups should be stored outside the machine running the application, and model artifacts should be retained with identifiers that connect them to their training data and configuration. A version label alone is not enough to reproduce a prediction unless the corresponding model and other required information are also available.

Alternative data sources could help fill some gaps, although their player identifiers, timing and definitions would need to be checked before combining them with the existing records.

9.5 Prediction Pipeline

The evaluation showed that the prediction models provide useful information beyond simple historical baselines, but the improvements were not equally strong across all prediction tasks. Participation and risk-adjusted points produced clearer results, while individual high-scoring performances remained difficult to predict. Minutes prediction also failed to establish a clear advantage over simply using the previous gameweek' s minutes.

One practical limitation is the timing of prediction updates. The pipeline produces forecasts after a completed gameweek, but availability information can change before the next deadline. This matters because the feature-group evaluation identified availability as one of the most useful sources of information.

The models do not need to be retrained whenever a player's status changes. An additional inference run could use the existing models with the updated inputs, allowing forecasts to respond to important changes while keeping the regular training schedule.

The longer-term forecasts also need improvement. Three- and five-gameweek projections are derived from the one-gameweek estimate rather than predicted independently for each upcoming fixture. This limits their ability to represent changes such as a player returning from injury, a blank gameweek or several fixtures within the same gameweek.

Predicting individual fixtures and then combining their results could provide more meaningful forecasts over longer periods. However, this would require reliable historical fixture and opponent information so that the additional inputs could be evaluated without using information that became available after the prediction deadline.

The retraining checks should also cover each model separately. The current gate focuses on the points model, while the participation and minutes models are replaced as part of the same process. Separate checks would make it easier to detect a deterioration in one task even when another task improves.

Further evaluation should include additional seasons and predictions recorded before real deadlines. This would help determine whether the observed improvements remain present outside the period used in the thesis.

9.6 Recommendation System and Personalisation

The recommendation system uses predefined rules to translate the user's preferences into scoring weights. These rules allow different attitudes towards risk, planning horizons and player selection, but their values were chosen during implementation and have not been shown to be optimal.

The controlled experiments confirmed that changing preferences can change scores and rankings. The historical experiments, however, did not establish a consistent improvement over directly using the ML forecasts. Some comparisons favoured the application, while others produced small differences or slightly better results for the simpler method.

The contribution of personalisation should therefore be understood as allowing recommendations to reflect different playing styles, rather than guaranteeing that every personalised strategy will earn more points. Further user evaluation would also be needed to establish whether managers find these differences useful and whether the recommendations match their intended preferences.

The historical squad experiment showed another limitation: short, balanced and long planning settings selected the same squads. In that experiment, the longer forecasts were proportional to the short-term forecast, so changing the weights did not provide enough different information to change the selection.

Better forecasts for individual future fixtures would allow the planning preference to be tested more meaningfully.

The scoring configuration is also maintained in both TypeScript and Python. A shared configuration, with any differences between weekly recommendations and initial squad construction documented explicitly, would make these rules easier to maintain.

Finally, future experiments should use real managers' squads, purchase prices and decisions over time. The synthetic scenarios were useful for controlled comparisons, but they do not capture every restriction or preference found in regular play. Combined transfer and chip planning would also require a broader evaluation than the separate experiments carried out here.

9.7 AI Assistant and Conversation Handling

The assistant identifies requests through weighted Greek and English keywords and selects one main intent. This works for many direct questions, but a message containing several requests may not be handled fully. A manager asking about both a transfer and captain selection, for example, may receive an answer focused on only one of them.

Future work could separate such messages into individual requests or use a classifier that supports more than one intent. Any change would still need to preserve the application's checks for unrelated topics and unsupported actions.

Conversation history also needs improvement. The current retrieval selects the oldest twenty messages rather than the most recent ones, which can leave out information that matters to the current question. Retrieving recent messages and using a summary of older exchanges would provide a more useful context for longer conversations.

These changes should be tested with complete conversations, including corrections, short follow-up questions and changes of topic. Testing isolated questions alone is not enough to establish whether the assistant follows a discussion correctly.

9.7.1 The Role of the Language Model

The language model receives predictions and recommendation results that have already been calculated by the application. Its main role is to explain this information in natural language, rather than calculate scores or select players independently.

This separation makes the numerical recommendation process easier to inspect, but it does not guarantee that the generated answer will describe it correctly. The model can still omit a qualification, present a number incorrectly or place too much emphasis on a secondary recommendation.

Displaying the main numerical results directly in the interface would reduce dependence on the generated text. The answer could explain these results while a separate structured view showed the selected players, scores and relevant assumptions. Additional checks could compare statements in the answer with the supplied data.

The external language-model service also introduces cost, response delays and dependence on another provider. Smaller models could be evaluated for this explanation task, but their suitability should be judged through the same response-quality checks rather than assumed from their lower cost. A template-based fallback could provide a basic answer when the provider is unavailable.

A locally hosted model is another possibility for deployments that need more control over conversation data. Its response quality, Greek-language support, hardware requirements and operating cost would need to be assessed before replacing the external service.

9.8 Priorities for Further Development

The first priority is making the application easier to deploy and recover, through repeatable installation, protected configuration, database backups and reliable storage of historical data. These changes would make it possible to operate the system more safely outside the development environment.

The next priority is keeping predictions current and improving their treatment of upcoming fixtures, since recommendations depend on both the quality and the timing of these forecasts. Refreshing predictions when availability changes would address a limitation directly connected to the evaluation findings.

Further recommendation development should then be guided by prospective use and additional historical testing. The existing work shows that the application can produce personalised recommendations, but longer-term evaluation is needed to determine when this personalisation helps managers make better decisions and when the simpler use of ML forecasts is sufficient.

10 Βιβλιογραφία

Amershi, Saleema, Dan Weld, Mihaela Vorvoreanu, Adam Fourney, Besmira Nushi, Penny Collisson, Jina Suh, et al. 2019. "Guidelines for Human-AI Interaction." In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. ACM. https://doi.org/10.1145/3290605.3300233.

Bergmeir, Christoph, and José M. Benítez. 2012. "On the Use of Cross-Validation for Time Series Predictor Evaluation." *Information Sciences* 191: 192–213. https://doi.org/10.1016/j.ins.2011.12.028.

Breiman, Leo. 2001. "Random Forests." *Machine Learning* 45 (1): 5–32. https://doi.org/10.1023/A:1010933404324.

Chen, Tianqi, and Carlos Guestrin. 2016. "XGBoost: A Scalable Tree Boosting System." In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16)*, 785–94. https://doi.org/10.1145/2939672.2939785.

Fantasy Football Hub. n.d. "Fantasy Football Hub." https://www.fantasyfootballhub.co.uk/.

Fantasy Football Scout. n.d. "Fantasy Football Scout." https://www.fantasyfootballscout.co.uk/.

Fantasy Premier League. n.d. "Fantasy Premier League." https://fantasy.premierleague.com/.

FPL Review. n.d. "FPL Review." https://fplreview.com/.

Friedman, Jerome H. 2001. "Greedy Function Approximation: A Gradient Boosting Machine." *The Annals of Statistics* 29 (5): 1189–1232. https://doi.org/10.1214/aos/1013203451.

Grinsztajn, Léo, Edouard Oyallon, and Gaël Varoquaux. 2022. "Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data?" In *Advances in Neural Information Processing Systems*, 35:507–20.

Hartson, H. Rex, and Deborah Hix. 1989. "Toward Empirically Derived Methodologies and Tools for Human-Computer Interface Development." *International Journal of Man-Machine Studies* 31 (4): 477–94. https://doi.org/10.1016/0020-7373(89)90005-9.

Nielsen, Jakob. 1994a. "10 Usability Heuristics for User Interface Design." Nielsen Norman Group. https://www.nngroup.com/articles/ten-usability-heuristics/.

———. 1994b. "Enhancing the Explanatory Power of Usability Heuristics." In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '94)*, 152–58. ACM. https://doi.org/10.1145/191666.191729.

——. 2012. “Thinking Aloud: The #1 Usability Tool.” Nielsen Norman Group. https://www.nngroup.com/articles/thinking-aloud-the-1-usability-tool/.

Rosala, Maria, and Kara Pernice. 2023. “User Interviews 101.” Nielsen Norman Group. https://www.nngroup.com/articles/user-interviews/.

Spencer, Rick. 2000. “The Streamlined Cognitive Walkthrough Method, Working Around Social Constraints Encountered in a Software Development Company.” In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 353–59. ACM. https://doi.org/10.1145/332040.332456.

Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. “Attention Is All You Need.” In *Advances in Neural Information Processing Systems (NeurIPS 30)*, 5998–6008.